

TREINAMENTOS

Desenvolvimento Web com ASP.NET MVC 4

Desenvolvimento Web com ASP.NET MVC 4

25 de abril de 2016
As apostilas atualizadas estão disponíveis em **www.k19.com.br**

Esta apostila contém:

- 225 exercícios de fixação.
- 11 exercícios complementares.
- 1 desafios.
- 0 questões de prova.

Sumário i

Sobre a K19 1

Seguro Treinamento 2

Termo de Uso 3

Cursos 4

Preparação do ambiente 5

1 Banco de Dados 1
1.1 Sistemas Gerenciadores de Banco de Dados . . . 1
1.2 SQL Server . . . 2
1.3 Bases de Dados (*Databases*) . . . 2
1.4 Tabelas . . . 3
1.5 CRUD . . . 5
1.6 Chaves Primária e Estrangeira . . . 9
1.7 Exercícios de Fixação . . . 9

2 ADO.NET 23
2.1 Driver . . . 24
2.2 ODBC . . . 25
2.3 ODBC Manager . . . 25

2.4 Criando uma conexão . . . 26
2.5 Apagando uma base de dados . . . 27
2.6 Criando uma base de dados . . . 28
2.7 Criando uma tabela . . . 28
2.8 Inserindo registros . . . 29
2.9 Exercícios de Fixação . . . 29
2.10 Exercícios Complementares . . . 31
2.11 SQL Injection . . . 32
2.12 Exercícios de Fixação . . . 33
2.13 Exercícios Complementares . . . 34
2.14 Listando registros . . . 34
2.15 Exercícios de Fixação . . . 35
2.16 Exercícios Complementares . . . 36
2.17 Connection Factory . . . 36
2.18 Exercícios de Fixação . . . 37
2.19 Exercícios Complementares . . . 40

3 Entity Framework 43
3.1 Múltiplas sintaxes da linguagem SQL . . . 43
3.2 Orientação a Objetos VS Modelo Relacional . . . 43
3.3 Ferramentas ORM . . . 44
3.4 Instalando o Entity Framework . . . 45
3.5 Configuração . . . 46
3.6 Mapeamento . . . 48
3.7 Exercícios de Fixação . . . 53
3.8 Gerenciamento de entidades . . . 55
3.9 Exercícios de Fixação . . . 57
3.10 Tipos complexos . . . 59
3.11 Exercícios de Fixação . . . 61
3.12 Relacionamentos . . . 63
3.13 One to One . . . 63
3.14 Exercícios de Fixação . . . 65
3.15 One to Many . . . 66
3.16 Exercícios de Fixação . . . 68
3.17 Many to One . . . 69
3.18 Exercícios de Fixação . . . 70
3.19 Many to Many . . . 72
3.20 Exercícios de Fixação . . . 73
3.21 Relacionamentos Bidirecionais . . . 75
3.22 Exercícios de Fixação . . . 77
3.23 Consultas . . . 79
3.24 Exercícios de Fixação . . . 81
3.25 Sistema de Controle de Finanças Pessoais . . . 84
3.26 Exercícios de Fixação . . . 84
3.27 Desafios . . . 107

4 Visão Geral do ASP.NET MVC 109
4.1 Necessidades de uma aplicação web . . . 109
4.2 ASP.NET MVC . . . 110
4.3 MVC e Front Controller . . . 110

4.4 Visual Studio . . . 111
4.5 Exemplo de uma Aplicação Web . . . 113
4.6 Exercícios de Fixação . . . 114
4.7 Integração com Entity Framework . . . 118
4.8 Scaffold . . . 118
4.9 Exercícios de Fixação . . . 118

5 Camada de Apresentação 125
5.1 Razor e ASPX . . . 125
5.2 Exercícios de Fixação . . . 127
5.3 Exercícios Complementares . . . 128
5.4 ViewBag e Strogly Typed Views . . . 129
5.5 Exercícios de Fixação . . . 131
5.6 HTML Helpers . . . 134
5.7 Exercícios de Fixação . . . 143
5.8 Layouts . . . 147
5.9 Exercícios de Fixação . . . 151
5.10 Partial views . . . 156
5.11 Exercícios de Fixação . . . 158

6 Camada de Controle 161
6.1 Actions . . . 161
6.2 ActionResult . . . 162
6.3 Parâmetros . . . 163
6.4 Exercícios de Fixação . . . 164
6.5 TempData . . . 167
6.6 Exercícios de Fixação . . . 168
6.7 Rotas . . . 169
6.8 Exercícios de Fixação . . . 171

7 Validação 175
7.1 Controller . . . 175
7.2 View . . . 176
7.3 Exercícios de Fixação . . . 177
7.4 Anotações . . . 180
7.5 Validação no lado do Cliente . . . 181
7.6 Exercícios de Fixação . . . 182

8 Sessão 185
8.1 Identificando os navegadores . . . 185
8.2 Sessões no ASP.NET MVC . . . 185
8.3 Session Mode . . . 186
8.4 Exercícios de Fixação . . . 187

9 Filtros 193
9.1 Tipos de Filtros . . . 193
9.2 Filtro de Autorização . . . 194
9.3 Exercícios de Fixação . . . 196
9.4 Filtro de Autenticação . . . 202
9.5 Exercícios de Fixação . . . 204

9.6 Filtro de Ação . . . 206
9.7 Exercícios de Fixação . . . 207
9.8 Filtro de Resultado . . . 210
9.9 Exercícios de Fixação . . . 212
9.10 Filtro de Exceção . . . 212
9.11 Exercícios de Fixação . . . 213

10 Tratamento de Erros 217
10.1 Try-Catch . . . 218
10.2 Custom Errors . . . 218
10.3 Http Errors . . . 219
10.4 Exercícios de Fixação . . . 220

A ASP.NET Web API 225
A.1 REST . . . 225
A.2 Resources . . . 226
A.3 URIs . . . 226
A.4 Operações . . . 226
A.5 Media Type . . . 228
A.6 Exercícios de Fixação . . . 228

B Code First Migrations 233
B.1 Code First Migrations . . . 233
B.2 Exercícios de Fixação . . . 239
B.3 Acrescentando uma nova entidade . . . 243
B.4 Exercícios de Fixação . . . 245
B.5 Operações do Code First Migrations . . . 246
B.6 Exercícios de Fixação . . . 247
B.7 Migração para uma versão específica . . . 249
B.8 Gerando Script SQL . . . 250

C Respostas 251



Sobre a K19

A K19 é uma empresa especializada na capacitação de desenvolvedores de software. Sua equipe é composta por profissionais formados em Ciência da Computação pela Universidade de São Paulo (USP) e que possuem vasta experiência em treinamento de profissionais para área de TI.

O principal objetivo da K19 é oferecer treinamentos de máxima qualidade e relacionados às principais tecnologias utilizadas pelas empresas. Através desses treinamentos, seus alunos tornam-se capacitados para atuar no mercado de trabalho.

Visando a máxima qualidade, a K19 mantém as suas apostilas em constante renovação e melhoria, oferece instalações físicas apropriadas para o ensino e seus instrutores estão sempre atualizados didática e tecnicamente.



Seguro Treinamento

Na K19 o aluno faz o curso quantas vezes quiser!

Comprometida com o aprendizado e com a satisfação dos seus alunos, a K19 é a única que possui o Seguro Treinamento. Ao contratar um curso, o aluno poderá refazê-lo quantas vezes desejar mediante a disponibilidade de vagas e pagamento da franquia do Seguro Treinamento.

As vagas não preenchidas até um dia antes do início de uma turma da K19 serão destinadas ao alunos que desejam utilizar o Seguro Treinamento. O valor da franquia para utilizar o Seguro Treinamento é 10% do valor total do curso.



Termo de Uso

Termo de Uso

Todo o conteúdo desta apostila é propriedade da K19 Treinamentos. A apostila pode ser utilizada livremente para estudo pessoal . Além disso, este material didático pode ser utilizado como material de apoio em cursos de ensino superior desde que a instituição correspondente seja reconhecida pelo MEC (Ministério da Educação) e que a K19 seja citada explicitamente como proprietária do material.

É proibida qualquer utilização desse material que não se enquadre nas condições acima sem o prévio consentimento formal, por escrito, da K19 Treinamentos. O uso indevido está sujeito às medidas legais cabíveis.



Conheça os nossos cursos



K01- Lógica de Programação



K02 - Desenvolvimento Web com HTML, CSS e JavaScript



K03 - SQL e Modelo Relacional



K11 - Orientação a Objetos em Java



K12 - Desenvolvimento Web com JSF2 e JPA2



K21 - Persistência com JPA2 e Hibernate



K22 - Desenvolvimento Web Avançado com JFS2, EJB3.1 e CDI



K23 - Integração de Sistemas com Webservices, JMS e EJB



K41 - Desenvolvimento Mobile com Android



K51 - Design Patterns em Java



K52 - Desenvolvimento Web com Struts



K31 - C# e Orientação a Objetos



K32 - Desenvolvimento Web com ASP.NET MVC

www.k19.com.br/cursos

Preparação do ambiente

Para realizar os exercícios desta apostila, você deve utilizar o sistema operacional Windows 7 ou Windows 8. Siga os tutoriais abaixo para preparar o seu ambiente.

- www.k19.com.br/artigos/como-instalar-o-visual-studio-express-2013-para-windows-desktop
- www.k19.com.br/artigos/como-instalar-o-visual-studio-express-2013-para-web
- www.k19.com.br/artigos/como-instalar-o-sql-server-express-2012/

CAPÍTULO 1

BANCO DE DADOS

Em geral, as aplicações necessitam armazenar dados de forma persistente para consultá-los posteriormente. Por exemplo, a aplicação de uma livraria precisa armazenar os dados dos livros e dos autores de forma persistente.

Suponha que esses dados sejam armazenados em arquivos do sistema operacional. Vários fatores importantes nos levam a descartar tal opção. A seguir, apresentamos as principais dificuldades a serem consideradas na persistência de dados.

Segurança: O acesso às informações potencialmente confidenciais deve ser controlado de forma que apenas usuários e sistemas autorizados possam manipulá-las.

Integridade: Restrições relacionadas aos dados armazenados devem ser respeitadas para que as informações estejam sempre consistentes.

Consulta: O tempo gasto para realizar as consultas aos dados armazenados deve ser o menor possível.

Concorrência: Em geral, diversos sistemas e usuários acessarão concorrentemente as informações armazenadas. Apesar disso, a integridade dos dados deve ser preservada.

Considerando todos esses aspectos, concluímos que um sistema complexo seria necessário para persistir as informações de uma aplicação de maneira adequada. Felizmente, tal tipo de sistema já existe e é conhecido como **Sistema Gerenciador de Banco de Dados** (SGBD).

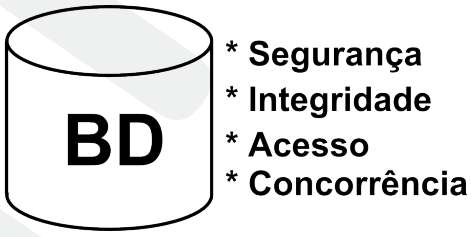


Figura 1.1: Sistema Gerenciador de Banco de Dados

Sistemas Gerenciadores de Banco de Dados

No mercado, há diversas opções de sistemas gerenciadores de banco de dados. Os mais populares são:

- Oracle Database

- SQL Server
- MySQL Server
- PostgreSQL



SQL Server

Neste treinamento, utilizaremos o SQL Server Express, que é mantido pela Microsoft. O SQL Server Express pode ser obtido a partir do site:

`http://www.microsoft.com/express/Database/`.

Microsoft SQL Server Management Studio Express

Para interagir com o SQL Server Express, utilizaremos um cliente com interface gráfica chamado de Microsoft SQL Server Management Studio Express.



Bases de Dados (*Databases*)

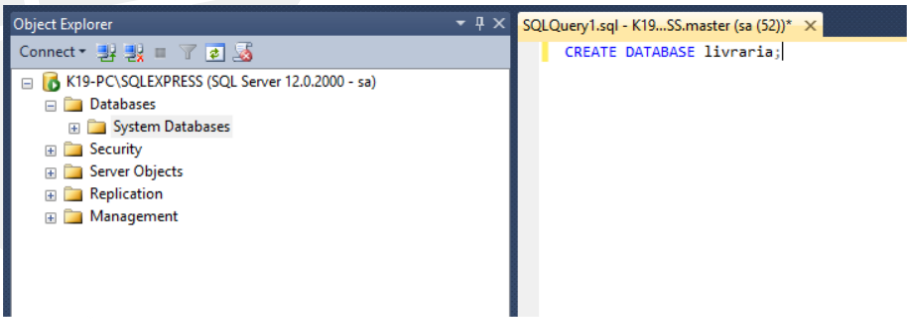
Um sistema gerenciador de banco de dados é capaz de gerenciar informações de diversos sistemas ao mesmo tempo. Por exemplo, as informações dos clientes de um banco, além dos produtos de uma loja virtual ou dos livros de uma livraria.

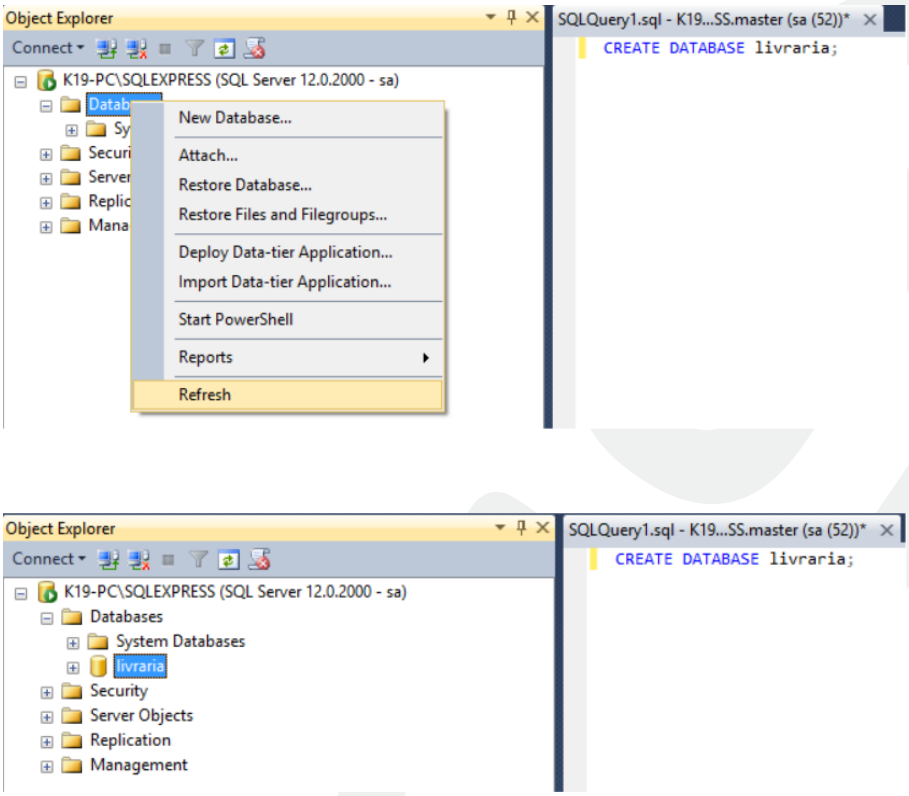
Suponha que os dados fossem mantidos sem nenhuma separação lógica. Implementar regras de segurança específicas seria extremamente complexo. Tais regras criam restrições quanto ao conteúdo que pode ser acessado por cada usuário. Por exemplo, determinado usuário poderia ter permissão de acesso aos dados dos clientes do banco, mas não às informações dos produtos da loja virtual, ou dos livros da livraria.

Para obter uma organização melhor, os dados são armazenados separadamente em um SGDB. Daí surge o conceito de **base de dados** (database). Uma base de dados é um agrupamento lógico das informações de um determinado domínio.

Criando uma base de dados no SQL Server Express

Para criar uma base de dados no SQL Server Express, utilizamos o comando **CREATE DATABASE**.





Repare que além da base de dados **livraria** há outras bases. Essas bases foram criadas automaticamente pelo próprio SQL Server Express para teste ou para guardar algumas configurações.

Quando uma base de dados não é mais necessária, ela pode ser removida através do comando **DROP DATABASE**.

Tabelas

Um servidor de banco de dados é dividido em bases de dados com o intuito de separar as informações de domínios diferentes. Nessa mesma linha de raciocínio, podemos dividir os dados de uma base a fim de agrupá-los segundo as suas correlações. Essa separação é feita através de **tabelas**. Por exemplo, no sistema de um banco, é interessante separar o saldo e o limite de uma conta, do nome e CPF de um cliente. Então, poderíamos criar uma tabela para os dados relacionados às contas e outra para os dados relacionados aos clientes.

Cliente		
nome	idade	cpf
José	27	31875638735
Maria	32	30045667856

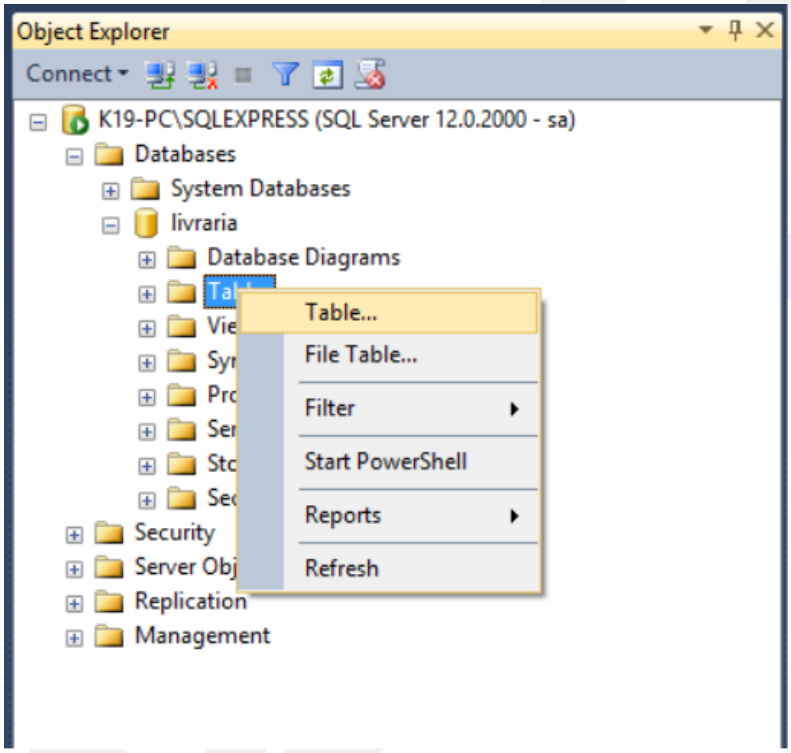
Conta		
numero	saldo	limite
1	1000	500
2	2000	700

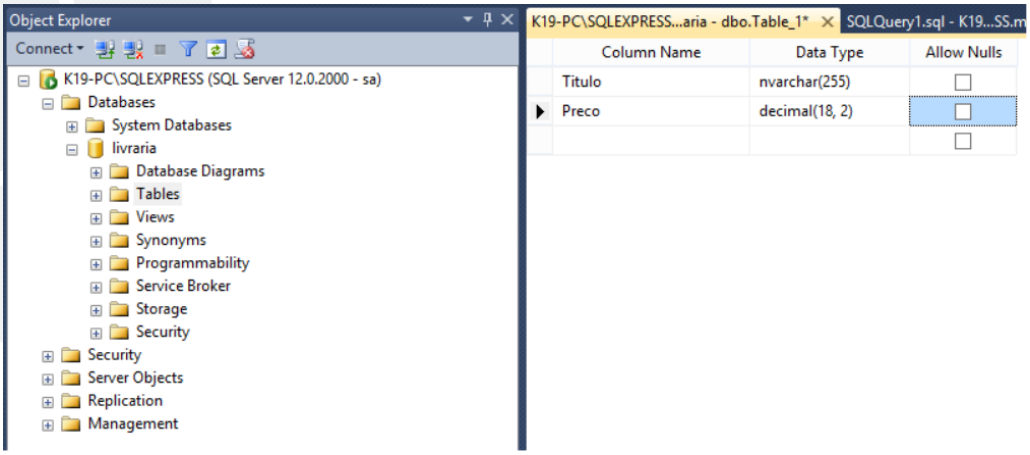
Tabela 1.1: Tabelas para armazenar os dados relacionados aos clientes e às contas

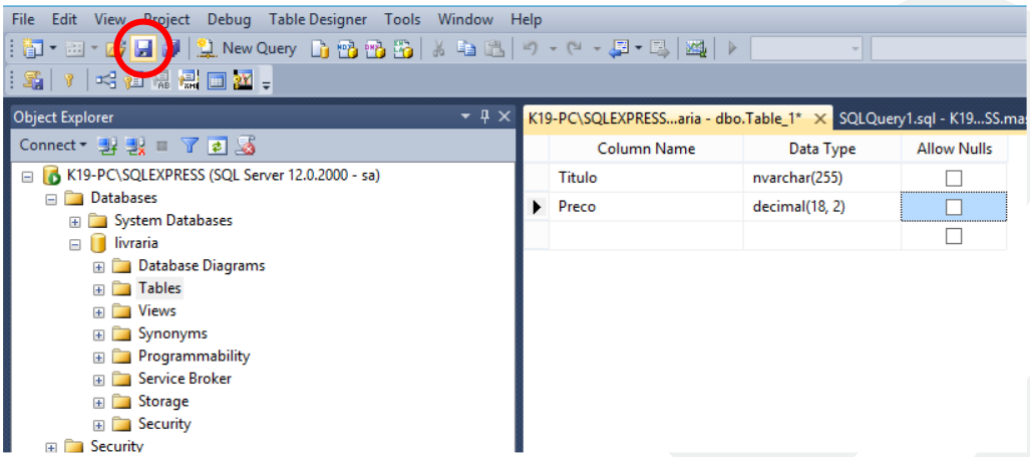
Uma tabela é formada por **registros** (linhas) e os registros são formados por **campos** (colunas). Por exemplo, considere uma tabela para armazenar as informações dos clientes de um banco. Cada registro dessa tabela armazena em seus campos os dados de um determinado cliente.

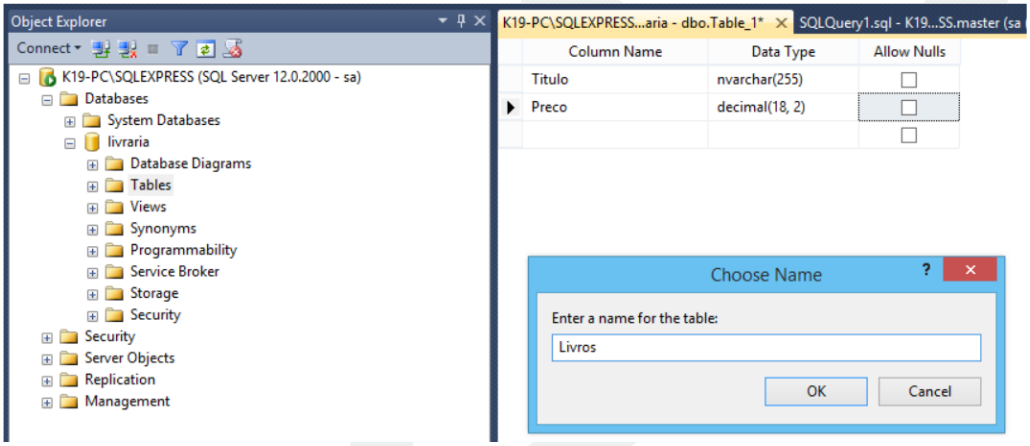
Criando tabelas no SQL Server Express

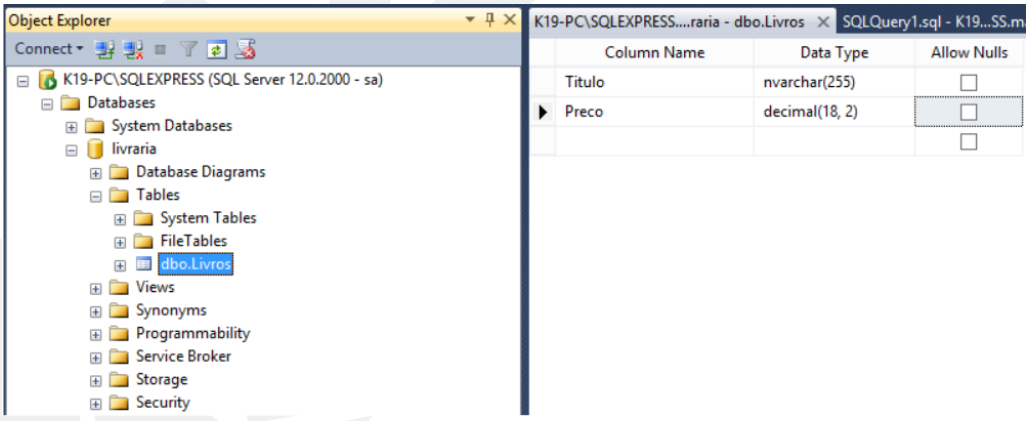
As tabelas no SQL Server Express são criadas através do comando **CREATE TABLE**. Na criação de uma tabela é necessário definir quais são os nomes e os tipos das colunas.











No SQL Server os nomes das tabelas são precedidas pelo ID do usuário que possui a tabela. No caso do usuário **sa**, o ID é **dbo**. Portanto o nome da tabela Livros fica *dbo.Livros*.

Se uma tabela não for mais desejada ela pode ser removida através do comando **DROP TABLE**.

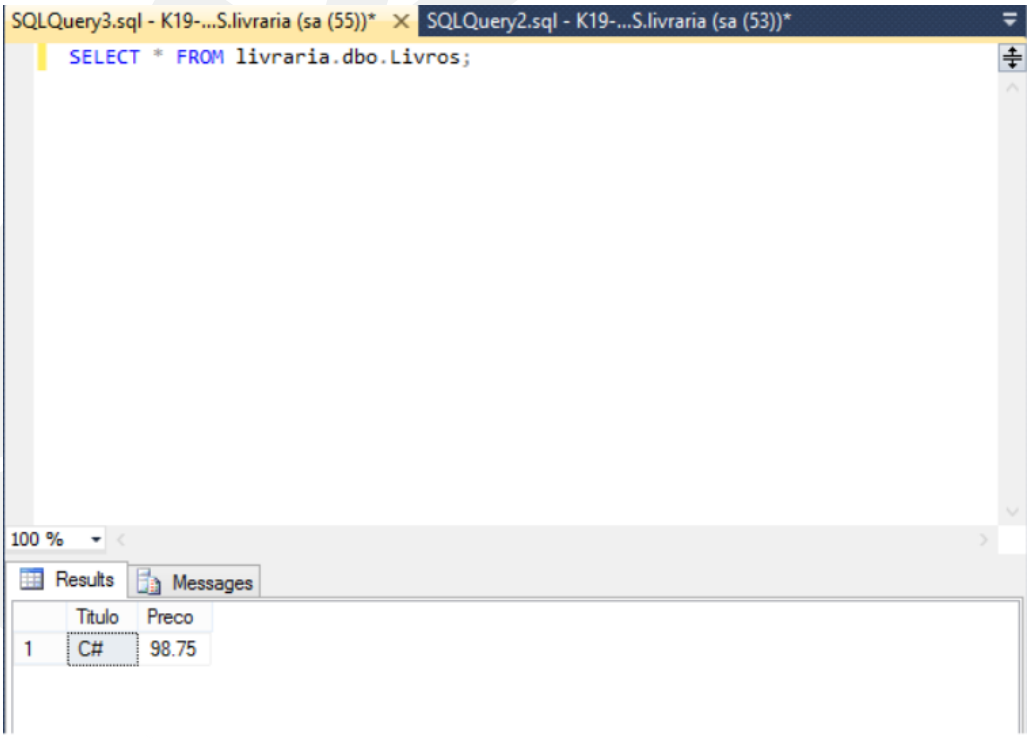


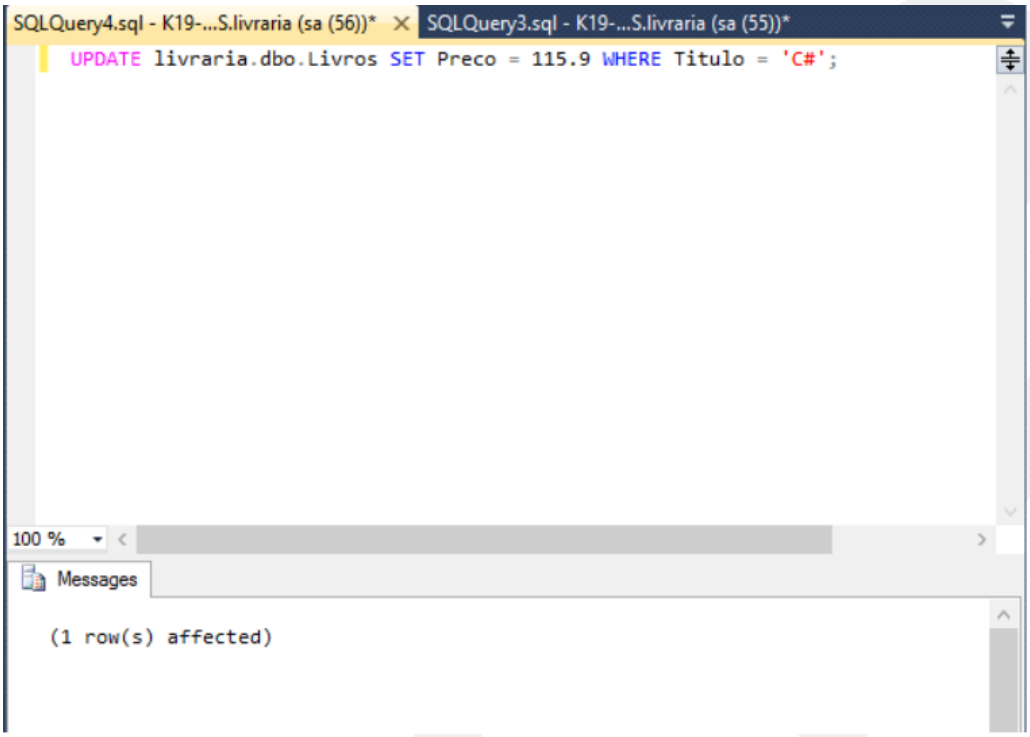
CRUD

As operações básicas para manipular os dados persistidos são: inserir, ler, alterar e remover.

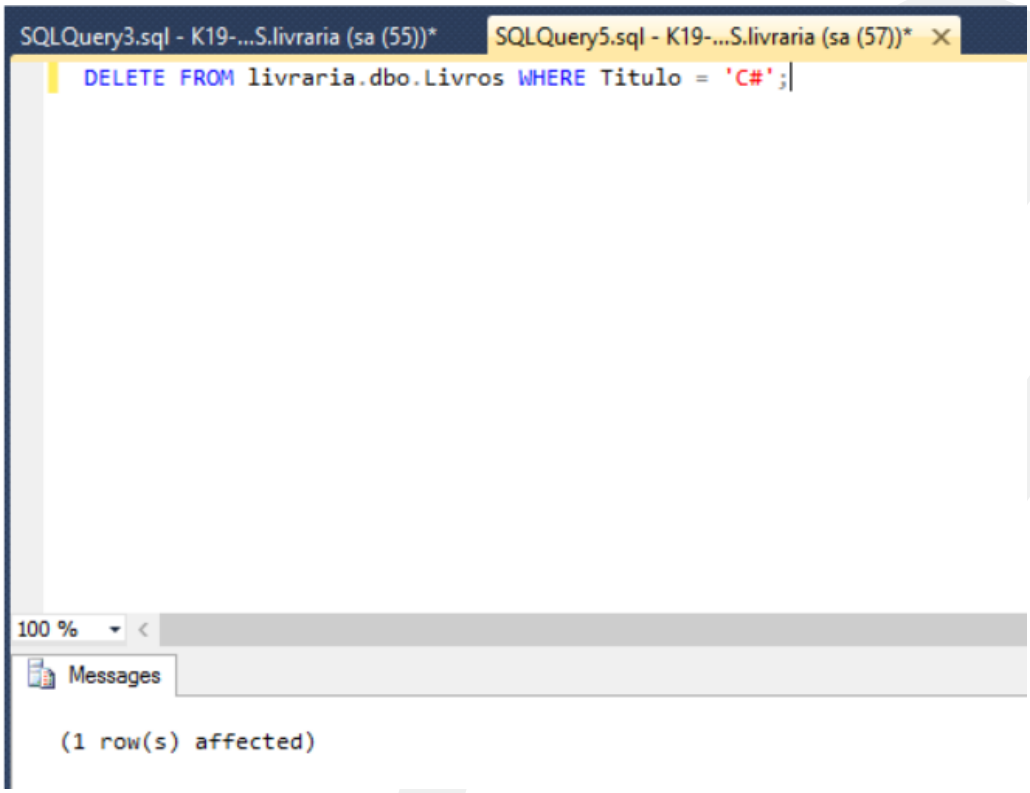
Essas operações são realizadas através de uma linguagem de consulta denominada **SQL** (*Structured Query Language*). Essa linguagem oferece quatro comandos básicos: **INSERT**, **SELECT**, **UPDATE** e **DELETE**. Esses comandos são utilizados para inserir, ler, alterar e remover registros, respectivamente.

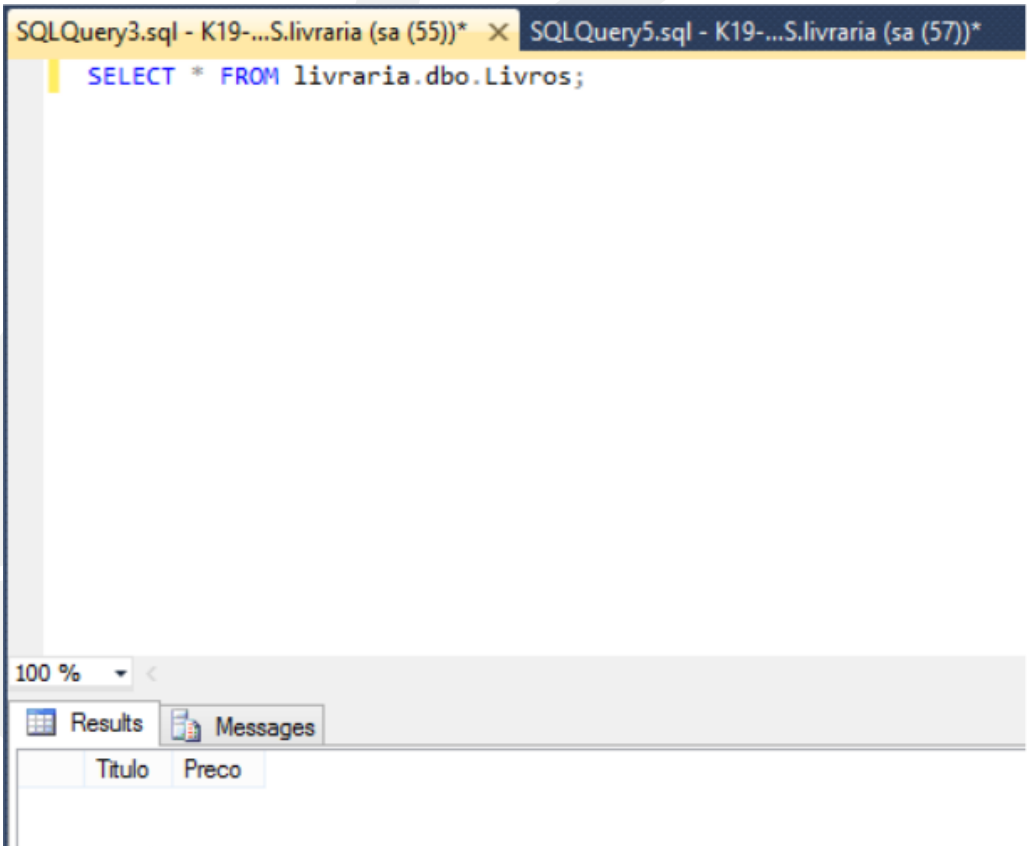





SQLQuery4.sql - K19-...S.livraria (sa (56))*
SQLQuery3.sql - K19-...S.livraria (sa (55))*
UPDATE livraria.dbo.Livros SET Preco = 115.9 WHERE Titulo = 'C#';
100 %
Messages
(1 row(s) affected)


SQLQuery4.sql - K19-...S.livraria (sa (56))*
SQLQuery3.sql - K19-...S.livraria (sa (55))*
SELECT * FROM livraria.dbo.Livros;
100 %
Results
Messages
Titulo
Preco
1
C#
115.90


SQLQuery3.sql - K19-...S.livraria (sa (55))*
SQLQuery5.sql - K19-...S.livraria (sa (57))*
DELETE FROM livraria.dbo.Livros WHERE Titulo = 'C#';
100 %
Messages
(1 row(s) affected)


SQLQuery3.sql - K19-...S.livraria (sa (55))*
SQLQuery5.sql - K19-...S.livraria (sa (57))*
SELECT * FROM livraria.dbo.Livros;
100 %
Results
Messages
Titulo
Preco



Chaves Primária e Estrangeira

Suponha que os livros da nossa livraria sejam classificados por editoras. As editoras possuem nome e telefone. Para armazenar esses dados, uma nova tabela deveria ser criada.

Nesse momento, teríamos duas tabelas (Livro e Editora). Constantemente, a aplicação da livraria deverá descobrir qual é a editora de um determinado livro ou quais são os livros de uma determinada editora. Para isso, os registros da tabela Editora devem estar relacionados aos da tabela Livro.

Na tabela Livro, poderíamos adicionar uma coluna para armazenar o nome da editora dos livros. Dessa forma, se alguém quiser recuperar as informações da editora de um determinado livro, deve consultar a tabela Livro para obter o nome da editora correspondente. Depois, com esse nome, deve consultar a tabela Editora para obter as informações da editora.

Porém, há um problema nessa abordagem. A tabela Editora aceita duas editoras com o mesmo nome. Dessa forma, eventualmente, não conseguiríamos descobrir os dados corretos da editora de um determinado livro. Para resolver esse problema, deveríamos criar uma restrição na tabela Editora que proíba a inserção de editoras com o mesmo nome.

Para resolver esse problema no SQL Server Express, poderíamos adicionar a propriedade **UNIQUE** no campo nome da tabela Editora. Porém, ainda teríamos mais um problema. Na tabela Livro, poderíamos adicionar registros vinculados a editoras inexistentes, pois não há nenhuma relação explícita entre as tabelas. Para solucionar esses problemas, devemos utilizar o conceito de **chave primária** e **chave estrangeira**.

Toda tabela pode ter uma chave primária, que é um conjunto de um ou mais campos que devem ser únicos para cada registro. Normalmente, um campo numérico é escolhido para ser a chave primária de uma tabela, pois as consultas podem ser realizadas com melhor desempenho.

Então, poderíamos adicionar um campo numérico na tabela Editora e torná-lo chave primária. Vamos chamar esse campo de **id**. Na tabela Livro, podemos adicionar um campo numérico chamado **editora_id** que deve ser utilizado para guardar o valor da chave primária da editora correspondente ao livro. Além disso, o campo editora_id deve estar explicitamente vinculado com o campo id da tabela Editora. Para estabelecer esse vínculo, o campo editora_id da tabela Livro deve ser uma chave estrangeira associada à chave primária da tabela Editora.

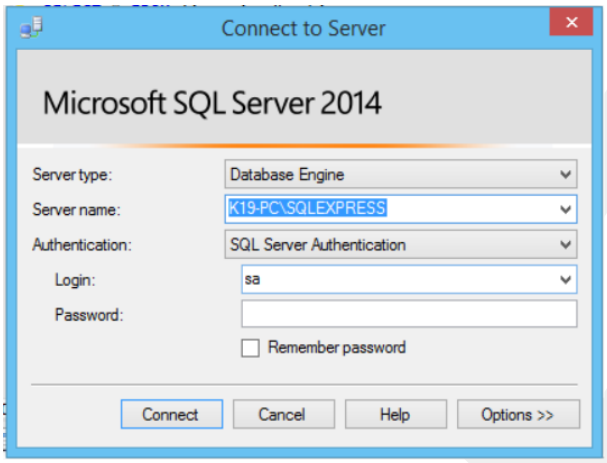
Uma chave estrangeira é um conjunto de uma ou mais colunas de uma tabela que possuem valores iguais aos da chave primária de outra tabela.

Com a definição da chave estrangeira, um livro não pode ser inserido com o valor do campo editora_id inválido. Caso tentássemos fazer isso, obteríamos uma mensagem de erro.

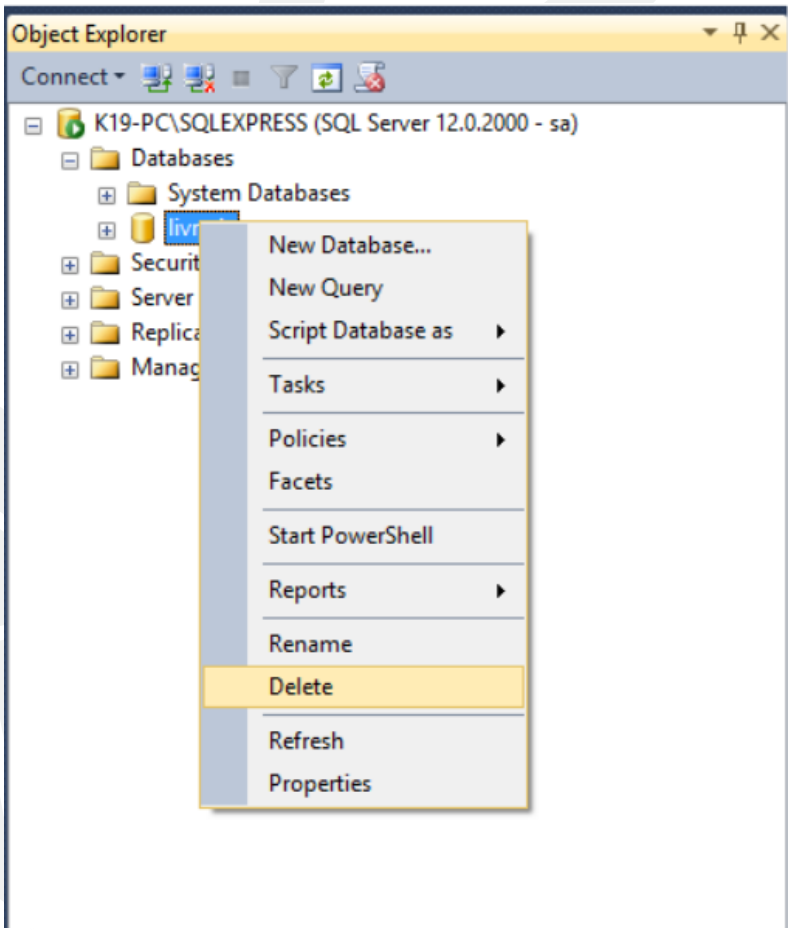


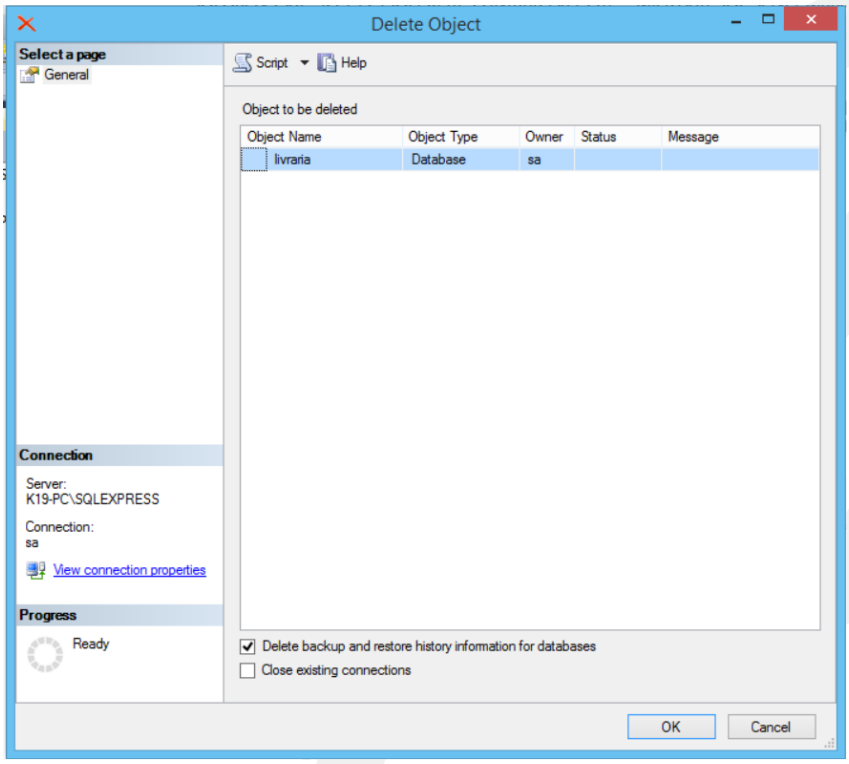
Exercícios de Fixação

1 Abra o **Microsoft SQL Server Management Studio Express** utilizando **NOME_DA_MAQUINA SQLEXPRESS** como Server Name, **SQL Server Authentication** como Authentication, **sa** como Login e **sa** como Password.

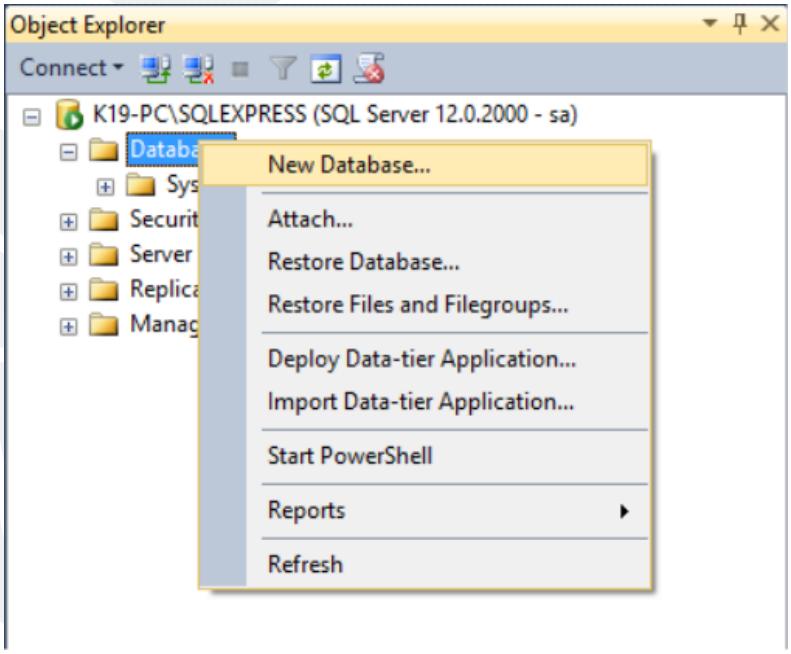


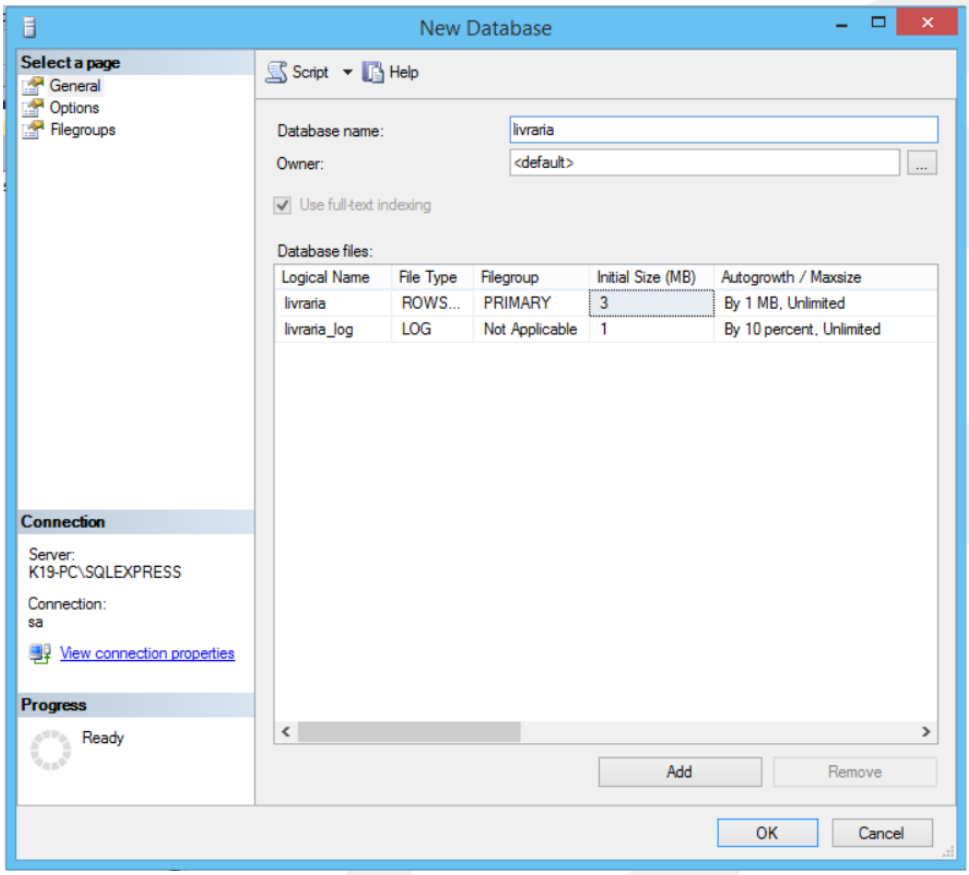
2 Caso exista uma base de dados chamada **Livraria**, remova-a conforme a figura abaixo:



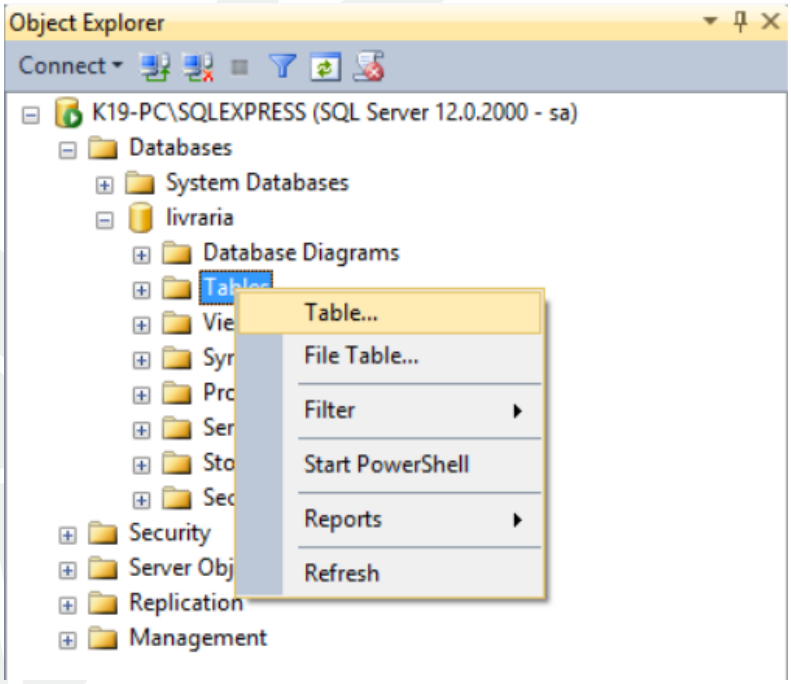


3 Crie uma nova base de dados chamada **livraria**, conforme mostrado na figura abaixo. Você vai utilizar esta base nos exercícios seguintes.

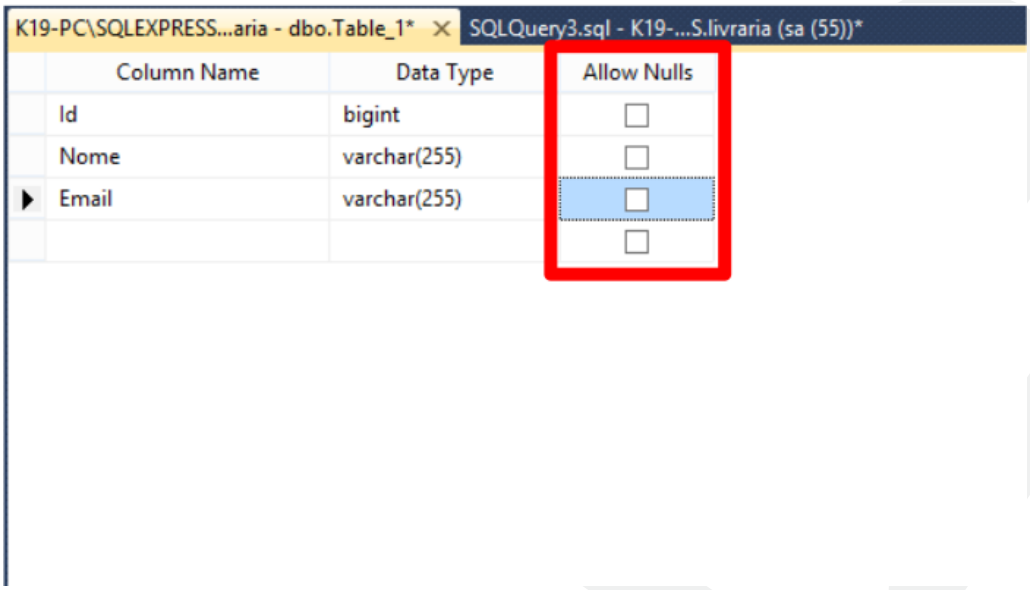




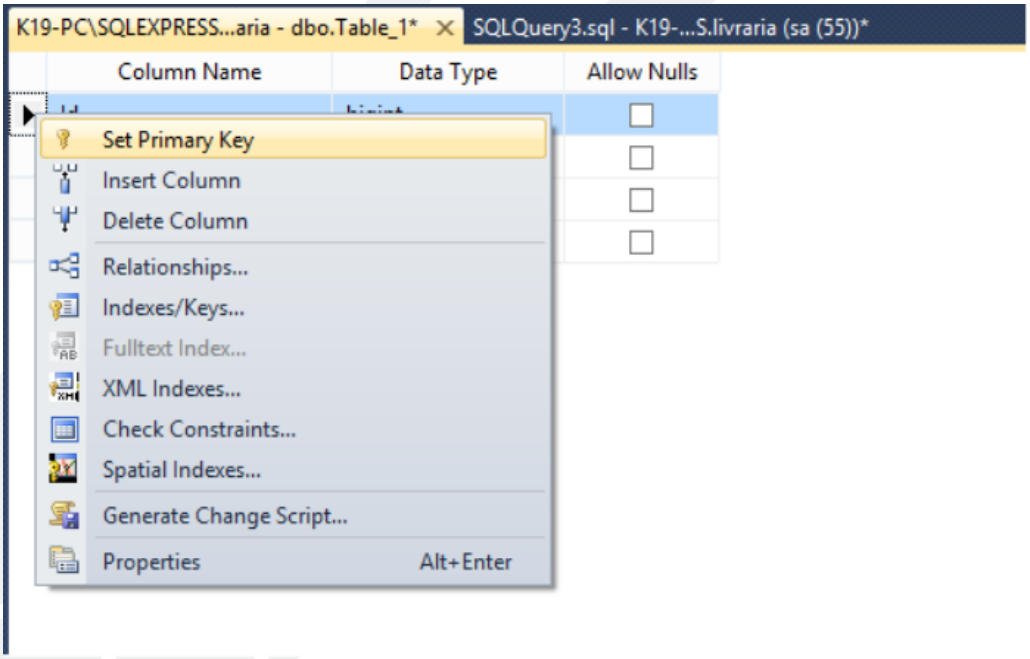
4 Crie uma tabela chamada **Editoras** conforme as figuras abaixo.



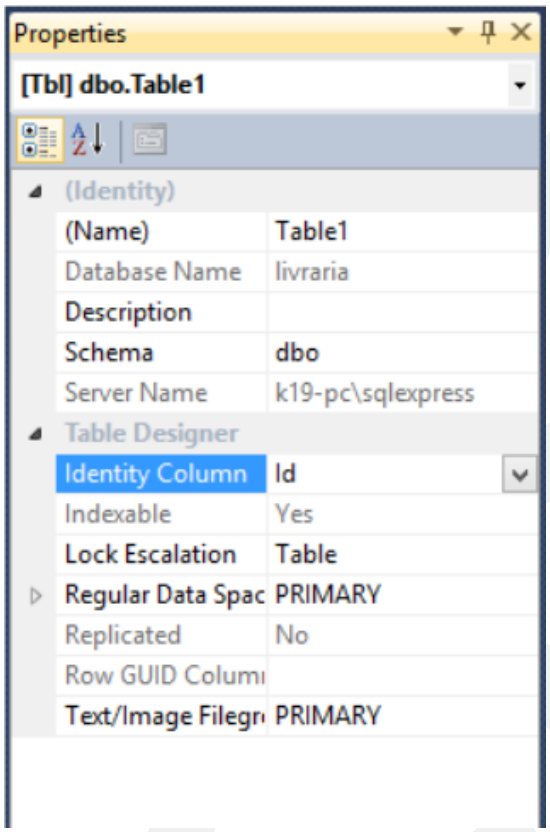
Altere os campos para torná-los obrigatórios, NÃO permitindo que eles fiquem em branco *NULL*.



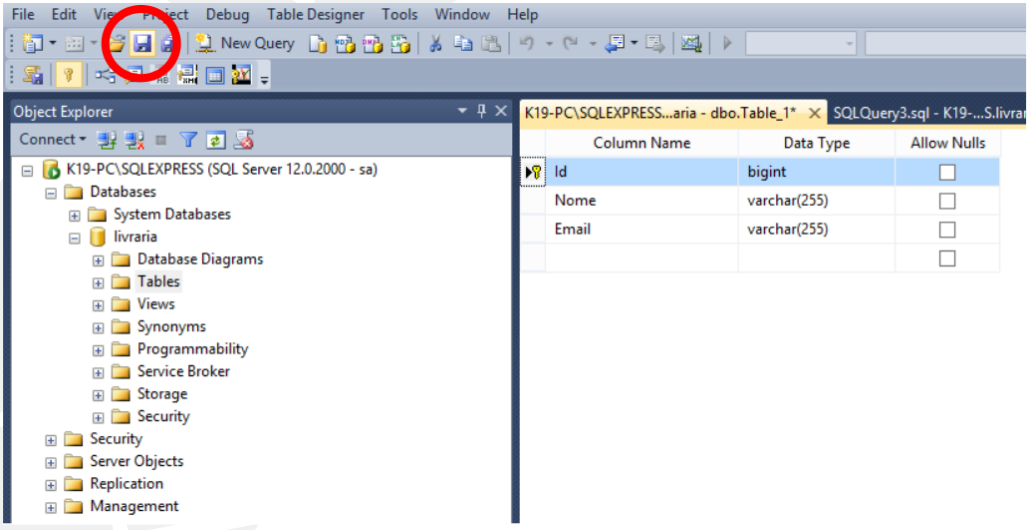
Além disso o campo **Id** deve ser uma chave primária.



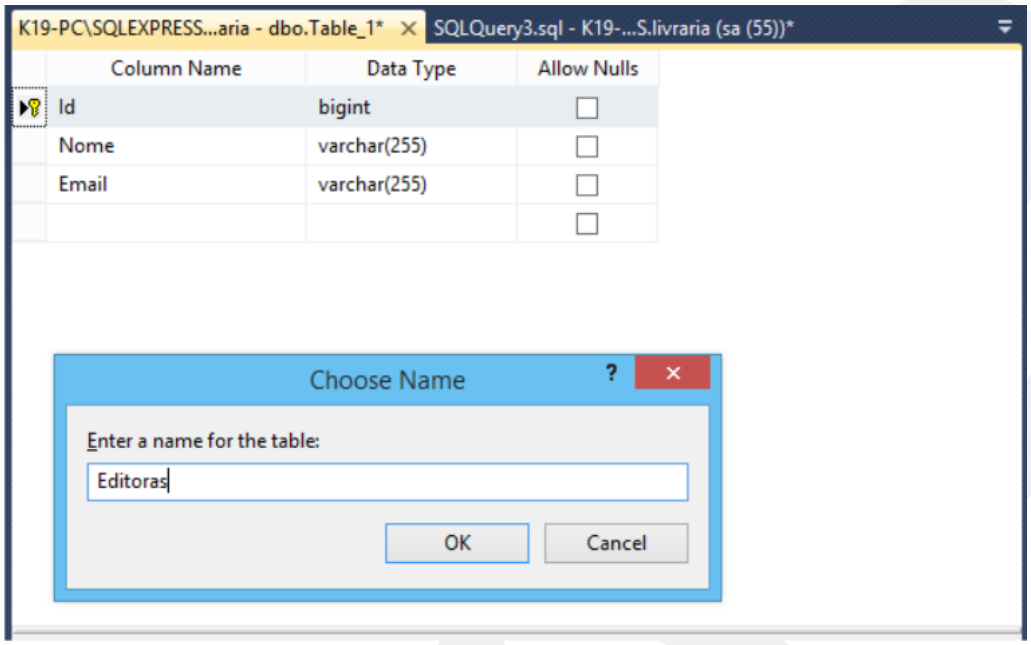
O campo **Id** dever ser incrementado automaticamente. Defina ele com a propriedade **Identity** segundo a figura abaixo:



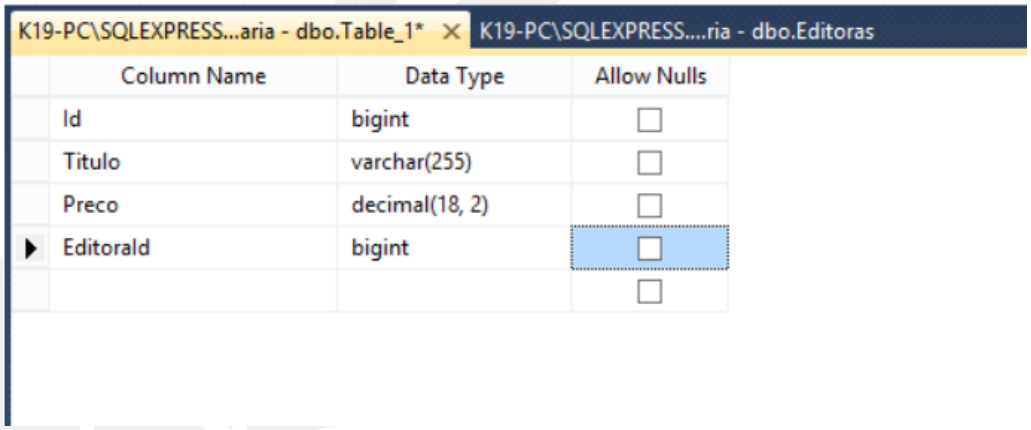
Clique no botão *Salvar*.



Defina o nome da tabela como `Editoras`.

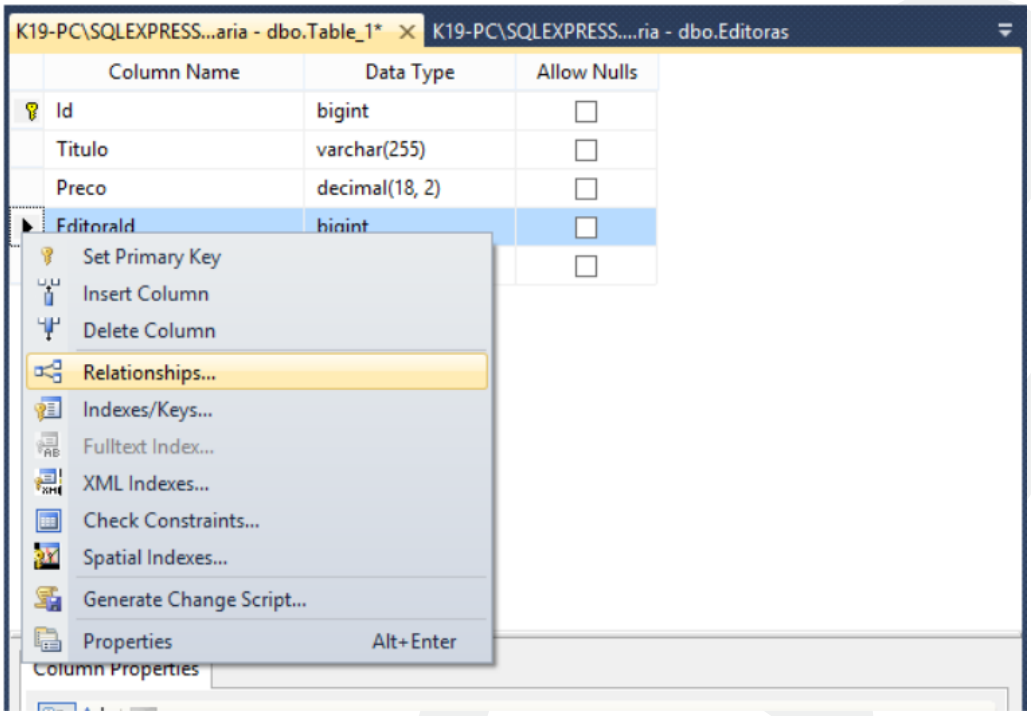


5 Crie uma tabela chamada **Livros** conforme as figuras abaixo:

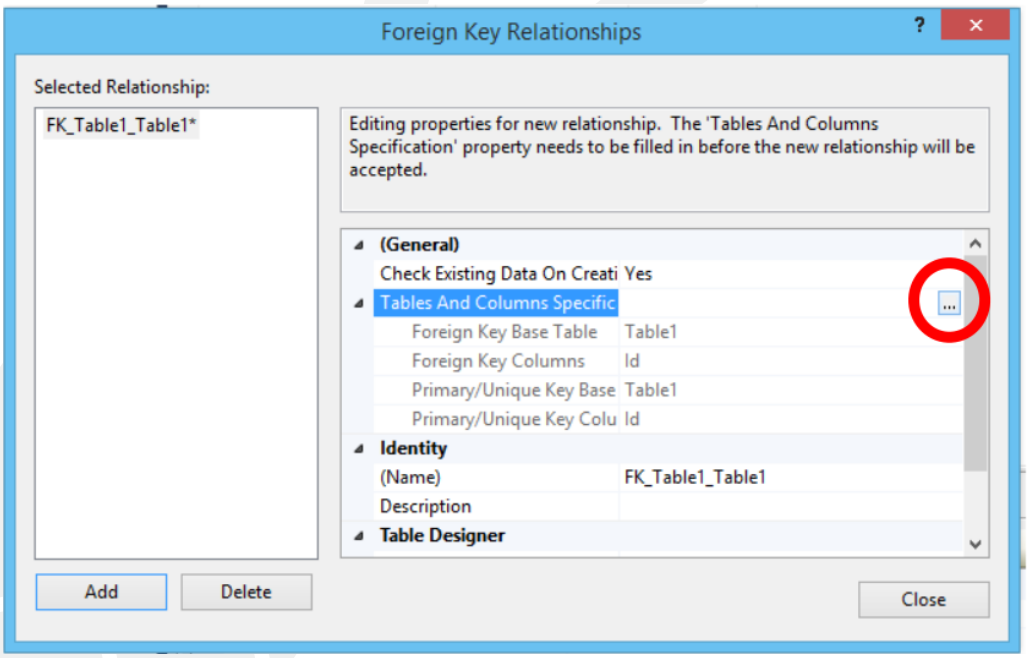


Lembrando de NÃO marcar a opção **ALLOW NULL**. Além disso o campo **Id** deve ser uma chave primária e automaticamente incrementada.

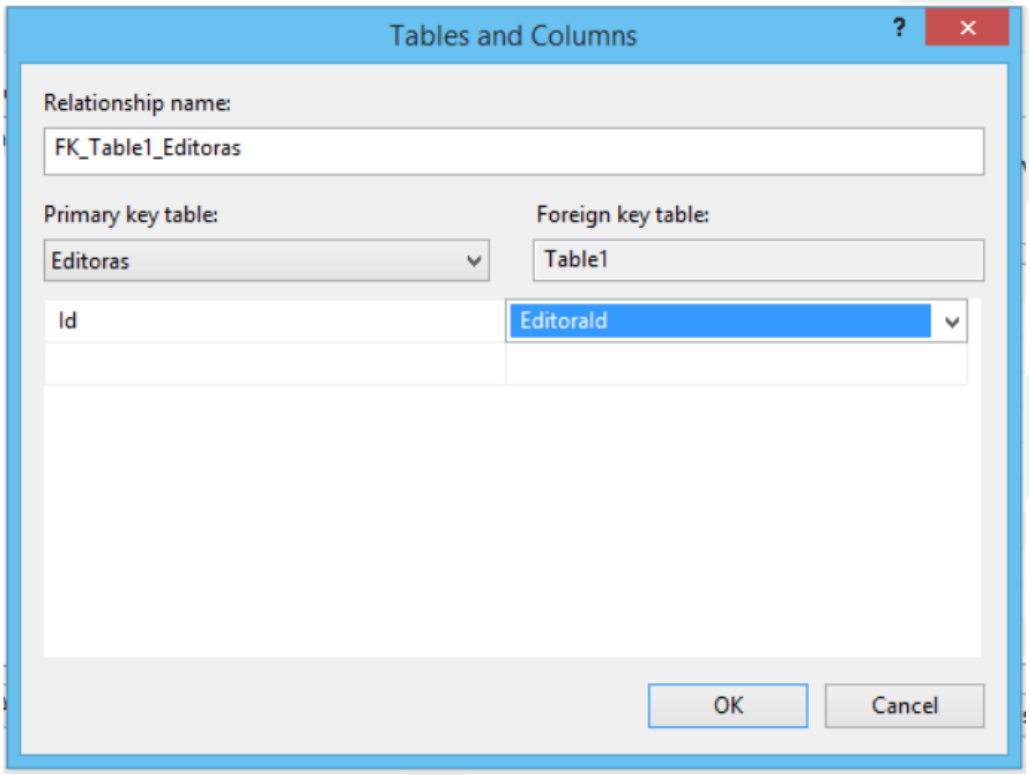
Você precisa tornar o campo **EditoraId** uma chave estrangeira. Clique com o botão direito sobre a coluna **EditoraId** e selecione a opção **Relantioships...**, conforme a figura abaixo:



Devemos acrescentar o relacionamento entre livro e editora. Clique em **Add** e posteriormente no botão à direita na linha *Tables and Columns Specification.*



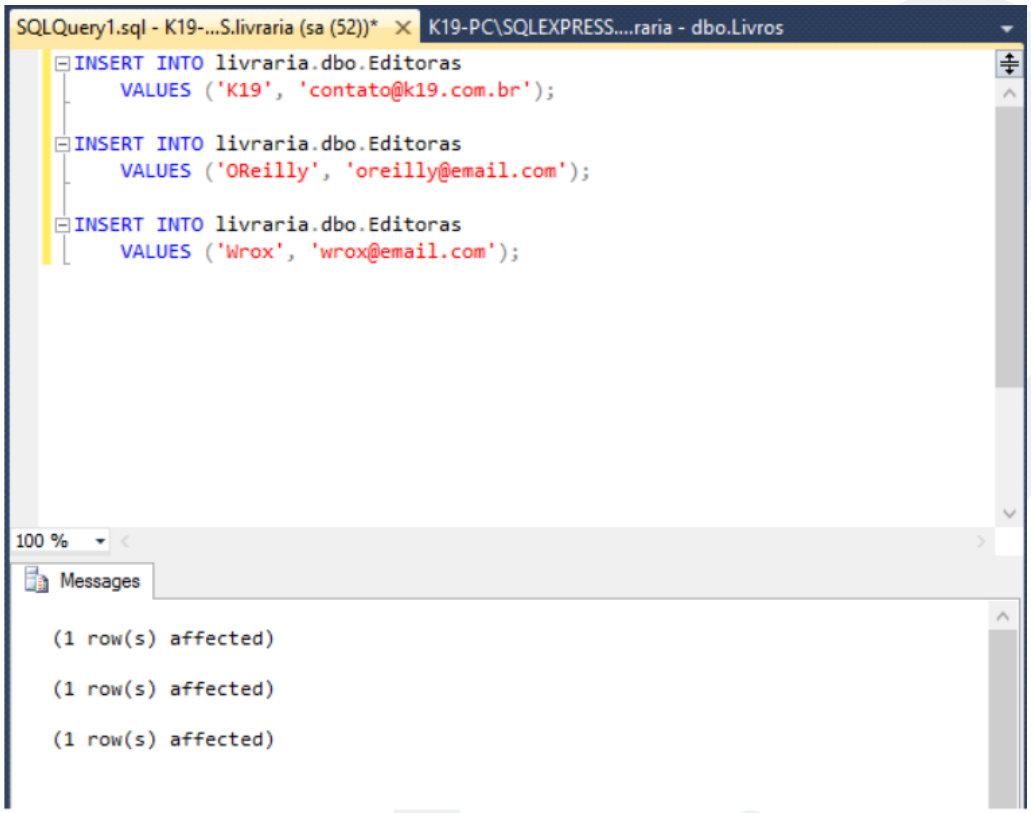
Devemos informar qual é a chave primária que a coluna *EditoraId* da tabela Livros faz referência. Para isto, informe a tabela Editoras como **Primary Key Table** e indique a coluna *Id* como a chave primária referenciada. Selecione a coluna **EditoraId** como a coluna que irá fazer referência a chave primária da tabela Editoras.



6 Adicione alguns registros na tabela Editoras. Veja exemplos na figura abaixo:

```
1 INSERT INTO livraria.dbo.Editoras
2   VALUES ('K19', 'contato@k19.com.br');
3
4 INSERT INTO livraria.dbo.Editoras
5   VALUES ('OReilly', 'oreilly@email.com');
6
7 INSERT INTO livraria.dbo.Editoras
8   VALUES ('Wrox', 'wrox@email.com');
```

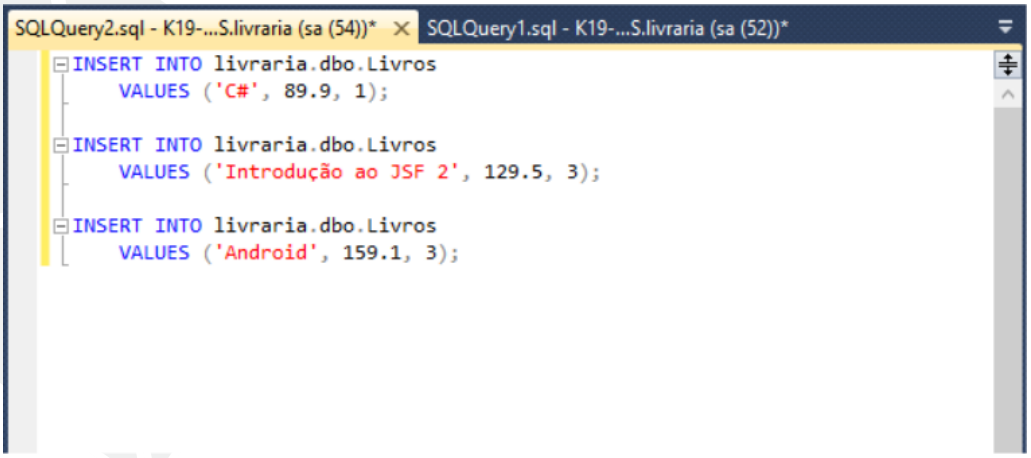
Código SQL 1.1: www.k19.com.br/exercicios/k32/01/006/adicionando-registros-editoras.sql



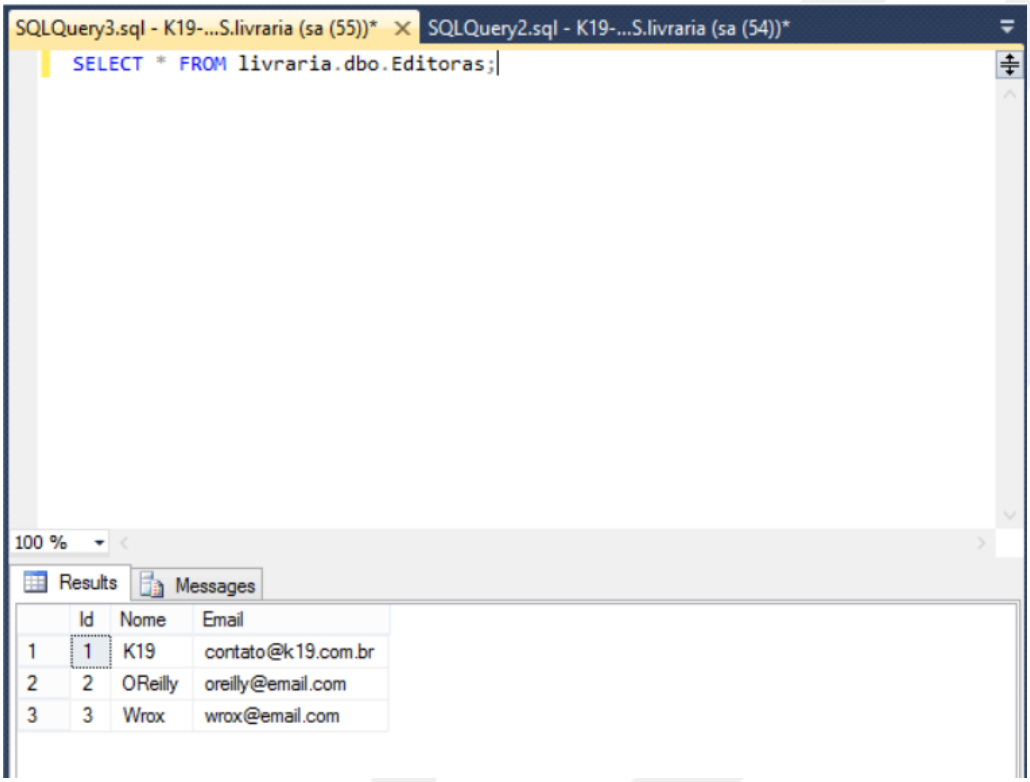
7 Adicione alguns registros na tabela Livros. Veja exemplos na figura abaixo:

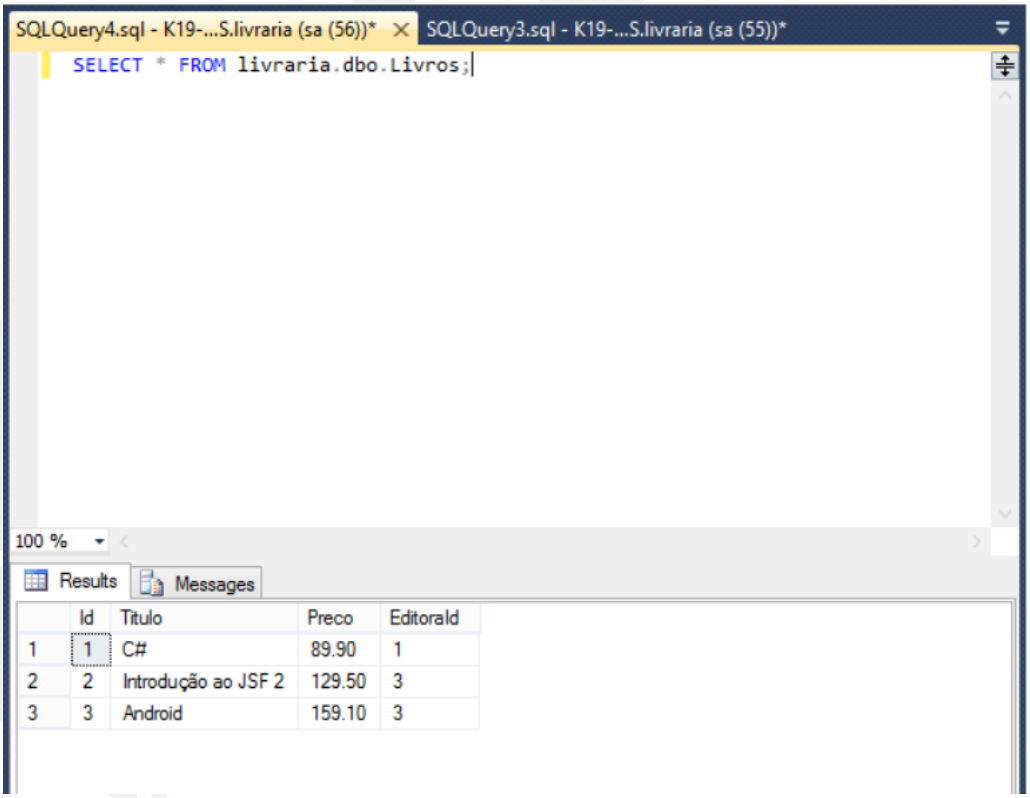
```
1 INSERT INTO livraria.dbo.Livros
2   VALUES ('C#', 89.9, 1);
3
4 INSERT INTO livraria.dbo.Livros
5   VALUES ('Introdução ao JSF 2', 129.5, 3);
6
7 INSERT INTO livraria.dbo.Livros
8   VALUES ('Android', 159.1, 3);
```

Código SQL 1.2: www.k19.com.br/exercicios/k32/01/006/adicionando-registros-livros.sql

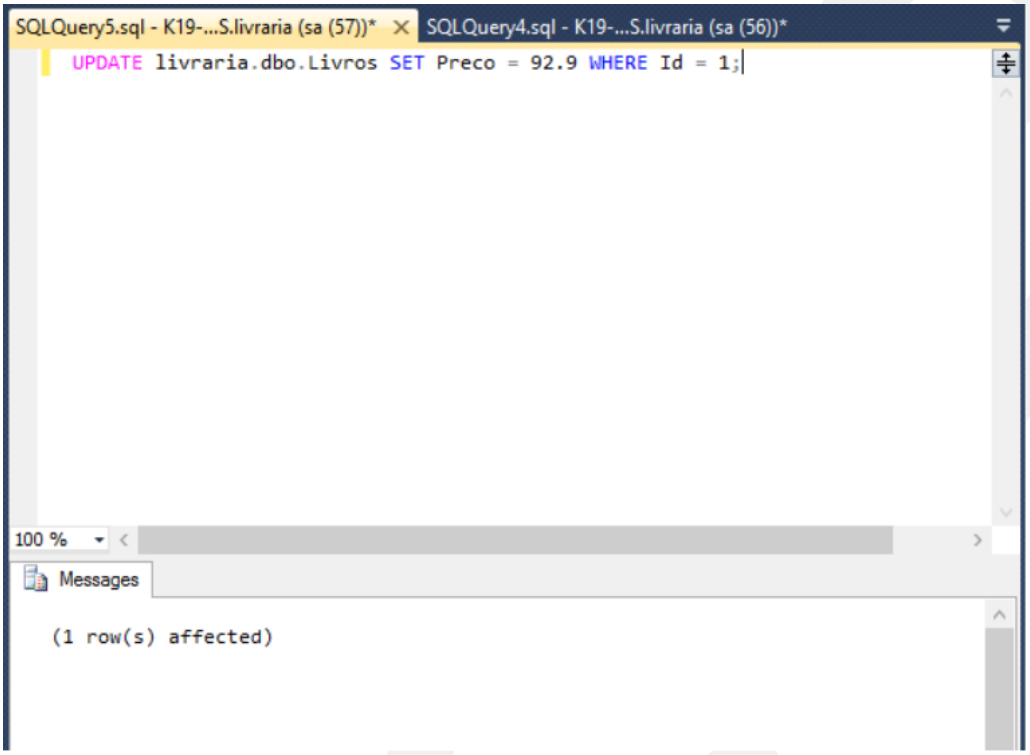


8 Consulte os registros da tabela Editoras, e em seguida consulte a tabela Livros. Veja exemplos logo abaixo:

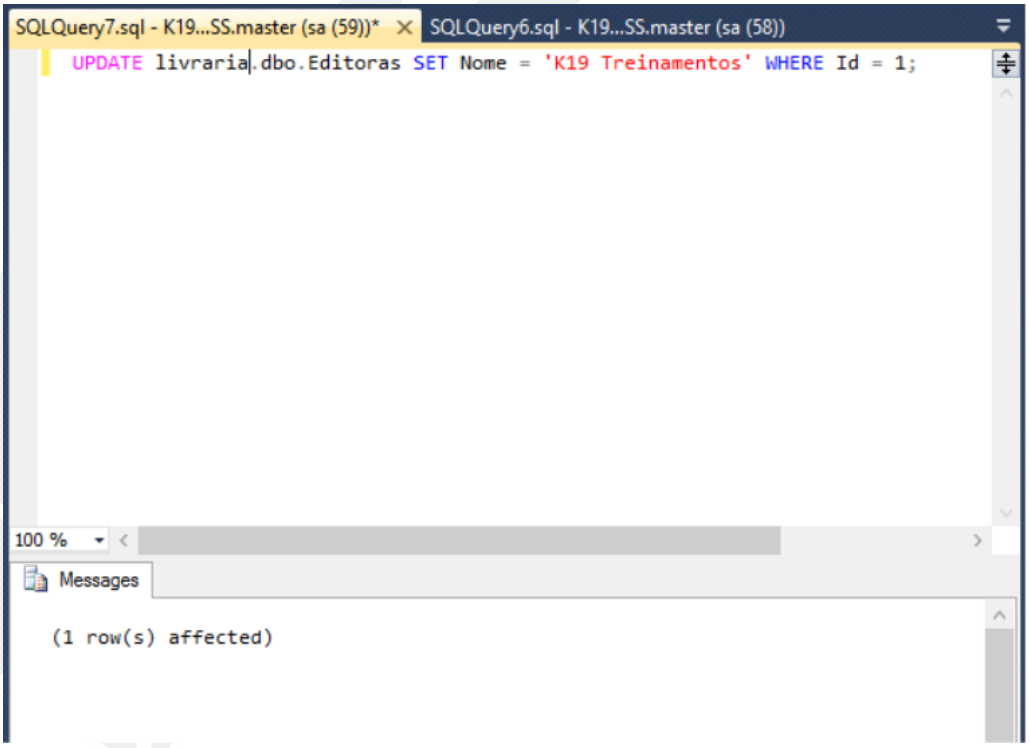




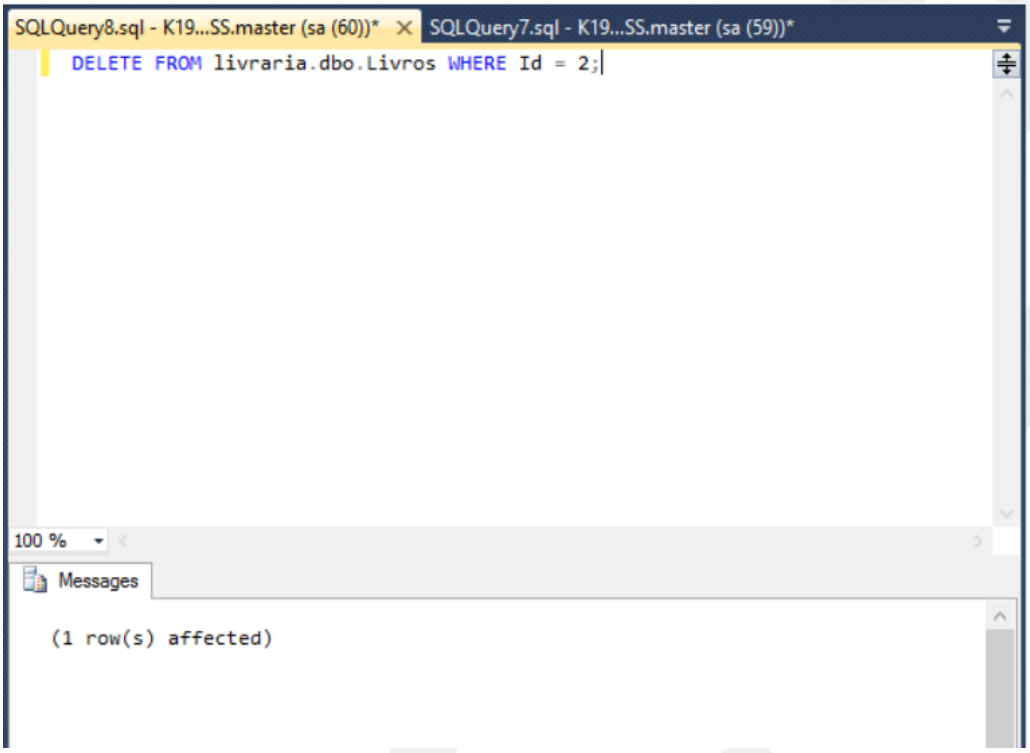
9 Altere alguns dos registros da tabela Livros. Veja o exemplo abaixo:



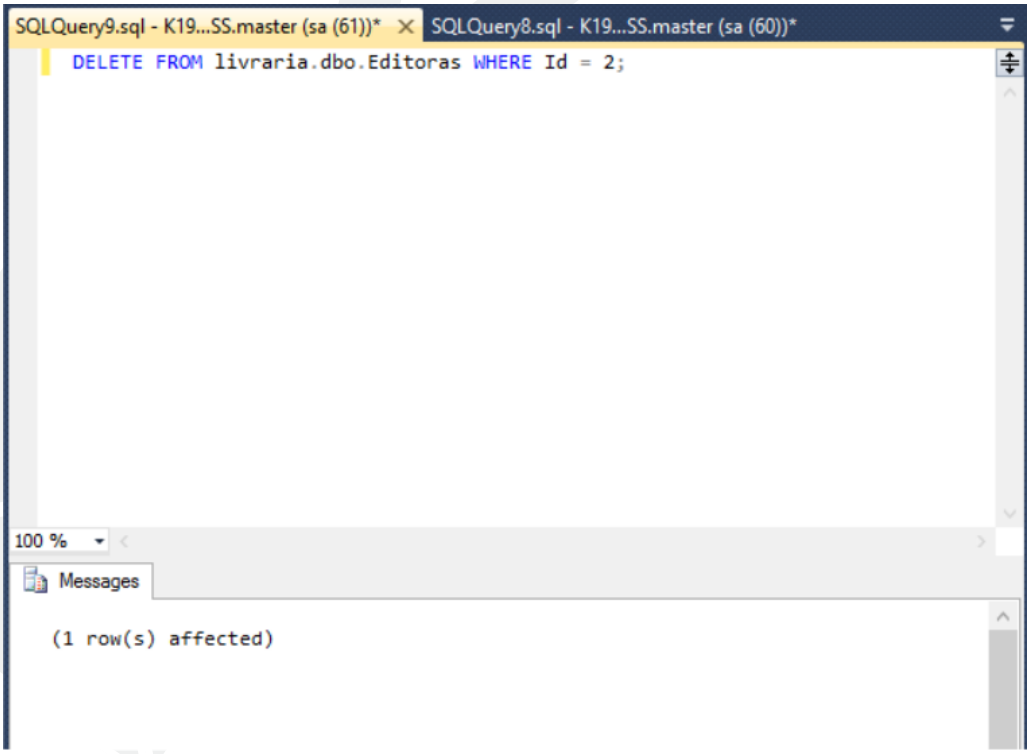
10 Altere alguns dos registros da tabela Editoras. Veja o exemplo abaixo:



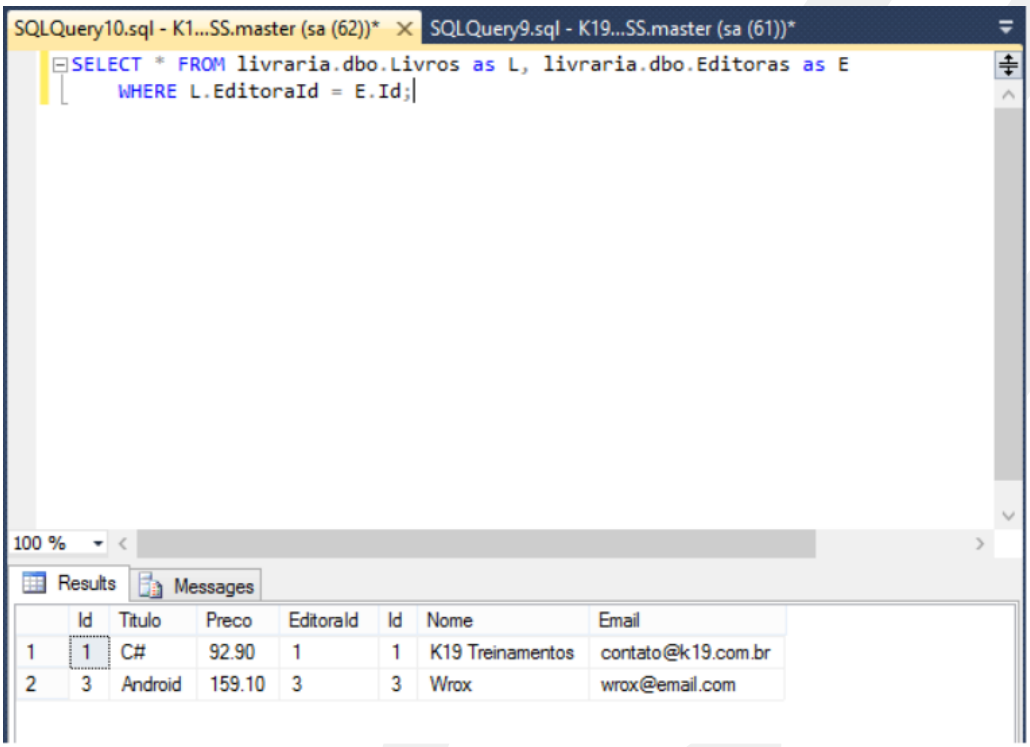
11 Remova alguns registros da tabela Livros. Veja o exemplo abaixo:



12 Remova alguns registros da tabela Editoras. Preste atenção para não remover uma editora que tenha algum livro relacionado já adicionado no banco. Veja o exemplo abaixo:



13 Faça uma consulta para buscar todos os livros de uma determinada editora. Veja um exemplo na figura abaixo:



ADO.NET

CAPÍTULO 2

No capítulo anterior, aprendemos que utilizar bancos de dados é uma ótima alternativa para armazenar os dados de uma aplicação. Entretanto, você deve ter percebido que as interfaces disponíveis para interagir com o SQL Server Express não podem ser utilizadas por qualquer pessoa. Para utilizá-las, é necessário conhecer a linguagem SQL e os conceitos do modelo relacional. Em geral, as interfaces dos outros SGDBs exigem os mesmos conhecimentos.

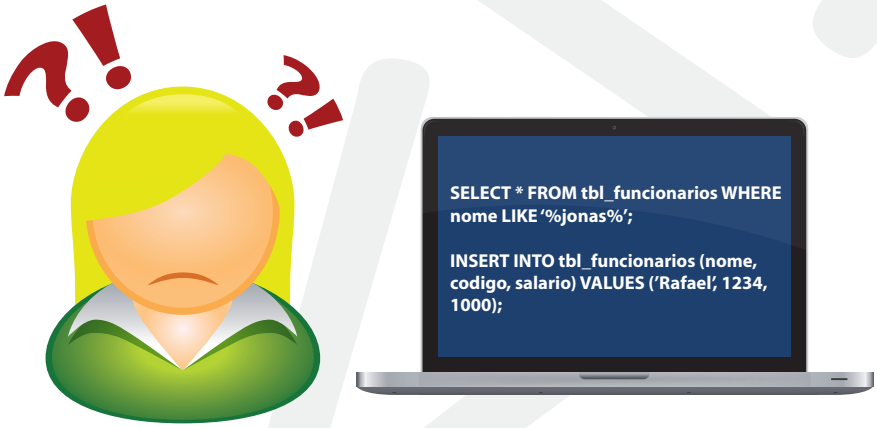


Figura 2.1: Usuários comuns não possuem conhecimento sobre SQL ou sobre o modelo relacional

Para resolver esse problema, podemos desenvolver aplicações com interfaces que não exijam conhecimentos técnicos de SQL ou do modelo relacional para serem utilizadas. Dessa forma, usuários comuns poderiam manipular as informações do banco de dados através dessas aplicações. Nessa abordagem, os usuários interagem com as aplicações e as aplicações interagem com os SGDBs.



Figura 2.2: Usuários comuns devem utilizar interfaces simples



Driver

As aplicações interagem com os SGDBs através de troca de mensagens. Os SGDBs definem o formato das mensagens. Para não sobrecarregar o canal de comunicação entre as aplicações e os SGDBs, as mensagens trocadas devem ocupar o menor espaço possível. Geralmente, protocolos binários são mais apropriados para reduzir o tamanho das mensagens e consequentemente diminuir a carga do canal de comunicação. Por isso, os SGDBs utilizam protocolos binários.

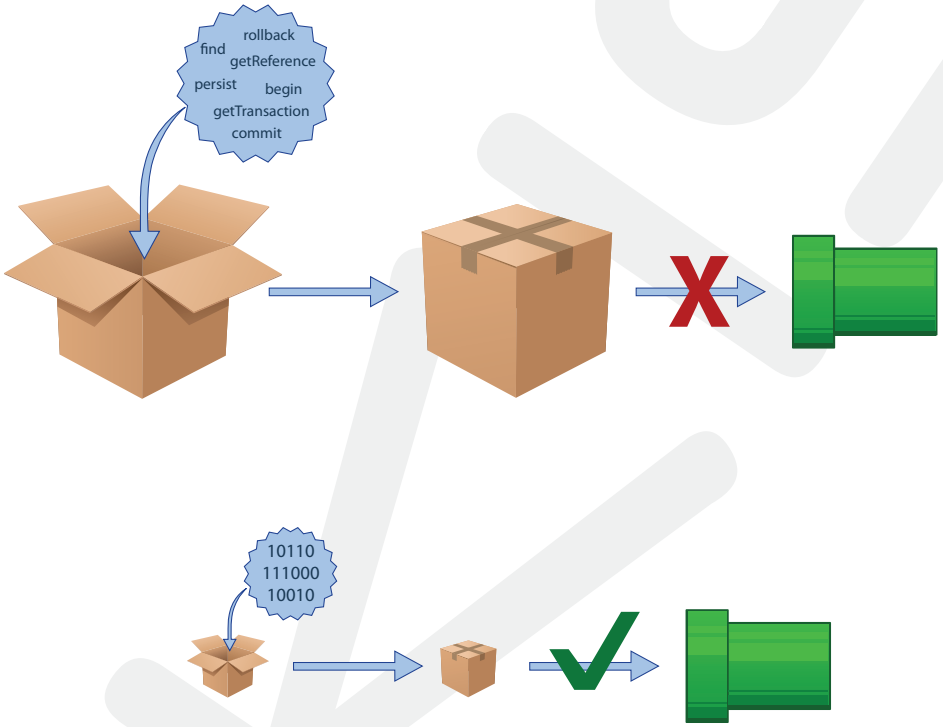


Figura 2.3: Diminuindo o tamanho das mensagens para não sobrecarregar o meio de comunicação

Mensagens binárias são facilmente interpretadas por computadores. Por outro lado, são complexas para um ser humano compreender. Dessa forma, o trabalho dos desenvolvedores seria muito complexo, aumentando o custo para o desenvolvimento e manutenção das aplicações.

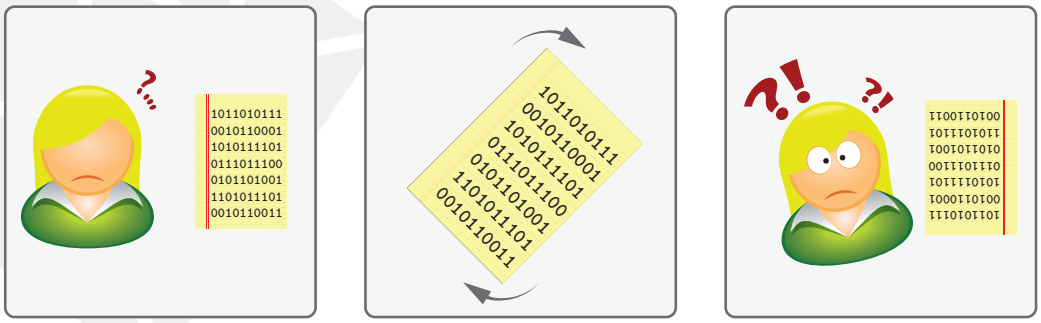


Figura 2.4: Mensagens binárias são altamente complexas para os seres humanos

Para resolver esse problema e facilitar o desenvolvimento das aplicações, as empresas proprietárias dos SGDBs, normalmente, desenvolvem e distribuem **drivers de conexão**. Um driver de conexão atua como um intermediário entre as aplicações e os SGDBs.

Os drivers de conexão são "tradutores" de comandos escritos em uma determinada linguagem de programação para comandos definidos de acordo com o protocolo de um SGDB. Utilizando um driver de conexão, os desenvolvedores das aplicações não manipulam diretamente as mensagens binárias trocadas entre as aplicações e os SGDBs.



Mais Sobre
Em alguns casos, o protocolo binário de um determinado SGDB é fechado. Consequentemente, a única maneira de se comunicar com ele é através de um driver de conexão oferecido pelo fabricante desse SGDB.



ODBC

Suponha que os proprietários dos bancos de dados desenvolvessem os drivers de maneira totalmente independente. Consequentemente, cada driver teria sua própria interface, ou seja, seu próprio conjunto de instruções. Dessa maneira, o desenvolvedor da aplicação precisa conhecer as instruções de cada um dos drivers dos respectivos bancos que ele for utilizar.

Para facilitar o trabalho do desenvolvedor da aplicação, a Microsoft® definiu uma especificação chamada ODBC (Open Database Connectivity) para padronizar a interface dos drivers de conexão. Assim, quando uma empresa proprietária de um banco de dados pretende desenvolver um driver, ela segue essa especificação com o intuito de popularizá-lo.

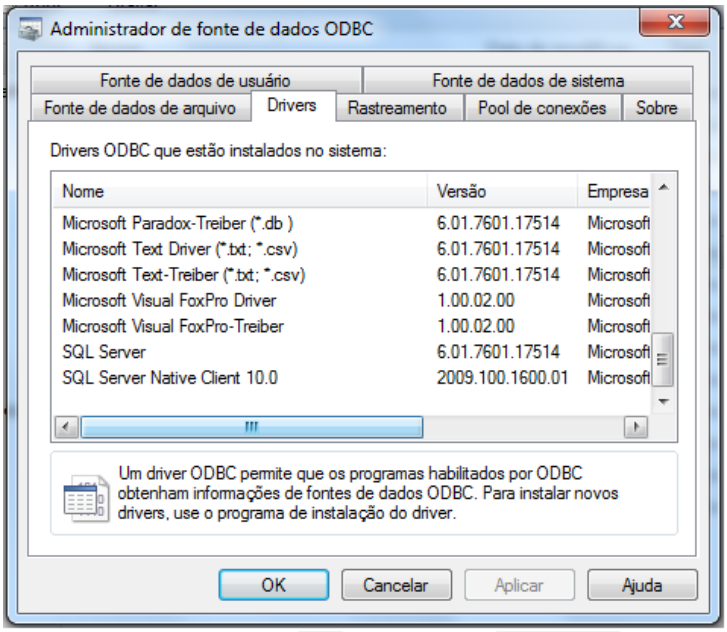
Os drivers de conexão que respeitam a especificação ODBC, ou seja, possuem um conjunto de comandos padronizados, são chamados de drivers de conexão ODBC.



ODBC Manager

Para que drivers ODBC possam ser instalados em uma máquina e as aplicações consigam utilizá-los é necessário ter o ODBC Manager, que já vem instalado no Windows®.

O driver de conexão ODBC já está disponível para utilização, podemos consultar o ODBC Manager do Windows®. O ODBC Manager pode ser executado através do item Ferramentas Administrativas do Painel de Controle.



Criando uma conexão

Com o driver de conexão ODBC instalado na máquina já é possível criar uma conexão com o SGBD correspondente. Abaixo, estão as informações geralmente utilizadas para a criação de uma conexão ODBC.

- Nome do Driver ODBC.
- Endereço (IP e porta) do SGDB.
- Nome da base de dados.
- Um usuário do SGBD.
- Senha do usuário.

Todas essas informações são definidas na chamada **string de conexão** ou **url de conexão**.

```
1 string stringDeConexao = @"driver={SQL Server};
2   server=.\SQLEXPRESS;database=livraria;uid=sa;pwd=sa";
```

Código C# 2.1: Definindo a string de conexão

Para utilizar Windows Authentication, ou seja, utilizar o mesmo usuário e senha do Windows para conectar no SGBD, podemos acrescentar a propriedade `Trusted_Connection` na string de conexão. Nesse caso, o usuário do SGBD e a senha não são informados.

```
1 string stringDeConexao = @"driver={SQL Server};
2   server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
```

Código C# 2.2: Definindo a string de conexão

Com a string de conexão definida, podemos utilizar a classe `OdbcConnection` do namespace `System.Data.Odbc` que pertence ao *.NET Framework* para criar objetos que representem as conexões com o SGBD. É importante ter em mente que a conexão com o SGBD não é estabelecida quando um objeto da classe `OdbcConnection` é criado. Para abrir a conexão, é necessário utilizar o método `Open()`.

```
1 string stringDeConexao = @"driver={SQL Server};
2   server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
3
4 OdbcConnection conexao = new OdbcConnection(stringDeConexao);
5
6 conexao.Open();
```

Código C# 2.3: Criando uma conexão ODBC



Apagando uma base de dados

Após estabelecer uma conexão ODBC, podemos executar operações. A primeira operação que realizaremos é apagar uma base de dados. O primeiro passo para executar essa operação é definir o código SQL correspondente.

```
1 string sql = @"IF EXISTS
2   (SELECT name FROM master.dbo.sysdatabases WHERE name = 'livraria')
3   DROP DATABASE livraria";
```



Mais Sobre
O caractere "@" antes de um valor literal do tipo string indica que os caracteres dentro da string não devem ser processados como caracteres especiais.

Em seguida, devemos criar um objeto da classe `OdbcCommand` a partir do código sql e da conexão previamente criados. O comando não é executado quando os objetos dessa classe são instanciados. O método `ExecuteNonQuery()` pode ser utilizado para executar o comando no SGBD. Esse método deve ser chamado quando a conexão estiver aberta.

```
1 // criando um comando ODBC
2 OdbcCommand comando = new OdbcCommand(sql, conexao);
3
4 // abrindo a conexão
5 conexao.Open();
6
7 // executanto o comando
8 comando.ExecuteNonQuery();
```

Código C# 2.5: Criando um comando ODBC



Importante
A mesma conexão pode ser reaproveitada para executar várias operações. Quando não houver mais operações a serem executadas, devemos finalizar a conexão ODBC através do método `Close()`. Finalizar as conexões ODBC que não são mais necessárias é importante pois libera recursos no SGBD.

```
1 conexao.close();
```

Código C# 2.6: Finalizando uma conexão ODBC



Mais Sobre

Em C#, para evitar que o uma conexão não seja fechada após a sua utilização, podemos aplicar um bloco **using**.

```
1 using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
2 {
3   // utiliza a conexao
4 }
```

Código C# 2.7: Utilizando um bloco using

No código acima, quando o bloco `using` que está associado à conexão ODBC terminar, automaticamente, essa conexão será fechada.



Criando uma base de dados

O procedimento para criar uma base de dados é muito parecido com o procedimento para apagar uma base de dados.

```
1  // código SQL da operação
2  string sql = @"CREATE DATABASE livraria";
3
4  // criando um comando ODBC
5  OdbcCommand comando = new OdbcCommand(sql, conexao);
6
7  // abrindo a conexão
8  conexao.Open();
9
10 // executanto o comando
11 comando.ExecuteNonQuery();
```



Criando uma tabela

Analogamente, ao que vimos nas seções anteriores, podemos criar uma tabela através de um conexão ODBC.

```
1  string sql =
2    "CREATE TABLE Editoras (" +
3      "id BIGINT IDENTITY(1,1)," +
4      "nome VARCHAR (255) NOT NULL," +
5      "email VARCHAR (255) NOT NULL," +
6      "CONSTRAINT PK_Editoras PRIMARY KEY CLUSTERED (id asc)" +
7    ")";
8
9  OdbcCommand comando = new OdbcCommand(sql, conexao);
10
11 conexao.Open();
```

```
12
13 comando.ExecuteNonQuery();
```



Inserindo registros

Também podemos adicionar dados nas tabelas utilizando conexões ODBC.

```
1 string sql = @"INSERT INTO Editoras (Nome, Email)
2   VALUES  ('K19 Treinamentos', 'contato@k19.com.br')";
3
4 OdbcCommand comando = new OdbcCommand(sql, conexao);
5
6 conexao.Open();
7
8 comando.ExecuteNonQuery();
```

Normalmente, optamos por deixar os SGBDs gerar os valores das chaves primárias dos registros inseridos em uma tabela. Esses valores podem ser obtidos através dos recursos ADO.NET.

Veja o código a seguir. Na linha 2, o trecho "OUTPUT INSERTED.id" indica que a operação devolverá o valor do campo "id" no registro que será inserido. Na linha 9, o método `ExecuteScalar()` é utilizado para executar o comando no SGBD e devolve o resultado da operação.

```
1 string sql = @"INSERT INTO Editora (nome, email)
2   OUTPUT INSERTED.id
3   VALUES ('K19','contato@k19.com.br')";
4
5 OdbcCommand comando = new OdbcCommand(sql, conexao);
6
7 conexao.Open();
8
9 var id = comando.ExecuteScalar();
```



Exercícios de Fixação

1. Crie um projeto do tipo `Console Application` no `Visual Studio` chamado **Odbc**.

2. Crie uma classe chamada **CriaBaseDeDadosLivraria** no projeto **Odbc** com o seguinte conteúdo.

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class CriaBaseDeDadosLivraria
6    {
7      static void Main(string[] args)
8      {
9        string stringDeConexao = @"driver={SQL Server};
10         server=.\SQLEXPRESS;Trusted_Connection=Yes";
11
12       using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
13       {
14         conexao.Open();
```

```
15
16          string sql = @"IF EXISTS
17            (SELECT name FROM master.dbo.sysdatabases WHERE name = 'livraria')
18            DROP DATABASE livraria";
19          OdbcCommand command = new OdbcCommand(sql, conexao);
20          command.ExecuteNonQuery();
21
22          sql = @"CREATE DATABASE livraria";
23          command = new OdbcCommand(sql, conexao);
24          command.ExecuteNonQuery();
25        }
26      }
27    }
28  }
```

Código C# 2.12: www.k19.com.br/exercicios/k32/02/002/CriaBaseDeDadosLivraria.cs

Selecione a classe **CriaBaseDeDadosLivraria** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se a base dados livraria não existe no SGBD.

3 Crie uma classe chamada **CriaTabelaEditora** no projeto **Odbc** com o seguinte conteúdo.

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class CriaTabelaEditora
6    {
7      static void Main(string[] args)
8      {
9        string stringDeConexao = @"driver={SQL Server};
10         server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
11
12       using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
13       {
14         conexao.Open();
15
16         string sql =
17           "CREATE TABLE Editoras (" +
18             "id BIGINT IDENTITY(1,1)," +
19             "nome VARCHAR (255) NOT NULL," +
20             "email VARCHAR (255) NOT NULL," +
21             "CONSTRAINT PK_Editoras PRIMARY KEY CLUSTERED (id asc)" +
22           ")";
23         OdbcCommand command = new OdbcCommand(sql, conexao);
24         command.ExecuteNonQuery();
25       }
26     }
27   }
28 }
```

Código C# 2.13: www.k19.com.br/exercicios/k32/02/003/CriaTabelaEditora.cs

Selecione a classe **CriaTabelaEditora** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se a tabela Editora foi criada corretamente.

4 Crie uma classe chamada **Editora** no projeto **Odbc** com o seguinte conteúdo.

```
1  namespace Odbc
2  {
3    class Editora
4    {
```

```
5       public long? Id { get; set; }
6       public string Nome { get; set; }
7       public string Email { get; set; }
8     }
9   }
```

Código C# 2.14: www.k19.com.br/exercicios/k32/02/004/Editora.cs

5 Crie uma classe chamada **InsereEditora** no projeto **Odbc** com o seguinte conteúdo.

```
1   using System.Data.Odbc;
2
3   namespace Odbc
4   {
5     class InsereEditora
6     {
7       static void Main(string[] args)
8       {
9         string stringDeConexao = @"driver={SQL Server};
10          server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
11
12        Editora e = new Editora();
13
14        System.Console.Write("Digite o Nome da Editora:");
15        e.Nome = System.Console.ReadLine();
16
17        System.Console.Write("Digite o Email da Editora:");
18        e.Email = System.Console.ReadLine();
19
20        string sql =
21          @"INSERT INTO Editoras (Nome, Email)
22          OUTPUT INSERTED.id
23          VALUES('" + e.Nome + @"', '" + e.Email + @"')";
24
25        using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
26        {
27          OdbcCommand command = new OdbcCommand(sql, conexao);
28          conexao.Open();
29          e.Id = command.ExecuteScalar() as long?;
30
31          System.Console.WriteLine("Editora cadastrada com id: " + e.Id);
32        }
33      }
34    }
35  }
```

Código C# 2.15: www.k19.com.br/exercicios/k32/02/005/InsereEditora.cs

Selecione a classe **InsereEditora** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Editoras.



Exercícios Complementares

1 Crie uma classe chamada **CriaTabelaLivro** para adicionar a tabela **Livro** na base de dados **livraria**.

2 Crie uma classe chamada **Livro** para modelar os livros da livraria.

3 Crie uma classe chamada `InsereLivro` no projeto **Odbc** para inserir registros na tabela `Livros`.



SQL Injection

A implementação da inserção de registros feita anteriormente possui uma falha grave. Os dados obtidos do usuário através do teclado não são tratados antes de serem enviados para o SGDB.

Esses dados podem conter caracteres especiais. Se esses caracteres não são tratados, o comportamento esperado da operação é afetado. Eventualmente, registros não são inseridos como deveriam ou brechas de segurança podem se abrir.

Por exemplo, considere a classe `InsereEditora` do exercício de fixação. Se o usuário digitar "O'Reilly" e "oreilly@email.com", o código SQL gerado pela aplicação seria:

```
1 INSERT INTO Editoras (nome, email) VALUES ('O'Reilly', 'oreilly@email.com')
```

Observe que o caractere aspas simples aparece cinco vezes no código SQL acima. O SGDB não saberia dizer onde de fato termina o nome da editora. Ao tentar executar esse código, um erro de sintaxe é lançado pelo SQL Server. Para resolver esse problema manualmente, devemos adicionar dois caracteres "'' seguidos.

```
1 INSERT INTO Editoras (nome, email) VALUES ('O''Reilly', 'oreilly@email.com')
```

Os valores recebidos dos usuários devem ser analisados e os caracteres especiais contidos nesses valores devem ser tratados. Esse processo é extremamente trabalhoso, pois o conjunto de caracteres especiais e a forma de tratá-los é diferente em cada SGDB.

A responsabilidade do tratamento dos caracteres especiais contidos nos valores de entrada dos usuários pode ser repassada para os drivers ODBC. Dessa forma, o código das aplicações se torna independente das particularidades desse processo para cada SGDB.



Mais Sobre
O processo de tratamento dos caracteres especiais das entradas dos usuários é denominado **sanitize**.

```
1  // pegando os dados da editora pelo teclado
2  string nome = System.Console.ReadLine();
3  string email = System.Console.ReadLine();
4
5  // definindo a sentença SQL com parâmetros
6  string sql =
7    @"INSERT INTO Editoras (Nome, Email) VALUES (?, ?);";
8
9  // criando um comando odbc
10 OdbcCommand comando = new OdbcCommand(sql, conexao);
11
12 // atribuindo valores aos parâmetros
13 comando.Parameters.AddWithValue("@Nome", nome);
14 comando.Parameters.AddWithValue("@Email", email);
```

Código C# 2.21: "Sanitizando" as entradas dos usuários

Observe que a sentença SQL foi definida com parâmetros através do caractere "**?**". Antes de executar o comando, é necessário atribuir valores aos parâmetros. Isso é feito com o método `AddWithValue()`. Esse método realiza a tarefa de "sanitizar" os valores enviados pelo usuário.



Exercícios de Fixação

6 Tente causar um erro de SQL Injection ao inserir editoras com a classe `InsereEditora`. (Dica: tente entradas com aspas simples)

7 Altere o código da classe `InsereEditora` para eliminar o problema do SQL Injection. Observe o código abaixo:

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class InsereEditora
6    {
7      static void Main(string[] args)
8      {
9        string stringDeConexao = @"driver={SQL Server};
10         server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
11
12       Editora e = new Editora();
13
14       System.Console.Write("Digite o Nome da Editora:");
15       e.Nome = System.Console.ReadLine();
16
17       System.Console.Write("Digite o Email da Editora:");
18       e.Email = System.Console.ReadLine();
19
20       string sql =
21         @"INSERT INTO Editoras (Nome, Email)
22         OUTPUT INSERTED.id
23         VALUES(?, ?)";
24
25       using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
26       {
27         OdbcCommand command = new OdbcCommand(sql, conexao);
28
29         command.Parameters.AddWithValue("@Nome", e.Nome);
30         command.Parameters.AddWithValue("@Email", e.Email);
31
32         conexao.Open();
33         e.Id = command.ExecuteScalar() as long?;
34
35         System.Console.WriteLine("Editora cadastrada com id: " + e.Id);
36       }
37     }
38   }
39 }
```

Código C# 2.22: www.k19.com.br/exercicios/k32/02/007/InsereEditora.cs

Selecione a classe **InsereEditora** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Editoras.

8 Agora tente causar novamente o problema de SQL Injection ao inserir novas editoras.



Exercícios Complementares

4 Provoque um erro de SQL Injection na classe `InsereLivro`. (Dica: tente entradas com aspas simples.)

5 Altere o código para eliminar o problema do SQL Injection.

6 Agora tente causar novamente o problema de SQL Injection ao inserir novos livros. Selecione a classe **InsereLivro** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Livros.



Listando registros

Depois de inserir alguns registros, é interessante consultar os dados das tabelas para conferir se a inserção foi realizada com sucesso.

O processo para executar um comando de consulta é parecido com o de inserção. É necessário definir a sentença SQL e criar um objeto da classe `OdbcCommand`.

```
1 string sql = @"SELECT * FROM Editoras";
2
3 OdbcCommand comando = new OdbcCommand(sql, conexao);
4
5 OdbcDataReader resultado = comando.ExecuteReader();
```

Código C# 2.24: Criando um comando de seleção

A diferença é que para executar um comando de consulta é necessário utilizar o método `ExecuteReader()` ao invés do `ExecuteNonQuery()`. Esse método devolve um objeto da classe *System.Data.Odbc.OdbcDataReader*.

Os dados contidos no `OdbcDataReader` podem ser acessados através dos nomes das colunas.

```
1 string nome = resultado["Nome"] as string;
2 string email = resultado["Email"] as string;
```

Código C# 2.25: Recuperando os campos do primeiro registro do resultado

O código acima mostra como os campos dos registros recuperados são acessados. O método `Read()` é utilizado para percorrer esses registros.

```
1 resultado.Read();
2
3 string nome1 = resultado["nome"] as string;
4 string email1 = resultado["email"] as string;
5
6 resultado.Read();
```

```
7
8 string nome2 = resultado["nome"] as string;
9 string email2 = resultado["email"] as string;
```

O próprio método `Read()` devolve um valor booleano para indicar se o reader conseguiu avançar para o próximo registro. Quando esse método devolver `false` significa que não há mais registros.

```
1 while(resultado.Read())
2 {
3   long? id = resultado["id"] as long?;
4   string nome = resultado["nome"] as string;
5   string email = resultado["email"] as string;
6 }
```



Exercícios de Fixação

9 Insira algumas editoras utilizando a classe `InsereEditora` que você criou nos exercícios anteriores.

10 Adicione uma nova classe ao projeto **Odbc** chamada `ListaEditoras`. O objetivo é listar as editoras que foram salvas no banco. Veja o código dessa classe.

```
1  using System.Data.Odbc;
2  using System.Collections.Generic;
3
4  namespace Odbc
5  {
6    class ListaEditoras
7    {
8      static void Main(string[] args)
9      {
10       string stringDeConexao = @"driver={SQL Server};
11         server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
12
13       using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
14       {
15         string sql = "SELECT * FROM Editoras";
16         OdbcCommand command = new OdbcCommand(sql, conexao);
17         conexao.Open();
18         OdbcDataReader resultado = command.ExecuteReader();
19
20         List<Editora> lista = new List<Editora>();
21
22         while (resultado.Read())
23         {
24           Editora e = new Editora();
25
26           e.Id = resultado["Id"] as long?;
27           e.Nome = resultado["Nome"] as string;
28           e.Email = resultado["Email"] as string;
29
30           lista.Add(e);
31         }
32
33         foreach(Editora e in lista)
34         {
35           System.Console.WriteLine("{0} : {1} - {2}\n", e.Id, e.Nome, e.Email);
36         }
37       }
```

```
38      }
39    }
40  }
```

Código C# 2.28: www.k19.com.br/exercicios/k32/02/010/ListaEditoras.cs

Selecione a classe **ListaEditoras** como Startup Object do projeto **Odbc**. Execute esse projeto e veja a saída do programa.



Exercícios Complementares

7 Crie uma classe para listar os livros cadastrados na base de dados.



Connection Factory

Você deve ter percebido que para cada ação executada no banco de dados, nós precisamos criar uma conexão. Isso gera um problema relacionado à string de conexão ficar armazenada em diversos locais. Imagine que o driver do banco foi atualizado e mudamos a sua versão. Isso implicaria fazer diversas alterações no código em cada ocorrência da string de conexão, tornando o código mais suscetível a erros e dificultando a sua manutenção.

Para resolver esta situação, nós poderíamos criar uma classe responsável pela criação e distribuição de conexões, mantendo assim uma única referência para a string de conexão, e qualquer alteração no modo em que nos conectamos à base de dados, só implica mudanças nesta classe.

```
1  static class ConnectionFactory
2  {
3    public static OdbcConnection CreateConnection()
4    {
5      string driver = @"SQL Server";
6      string servidor = @".\SQLEXPRESS";
7      string baseDeDados = @"livraria";
8      string usuario = @"sa";
9      string senha = @"sa";
10
11     StringBuilder connectionString = new StringBuilder();
12     connectionString.Append("driver=");
13     connectionString.Append(driver);
14     connectionString.Append(";server=");
15     connectionString.Append(servidor);
16     connectionString.Append(";database=");
17     connectionString.Append(baseDeDados);
18     connectionString.Append(";uid=");
19     connectionString.Append(usuario);
20     connectionString.Append(";pwd=");
21     connectionString.Append(senha);
22
23     return new OdbcConnection(connectionString.ToString());
24   }
25 }
```

Código C# 2.30: ConnectionFactory.cs

Agora podemos obter uma nova conexão apenas chamando `ConnectionFactory.CreateConnection()`.

O resto do sistema não precisa mais conhecer os detalhes sobre a conexão com o banco de dados, diminuindo o acoplamento da aplicação.



Exercícios de Fixação

11 Adicione uma nova classe chamada `ConnectionFactory` com seguinte código:

```
using System;
using System.Data.Odbc;
using System.Text;

namespace Odbc
{
  static class ConnectionFactory
  {
    public static OdbcConnection CreateConnection()
    {
      string driver = @"SQL Server";
      string servidor = @".\SQLEXPRESS";
      string baseDeDados = @"livraria";

      StringBuilder connectionString = new StringBuilder();
      connectionString.Append("driver=");
      connectionString.Append(driver);
      connectionString.Append(";server=");
      connectionString.Append(servidor);
      connectionString.Append(";database=");
      connectionString.Append(baseDeDados);
      connectionString.Append(";Trusted_Connection=Yes");

      return new OdbcConnection(connectionString.ToString());
    }
  }
}
```

Código C# 2.31: www.k19.com.br/exercicios/k32/02/011/ConnectionFactory.cs

12 Altere as classes **CriaBaseDeDadosLivraria**, **CriaTabelaEditora**, **InsereEditora** e **ListaEditoras** para que elas utilizem a fábrica de conexão. Depois, para testar, adicione uma editora e recupere a lista de editoras novamente.

```
using System.Data.Odbc;

namespace Odbc
{
  class CriaBaseDeDadosLivraria
  {
    static void Main(string[] args)
    {
      using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
      {
        conexao.Open();

        string sql = @"IF EXISTS
          (SELECT name FROM master.dbo.sysdatabases WHERE name = 'livraria')
          DROP DATABASE livraria";
        OdbcCommand command = new OdbcCommand(sql, conexao);
        command.ExecuteNonQuery();

        sql = @"CREATE DATABASE livraria";
```

```
20          command = new OdbcCommand(sql, conexao);
21          command.ExecuteNonQuery();
22        }
23      }
24    }
25  }
```

Código C# 2.32: www.k19.com.br/exercicios/k32/02/012/CriaBaseDeDadosLivraria.cs

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class CriaTabelaEditora
6    {
7      static void Main(string[] args)
8      {
9        using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
10       {
11         conexao.Open();
12
13         string sql =
14           "CREATE TABLE Editoras (" +
15             "id BIGINT IDENTITY(1,1)," +
16             "nome VARCHAR (255) NOT NULL," +
17             "email VARCHAR (255) NOT NULL," +
18             "CONSTRAINT PK_Editoras PRIMARY KEY CLUSTERED (id asc)" +
19           ")";
20         OdbcCommand command = new OdbcCommand(sql, conexao);
21         command.ExecuteNonQuery();
22       }
23     }
24   }
25 }
```

Código C# 2.33: www.k19.com.br/exercicios/k32/02/012/CriaTabelaEditora.cs

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class InsereEditora
6    {
7      static void Main(string[] args)
8      {
9        Editora e = new Editora();
10
11       System.Console.Write("Digite o Nome da Editora:");
12       e.Nome = System.Console.ReadLine();
13
14       System.Console.Write("Digite o Email da Editora:");
15       e.Email = System.Console.ReadLine();
16
17       string sql =
18         @"INSERT INTO Editoras (Nome, Email)
19         OUTPUT INSERTED.id
20         VALUES(?, ?)";
21
22       using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
23       {
24         OdbcCommand command = new OdbcCommand(sql, conexao);
25
26         command.Parameters.AddWithValue("@Nome", e.Nome);
27         command.Parameters.AddWithValue("@Email", e.Email);
28
29         conexao.Open();
30         e.Id = command.ExecuteScalar() as long?;
31
```

```
32          System.Console.WriteLine("Editora cadastrada com id: " + e.Id);
33        }
34      }
35    }
36  }
```

Código C# 2.34: www.k19.com.br/exercicios/k32/02/012/InsereEditora.cs

```
1  using System.Data.Odbc;
2  using System.Collections.Generic;
3
4  namespace Odbc
5  {
6    class ListaEditoras
7    {
8      static void Main(string[] args)
9      {
10       using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
11       {
12         string sql = "SELECT * FROM Editoras";
13         OdbcCommand command = new OdbcCommand(sql, conexao);
14         conexao.Open();
15         OdbcDataReader resultado = command.ExecuteReader();
16
17                 List<Editora> lista = new List<Editora>();
18
19         while (resultado.Read())
20         {
21           Editora e = new Editora();
22
23           e.Id = resultado["Id"] as long?;
24           e.Nome = resultado["Nome"] as string;
25           e.Email = resultado["Email"] as string;
26
27           lista.Add(e);
28         }
29
30         foreach(Editora e in lista)
31         {
32           System.Console.WriteLine("{0} : {1} - {2}\n", e.Id, e.Nome, e.Email);
33         }
34       }
35     }
36   }
37 }
```

Código C# 2.35: www.k19.com.br/exercicios/k32/02/012/ListaEditoras.cs

13 Crie uma classe chamada **AlteraEditora** no projeto **Odbc** com o seguinte conteúdo.

```
1  using System;
2  using System.Data.Odbc;
3
4  namespace Odbc
5  {
6    class AlteraEditora
7    {
8      static void Main(string[] args)
9      {
10       using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
11       {
12         Editora e = new Editora();
13
14         System.Console.WriteLine("Digite o id da editora que deseja alterar: ");
15         e.Id = Convert.ToInt64(System.Console.ReadLine());
16
```

```
17          System.Console.Write("Digite o novo nome da editora:");
18          e.Nome = System.Console.ReadLine();
19
20          System.Console.Write("Digite o novo email da Editora:");
21          e.Email = System.Console.ReadLine();
22
23          string sql = "UPDATE Editoras SET nome = ?, email = ? WHERE id = ?";
24
25          OdbcCommand command = new OdbcCommand(sql, conexao);
26
27          command.Parameters.AddWithValue("@Nome", e.Nome);
28          command.Parameters.AddWithValue("@Email", e.Email);
29          command.Parameters.AddWithValue("@Id", e.Id);
30
31          conexao.Open();
32
33          command.ExecuteNonQuery();
34        }
35      }
36    }
37  }
```

Código C# 2.36: www.k19.com.br/exercicios/k32/02/013/AlteraEditora.cs

Execute a classe **AlteraEditora** e verifique se o registro foi alterado com sucesso na tabela Editora.

14 Crie uma classe chamada **RemoveEditora** no projeto **Odbc** com o seguinte conteúdo.

```
1   using System;
2   using System.Data.Odbc;
3
4   namespace Odbc
5   {
6     class RemoveEditora
7     {
8       static void Main(string[] args)
9       {
10        using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
11        {
12          System.Console.WriteLine("Digite o id da editora que deseja remover: ");
13          long id = Convert.ToInt64(System.Console.ReadLine());
14
15          string sql = "DELETE FROM Editoras WHERE id = ?";
16
17          OdbcCommand command = new OdbcCommand(sql, conexao);
18
19          command.Parameters.AddWithValue("@Id", id);
20
21          conexao.Open();
22
23          command.ExecuteNonQuery();
24        }
25      }
26    }
27  }
```

Código C# 2.37: www.k19.com.br/exercicios/k32/02/014/RemoveEditora.cs

Execute a classe **RemoveEditora** e verifique se o registro foi removido com sucesso na tabela Editora.



Exercícios Complementares

8 Altere as classes **CriaTabelaLivro**, **InsereLivro** e **ListaLivros** para que elas utilizem a fábrica de conexão. Para testar, adicione um livro e recupere a lista de livros novamente.

9 Implemente a alteração dos dados dos livros pelo `id`.

10 Implemente a remoção de livros pelo `id`.

ENTITY FRAMEWORK

CAPÍTULO 3

Múltiplas sintaxes da linguagem SQL

No capítulo anterior, utilizamos conexões ODBC para fazer uma aplicação C# interagir com os SGDBs. Nessa interação, as consultas são definidas com a linguagem SQL. A sintaxe dessa linguagem é diferente em cada SGDB. Dessa forma, a complexidade do trabalho dos desenvolvedores aumenta. Para resolver esse problema, as consultas deveriam ser definidas através de um mecanismo independente da linguagem SQL.

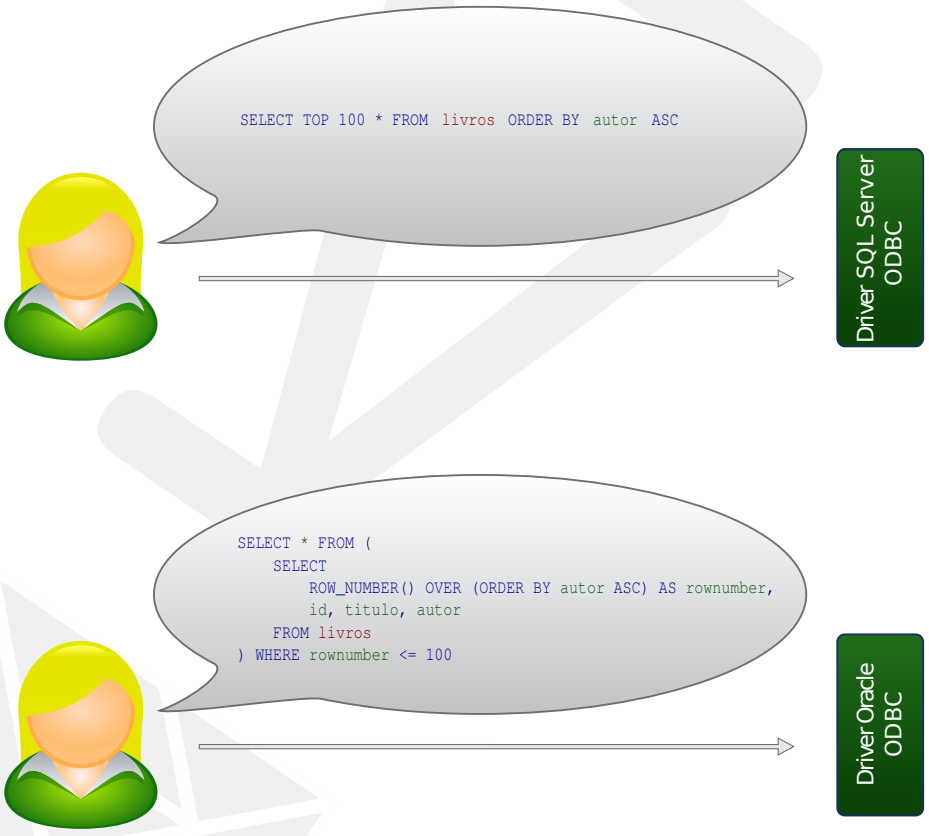


Figura 3.1: Diferentes sintaxes da linguagem SQL

Orientação a Objetos VS Modelo Relacional

Outro problema na comunicação entre uma aplicação C# e um SGDB é o conflito de paradigmas. Os SGDBs são organizados seguindo o modelo relacional. Por outro lado, as aplicações C# utilizam

o modelo orientado a objetos.

A transição de dados entre o modelo relacional e o modelo orientado a objetos não é simples. Para realizar essa transição, é necessário definir um mapeamento entre os conceitos desses dois paradigmas. Por exemplo, classes podem ser mapeadas para tabelas, objetos para registros, atributos para campos e referência entre objetos para chaves estrangeiras.

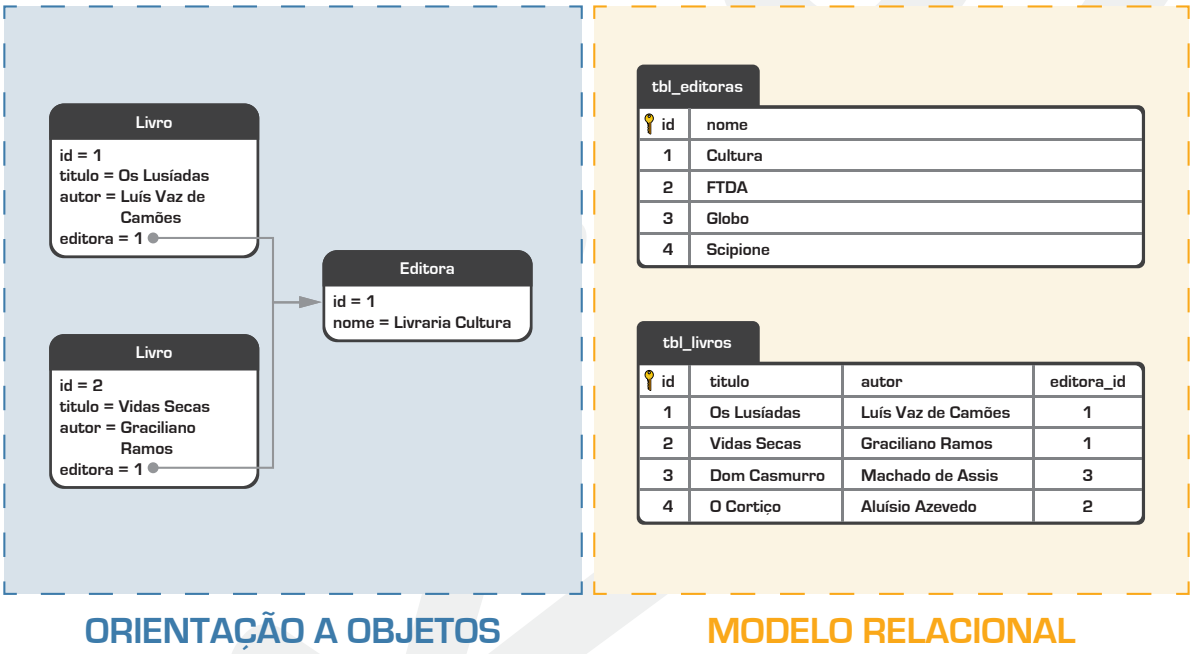


Figura 3.2: Modelo Orientado a Objetos vs Modelo Relacional



Ferramentas ORM

Para facilitar a comunicação entre aplicações C# que seguem o modelo orientado a objetos e os SGDBs que seguem o modelo relacional, podemos utilizar ferramentas que automatizam a transição de dados entre as aplicações e os SGDBs. Essas ferramentas são conhecidas como ferramentas **ORM** (*Object Relational Mapper*).

As ferramentas ORM oferecem mecanismos de consultas independentes da linguagem SQL. Dessa forma, o acoplamento entre as aplicações e os SGDBs diminui drasticamente. As principais ferramentas ORM para C# utilizadas no mercado são o Entity Framework e o NHibernate. Nesse curso, utilizaremos o Entity Framework 5.0.0.

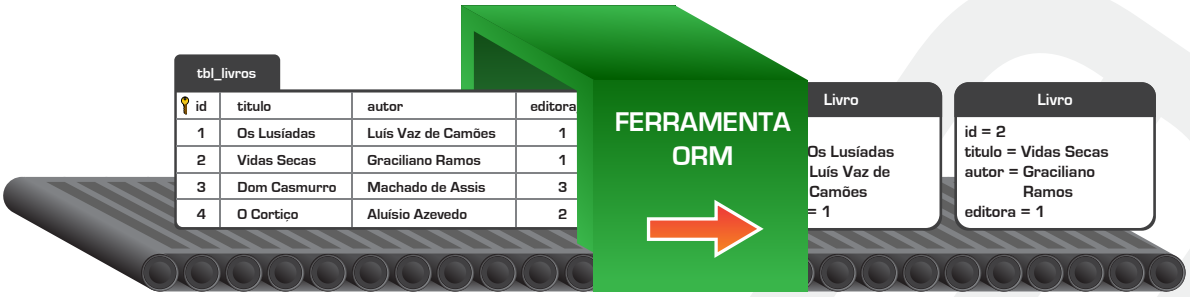


Figura 3.3: Transformação dos dados do Modelo Relacional para o Modelo Orientado a Objetos



Figura 3.4: Transformação dos dados do Modelo Orientado a Objetos para o Modelo Relacional



Instalando o Entity Framework

A instalação do Entity Framework em um projeto do **Visual Studio** pode ser facilmente realizada através do **Package Manager Console**. Veja o comando de instalação a seguir.

```
PM> Install-Package EntityFramework -Version 6.1.3
```

Terminal 3.1: Package Manager Console



Mais Sobre

O Entity Framework também pode ser instalado através do **Manage NuGet Packages**. Basicamente, o Manage NuGet Packages é a alternativa visual ao Package Manager Console.

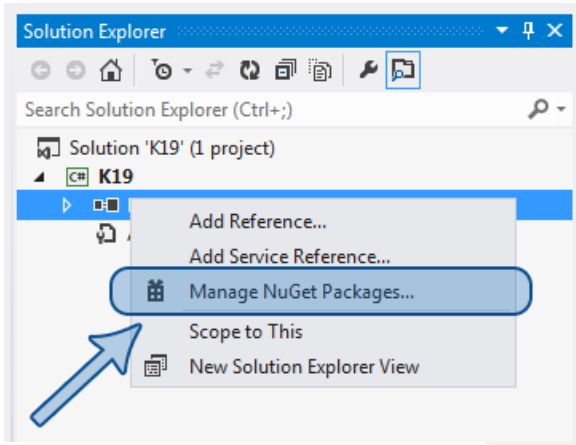


Figura 3.5: Manage NuGet Package

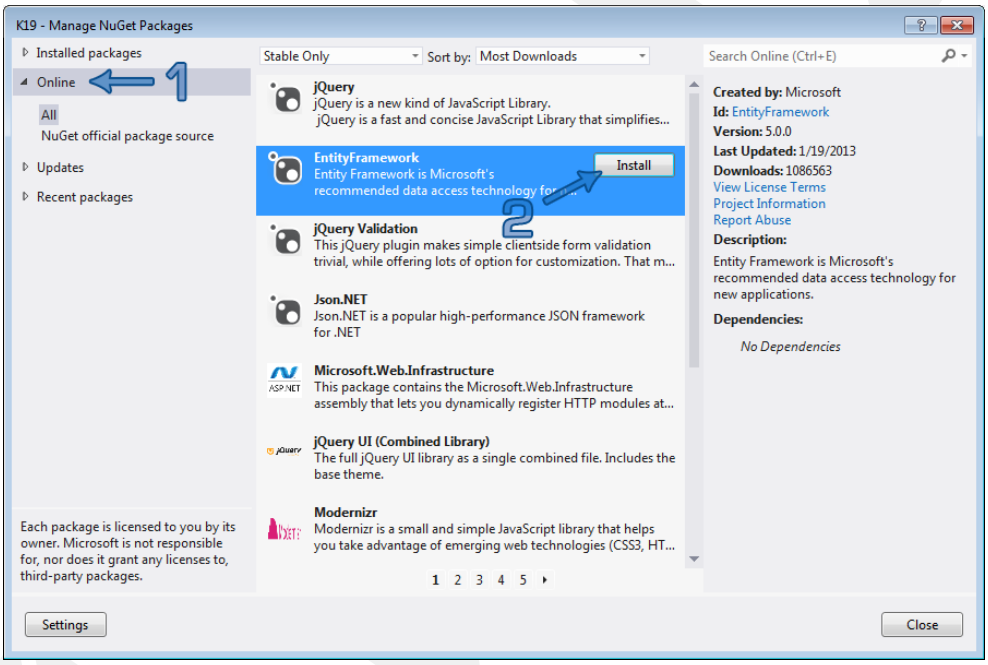


Figura 3.6: Manage NuGet Package

Configuração

Para utilizar os recursos do Entity Framework, devemos definir pelo menos um `DbContext`. Para isso, basta criar uma classe e herdar da classe `DbContext`.

```
1 class K19Context : DbContext
2 {
3
4 }
```

Código C# 3.1: K19Context.cs

O Entity Framework é fortemente baseado no conceito de "Convenção sobre Configuração". Dessa forma, nenhuma configuração é necessária a não ser que seja necessário alterar o comportamento de um `DbContext`.

Por padrão, um `DbContext` utilizará o SQL Express que está rodando na mesma máquina que a aplicação. Se o SQL Express não estiver rodando, o LocalDb será utilizado. Para alterar o comportamento de um `DbContext`, podemos definir uma string de conexão no arquivo `app.config` ou no arquivo `web.config` com o mesmo nome da classe que define esse `DbContext`. Pode ser utilizado o fully qualified name ou o unqualified name dessa classe. Nessa abordagem, podemos alterar a forma de autenticação que por padrão é Windows Authentication.

```
<configuration>
  <connectionStrings>
    <add name="K19Context"
         providerName="System.Data.SqlClient"
         connectionString="[STRING DE CONEXAO]"/>
  </connectionStrings>
</configuration>
```

Código XML 3.1: app.config ou web.config

Por padrão, o nome da base de dados utilizada por um `DbContext` é igual ao fully qualified name da classe que define esse `DbContext`. Portanto, no exemplo a seguir, o nome da base de dados utilizada pelo `K19Context` é `Sistema.K19Context`.

```
namespace Sistema
{
  class K19Context : DbContext
  {

  }
}
```

Código C# 3.2: K19Context.cs

Podemos modificar o nome da base de dados escolhido por padrão. Para isso, basta adicionar um construtor na classe que define o `DbContext` semelhante ao mostrado no código a seguir.

```
namespace Sistema
{
  class K19Context : DbContext
  {
    public K19Context() : base("K19")
    {

    }
  }
}
```

Código C# 3.3: K19Context.cs

Há quatro estratégias para criar a base de dados e as tabelas.

CreateDatabaseIfNotExists (Padrão): Na inicialização do `DbContext`, se a base de dados não existir, ela será criada. Por outro lado, se a base de dados existir e houver alguma alteração no modelo, uma exceção será lançada para interromper a inicialização do `DbContext`.

DropCreateDatabaseIfModelChanges: Na inicialização do `DbContext`, se a base de dados não existir, ela será criada. Por outro lado, se a base de dados existir e houver alguma alteração no modelo, essa base de dados será apagada e uma nova será criada.

DropCreateDatabaseAlways: Na inicialização do `DbContext`, se a base de dados não existir, ela será criada. Por outro lado, se a base de dados existir, ela será apagada e uma nova será criada.

Custom DB Initializer: É possível definir um procedimento de inicialização personalizado a partir de um dos procedimentos acima. Basta criar uma classe e herdar da classe `CreateDatabaseIfNotExists`, `DropCreateDatabaseIfModelChanges` ou `DropCreateDatabaseAlways`.

Para mudar a estratégia padrão que é a `CreateDatabaseIfNotExists`, podemos utilizar o método `SetInitializer()` no construtor do `DbContext` correspondente.

```
1 class K19Context : DbContext
2 {
3   public K19Context()
4   {
5     DropCreateDatabaseIfModelChanges<K19Context> initializer =
6       new DropCreateDatabaseIfModelChanges<K19Context>();
7     Database.SetInitializer<K19Context>(initializer);
8   }
9 }
```

Código C# 3.4: K19Context.cs



Mapeamento

Por padrão, no Entity Framework, o mapeamento das entidades segue diversas convenções. Considere as entidades `Livro` e `Editora` definidas abaixo.

```
1 class Livro
2 {
3   public long LivroId { get; set; }
4   public string Titulo { get; set; }
5   public double Preco { get; set; }
6 }
```

Código C# 3.5: Livro.cs

```
1 class Editora
2 {
3   public long EditoraId { get; set; }
4   public string Nome { get; set; }
5   public string Email { get; set; }
6 }
```

Código C# 3.6: Editora.cs

As entidades devem ser "registradas" em um **DbContext**. Para registrar uma entidade, basta definir uma propriedade do tipo **DbSet**. Veja o exemplo abaixo.

```
1 namespace EF
2 {
3   class K19Context : DbContext
4   {
```

```
5       public DbSet<Editora> Editoras { get; set; }
6       public DbSet<Livro> Livros { get; set; }
7     }
8   }
```

Código C# 3.7: K19Context.cs

De acordo com as convenções do Entity Framework, as entidades `Editora` e `Livro` serão mapeadas para tabelas chamadas **Editoras** e **Livroes** respectivamente. Observe que os nomes das entidades foram pluralizados seguindo as regras da língua inglesa para definir os nomes das tabelas.

Cada propriedade da entidade `Editora` será mapeada para uma coluna com o mesmo nome na tabela `Editoras`. Os tipos dessas colunas são escolhidos automaticamente. Analogamente, cada propriedade da entidade `Livro` será mapeada para uma coluna com o mesmo nome na tabela `Livroes`. Os tipos dessas colunas também são escolhidos automaticamente.

Por convenção, a coluna correspondente à propriedade `LivroId` será definida como chave primária da tabela `Livroes`. Analogamente, a coluna correspondente à propriedade `EditoraId` será definida como chave primária da tabela `Editoras`. Além disso, também por convenção, essas chaves primárias serão geradas automaticamente pelo banco de dados.



Mais Sobre

Para desabilitar a pluralização aplicada no nome das tabelas, é necessário sobrescrever o método **OnModelCreating()** da classe `DbContext`.

```
1  class K19Context : DbContext
2  {
3    public DbSet<Editora> Editoras { get; set; }
4    public DbSet<Livro> Livros { get; set; }
5
6    protected override void OnModelCreating( DbModelBuilder dbModelBuilder)
7    {
8      dbModelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
9    }
10 }
```

Código C# 3.8: K19Context.cs

A seguir, veremos as principais convenções de mapeamento do Entity Framework e como modificá-las.

Primary Key

O Entity Framework assumirá como identificador de uma entidade, a propriedade cujo nome seja **ID** ou **EntidadeID** escrito com letras maiúsculas e/ou minúsculas. Essa propriedade será mapeada para uma coluna definida no banco de dados como chave primária da tabela correspondente a entidade. Veja os exemplos abaixo.

```
1  class Livro
2  {
3    public int LivroID { get; set; }
4  }
```

Código C# 3.9: Livro.cs

```
1 class Livro
2 {
3   public int ID { get; set; }
4 }
```

Código C# 3.10: Livro.cs

Se o tipo da propriedade utilizada como identificador de uma entidade for numérico, os valores da coluna correspondente a essa propriedade serão gerados pelo banco de dados. Isso significa que, no SQL Server, a coluna será definida com a opção **Identity**.

Key

Considere a seguinte classe.

```
1 class Aluno
2 {
3   public int Codigo { get; set; }
4
5   public string Nome { get; set; }
6 }
```

Código C# 3.11: Aluno.cs

Nenhuma propriedade se enquadra nas convenções de nomenclatura de chave primária. Contudo, é possível definir explicitamente, qual propriedade será mapeada como chave primária através da anotação **Key**. No exemplo abaixo, a propriedade `Codigo` foi definida como chave da entidade.

```
1 class Aluno
2 {
3   [Key]
4   public int Codigo { get; set; }
5
6   public string Nome { get; set; }
7 }
```

Código C# 3.12: Aluno.cs

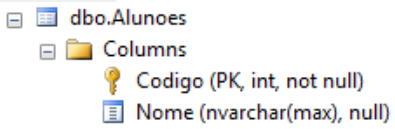


Figura 3.7: Escolhendo a chave primária explicitamente

DatabaseGenerated

Por padrão, quando a propriedade correspondente à chave primária de uma tabela é numérica, os valores dessa chave serão gerados pelo banco de dados. Podemos alterar esse comportamento através da anotação **DatabaseGenerated**.

```
1 class Aluno
2 {
3   [DatabaseGenerated(DatabaseGeneratedOption.None)]
4   public int AlunoID { get; set; }
5
6   public string Nome { get; set; }
7 }
```

Código C# 3.13: Aluno.cs

Table e Column

No Entity Framework, os nomes das tabelas e das colunas são definidos por convenção. Contudo, podemos personalizá-los de acordo com a necessidade através das anotações **Table** e **Column**.

```
1 [Table("tbl_alunos")]
2 class Aluno
3 {
4   public int AlunoID { get; set; }
5
6   [Column("nome_do_aluno")]
7   public string Nome { get; set; }
8 }
```

Código C# 3.14: Aluno.cs

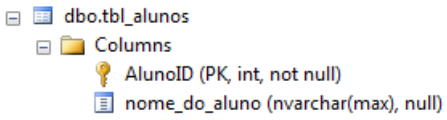


Figura 3.8: Personalizando os nomes das tabelas e das colunas

A anotação `Column` também permite a escolha do tipo da coluna correspondente à propriedade anotada.

```
1 class Produto
2 {
3   public int ProdutoID { get; set; }
4
5   [Column("descricao_do_produto", TypeName="text")]
6   public string Descricao { get; set; }
7 }
```

Código C# 3.15: Produto.cs

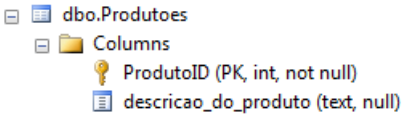


Figura 3.9: Personalizando o tipo de uma coluna

Required

Uma propriedade pode ser definida explicitamente como obrigatória através da anotação **Required**. No banco de dados, as colunas correspondentes às propriedades obrigatórias serão definidas com a opção **not null**.

```
1 class Aluno
2 {
3   public int AlunoID { get; set; }
4
5   [Required]
```

```
6   public string Nome { get; set; }
7 }
```

Código C# 3.16: Aluno.cs

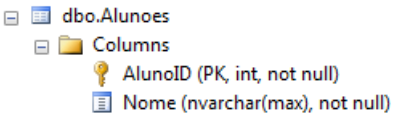


Figura 3.10: Definindo propriedades obrigatórias

MaxLength

A quantidade máxima de caracteres de uma string pode ser definida através da anotação **MaxLength**.

```
1  class Aluno
2  {
3    public int AlunoID { get; set; }
4
5    [MaxLength(30)]
6    public string Nome { get; set; }
7  }
```

Código C# 3.17: Aluno.cs

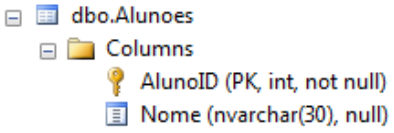


Figura 3.11: Definindo a quantidade máxima de caracteres de uma propriedade

NotMapped

Considere a seguinte classe.

```
1  class Aluno
2  {
3    public int AlunoID { get; set; }
4
5    public DateTime DataDeNascimento { get; set; }
6
7    public int Idade { get; set; }
8
9  }
```

Código C# 3.18: Aluno.cs

Observe que a propriedade `Idade` pode ser calculada a partir da propriedade `DataDeNascimento`. Portanto, não é necessário armazenar o valor da idade no banco de dados. Nesses casos, podemos utilizar a anotação **NotMapped** e indicar quais propriedades não devem ser mapeadas nas tabelas.

```
1  class Aluno
2  {
3    public int AlunoID { get; set; }
4
5    public DateTime DataDeNascimento { get; set; }
```

```
6
7     [NotMapped]
8     public int Idade { get; set; }
9
10 }
```

Código C# 3.19: Aluno.cs

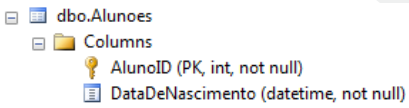


Figura 3.12: Excluindo uma propriedade no mapeamento



Exercícios de Fixação

1 Abra o `Visual Studio`. Crie um projeto do tipo **Console Application** em **C#** chamado **EF**.

2 Instale o Entity Framework no projeto **EF** utilizando o `Manage Nuget Packages`. Siga o exemplo das imagens abaixo.

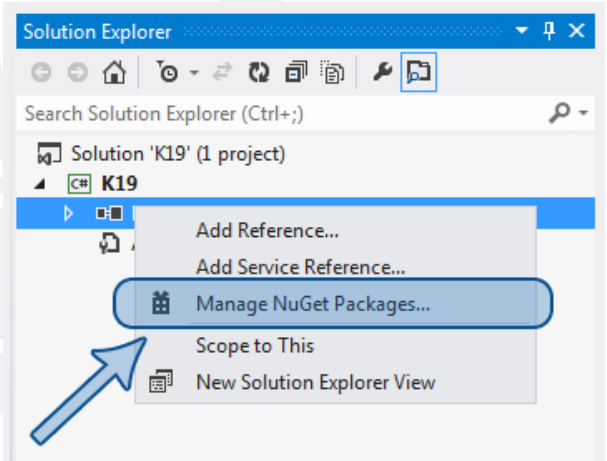


Figura 3.13: Manage NuGet Package

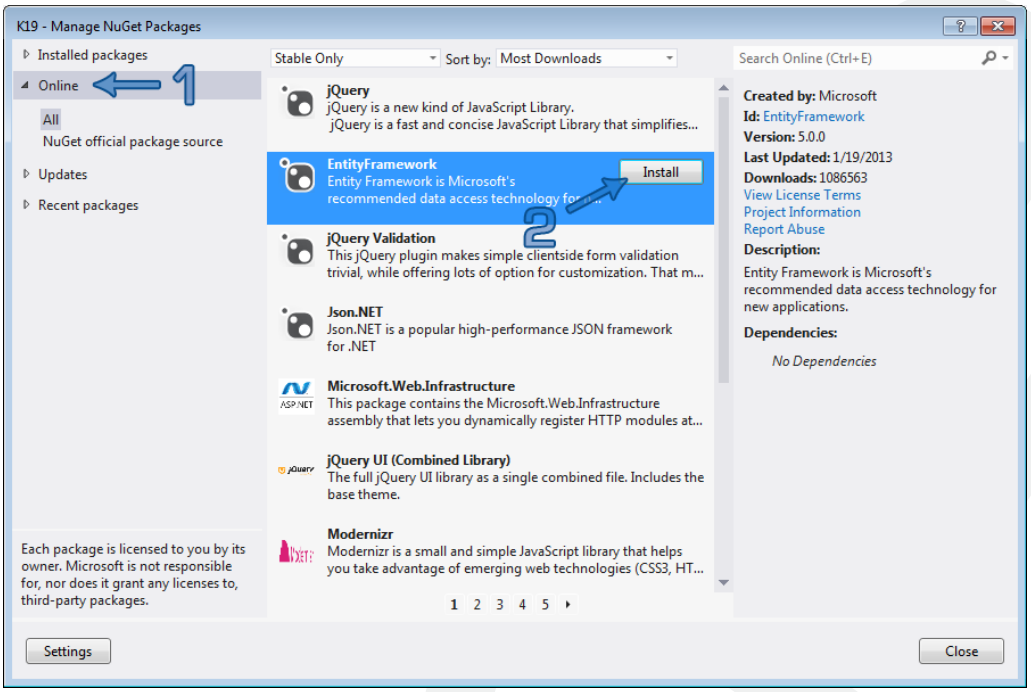


Figura 3.14: Manage NuGet Package

3 Crie uma classe chamada **Editora** no projeto **EF**.

```
1 namespace EF
2 {
3   class Editora
4   {
5     public long Id { get; set; }
6     public string Nome { get; set; }
7     public string Email { get; set; }
8   }
9 }
```

Código C# 3.20: www.k19.com.br/exercicios/k32/03/003/Editora.cs

4 Crie uma classe chamada **EFContext** no projeto **EF**.

```
1 using System.Data.Entity;
2
3 namespace EF
4 {
5   class EFContext : DbContext
6   {
7     public DbSet<Editora> Editoras { get; set; }
8   }
9 }
```

Código C# 3.21: www.k19.com.br/exercicios/k32/03/004/EFContext.cs

5 Crie uma classe chamada **Teste** no projeto **EF**.

```
1 namespace EF
```

```
{
  class Teste
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        Editora e = new Editora
        {
          Nome = "K19",
          Email = "contato@k19.com.br"
        }

        ctx.Editoras.Add(e);

        ctx.SaveChanges();
      }
    }
  }
}
```

Código C# 3.22: www.k19.com.br/exercicios/k32/03/005/Teste.cs

Selecione a classe **Teste** como Startup Object do projeto **EF**. Execute esse projeto e verifique se a base de dados EF.EFContext e a tabela Editoras foram criadas com sucesso.



Gerenciamento de entidades

As instâncias das entidades de uma aplicação que utiliza o Entity Framework são administradas por objetos do tipo `DbContext`. Esses objetos são responsáveis pelas operações de inserção, remoção, alteração e consulta. A seguir, mostraremos o funcionamento básico de um `DbContext`.

A primeira regra básica sobre os DbContexts é sempre utilizá-los dentro de um bloco `using` para evitar que os recursos utilizados por eles não sejam devidamente fechados.

```
using (var context = new K19Context())
{
  // utilizando o DbContext
}
```

Código C# 3.23: Utilizando um DbContext dentro do bloco `using`

Persistindo

Para indicar que determinado objeto deve ser persistido no banco de dados, devemos utilizar o método `System.Data.Entity.DbSet.Add()`, passando o objeto em questão como parâmetro. O método `Add()` adiciona o objeto ao contexto com o estado **Added**.

Para armazenar de fato as informações de um objeto no banco de dados, utilizamos o método `SaveChanges()` do `DbContext`. Todos os objetos do contexto que estejam no estado `Added` são inseridos no banco de dados quando o método `SaveChanges()` é chamado.

```
Editora editora = new Editora { Nome = "K19", Email = "contato@k19.com.br" };

// Adiciona a editora criada ao contexto com estado Added
context.Editoras.Add(editora);

// Sincroniza o contexto com o banco de dados
```

```
7 context.SaveChanges();
```

Código C# 3.24: Persistindo um objeto

Buscando

Para obter um objeto que contenha informações do banco de dados, basta utilizar o método `Find()` do `DbSet`. Devemos passar o identificador do objeto que desejamos recuperar como parâmetro para esse método.

```
1 Editora editora = context.Editoras.Find(1);
```

Código C# 3.25: Buscando uma editora com o identificador 1

Removendo

Para indicar que determinado objeto deve ser removido, basta utilizar o método `System.Data.Entity.DbSet.Remove()`. Esse método marca o objeto para ser removido do banco de dados, colocando-o no estado **Deleted**.

Quando o método `SaveChanges()` é chamado, todas os objetos no estado `Deleted` são removidos do banco de dados.

```
1 Editora editora = context.Editoras.Find(1);
2
3 // Marca a editora que deve ser removida
4 context.Editoras.Remove(editora);
5
6 // Sincroniza o contexto com o banco de dados
7 context.SaveChanges();
```

Código C# 3.26: Removendo uma editora do banco de dados

Atualizando

Para alterar os dados do registro correspondente a um objeto, basta modificar as suas propriedades. Quando as propriedades de um objeto do contexto são alteradas, o estado **Modified** é associado a este objeto. Objetos no estado `Modified` são atualizados no banco de dados quando o método `SaveChanges()` é chamado.

```
1 Editora editora = context.Editoras.Find(1);
2
3 // Modificando as propriedades de um objeto
4 editora.Nome = "Abril S/A";
5
6 // Sincroniza o contexto com o banco de dados
7 context.SaveChanges();
```

Código C# 3.27: Alterando o nome de uma editora

Listando

Para obter uma listagem com todos os objetos referentes aos registros de uma tabela, podemos utilizar a Language Integrated Query (LINQ), que nos permite escrever consultas dentro do código C#.

```
var consulta = from e in context.Editoras
               where e.Nome.Contains("A")
               select e;

// Equivalente a: SELECT * FROM Editoras e where e.Nome Like 'A%'
foreach (var item in consulta)
{
  System.Console.WriteLine("Editora: " + item.Nome);
}
```

Código C# 3.28: Utilizando LINQ para fazer uma consulta



Exercícios de Fixação

6 Crie uma classe chamada **InsereEditoraComEF** no projeto **EF**.

```
namespace EF
{
  class InsereEditoraComEF
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        Editora e = new Editora();

        System.Console.Write("Digite o Nome da Editora:");
        e.Nome = System.Console.ReadLine();

        System.Console.Write("Digite o Email da Editora:");
        e.Email = System.Console.ReadLine();

        ctx.Editoras.Add(e);

        ctx.SaveChanges();

        System.Console.WriteLine("Editora cadastrada com id: " + e.Id);
      }
    }
  }
}
```

Código C# 3.29: www.k19.com.br/exercicios/k32/03/006/InsereEditoraComEF.cs

Selecione a classe **InsereEditoraComEF** como Startup Object do projeto **EF**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Editoras.

7 Crie uma classe chamada **ListaEditorasComEF** no projeto **EF**.

```
namespace EF
{
  class ListaEditorasComEF
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        foreach(Editora e in ctx.Editoras)
        {
```

```
11            System.Console.WriteLine("Editora {0} - {1}", e.Nome, e.Email);
12          }
13        }
14      }
15    }
16  }
```

Código C# 3.30: www.k19.com.br/exercicios/k32/03/007/ListaEditorasComEF.cs

Selecione a classe **ListaEditorasComEF** como Startup Object do projeto **EF**. Execute esse projeto e veja a saída do programa.

8 Crie uma classe chamada **AlteraEditoraComEF** no projeto **EF**.

```
1  using System;
2
3  namespace EF
4  {
5    class AlteraEditoraComEF
6    {
7      static void Main(string[] args)
8      {
9        using(EFContext ctx = new EFContext())
10       {
11         Console.Write("Digite o Id da Editora que deseja alterar:");
12         long id = Convert.ToInt64(Console.ReadLine());
13
14         Editora e = ctx.Editoras.Find(id);
15
16         Console.Write("Digite o novo Nome da Editora:");
17         e.Nome = Console.ReadLine();
18
19         Console.Write("Digite o novo Email da Editora:");
20         e.Email = Console.ReadLine();
21
22         ctx.SaveChanges();
23       }
24     }
25   }
26 }
```

Código C# 3.31: www.k19.com.br/exercicios/k32/03/008/AlteraEditoraComEF.cs

Selecione a classe **AlteraEditoraComEF** como Startup Object do projeto **EF**. Execute esse projeto e verifique se o registro foi alterado com sucesso na tabela Editoras.

9 Crie uma classe chamada **RemoveEditoraComEF** no projeto **EF**.

```
1  using System;
2
3  namespace EF
4  {
5    class RemoveEditoraComEF
6    {
7      static void Main(string[] args)
8      {
9        using(EFContext ctx = new EFContext())
10       {
11         Console.Write("Digite o Id da Editora que deseja remover:");
12         long id = Convert.ToInt64(Console.ReadLine());
13
14         Editora e = ctx.Editoras.Find(id);
```

```
15
16          ctx.Editoras.Remove(e);
17
18          ctx.SaveChanges();
19        }
20      }
21    }
22  }
```

Código C# 3.32: www.k19.com.br/exercicios/k32/03/009/RemoveEditoraComEF.cs

Selecione a classe **RemoveEditoraComEF** como Startup Object do projeto **EF**. Execute esse projeto e verifique se o registro foi removido com sucesso na tabela Editoras.



Tipos complexos

Considere que é necessário cadastrar os endereços das editoras no sistema da livraria. Esses endereços devem ser formados por país, estado, cidade, logradouro, número, complemento e CEP. Essas informações podem ser definidas na classe `Editora`.

```
1  class Editora
2  {
3    public long Id { get; set; }
4
5    public string Nome { get; set; }
6
7    public string Email { get; set; }
8
9    public string Pais { get; set; }
10
11   public string Estado { get; set; }
12
13   public string Cidade { get; set; }
14
15   public string Logradouro { get; set; }
16
17   public string Numero { get; set; }
18
19   public string Complemento { get; set; }
20
21   public string Cep { get; set; }
22 }
```

Código C# 3.33: Editora.cs

Cada propriedade acima será mapeada para uma coluna da tabela correspondente a classe `Editora`.

Agora, considere que o sistema da livraria deve permitir também o cadastro dos endereços dos clientes. Seguindo a mesma abordagem, as informações dos endereços dos clientes podem ser definidas na classe `Cliente`.

```
1  class Cliente
2  {
3    public long Id { get; set; }
4
5    public string Nome { get; set; }
6
7    public string Cpf { get; set; }
8
9    public string Pais { get; set; }
10
```

```
11    public string Estado { get; set; }
12
13    public string Cidade { get; set; }
14
15    public string Logradouro { get; set; }
16
17    public string Numero { get; set; }
18
19    public string Complemento { get; set; }
20
21    public string Cep { get; set; }
22 }
```

Código C# 3.34: Cliente.cs

Cada atributo acima será mapeado para uma coluna da tabela correspondente a classe `Cliente`.

Contudo, podemos melhorar a modelagem das classes `Editora` e `Cliente` criando uma terceira classe para definir os edereços.

```
1  class Endereco
2  {
3    public string Pais { get; set; }
4
5    public string Estado { get; set; }
6
7    public string Cidade { get; set; }
8
9    public string Logradouro { get; set; }
10
11   public string Numero { get; set; }
12
13   public string Complemento { get; set; }
14
15   public string Cep { get; set; }
16 }
```

Código C# 3.35: Endereco.cs

```
1  class Editora
2  {
3    public long Id { get; set; }
4
5    public string Nome { get; set; }
6
7    public string Email { get; set; }
8
9    public Endereco Endereco { get; set; }
10 }
```

Código C# 3.36: Editora.cs

```
1  class Cliente
2  {
3    public long Id { get; set; }
4
5    public string Nome { get; set; }
6
7    public string Cpf { get; set; }
8
9    public Endereco Endereco { get; set; }
10 }
```

Código C# 3.37: Cliente.cs

A criação da classe `Endereco` simplifica a modelagem das classes `Editora` e `Cliente`. Permite um melhor reaproveitamento de código.

As propriedades da classe `Endereco` serão mapeadas para colunas nas tabelas correspondentes às classes `Editora` e `Cliente`.



Exercícios de Fixação

10 Crie uma classe chamada **Endereco** no projeto **EF**.

```
namespace EF
{
  class Endereco
  {
    public string Pais { get; set; }

    public string Estado { get; set; }

    public string Cidade { get; set; }

    public string Logradouro { get; set; }

    public string Numero { get; set; }

    public string Complemento { get; set; }

    public string Cep { get; set; }
  }
}
```

Código C# 3.38: www.k19.com.br/exercicios/k32/03/010/Endereco.cs

11 Crie uma classe chamada **Cliente** no projeto **EF**.

```
namespace EF
{
  class Cliente
  {
    public long Id { get; set; }

    public string Nome { get; set; }

    public string Cpf { get; set; }

    public Endereco Endereco { get; set; }
  }
}
```

Código C# 3.39: www.k19.com.br/exercicios/k32/03/011/Cliente.cs

12 Altere a classe `EFContext`.

```
using System.Data.Entity;

namespace EF
{
  class EFContext : DbContext
```

```
6   {
7     public DbSet<Editora> Editoras { get; set; }
8     public DbSet<Cliente> Clientes { get; set; }
9
10    public EFContext()
11    {
12      DropCreateDatabaseIfModelChanges<EFContext> initializer =
13        new DropCreateDatabaseIfModelChanges<EFContext>();
14      Database.SetInitializer<EFContext>(initializer);
15    }
16  }
17 }
```

Código C# 3.40: www.k19.com.br/exercicios/k32/03/012/EFContext.cs

13 Crie uma classe chamada **InsereClienteComEF** no projeto **EF**.

```
1  namespace EF
2  {
3    class InsereClienteComEF
4    {
5      static void Main(string[] args)
6      {
7        using(EFContext ctx = new EFContext())
8        {
9          Cliente c = new Cliente();
10
11         System.Console.Write("Digite o Nome do Cliente:");
12         c.Nome = System.Console.ReadLine();
13
14         System.Console.Write("Digite o CPF do Cliente:");
15         c.Cpf = System.Console.ReadLine();
16
17         Endereco e = new Endereco();
18
19         c.Endereco = e;
20
21         System.Console.WriteLine("Digite o endereço do cliente: ");
22
23         System.Console.Write("País: ");
24         e.Pais = System.Console.ReadLine();
25
26         System.Console.Write("Estado: ");
27         e.Estado = System.Console.ReadLine();
28
29         System.Console.Write("Cidade: ");
30         e.Cidade = System.Console.ReadLine();
31
32         System.Console.Write("Logradouro: ");
33         e.Logradouro = System.Console.ReadLine();
34
35         System.Console.Write("Número: ");
36         e.Numero = System.Console.ReadLine();
37
38         System.Console.Write("Complemento: ");
39         e.Complemento = System.Console.ReadLine();
40
41         System.Console.Write("Cep: ");
42         e.Cep = System.Console.ReadLine();
43
44         ctx.Clientes.Add(c);
45
46         ctx.SaveChanges();
47
48         System.Console.WriteLine("Cliente cadastrado com id: " + c.Id);
49       }
50     }
```

```
51     }
52 }
```

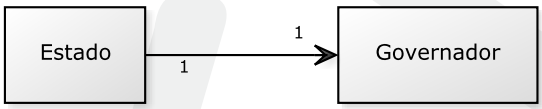
Código C# 3.41: www.k19.com.br/exercicios/k32/03/013/InsereClienteComEEcs

Selecione a classe **InsereClienteComEF** como Startup Object do projeto **EF**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Clientes.

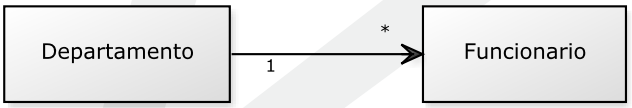
Relacionamentos

Os relacionamentos entre as entidades de um domínio devem ser expressos na modelagem através de vínculos entre classes. Podemos classificar os relacionamentos de acordo com a cardinalidade.

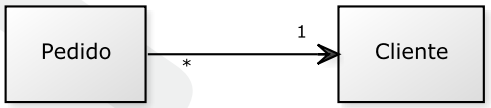
One to One (Um para Um): Por exemplo, um estado é governado por apenas um governador e um governador governa apenas um estado.



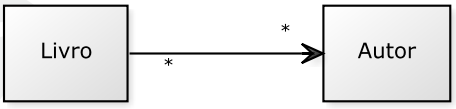
One to Many (Um para Muitos): Por exemplo, um departamento possui muitos funcionários e um funcionário trabalha em apenas em um departamento.



Many to One (Muitos para Um): Por exemplo, um pedido pertence a apenas um cliente e um cliente faz muitos pedidos.



Many to Many (Muitos para Muitos): Por exemplo, um livro possui muitos autores e um autor possui muitos livros.



One to One

Suponha que em nosso domínio existam duas entidades: Estado e Governador. Devemos criar uma classe para cada entidade e aplicar nelas as anotações básicas de mapeamento.

```
1 class Estado
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.42: Estado.cs

```
1 class Governador
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.43: Governador.cs

Como existe um relacionamento entre estados e governadores, devemos expressar esse vínculo através de uma propriedade que pode ser inserido na classe `Estado`.

```
1 class Estado
2 {
3   public long Id { get; set; }
4
5   public Governador Governador { get; set; }
6 }
```

Código C# 3.44: Estado.cs

Nesse caso, uma chave estrangeira será adicionada na na tabela correspondente à classe `Estado` com o nome `Governador_Id`. Essa chave estará vinculada à chave primária da tabela correspondente à classe `Governador`. Contudo, o Entity Framework não adiciona a restrição "unique" nessa chave estrangeira. Dessa forma, um governador poderá ser associado a vários estados. Descaracterizando o relacionamento "One to One". Para contornar esse problema, podemos adicionar essa restrição diretamente implementado um "Custom DB Initializer".

```
1 class CustomDBInitializer : DropCreateDatabaseIfModelChanges<K19Context>
2 {
3   protected override void Seed(K19Context ctx)
4   {
5     ctx.Database.ExecuteSqlCommand(
6       "ALTER TABLE Estadoes ADD CONSTRAINT governador UNIQUE(Governador_Id)");
7     base.Seed(ctx);
8   }
9 }
```

Código C# 3.45: CustomDBInitializer.cs

No método `Seed()` do "Custom DB Initializer", executamos um comando escrito diretamente em SQL. Esse comando adicionar a restrição "unique" na chave estrangeira da tabela correspondente à classe `Estado`.

O próximo passo é vincular esse "Custom DB Initializer" ao `DbContext`.

```
1 class K19Context : DbContext
2 {
3   // Propriedades
4
5   public K19Context()
6   {
7     CustomDBInitializer initializer = new CustomDBInitializer();
8     Database.SetInitializer<K19Context>(initializer);
```

```
9     }
10 }
```

Código C# 3.46: K19Context.cs

Podemos alterar o nome padrão da chave estrangeira utilizando as anotações **ForeignKey** e **Column**. A primeira anotação deve ser aplicada à propriedade que define o relacionamento. A segunda anotação deve ser aplicada a uma propriedade que será mapeada para a chave estrangeira.

```
1  class Estado
2  {
3    public long Id { get; set; }
4
5    [Column("GOV_ID")]
6    public long GovernadorId { get; set; }
7
8    [ForeignKey("GovernadorId")]
9    public Governador Governador { get; set; }
10 }
```

Código C# 3.47: Estado.cs



Exercícios de Fixação

14 Crie duas entidades no projeto `EF`: `Estado` e `Governador`.

```
1  namespace EF
2  {
3    class Governador
4    {
5      public long Id { get; set; }
6
7      public string Nome { get; set; }
8    }
9  }
```

Código C# 3.48: www.k19.com.br/exercicios/k32/03/014/Governador.cs

```
1  namespace EF
2  {
3    class Estado
4    {
5      public long Id { get; set; }
6
7      public string Nome { get; set; }
8
9      public Governador Governador { get; set; }
10   }
11 }
```

Código C# 3.49: www.k19.com.br/exercicios/k32/03/014/Estado.cs

15 Altere a classe `EFContext`.

```
1  using System.Data.Entity;
2
3  namespace EF
```

```
4  {
5    class EFContext : DbContext
6    {
7      public DbSet<Editora> Editoras { get; set; }
8      public DbSet<Cliente> Clientes { get; set; }
9      public DbSet<Estado> Estados { get; set; }
10     public DbSet<Governador> Governadores { get; set; }
11
12     public EFContext()
13     {
14       DropCreateDatabaseIfModelChanges<EFContext> initializer =
15         new DropCreateDatabaseIfModelChanges<EFContext>();
16       Database.SetInitializer<EFContext>(initializer);
17     }
18   }
19 }
```

Código C# 3.50: www.k19.com.br/exercicios/k32/03/015/EFContext.cs

16 Adicione um governador e um estado no banco de dados. Crie uma classe chamada `AdicionaEstadoGovernador` no projeto `EF`.

```
1  namespace EF
2  {
3    class AdicionaEstadoGovernador
4    {
5      static void Main(string[] args)
6      {
7        using(EFContext ctx = new EFContext())
8        {
9          Governador g = new Governador();
10         System.Console.WriteLine("Digite o nome do governador:");
11         g.Nome = System.Console.ReadLine();
12
13         Estado e = new Estado();
14         System.Console.WriteLine("Digite o nome do estado:");
15         e.Nome = System.Console.ReadLine();
16
17         e.Governador = g;
18
19         ctx.Estados.Add(e);
20
21         ctx.SaveChanges();
22
23         System.Console.WriteLine("Governador cadastrado com id: " + g.Id);
24         System.Console.WriteLine("Estado cadastrado com id: " + e.Id);
25       }
26     }
27   }
28 }
```

Código C# 3.51: www.k19.com.br/exercicios/k32/03/016/AdicionaEstadoGovernador.cs

Selecione a classe **AdicionaEstadoGovernador** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as tabelas Estadoes e Governadors foram criadas com sucesso.



One to Many

Suponha que em nosso domínio existam as entidades Departamento e Funcionário. Criaríamos duas classes com as anotações básicas de mapeamento.

```
1 class Departamento
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.52: Departamento.cs

```
1 class Funcionario
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.53: Funcionario.cs

Como existe um relacionamento entre departamentos e funcionários, devemos expressar esse vínculo através de uma propriedade que pode ser inserido na classe `Departamento`. Supondo que um departamento possa ter muitos funcionários, devemos utilizar uma coleção para expressar esse relacionamento.

```
1 class Departamento
2 {
3   public long Id { get; set; }
4
5   public ICollection<Funcionario> Funcionarios { get; set; }
6 }
```

Código C# 3.54: Departamento.cs

Nesse caso, uma chave estrangeira será adicionada na tabela correspondente à classe `Funcionario` com o nome `Departamento_Id`. Essa chave estará vinculada à chave primária da tabela correspondente à classe `Departamento`.

Podemos alterar o nome padrão da chave estrangeira utilizando as anotações **ForeignKey** e **Column**. A primeira anotação deve ser aplicada à propriedade que define o relacionamento. A segunda anotação deve ser aplicada a uma propriedade que será mapeada para a chave estrangeira. Essa propriedade deve ser definida na classe `Funcionario`.

```
1 class Departamento
2 {
3   public long Id { get; set; }
4
5   [ForeignKey("DepartamentoId")]
6   public ICollection<Funcionario> Funcionarios { get; set; }
7 }
```

Código C# 3.55: Departamento.cs

```
1 class Funcionario
2 {
3   public long Id { get; set; }
4
5   [Column("DEP_ID")]
6   public long DepartamentoId { get; set; }
7
8 }
```

Código C# 3.56: Funcionario.cs



Exercícios de Fixação

17 Crie duas entidades no projeto EF: `Funcionario` e `Departamento`.

```
1  namespace EF
2  {
3    class Funcionario
4    {
5      public long Id { get; set; }
6      public string Nome { get; set; }
7    }
8  }
```

Código C# 3.57: www.k19.com.br/exercicios/k32/03/017/Funcionario.cs

```
1  using System.Collections.Generic;
2
3  namespace EF
4  {
5    class Departamento
6    {
7      public long Id { get; set; }
8
9      public string Nome { get; set; }
10
11     public ICollection<Funcionario> Funcionarios { get; set; }
12   }
13 }
```

Código C# 3.58: www.k19.com.br/exercicios/k32/03/017/Departamento.cs

18 Altere a classe `EFContext`.

```
1  using System.Data.Entity;
2
3  namespace EF
4  {
5    class EFContext : DbContext
6    {
7      public DbSet<Editora> Editoras { get; set; }
8      public DbSet<Cliente> Clientes { get; set; }
9      public DbSet<Estado> Estados { get; set; }
10     public DbSet<Governador> Governadores { get; set; }
11     public DbSet<Funcionario> Funcionarios { get; set; }
12     public DbSet<Departamento> Departamentos { get; set; }
13
14     public EFContext()
15     {
16       DropCreateDatabaseIfModelChanges<EFContext> initializer =
17         new DropCreateDatabaseIfModelChanges<EFContext>();
18       Database.SetInitializer<EFContext>(initializer);
19     }
20   }
21 }
```

Código C# 3.59: www.k19.com.br/exercicios/k32/03/018/EFContext.cs

19 Adicione um departamento e um funcionário no banco de dados. Crie uma classe chamada `AdicionaDepartamentoFuncionario` no projeto `EF`.

```
using System.Collections.Generic;

namespace EF
{
  class AdicionaDepartamentoFuncionario
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        Funcionario f1 = new Funcionario();
        System.Console.WriteLine("Digite o nome do primeiro funcionário:");
        f1.Nome = System.Console.ReadLine();

        Funcionario f2 = new Funcionario();
        System.Console.WriteLine("Digite o nome do segundo funcionário:");
        f2.Nome = System.Console.ReadLine();

        Departamento d = new Departamento();
        System.Console.WriteLine("Digite o nome do departamento:");
        d.Nome = System.Console.ReadLine();

        d.Funcionarios = new List<Funcionario>();

        d.Funcionarios.Add(f1);
        d.Funcionarios.Add(f2);

        ctx.Departamentos.Add(d);

        ctx.SaveChanges();

        System.Console.WriteLine("Primeiro funcionário cadastrado com id :" + f1.Id);
        System.Console.WriteLine("Segundo funcionário cadastrado com id :" + f2.Id);
        System.Console.WriteLine("Departamento cadastrado com id :" + d.Id);
      }
    }
  }
}
```

Código C# 3.60: www.k19.com.br/exercicios/k32/03/019/AdicionaDepartamentoFuncionario.cs

Selecione a classe **AdicionaDepartamentoFuncionario** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as tabelas Funcionarioes e Departamentoes foram criadas com sucesso.



Many to One

Suponha que em nosso domínio existam as entidades Pedido e Cliente. As duas classes que modelariam essas entidades seriam definidas com as anotações principais de mapeamento.

```
class Pedido
{
  public long Id { get; set; }
}
```

Código C# 3.61: Pedido.cs

```
class Cliente
{
```

```
3   public long Id { get; set; }
4 }
```

Código C# 3.62: Cliente.cs

Como existe um relacionamento entre pedidos e clientes, devemos expressar esse vínculo através de uma propriedade que pode ser inserido na classe `Pedido`. Supondo que um pedido pertença a um único cliente, devemos utilizar uma propriedade simples para expressar esse relacionamento.

```
1 class Pedido
2 {
3   public long Id { get; set; }
4
5   public Cliente Cliente { get; set; }
6 }
```

Código C# 3.63: Pedido.cs

Nesse caso, uma chave estrangeira será adicionada na na tabela correspondente à classe `Pedido` com o nome `Cliente_Id`. Essa chave estará vinculada à chave primária da tabela correspondente à classe `Cliente`.

Podemos alterar o nome padrão da chave estrangeira utilizando as anotações **ForeignKey** e **Column**. A primeira anotação deve ser aplicada à propriedade que define o relacionamento. A segunda anotação deve ser aplicada a uma propriedade que será mapeada para a chave estrangeira.

```
1  class Pedido
2  {
3    public long Id { get; set; }
4
5    [Column("CLI_ID")]
6    public long ClienteId { get; set; }
7
8    [ForeignKey("ClienteId")]
9    public Cliente Cliente { get; set; }
10 }
```

Código C# 3.64: Pedido.cs



Exercícios de Fixação

20 Crie duas classes no projeto `EF`: `Pedido` e `Consumidor`.

```
1 namespace EF
2 {
3   class Consumidor
4   {
5     public long Id { get; set; }
6
7     public string Nome { get; set; }
8   }
9 }
```

Código C# 3.65: www.k19.com.br/exercicios/k32/03/020/Consumidor.cs

```
1 namespace EF
```

```
2 {
3   class Pedido
4   {
5     public long Id { get; set; }
6
7     public Consumidor Consumidor { get; set; }
8   }
9 }
```

Código C# 3.66: www.k19.com.br/exercicios/k32/03/020/Pedido.cs

21 Altere a classe `EFContext`.

```
1  using System.Data.Entity;
2
3  namespace EF
4  {
5    class EFContext : DbContext
6    {
7      public DbSet<Editora> Editoras { get; set; }
8      public DbSet<Cliente> Clientes { get; set; }
9      public DbSet<Estado> Estados { get; set; }
10     public DbSet<Governador> Governadores { get; set; }
11     public DbSet<Funcionario> Funcionarios { get; set; }
12     public DbSet<Departamento> Departamentos { get; set; }
13     public DbSet<Consumidor> Consumidores { get; set; }
14     public DbSet<Pedido> Pedidos { get; set; }
15
16     public EFContext()
17     {
18       DropCreateDatabaseIfModelChanges<EFContext> initializer =
19         new DropCreateDatabaseIfModelChanges<EFContext>();
20       Database.SetInitializer<EFContext>(initializer);
21     }
22   }
23 }
```

Código C# 3.67: www.k19.com.br/exercicios/k32/03/021/EFContext.cs

22 Adicione um cliente e um departamento no banco de dados. Crie uma classe chamada `AdicionaPedidoConsumidor` no projeto `EF`.

```
1  namespace EF
2  {
3    class AdicionaPedidoConsumidor
4    {
5      static void Main(string[] args)
6      {
7        using(EFContext ctx = new EFContext())
8        {
9          Consumidor c = new Consumidor();
10         System.Console.WriteLine("Digite o nome do consumidor");
11         c.Nome = System.Console.ReadLine();
12
13         Pedido p = new Pedido();
14
15         p.Consumidor = c;
16
17         ctx.Pedidos.Add(p);
18
19         ctx.SaveChanges();
20
21         System.Console.WriteLine("Consumidor cadastrado com id: " + c.Id);
22         System.Console.WriteLine("Pedido cadastrado com id: " + p.Id);
```

```
23        }
24      }
25    }
26  }
```

Código C# 3.68: www.k19.com.br/exercicios/k32/03/022/AdicionaPedidoConsumidor.cs

Selecione a classe **AdicionaPedidoConsumidor** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as tabelas Pedidoes e Consumidors foram criadas com sucesso.



Many to Many

Suponha que em nosso domínio existam as entidades Livro e Autor. As classes com as anotações básicas de mapeamento seriam mais ou menos assim:

```
1 class Livro
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.69: Livro.cs

```
1 class Autor
2 {
3   public long Id { get; set; }
4 }
```

Código C# 3.70: Autor.cs

Como existe um relacionamento entre livros e autores, devemos expressar esse vínculo através de uma propriedade que pode ser inserida na classe `Livro`. Supondo que um livro possa ser escrito por muitos autores, devemos utilizar uma coleção para expressar esse relacionamento.

```
1 class Livro
2 {
3   public long Id { get; set; }
4
5   public ICollection<Autor> Autores { get; set; }
6 }
```

Código C# 3.71: Livro.cs

Para o Entity Framework assumir que o relacionamento é um "Muitos para Muitos", podemos adicionar uma propriedade na classe `Autor`. Essa propriedade será automaticamente vinculada à propridade na classe `Livro`.

```
1 class Autor
2 {
3   public long Id { get; set; }
4
5   public ICollection<Livro> Livros { get; set; }
6 }
```

Código C# 3.72: Autor.cs

Nesse caso, uma tabela para relacionar os registros das tabelas relacionadas às classes `Livro` e `Autor` será criada. Nessa tabela, haverá uma chave estrangeira vinculada à chave primária da tabela correspondente à classe `Livro` e outra vinculada à chave primária da tabela correspondente à classe `Autor`.

Podemos alterar o nome padrão da tabela de relacionamento reescrevendo o método `OnModelCreating()` no `DbContext`.

```
class EFContext : DbContext
{
  public DbSet<Autor> Autores { get; set; }
  public DbSet<Livro> Livros { get; set; }

  protected override void OnModelCreating(DbModelBuilder modelBuilder)
  {
      modelBuilder.Entity<Autor>()
          .HasMany(autor => autor.Livros)
          .WithMany(livro => livro.Autores)
        .Map(x =>
        {
            x.ToTable("livros_e_autores");
            x.MapLeftKey("autor_id");
            x.MapRightKey("livro_id");
        });
  }
}
```

Código C# 3.73: K19Context.cs



Exercícios de Fixação

23 Crie duas entidades no projeto `EF`: `Livro` e `Autor`.

```
using System.Collections.Generic;

namespace EF
{
  class Autor
  {
    public long Id { get; set; }

    public string Nome { get; set; }

    public ICollection<Livro> Livros { get; set; }
  }
}
```

Código C# 3.74: www.k19.com.br/exercicios/k32/03/023/Autor.cs

```
using System.Collections.Generic;

namespace EF
{
  class Livro
  {
    public long Id { get; set; }

    public string Nome { get; set; }

    public ICollection<Autor> Autores { get; set; }
```

```
12    }
13  }
```

Código C# 3.75: www.k19.com.br/exercicios/k32/03/023/Livro.cs

24 Altere a classe `EFContext`.

```
1  using System.Data.Entity;
2
3  namespace EF
4  {
5    class EFContext : DbContext
6    {
7      public DbSet<Editora> Editoras { get; set; }
8      public DbSet<Cliente> Clientes { get; set; }
9      public DbSet<Estado> Estados { get; set; }
10     public DbSet<Governador> Governadores { get; set; }
11     public DbSet<Funcionario> Funcionarios { get; set; }
12     public DbSet<Departamento> Departamentos { get; set; }
13     public DbSet<Consumidor> Consumidores { get; set; }
14     public DbSet<Pedido> Pedidos { get; set; }
15     public DbSet<Autor> Autores { get; set; }
16     public DbSet<Livro> Livros { get; set; }
17
18     public EFContext()
19     {
20       DropCreateDatabaseIfModelChanges<EFContext> initializer =
21         new DropCreateDatabaseIfModelChanges<EFContext>();
22       Database.SetInitializer<EFContext>(initializer);
23     }
24   }
25 }
```

Código C# 3.76: www.k19.com.br/exercicios/k32/03/024/EFContext.cs

25 Adicione um livro e um autor no banco de dados. Crie uma classe chamada `AdicionaLivroAutor` no projeto `EF`.

```
1  using System.Collections.Generic;
2
3  namespace EF
4  {
5    class AdicionaLivroAutor
6    {
7      static void Main(string[] args)
8      {
9        using(EFContext ctx = new EFContext())
10       {
11         Autor autor = new Autor();
12         System.Console.WriteLine("Digite o nome do autor");
13         autor.Nome = System.Console.ReadLine();
14
15         Livro livro = new Livro();
16         System.Console.WriteLine("Digite o nome do livro");
17         livro.Nome = System.Console.ReadLine();
18
19         autor.Livros = new List<Livro>();
20         livro.Autores = new List<Autor>();
21
22         autor.Livros.Add(livro);
23         livro.Autores.Add(autor);
24
25         ctx.Livros.Add(livro);
26         ctx.Autores.Add(autor);
```

```
27
28          ctx.SaveChanges();
29
30          System.Console.WriteLine("Autor cadastrado com id: " + autor.Id);
31          System.Console.WriteLine("Livro cadastrado com id: " + livro.Id);
32        }
33      }
34    }
35  }
```

Código C# 3.77: www.k19.com.br/exercicios/k32/03/025/AdicionaLivroAutor.cs

Selecione a classe **AdicionaLivroAutor** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as tabelas Autors, Livroes e Livro_Autors foram criadas com sucesso.



Relacionamentos Bidirecionais

Quando expressamos um relacionamento colocando uma propriedade em uma das entidades, podemos acessar a outra entidade a partir da primeira. Por exemplo, considere o relacionamento entre governadores e estados.

```
1  namespace EF
2  {
3    class Estado
4    {
5      public long Id { get; set; }
6
7      public string Nome { get; set; }
8
9      public Governador Governador { get; set; }
10   }
11 }
```

Código C# 3.78: Estado.cs

Como o relacionamento está definido na classe `Estado`, podemos acessar o governador a partir de um estado.

```
1  Estado e = ctx.Estados.Find(1);
2  Governador g = e.Governador;
```

Código C# 3.79: Acessando o governador a partir de um estado

Também podemos expressar o relacionamento na classe `Governador`. Dessa forma, poderíamos acessar um estado a partir de um governador.

```
1  namespace EF
2  {
3    class Governador
4    {
5      public long Id { get; set; }
6
7      public string Nome { get; set; }
8
9      public Estado Estado { get; set; }
10   }
11 }
```

Código C# 3.80: Governador.cs

```
1 Governador g = ctx.Governadores(1);
2 Estado e = g.Estado;
```

Código C# 3.81: Acessando um estado a partir de um governador



Mais Sobre

Considere as seguintes entidades.

```
1  class Pessoa
2  {
3    public int PessoaID { get; set; }
4
5    public string Nome { get; set; }
6
7    public ICollection<Livro> LivrosPublicados { get; set; }
8
9    public ICollection<Livro> LivrosRevisados { get; set; }
10 }
```

Código C# 3.82: Pessoa.cs

```
1  class Livro
2  {
3    public int LivroID { get; set; }
4
5    public string Titulo { get; set; }
6
7    public Pessoa Autor { get; set; }
8
9    public Pessoa Revisor { get; set; }
10 }
```

Código C# 3.83: Livro.cs

Nesse caso, como o Entity Framework não consegue combinar as propriedades das duas entidades para estabelecer dois relacionamentos bidirecionais, ele criará no banco de dados quatro chaves estrangeiras, uma para cada propriedade de navegação das entidades `Pessoa` e `Livro`.

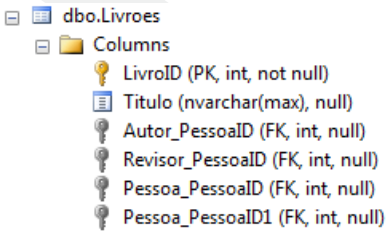


Figura 3.15: Relacionamentos não combinados

Para agrupar as propriedades de navegação das entidades `Pessoa` e `Livro`, devemos utilizar a anotação **InverseProperty**. Observe o código a seguir.

```
1 class Pessoa
2 {
3   public int PessoaID { get; set; }
```

```
4
5     public string Nome { get; set; }
6
7     [InverseProperty("Autor")]
8     public ICollection<Livro> LivrosPublicados { get; set; }
9
10    [InverseProperty("Revisor")]
11    public ICollection<Livro> LivrosRevisados { get; set; }
12  }
```

Código C# 3.84: Pessoa.cs

```
1  class Livro
2  {
3    public int LivroID { get; set; }
4
5    public string Titulo { get; set; }
6
7    public Pessoa Autor { get; set; }
8
9    public Pessoa Revisor { get; set; }
10 }
```

Código C# 3.85: Livro.cs

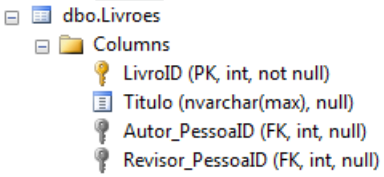


Figura 3.16: Relacionamentos combinados com `InverseProperty`



Exercícios de Fixação

26 Considere um sistema de cobrança de ligações telefônicas. Nesse sistema, temos uma entidade chamada Ligação e uma entidade chamada Fatura. Cada ligação está associada a uma única fatura, enquanto que uma fatura está associada a múltiplas ligações. Implemente classes para modelar essas duas entidades no projeto EF.

```
1  namespace EF
2  {
3    class Ligacao
4    {
5      public long Id { get; set; }
6
7      public int Duracao { get; set; }
8
9      public Fatura Fatura { get; set; }
10   }
11 }
```

Código C# 3.86: www.k19.com.br/exercicios/k32/03/026/Ligacao.cs

```
using System.Collections.Generic;

namespace EF
{
  class Fatura
  {
    public long Id { get; set; }

    public ICollection<Ligacao> Ligacoes { get; set; }
  }
}
```

Código C# 3.87: www.k19.com.br/exercicios/k32/03/026/Fatura.cs

27 Altere a classe `EFContext`.

```
using System.Data.Entity;

namespace EF
{
  class EFContext : DbContext
  {
    public DbSet<Editora> Editoras { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Estado> Estados { get; set; }
    public DbSet<Governador> Governadores { get; set; }
    public DbSet<Funcionario> Funcionarios { get; set; }
    public DbSet<Departamento> Departamentos { get; set; }
    public DbSet<Consumidor> Consumidores { get; set; }
    public DbSet<Pedido> Pedidos { get; set; }
    public DbSet<Autor> Autores { get; set; }
    public DbSet<Livro> Livros { get; set; }
    public DbSet<Ligacao> Ligacoes { get; set; }
    public DbSet<Fatura> Faturas { get; set; }

    public EFContext()
    {
      DropCreateDatabaseIfModelChanges<EFContext> initializer =
        new DropCreateDatabaseIfModelChanges<EFContext>();
      Database.SetInitializer<EFContext>(initializer);
    }
  }
}
```

Código C# 3.88: www.k19.com.br/exercicios/k32/03/027/EFContext.cs

28 Faça um teste para adicionar algumas ligações e uma fatura. Adicione no projeto `EF` uma classe chamada `AdicionaFaturaLigacao`.

```
using System.Collections.Generic;

namespace EF
{
  class AdicionaFaturaLigacao
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        Ligacao ligacao1 = new Ligacao();
        System.Console.WriteLine("Digite o tempo de duração da primeira ligação: ");
        ligacao1.Duracao = System.Convert.ToInt32(System.Console.ReadLine());

```

```
15          Ligacao ligacao2 = new Ligacao();
16          System.Console.WriteLine("Digite o tempo de duração da segunda ligação: ");
17          ligacao2.Duracao = System.Convert.ToInt32(System.Console.ReadLine());
18
19          Fatura fatura = new Fatura();
20          fatura.Ligacoes = new List<Ligacao>();
21
22          ligacao1.Fatura = fatura;
23          ligacao2.Fatura = fatura;
24
25          fatura.Ligacoes.Add(ligacao1);
26          fatura.Ligacoes.Add(ligacao2);
27
28          ctx.Ligacoes.Add(ligacao1);
29          ctx.Ligacoes.Add(ligacao2);
30          ctx.Faturas.Add(fatura);
31
32          ctx.SaveChanges();
33
34          System.Console.WriteLine(
35            "Primeira ligação cadastrada com id: " + ligacao1.Id);
36          System.Console.WriteLine(
37            "Segunda ligação cadastrada com id: " + ligacao2.Id);
38          System.Console.WriteLine(
39            "Fatura cadastrada com id: " + fatura.Id);
40        }
41      }
42    }
43  }
```

Código C# 3.89: www.k19.com.br/exercicios/k32/03/028/AdicionaFaturaLigacao.cs

Selecione a classe **AdicionaFaturaLigacao** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as tabelas Ligacaoes e Faturas foram criadas com sucesso.



Consultas

O Entity Framework permite que as aplicações criem consultas utilizando a linguagem integrada **LINQ**. Essas consultas são traduzidas para a linguagem SQL do banco de dados que estiver sendo utilizado. Dessa forma, podemos definir uma consulta uma única vez em LINQ é utilizá-la em vários bancos de dados diferentes.

Recuperando a quantidade de registros

```
1 int resultado =
2   ctx.Editoras
3     .Count();
```

Código C# 3.90: Quantidade de editoras

```
1 int resultado =
2   ctx.Livros
3     .Count(x => x.Preco >= 100);
```

Código C# 3.91: Quantidade de livros com preço maior ou igual a 100

Somando valores

```
1 double resultado =
```

```
2   ctx.Livros
3     .Sum(x => x.Preco);
```

Código C# 3.92: Soma dos preços dos livros

```
1 double resultado =
2   ctx.Livros
3     .Where(x => x.Preco >= 100)
4     .Sum(x => x.Preco);
```

Código C# 3.93: Soma dos preços dos livros com preço maior ou igual a 100

Recuperando valores mínimos ou máximos

```
1 double resultado =
2   ctx.Livros
3     .Min(x => x.Preco);
```

Código C# 3.94: O preço do livro mais barato

```
1 double resultado =
2   ctx.Livros
3     .Where(x => x.Preco >= 100)
4     .Min(x => x.Preco);
```

Código C# 3.95: O menor preço de um livro que seja maior ou igual a 100

```
1 double resultado =
2   ctx.Livros
3     .Max(x => x.Preco);
```

Código C# 3.96: O preço do livro mais caro

```
1 double resultado =
2   ctx.Livros
3     .Where(x => x.Preco <= 100)
4     .Max(x => x.Preco);
```

Código C# 3.97: O maior preço de um livro que seja menor ou igual a 100

Recuperando médias

```
1 double resultado =
2   ctx.Livros
3     .Average(x => x.Preco);
```

Código C# 3.98: O preço médio dos livros

```
1 double resultado =
2   ctx.Livros
3     .Where(x => x.Preco >= 100)
4     .Average(x => x.Preco);
```

Código C# 3.99: O preço médio dos livros com preço maior ou igual a 100

Recuperando listas

```
1 var resultado = ctx.Livros;
```

Código C# 3.100: Lista de livros

```
1 var resultado =
2   ctx.Livros
3     .Where(x => x.Preco >= 100);
```

Código C# 3.101: Lista do livros com preço maior ou igual a 100

```
1 var resultado =
2   ctx.Livros
3     .Select(x => x.Nome);
```

Código C# 3.102: Lista dos nomes dos livros

```
1 var resultado =
2   ctx.Livros
3     .Where(x => X.Preco >= 100)
4     .Select(x => x.Nome);
```

Código C# 3.103: Lista dos nomes dos livros com preço maior ou igual a 100

Ordenando resultados

```
1 var resultado =
2   ctx.Livros
3     .OrderBy(x => x.Nome);
```

Código C# 3.104: Lista dos livros ordenados pelo nome

```
1 var resultado =
2   ctx.Livros
3     .OrderBy(x => x.Nome)
4     .Select(x => x.Nome);
```

Código C# 3.105: Lista ordenada dos nomes dos livros

Limitando resultados

```
1 var resultado =
2   ctx.Livros
3     .OrderBy(x => x.Nome)
4     .Take(10);
```

Código C# 3.106: Lista dos 10 primeiros livros ordenados pelo nome

```
1 var resultado =
2   ctx.Livros
3     .OrderBy(x => x.Nome)
4     .Skip(2);
```

Código C# 3.107: Lista dos livros ordenados pelo nome a partir do terceiro



Exercícios de Fixação

29 Crie uma classe chamada **Revista** no projeto **EF**.

```
namespace EF
{
  class Revista
  {
    public long Id { get; set; }

    public string Nome { get; set; }

    public double Preco { get; set; }
  }
}
```

Código C# 3.108: www.k19.com.br/exercicios/k32/03/029/Revista.cs

30 Altere a classe `EFContext`.

```
using System.Data.Entity;

namespace EF
{
  class EFContext : DbContext
  {
    public DbSet<Editora> Editoras { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Estado> Estados { get; set; }
    public DbSet<Governador> Governadores { get; set; }
    public DbSet<Funcionario> Funcionarios { get; set; }
    public DbSet<Departamento> Departamentos { get; set; }
    public DbSet<Consumidor> Consumidores { get; set; }
    public DbSet<Pedido> Pedidos { get; set; }
    public DbSet<Autor> Autores { get; set; }
    public DbSet<Livro> Livros { get; set; }
    public DbSet<Ligacao> Ligacoes { get; set; }
    public DbSet<Fatura> Faturas { get; set; }
    public DbSet<Revista> Revistas { get; set; }

    public EFContext()
    {
      DropCreateDatabaseIfModelChanges<EFContext> initializer =
        new DropCreateDatabaseIfModelChanges<EFContext>();
      Database.SetInitializer<EFContext>(initializer);
    }
  }
}
```

Código C# 3.109: www.k19.com.br/exercicios/k32/03/030/EFContext.cs

31 Adicione diversas revistas. Crie uma classe chamada `AdicionaRevista` no projeto `EF`.

```
namespace EF
{
  class AdicionaRevista
  {
    static void Main(string[] args)
    {
      using(EFContext ctx = new EFContext())
      {
        for(int i = 0; i < 15; i++) {
          Revista revista = new Revista();
          revista.Nome = "Revista " + (i + 1);
          revista.Preco = 10.0 * (i + 1);

```

```
14            ctx.Revistas.Add(revista);
15          }
16
17          ctx.SaveChanges();
18        }
19      }
20    }
21  }
```

Código C# 3.110: www.k19.com.br/exercicios/k32/03/031/AdicionaRevista.cs

Selecione a classe **AdicionaRevista** como Startup Object do projeto **EF**. Execute esse projeto e verifique se as revistas foram adicionadas no banco de dados.

32 Realize diversas consultas. Crie uma classe chamada `Consultas` no projeto `EF`.

```
1  using System.Linq;
2
3  namespace EF
4  {
5    class Consultas
6    {
7      static void Main(string[] args)
8      {
9        using(EFContext ctx = new EFContext())
10       {
11         /* Recuperando a quantidade de revistas */
12         int contador = ctx.Revistas.Count();
13
14         System.Console.WriteLine("Há " + contador + " revistas.\n");
15
16         /* Recuperando a quantidade de revistas com preço acima de 100 */
17         contador = ctx.Revistas.Count(x => x.Preco > 100);
18
19         System.Console.WriteLine(
20           "Há " + contador + " revistas com preço acima de 100.\n");
21
22         /* Somando os preços das revistas */
23         double soma = ctx.Revistas.Sum(x => x.Preco);
24
25         System.Console.WriteLine(
26           "A soma dos preços das revistas é " + soma + "\n");
27
28         /* Recuperando o preço da revista mais cara */
29         double precoMaximo = ctx.Revistas.Max(x => x.Preco);
30
31         System.Console.WriteLine(
32           "O preço da revista mais cara é " + precoMaximo + "\n");
33
34         /* Recuperando a média dos preços das revistas */
35         double media = ctx.Revistas.Average(x => x.Preco);
36
37         System.Console.WriteLine(
38           "A média dos preços das revistas é " + media + "\n");
39
40         /* Recuperando todas as revistas */
41         var revistas = ctx.Revistas;
42
43         System.Console.WriteLine("Todas as revistas: ");
44
45         foreach(Revista revista in revistas)
46         {
47           System.Console.WriteLine("Id: " + revista.Id);
48           System.Console.WriteLine("Nome: " + revista.Nome);
49           System.Console.WriteLine("Preço: " + revista.Preco);
50           System.Console.WriteLine();
```

```
51          }
52
53          /* Recuperando as revistas com preço acima de 100 */
54          var revistasCaras = ctx.Revistas.Where(x => x.Preco > 100);
55
56          System.Console.WriteLine("Revistas com preço acima de 100: ");
57
58          foreach(Revista revista in revistasCaras)
59          {
60            System.Console.WriteLine("Id: " + revista.Id);
61            System.Console.WriteLine("Nome: " + revista.Nome);
62            System.Console.WriteLine("Preço: " + revista.Preco);
63            System.Console.WriteLine();
64          }
65        }
66      }
67    }
68  }
```

Código C# 3.111: www.k19.com.br/exercicios/k32/03/032/Consultas.cs

Selecione a classe **Consultas** como Startup Object do projeto **EF**. Execute esse projeto e verifique as informações exibidas na saída padrão.



Sistema de Controle de Finanças Pessoais

Para praticar os conhecimentos obtidos nesse capítulo, vamos implementar um pequeno sistema de controle de finanças pessoais.



Exercícios de Fixação

33 Crie duas entidades no projeto `EF`: `Despesa` e `Receita`.

```
1  using System;
2
3  namespace EF
4  {
5    class Despesa
6    {
7      public long Id { get; set; }
8
9      public string Nome { get; set; }
10
11     public double Valor { get; set; }
12
13     public DateTime Data { get; set; }
14
15     public string Tipo { get; set; }
16
17     public override string ToString()
18     {
19       string s = this.Nome + " - ";
20       s += this.Valor + " - ";
21       s += this.Tipo + " - ";
22       s += this.Data.ToString("dd/MM/yyyy");
23
24       return s;
25     }
```

```
26    }
27  }
```

Código C# 3.112: www.k19.com.br/exercicios/k32/03/033/Despesa.cs

```
1  using System;
2
3  namespace EF
4  {
5    class Receita
6    {
7      public long Id { get; set; }
8
9      public string Nome { get; set; }
10
11     public double Valor { get; set; }
12
13     public DateTime Data { get; set; }
14
15     public string Tipo { get; set; }
16
17     public override string ToString()
18     {
19       string s = this.Nome + " - ";
20       s += this.Valor + " - ";
21       s += this.Tipo + " - ";
22       s += this.Data.ToString("dd/MM/yyyy");
23
24       return s;
25     }
26   }
27 }
```

Código C# 3.113: www.k19.com.br/exercicios/k32/03/033/Receita.cs

34 Crie um contexto no projeto EF: `GranaContext`.

```
1  using System.Data.Entity;
2
3  namespace EF
4  {
5    class GranaContext : DbContext
6    {
7      public DbSet<Despesa> Despesas { get; set; }
8      public DbSet<Receita> Receitas { get; set; }
9
10     public GranaContext()
11     {
12       DropCreateDatabaseIfModelChanges<GranaContext> initializer =
13         new DropCreateDatabaseIfModelChanges<GranaContext>();
14       Database.SetInitializer<GranaContext>(initializer);
15     }
16   }
17 }
```

Código C# 3.114: www.k19.com.br/exercicios/k32/03/034/GranaContext.cs

35 Crie um repositório no projeto EF chamado `DespesaRepositorio`.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4
```

```
5  namespace EF
6  {
7    class DespesaRepositorio
8    {
9
10     private GranaContext ctx;
11
12     public DespesaRepositorio(GranaContext ctx)
13     {
14       this.ctx = ctx;
15     }
16
17     public void Adiciona(Despesa despesa)
18     {
19       ctx.Despesas.Add(despesa);
20     }
21
22     public void Remove(Despesa despesa)
23     {
24       ctx.Despesas.Remove(despesa);
25     }
26
27     public double? SomaDespesasAte(DateTime data)
28     {
29       return ctx.Despesas
30         .Where(x => x.Data <= data)
31         .Sum(x => x.Valor);
32     }
33
34     public double? SomaDespesas(
35       DateTime dataInicial,
36       DateTime dataFinal)
37     {
38       return ctx.Despesas
39         .Where(x => x.Data >= dataInicial
40           && x.Data <= dataFinal)
41         .Sum(x => x.Valor);
42     }
43
44     public List<Despesa> BuscaPorPeriodo(
45       DateTime dataInicial,
46       DateTime dataFinal)
47     {
48       return ctx.Despesas
49         .Where(x => x.Data >= dataInicial
50           && x.Data <= dataFinal)
51         .ToList();
52     }
53
54     public List<Despesa> BuscaRecentes()
55     {
56       return ctx.Despesas
57         .OrderByDescending(x => x.Id)
58         .Take(10)
59         .ToList();
60     }
61   }
62 }
```

Código C# 3.115: www.k19.com.br/exercicios/k32/03/035/DespesaRepositorio.cs

36 Crie um repositório no projeto `EF` chamado `ReceitaRepositorio`.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4
```

```
5  namespace EF
6  {
7    class ReceitaRepositorio
8    {
9
10     private GranaContext ctx;
11
12     public ReceitaRepositorio(GranaContext ctx)
13     {
14       this.ctx = ctx;
15     }
16
17     public void Adiciona(Receita receita)
18     {
19       ctx.Receitas.Add(receita);
20     }
21
22     public void Remove(Receita receita)
23     {
24       ctx.Receitas.Remove(receita);
25     }
26
27     public double? SomaReceitasAte(DateTime data)
28     {
29       return ctx.Receitas
30         .Where(x => x.Data <= data)
31         .Sum(x => x.Valor);
32     }
33
34     public double? SomaReceitas(
35       DateTime dataInicial,
36       DateTime dataFinal)
37     {
38       return ctx.Receitas
39         .Where(x => x.Data >= dataInicial
40           && x.Data <= dataFinal)
41         .Sum(x => x.Valor);
42     }
43
44     public List<Receita> BuscaPorPeriodo(
45       DateTime dataInicial,
46       DateTime dataFinal)
47     {
48       return ctx.Receitas
49         .Where(x => x.Data >= dataInicial
50           && x.Data <= dataFinal)
51         .ToList();
52     }
53
54     public List<Receita> BuscaRecentes()
55     {
56       return ctx.Receitas
57         .OrderByDescending(x => x.Id)
58         .Take(10)
59         .ToList();
60     }
61   }
62 }
```

Código C# 3.116: www.k19.com.br/exercicios/k32/03/036/ReceitaRepositorio.cs

37 Crie uma interface chamada `Tela` no projeto `EF`.

```
1  namespace EF
2  {
3    interface Tela
4    {
```

```
5       Tela Mostra();
6
7       string Nome { get; }
8     }
9   }
```

Código C# 3.117: www.k19.com.br/exercicios/k32/03/037/Tela.cs

38 Crie uma classe chamada `TelaMenu` no projto `EF`.

```
1   using System.Collections.Generic;
2
3   namespace EF
4   {
5     class TelaMenu : Tela
6     {
7       public string Nome { get; set; }
8
9       private List<Tela> filhas = new List<Tela>();
10
11      public TelaMenu(string nome)
12      {
13        this.Nome = nome;
14      }
15
16      public Tela Mostra()
17      {
18        System.Console.WriteLine(">>> " + this.Nome + " <<<");
19        System.Console.WriteLine();
20
21        for (int i = 0; i < this.filhas.Count; i++)
22        {
23          System.Console.WriteLine((i + 1) + ". " + this.filhas[i].Nome);
24        }
25
26        System.Console.WriteLine();
27        System.Console.Write("Escolha a opção: ");
28        int indiceDaOpcao =
29          System.Convert.ToInt32(System.Console.ReadLine()) - 1;
30
31        System.Console.WriteLine();
32
33        return this.filhas[indiceDaOpcao];
34      }
35
36      public void AdicionaFilha(Tela tela)
37      {
38        this.filhas.Add(tela);
39      }
40    }
41  }
```

Código C# 3.118: www.k19.com.br/exercicios/k32/03/038/TelaMenu.cs

39 Crie uma classe chamada `TelaGeraDados` no projeto `EF`.

```
1   using System;
2
3   namespace EF
4   {
5     class TelaGeraDados : Tela
6     {
7       private Tela anterior;
8
```

```
9     public string Nome { get; set; }
10
11    public TelaGeraDados(Tela anterior)
12    {
13      this.anterior = anterior;
14      this.Nome = "Gera Dados";
15    }
16
17    public Tela Mostra()
18    {
19      System.Console.WriteLine(">>> " + this.Nome + " <<<");
20      System.Console.WriteLine();
21
22      System.Console.WriteLine("Gerando dados...");
23
24      using(GranaContext ctx = new GranaContext())
25      {
26        ReceitaRepositorio receitaRepositorio =
27          new ReceitaRepositorio(ctx);
28
29        DespesaRepositorio despesaRepositorio =
30          new DespesaRepositorio(ctx);
31
32        /* Receitas */
33        for (int i = 0; i < 36; i++)
34        {
35          Receita receita = new Receita();
36          receita.Nome = "Salário K19";
37          receita.Tipo = "Salário";
38          receita.Valor = 8000.0;
39
40          DateTime data = new DateTime(2014, 1, 10);
41          data = data.AddMonths(i);
42          receita.Data = data;
43
44          receitaRepositorio.Adiciona(receita);
45        }
46
47        for (int i = 0; i < 36; i++) {
48          Receita receita = new Receita();
49          receita.Nome = "VA";
50          receita.Tipo = "Vale Alimentação";
51          receita.Valor = 400.00;
52
53          DateTime data = new DateTime(2014, 1, 10);
54          data = data.AddMonths(i);
55          receita.Data = data;
56
57          receitaRepositorio.Adiciona(receita);
58        }
59
60        for (int i = 0; i < 36; i++) {
61          Receita receita = new Receita();
62          receita.Nome = "VR";
63          receita.Tipo = "Vale Refeição";
64          receita.Valor = 500.0;
65
66          DateTime data = new DateTime(2014, 1, 10);
67          data = data.AddMonths(i);
68          receita.Data = data;
69
70          receitaRepositorio.Adiciona(receita);
71        }
72
73        for (int i = 0; i < 36; i++) {
74          Receita receita = new Receita();
75          receita.Nome = "Aluguel Casa BH";
76          receita.Tipo = "Aluguel";
77          receita.Valor = 1000.0;
78
```

```
79            DateTime data = new DateTime(2014, 1, 10);
80            data = data.AddMonths(i);
81            receita.Data = data;
82
83            receitaRepositorio.Adiciona(receita);
84          }
85
86          /* Despesas */
87          for (int i = 0; i < 36; i++) {
88            Despesa despesa = new Despesa();
89            despesa.Nome = "Aluguel Apto SP";
90            despesa.Tipo = "Aluguel";
91            despesa.Valor = 1800.0;
92
93            DateTime data = new DateTime(2014, 1, 10);
94            data = data.AddMonths(i);
95            despesa.Data = data;
96
97            despesaRepositorio.Adiciona(despesa);
98          }
99
100         for (int i = 0; i < 36; i++) {
101           Despesa despesa = new Despesa();
102           despesa.Nome = "Compras Supermercado";
103           despesa.Tipo = "Alimentação";
104           despesa.Valor = 1000.0;
105
106           DateTime data = new DateTime(2014, 1, 10);
107           data = data.AddMonths(i);
108           despesa.Data = data;
109
110           despesaRepositorio.Adiciona(despesa);
111         }
112
113         for (int i = 0; i < 36; i++) {
114           Despesa despesa = new Despesa();
115           despesa.Nome = "Combustível";
116           despesa.Tipo = "Transporte";
117           despesa.Valor = 400.00;
118
119           DateTime data = new DateTime(2014, 1, 10);
120           data = data.AddMonths(i);
121           despesa.Data = data;
122
123           despesaRepositorio.Adiciona(despesa);
124         }
125
126         for (int i = 0; i < 36; i++) {
127           Despesa despesa = new Despesa();
128           despesa.Nome = "Cinema";
129           despesa.Tipo = "Lazer";
130           despesa.Valor = 200.0;
131
132           DateTime data = new DateTime(2014, 1, 10);
133           data = data.AddMonths(i);
134           despesa.Data = data;
135
136           despesaRepositorio.Adiciona(despesa);
137         }
138
139         ctx.SaveChanges();
140       }
141
142       System.Console.WriteLine("Dados gerados");
143       System.Console.WriteLine();
144
145       return this.anterior;
146     }
147   }
148 }
```

Código C# 3.119: www.k19.com.br/exercicios/k32/03/039/TelaGeraDados.cs

40 Crie uma classe chamada `TelaLimpaDados` no projeto `EF`.

```
namespace EF
{
  class TelaLimpaDados : Tela
  {
    private Tela anterior;

    public string Nome { get; set; }

    public TelaLimpaDados(Tela anterior)
    {
      this.anterior = anterior;
      this.Nome = "Limpa Dados";
    }

    public Tela Mostra()
    {
      System.Console.WriteLine(">>> " + this.Nome + " <<<");
      System.Console.WriteLine();

      System.Console.WriteLine("Limpando dados...");

      using(GranaContext ctx = new GranaContext())
      {
        ctx.Database.ExecuteSqlCommand("DELETE FROM Despesas");
        ctx.Database.ExecuteSqlCommand("DELETE FROM Receitas");

        ctx.SaveChanges();
      }

      System.Console.WriteLine("Dados limpos");
      System.Console.WriteLine();

      return this.anterior;
    }
  }
}
```

Código C# 3.120: www.k19.com.br/exercicios/k32/03/040/TelaLimpaDados.cs

41 Crie uma classe chamada `TelaSair` no projeto `EF`.

```
namespace EF
{
  class TelaSair : Tela
  {
    public string Nome { get; set; }

    public TelaSair()
    {
      this.Nome = "Sair";
    }

    public Tela Mostra()
    {
      return null;
    }
  }
}
```

Código C# 3.121: www.k19.com.br/exercicios/k32/03/041/TelaSair.cs

42 Crie uma classe chamada `TelaDespesaAdicionar` no projeto EF.

```
using System;
using System.Collections.Generic;
using System.Globalization;

namespace EF
{
  class TelaDespesaAdicionar : Tela
  {
    private Tela anterior;

    public string Nome { get; set; }

    public TelaDespesaAdicionar(Tela anterior)
    {
      this.anterior = anterior;
      this.Nome = "Adicionar Despesa";
    }

    public Tela Mostra()
    {
      System.Console.WriteLine(">>> " + this.Nome + " <<<");
      System.Console.WriteLine();

      System.Console.Write("Digite o nome: ");
      string nome = System.Console.ReadLine();

      double valor = -1;

      while (valor < 0)
      {
        System.Console.Write("Digite o valor: ");

        try
        {
          valor = System.Convert.ToDouble(System.Console.ReadLine());
        }
        catch (FormatException e)
        {
          System.Console.WriteLine("Valor incorreto");
          continue;
        }

        if (valor < 0)
        {
          System.Console.WriteLine("Valor incorreto");
        }
      }

      DateTime? data = null;

      while (data == null)
      {
        System.Console.Write("Digite a data (ex: 05/08/2016): ");

        try
        {
          CultureInfo cf = new CultureInfo("pt-BR");
          data = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
        }
        catch (FormatException e)
        {
```

```
62            System.Console.WriteLine("Data incorreta");
63          }
64        }
65
66        System.Console.WriteLine();
67        Dictionary<int, string> tipos = new Dictionary<int, string>();
68        tipos.Add(1, "Aluguel");
69        tipos.Add(2, "Alimentação");
70        tipos.Add(3, "Transporte");
71        tipos.Add(4, "Lazer");
72        tipos.Add(5, "Outros");
73
74        foreach (KeyValuePair<int, string> kvp in tipos)
75        {
76          System.Console.WriteLine(kvp.Key + ". " + kvp.Value);
77        }
78
79        int tipo = -1;
80
81        while (tipo < 1 || tipo > 5)
82        {
83          System.Console.Write("Escolha o tipo (número de 1 a 5): ");
84
85          try
86          {
87            tipo = System.Convert.ToInt32(System.Console.ReadLine());
88          }
89          catch (FormatException e)
90          {
91            System.Console.WriteLine("Tipo incorreto");
92            continue;
93          }
94
95          if (tipo < 1 || tipo > 5)
96          {
97            System.Console.WriteLine("Tipo incorreto");
98          }
99        }
100
101       using(GranaContext ctx = new GranaContext())
102       {
103         Despesa despesa = new Despesa();
104         despesa.Nome = nome;
105         despesa.Valor = valor;
106         despesa.Tipo = tipos[tipo];
107         despesa.Data = data.GetValueOrDefault();
108
109         DespesaRepositorio despesaRepositorio = new DespesaRepositorio(ctx);
110         despesaRepositorio.Adiciona(despesa);
111
112         ctx.SaveChanges();
113       }
114
115       System.Console.WriteLine("Despesa adicionada");
116
117       return this.anterior;
118     }
119   }
120 }
```

Código C# 3.122: www.k19.com.br/exercicios/k32/03/042/TelaDespesaAdicionar.cs

43 Crie uma classe chamada `TelaDespesaConsultaPeriodo` no projeto `EF`.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Globalization;
```

```
4
5  namespace EF
6  {
7    class TelaDespesaConsultaPeriodo : Tela
8    {
9      private Tela anterior;
10
11     public string Nome { get; set; }
12
13     public TelaDespesaConsultaPeriodo(Tela anterior)
14     {
15       this.anterior = anterior;
16       this.Nome = "Despesas por Período";
17     }
18
19     public Tela Mostra()
20     {
21       System.Console.WriteLine(">>> " + this.Nome + " <<<");
22       System.Console.WriteLine();
23
24       DateTime? dataInicial = null;
25
26       while (dataInicial == null)
27       {
28         System.Console.Write("Digite a data inicial (ex: 05/08/2016): ");
29
30         try
31         {
32           CultureInfo cf = new CultureInfo("pt-BR");
33           dataInicial = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
34         }
35         catch (FormatException e)
36         {
37           System.Console.WriteLine("Data incorreta");
38         }
39       }
40
41       DateTime? dataFinal = null;
42
43       while (dataFinal == null)
44       {
45         System.Console.Write("Digite a data final (ex: 05/08/2016): ");
46
47         try
48         {
49           CultureInfo cf = new CultureInfo("pt-BR");
50           dataFinal = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
51         }
52         catch (FormatException e)
53         {
54           System.Console.WriteLine("Data incorreta");
55         }
56       }
57
58       using(GranaContext ctx = new GranaContext())
59       {
60         DespesaRepositorio despesaRepositorio =
61             new DespesaRepositorio(ctx);
62
63         List<Despesa> despesas =
64           despesaRepositorio.BuscaPorPeriodo(
65             dataInicial.GetValueOrDefault(),
66             dataFinal.GetValueOrDefault());
67
68         int opcao = -1;
69
70         while(opcao < 0 || opcao > despesas.Count)
71         {
72           for (int i = 0; i < despesas.Count; i++)
73           {
```

```
74              System.Console.WriteLine((i + 1) + ". " + despesas[i]);
75            }
76
77            System.Console.WriteLine();
78            System.Console.WriteLine("Digite o número da despesa que deseja remover.");
79            System.Console.WriteLine("Digite 0 para voltar.");
80
81            try
82            {
83              opcao = System.Convert.ToInt32(System.Console.ReadLine());
84            }
85            catch (FormatException e)
86            {
87              System.Console.WriteLine("Opção incorreta");
88              continue;
89            }
90
91            if (opcao < 0 || opcao > 1000)
92            {
93              System.Console.WriteLine("Opção incorreto");
94              continue;
95            }
96
97            if(opcao != 0)
98            {
99              despesaRepositorio.Remove(despesas[opcao - 1]);
100
101             despesas.RemoveAt(opcao - 1);
102
103             opcao = -1;
104
105             ctx.SaveChanges();
106
107             System.Console.WriteLine();
108             System.Console.WriteLine("Despesa Removida");
109             System.Console.WriteLine();
110           }
111         }
112       }
113
114       System.Console.WriteLine();
115
116       return this.anterior;
117     }
118   }
119 }
```

Código C# 3.123: www.k19.com.br/exercicios/k32/03/043/TelaDespesaConsultaPeriodo.cs

44 Crie uma classe chamada `TelaDespesaConsultaRecentes` no projeto `EF`.

```
1  using System;
2  using System.Collections.Generic;
3
4  namespace EF
5  {
6    class TelaDespesaConsultaRecentes : Tela
7    {
8      private Tela anterior;
9
10     public string Nome { get; set; }
11
12     public TelaDespesaConsultaRecentes(Tela anterior)
13     {
14       this.anterior = anterior;
15       this.Nome = "Últimas Despesas Adicionadas";
16     }
```

```
17
18      public Tela Mostra()
19      {
20        System.Console.WriteLine(">>> " + this.Nome + " <<<");
21        System.Console.WriteLine();
22
23        using(GranaContext ctx = new GranaContext())
24        {
25          DespesaRepositorio despesaRepositorio =
26              new DespesaRepositorio(ctx);
27
28          List<Despesa> despesas =
29            despesaRepositorio.BuscaRecentes();
30
31          int opcao = -1;
32
33          while(opcao < 0 || opcao > despesas.Count)
34          {
35            for (int i = 0; i < despesas.Count; i++)
36            {
37              System.Console.WriteLine((i + 1) + ". " + despesas[i]);
38            }
39
40            System.Console.WriteLine();
41            System.Console.WriteLine("Digite o número da despesa que deseja remover.");
42            System.Console.WriteLine("Digite 0 para voltar.");
43
44            try
45            {
46              opcao = System.Convert.ToInt32(System.Console.ReadLine());
47            }
48            catch (FormatException e)
49            {
50              System.Console.WriteLine("Opção incorreta");
51              continue;
52            }
53
54            if (opcao < 0 || opcao > 1000)
55            {
56              System.Console.WriteLine("Opção incorreto");
57              continue;
58            }
59
60            if(opcao != 0)
61            {
62              despesaRepositorio.Remove(despesas[opcao - 1]);
63
64              despesas.RemoveAt(opcao - 1);
65
66              opcao = -1;
67
68              ctx.SaveChanges();
69
70              System.Console.WriteLine();
71              System.Console.WriteLine("Despesa Removida");
72              System.Console.WriteLine();
73            }
74          }
75        }
76
77        System.Console.WriteLine();
78
79        return this.anterior;
80      }
81    }
82  }
```

Código C# 3.124: www.k19.com.br/exercicios/k32/03/044/TelaDespesaConsultaRecentes.cs

45 Crie uma classe chamada `TelaReceitaAdicionar` no projeto `EF`.

```
using System;
usgin System.Collections.Generic;
using System.Globalization;

namespace EF
{
  class TelaReceitaAdicionar : Tela
  {
    private Tela anterior;

    public string Nome { get; set; }

    public TelaReceitaAdicionar(Tela anterior)
    {
      this.anterior = anterior;
      this.Nome = "Adicionar Receita";
    }

    public Tela Mostra()
    {
      System.Console.WriteLine(">>> " + this.Nome + " <<<");
      System.Console.WriteLine();

      System.Console.Write("Digite o nome: ");
      string nome = System.Console.ReadLine();

      double valor = -1;

      while (valor < 0)
      {
        System.Console.Write("Digite o valor: ");

        try
        {
          valor = System.Convert.ToDouble(System.Console.ReadLine());
        }
        catch (FormatException e)
        {
          System.Console.WriteLine("Valor incorreto");
          continue;
        }

        if (valor < 0)
        {
          System.Console.WriteLine("Valor incorreto");
        }
      }

      DateTime? data = null;

      while (data == null)
      {
        System.Console.Write("Digite a data (ex: 05/08/2016): ");

        try
        {
          CultureInfo cf = new CultureInfo("pt-BR");
          data = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
        }
        catch (FormatException e)
        {
          System.Console.WriteLine("Data incorreta");
        }
      }

      System.Console.WriteLine();
      Dictionary<int, string> tipos = new Dictionary<int, string>();
      tipos.Add(1, "Salário");
```

```
69        tipos.Add(2, "Vale Alimentação");
70        tipos.Add(3, "Vale Refeição");
71        tipos.Add(4, "Aluguel");
72        tipos.Add(5, "Outros");
73
74        foreach (KeyValuePair<int, string> kvp in tipos)
75        {
76          System.Console.WriteLine(kvp.Key + ". " + kvp.Value);
77        }
78
79        int tipo = -1;
80
81        while (tipo < 1 || tipo > 5)
82        {
83          System.Console.Write("Escolha o tipo (número de 1 a 5): ");
84
85          try
86          {
87            tipo = System.Convert.ToInt32(System.Console.ReadLine());
88          }
89          catch (FormatException e)
90          {
91            System.Console.WriteLine("Tipo incorreto");
92            continue;
93          }
94
95          if (tipo < 1 || tipo > 5)
96          {
97            System.Console.WriteLine("Tipo incorreto");
98          }
99        }
100
101       using(GranaContext ctx = new GranaContext())
102       {
103         Receita receita = new Receita();
104         receita.Nome = nome;
105         receita.Valor = valor;
106         receita.Tipo = tipos[tipo];
107         receita.Data = data.GetValueOrDefault();
108
109         ReceitaRepositorio receitaRepositorio = new ReceitaRepositorio(ctx);
110         receitaRepositorio.Adiciona(receita);
111
112         ctx.SaveChanges();
113       }
114
115       System.Console.WriteLine("Receita adicionada");
116       System.Console.WriteLine();
117
118       return this.anterior;
119     }
120   }
121 }
```

Código C# 3.125: www.k19.com.br/exercicios/k32/03/045/TelaReceitaAdicionar.cs

46 Crie uma classe chamada `TelaReceitaConsultaPeriodo` no projeto `EF`.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Globalization;
4
5  namespace EF
6  {
7    class TelaReceitaConsultaPeriodo : Tela
8    {
9      private Tela anterior;
```

```
10
11      public string Nome { get; set; }
12
13      public TelaReceitaConsultaPeriodo(Tela anterior)
14      {
15        this.anterior = anterior;
16        this.Nome = "Receitas por Período";
17      }
18
19      public Tela Mostra()
20      {
21        System.Console.WriteLine(">>> " + this.Nome + " <<<");
22        System.Console.WriteLine();
23
24        DateTime? dataInicial = null;
25
26        while (dataInicial == null)
27        {
28          System.Console.Write("Digite a data incial (ex: 05/08/2016): ");
29
30          try
31          {
32            CultureInfo cf = new CultureInfo("pt-BR");
33            dataInicial = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
34          }
35          catch (FormatException e)
36          {
37            System.Console.WriteLine("Data incorreta");
38          }
39        }
40
41        DateTime? dataFinal = null;
42
43        while (dataFinal == null)
44        {
45          System.Console.Write("Digite a data (ex: 05/08/2016): ");
46
47          try
48          {
49            CultureInfo cf = new CultureInfo("pt-BR");
50            dataFinal = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
51          }
52          catch (FormatException e)
53          {
54            System.Console.WriteLine("Data incorreta");
55          }
56        }
57
58        using(GranaContext ctx = new GranaContext())
59        {
60          ReceitaRepositorio receitaRepositorio =
61              new ReceitaRepositorio(ctx);
62
63          List<Receita> receitas =
64            receitaRepositorio.BuscaPorPeriodo(
65              dataInicial.GetValueOrDefault(),
66              dataFinal.GetValueOrDefault());
67
68          int opcao = -1;
69
70          while(opcao < 0 || opcao > receitas.Count)
71          {
72            for (int i = 0; i < receitas.Count; i++)
73            {
74              System.Console.WriteLine((i + 1) + ". " + receitas[i]);
75            }
76
77            System.Console.WriteLine();
78            System.Console.WriteLine("Digite o número da receita que deseja remover.");
79            System.Console.WriteLine("Digite 0 para voltar.");
```

```
80
81            try
82            {
83              opcao = System.Convert.ToInt32(System.Console.ReadLine());
84            }
85            catch (FormatException e)
86            {
87              System.Console.WriteLine("Opção incorreta");
88              continue;
89            }
90
91            if (opcao < 0 || opcao > 1000)
92            {
93              System.Console.WriteLine("Opção incorreto");
94              continue;
95            }
96
97            if(opcao != 0)
98            {
99              receitaRepositorio.Remove(receitas[opcao - 1]);
100
101             receitas.RemoveAt(opcao - 1);
102
103             opcao = -1;
104
105             ctx.SaveChanges();
106
107             System.Console.WriteLine();
108             System.Console.WriteLine("Receita Removida");
109             System.Console.WriteLine();
110           }
111         }
112       }
113       System.Console.WriteLine();
114
115       return this.anterior;
116     }
117   }
118 }
```

Código C# 3.126: www.k19.com.br/exercicios/k32/03/046/TelaReceitaConsultaPeriodo.cs

47 Crie uma classe chamada `TelaReceitaConsultaRecentes` no projeto `EF`.

```
1  using System;
2  using System.Collections.Generic;
3
4  namespace EF
5  {
6    class TelaReceitaConsultaRecentes : Tela
7    {
8      private Tela anterior;
9
10     public string Nome { get; set; }
11
12     public TelaReceitaConsultaRecentes(Tela anterior)
13     {
14       this.anterior = anterior;
15       this.Nome = "Últimas Receitas Adicionadas";
16     }
17
18     public Tela Mostra()
19     {
20       System.Console.WriteLine(">>> " + this.Nome + " <<<");
21       System.Console.WriteLine();
22
23       using(GranaContext ctx = new GranaContext())
```

```
24        {
25          ReceitaRepositorio receitaRepositorio =
26              new ReceitaRepositorio(ctx);
27
28          List<Receita> receitas =
29            receitaRepositorio.BuscaRecentes();
30
31          int opcao = -1;
32
33          while(opcao < 0 || opcao > receitas.Count)
34          {
35            for (int i = 0; i < receitas.Count; i++)
36            {
37              System.Console.WriteLine((i + 1) + ". " + receitas[i]);
38            }
39
40            System.Console.WriteLine();
41            System.Console.WriteLine("Digite o número da receita que deseja remover.");
42            System.Console.WriteLine("Digite 0 para voltar.");
43
44            try
45            {
46              opcao = System.Convert.ToInt32(System.Console.ReadLine());
47            }
48            catch (FormatException e)
49            {
50              System.Console.WriteLine("Opção incorreta");
51              continue;
52            }
53
54            if (opcao < 0 || opcao > 1000)
55            {
56              System.Console.WriteLine("Opção incorreto");
57              continue;
58            }
59
60            if(opcao != 0)
61            {
62              receitaRepositorio.Remove(receitas[opcao - 1]);
63
64              receitas.RemoveAt(opcao - 1);
65
66              opcao = -1;
67
68              ctx.SaveChanges();
69
70              System.Console.WriteLine();
71              System.Console.WriteLine("Receita Removida");
72              System.Console.WriteLine();
73            }
74          }
75        }
76
77        System.Console.WriteLine();
78
79        return this.anterior;
80      }
81    }
82  }
```

Código C# 3.127: www.k19.com.br/exercicios/k32/03/047/TelaReceitaConsultaRecentes.cs

48 Crie uma classe chamada `TelaRelatorioConsolidado` no projeto `EF`.

```
1  using System;
2  using System.Globalization;
3
```

```
4  namespace EF
5  {
6    class TelaRelatorioConsolidado : Tela
7    {
8      private Tela anterior;
9
10     public string Nome { get; set; }
11
12     public TelaRelatorioConsolidado(Tela anterior)
13     {
14       this.anterior = anterior;
15       this.Nome = "Relatório Consolidado";
16     }
17
18     public Tela Mostra()
19     {
20       System.Console.WriteLine(">>> " + this.Nome + " <<<");
21       System.Console.WriteLine();
22
23       DateTime? data = null;
24
25       while (data == null)
26       {
27         System.Console.Write("Digite a data (ex: 05/08/2016): ");
28
29         try
30         {
31           CultureInfo cf = new CultureInfo("pt-BR");
32           data = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
33         }
34         catch (FormatException e)
35         {
36           System.Console.WriteLine("Data incorreta");
37         }
38       }
39
40       using(GranaContext ctx = new GranaContext())
41       {
42         ReceitaRepositorio receitaRepositorio = new ReceitaRepositorio(ctx);
43
44         DespesaRepositorio despesaRepositorio = new DespesaRepositorio(ctx);
45
46         double? receitas =
47           receitaRepositorio.SomaReceitasAte(data.GetValueOrDefault());
48         double? despesas =
49           despesaRepositorio.SomaDespesasAte(data.GetValueOrDefault());
50
51         System.Console.WriteLine("Saldo: " + (receitas - despesas));
52       }
53
54       System.Console.WriteLine();
55
56       return this.anterior;
57     }
58   }
59 }
```

Código C# 3.128: www.k19.com.br/exercicios/k32/03/048/TelaRelatorioConsolidado.cs

49 Crie uma classe chamada `TelaRelatorioMensal` no projeto `EF`.

```
1  using System;
2
3  namespace EF
4  {
5    class TelaRelatorioMensal : Tela
6    {
```

```
7     private Tela anterior;
8
9     public string Nome { get; set; }
10
11    public TelaRelatorioMensal(Tela anterior)
12    {
13      this.anterior = anterior;
14      this.Nome = "Relatório Mensal";
15    }
16
17    public Tela Mostra()
18    {
19      System.Console.WriteLine(">>> " + this.Nome + " <<<");
20      System.Console.WriteLine();
21
22      int mes = -1;
23
24      while (mes < 1 || mes > 12)
25      {
26        System.Console.Write("Digite o mês (número de 1 a 12): ");
27
28        try
29        {
30          mes = System.Convert.ToInt32(System.Console.ReadLine());
31        }
32        catch (FormatException e)
33        {
34          System.Console.WriteLine("Mês incorreto");
35          continue;
36        }
37
38        if (mes < 1 || mes > 12)
39        {
40          System.Console.WriteLine("Mês incorreto");
41        }
42      }
43
44      int ano = -1;
45
46      while (ano < 1900 || ano > 3000)
47      {
48        System.Console.Write("Digite o ano (número de 1900 a 3000): ");
49
50        try
51        {
52          ano = System.Convert.ToInt32(System.Console.ReadLine());
53        }
54        catch (FormatException e)
55        {
56          System.Console.WriteLine("Ano incorreto");
57          continue;
58        }
59
60        if (ano < 1900 || ano > 3000)
61        {
62          System.Console.WriteLine("Ano incorreto");
63        }
64      }
65
66      DateTime dataInicial = new DateTime(ano, mes, 1);
67      DateTime dataFinal = new DateTime(ano, mes, 1);
68      dataFinal = dataFinal.AddMonths(1).AddDays(-1);
69
70      using(GranaContext ctx = new GranaContext())
71      {
72        ReceitaRepositorio receitaRepositorio = new ReceitaRepositorio(ctx);
73
74        DespesaRepositorio despesaRepositorio = new DespesaRepositorio(ctx);
75
76        double? receitas =
```

```
77            receitaRepositorio.SomaReceitas(dataInicial, dataFinal);
78          double? despesas =
79            despesaRepositorio.SomaDespesas(dataInicial, dataFinal);
80
81          System.Console.WriteLine("Receitas: " + receitas);
82          System.Console.WriteLine("Despesas: " + despesas);
83          System.Console.WriteLine("Saldo do Período: " + (receitas - despesas));
84        }
85
86        System.Console.WriteLine();
87
88        return this.anterior;
89      }
90    }
91  }
```

Código C# 3.129: www.k19.com.br/exercicios/k32/03/049/TelaRelatorioMensal.cs

50 Crie uma classe chamada `TelaRelatorioPersonalizado` no projeto `EF`.

```
1  using System;
2  using System.Globalization;
3
4  namespace EF
5  {
6    class TelaRelatorioPersonalizado : Tela
7    {
8      private Tela anterior;
9
10     public string Nome { get; set; }
11
12     public TelaRelatorioPersonalizado(Tela anterior)
13     {
14       this.anterior = anterior;
15       this.Nome = "Relatório Personalizado";
16     }
17
18     public Tela Mostra()
19     {
20       System.Console.WriteLine(">>> " + this.Nome + " <<<");
21       System.Console.WriteLine();
22
23       DateTime? dataInicial = null;
24
25       while (dataInicial == null)
26       {
27         System.Console.Write("Digite a data incial (ex: 05/08/2016): ");
28
29         try
30         {
31           CultureInfo cf = new CultureInfo("pt-BR");
32           dataInicial = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
33         }
34         catch (FormatException e)
35         {
36           System.Console.WriteLine("Data incorreta");
37         }
38       }
39
40       DateTime? dataFinal = null;
41
42       while (dataFinal == null)
43       {
44         System.Console.Write("Digite a data final (ex: 05/08/2016): ");
45
46         try
47         {
```

```
48            CultureInfo cf = new CultureInfo("pt-BR");
49            dataFinal = System.Convert.ToDateTime(System.Console.ReadLine(), cf);
50          }
51          catch (FormatException e)
52          {
53            System.Console.WriteLine("Data incorreta");
54          }
55        }
56
57        using(GranaContext ctx = new GranaContext())
58        {
59          ReceitaRepositorio receitaRepositorio = new ReceitaRepositorio(ctx);
60
61          DespesaRepositorio despesaRepositorio = new DespesaRepositorio(ctx);
62
63          double? receitas =
64            receitaRepositorio.SomaReceitas(
65              dataInicial.GetValueOrDefault(),
66              dataFinal.GetValueOrDefault());
67          double? despesas =
68            despesaRepositorio.SomaDespesas(
69              dataInicial.GetValueOrDefault(),
70              dataFinal.GetValueOrDefault());
71
72          System.Console.WriteLine("Receitas: " + receitas);
73          System.Console.WriteLine("Despesas: " + despesas);
74          System.Console.WriteLine("Saldo do Período: " + (receitas - despesas));
75        }
76
77        System.Console.WriteLine();
78
79        return this.anterior;
80      }
81    }
82  }
```

Código C# 3.130: www.k19.com.br/exercicios/k32/03/050/TelaRelatorioPersonalizado.cs

51 Crie uma classe chamada `Grana` no projeto `EF`.

```
1  namespace EF
2  {
3    class Grana
4    {
5      static void Main(string[] args)
6      {
7        /* Menu Principal */
8        TelaMenu principal =
9          new TelaMenu("Menu Principal");
10
11       TelaMenu relatorios =
12         new TelaMenu("Relatórios");
13       TelaMenu receitas =
14         new TelaMenu("Receitas");
15       TelaMenu despesas =
16         new TelaMenu("Despesas");
17       TelaGeraDados geraDados =
18         new TelaGeraDados(principal);
19       TelaLimpaDados limpaDados =
20         new TelaLimpaDados(principal);
21       TelaSair sair =
22         new TelaSair();
23
24       principal.AdicionaFilha(relatorios);
25       principal.AdicionaFilha(receitas);
26       principal.AdicionaFilha(despesas);
27       principal.AdicionaFilha(geraDados);
```

```
28        principal.AdicionaFilha(limpaDados);
29        principal.AdicionaFilha(sair);
30
31        /* Relatórios */
32        TelaRelatorioConsolidado consolidado =
33          new TelaRelatorioConsolidado(relatorios);
34        TelaRelatorioMensal mensal =
35          new TelaRelatorioMensal(relatorios);
36        TelaRelatorioPersonalizado personalizado =
37          new TelaRelatorioPersonalizado(relatorios);
38
39        relatorios.AdicionaFilha(consolidado);
40        relatorios.AdicionaFilha(mensal);
41        relatorios.AdicionaFilha(personalizado);
42        relatorios.AdicionaFilha(principal);
43
44        /* Receitas */
45        TelaReceitaAdicionar receitaAdicionar =
46          new TelaReceitaAdicionar(receitas);
47        TelaMenu receitaConsultar =
48          new TelaMenu("Consultar");
49
50        receitas.AdicionaFilha(receitaAdicionar);
51        receitas.AdicionaFilha(receitaConsultar);
52        receitas.AdicionaFilha(principal);
53
54        /* Receitas - Consultar */
55        TelaReceitaConsultaPeriodo receitaConsultaPeriodo =
56          new TelaReceitaConsultaPeriodo(receitaConsultar);
57        TelaReceitaConsultaRecentes receitaConsultaRecentes =
58          new TelaReceitaConsultaRecentes(receitaConsultar);
59
60        receitaConsultar.AdicionaFilha(receitaConsultaPeriodo);
61        receitaConsultar.AdicionaFilha(receitaConsultaRecentes);
62        receitaConsultar.AdicionaFilha(receitas);
63
64        /* Despesas */
65        TelaDespesaAdicionar despesaAdicionar =
66          new TelaDespesaAdicionar(despesas);
67        TelaMenu despesaConsultar =
68          new TelaMenu("Consultar");
69
70        despesas.AdicionaFilha(despesaAdicionar);
71        despesas.AdicionaFilha(despesaConsultar);
72        despesas.AdicionaFilha(principal);
73
74        /* Despesas - Consultar */
75        TelaDespesaConsultaPeriodo despesaConsultaPeriodo =
76          new TelaDespesaConsultaPeriodo(despesaConsultar);
77        TelaDespesaConsultaRecentes despesaConsultaRecentes =
78          new TelaDespesaConsultaRecentes(despesaConsultar);
79
80        despesaConsultar.AdicionaFilha(despesaConsultaPeriodo);
81        despesaConsultar.AdicionaFilha(despesaConsultaRecentes);
82        despesaConsultar.AdicionaFilha(despesas);
83
84        Tela atual = principal;
85        while (atual != null)
86        {
87          atual = atual.Mostra();
88          System.Console.Clear();
89        }
90      }
91    }
92  }
```

Código C# 3.131: www.k19.com.br/exercicios/k32/03/051/Grana.cs

Execute a classe `Grana` e teste todas as funcionalidades.



Desafios

1 Acrescente no sistema de controle de finanças pessoais um relatório que informe a soma dos valores das receitas de um determinado tipo e outro que informe a soma dos valores das despesas de um determinado tipo.

VISÃO GERAL DO ASP.NET MVC

CAPÍTULO 4



Necessidades de uma aplicação web

HTTP

Os usuários de uma aplicação web utilizam navegadores (*browsers*) para interagir com essa aplicação. A comunicação entre navegadores e uma aplicação web é realizada através de requisições e respostas definidas pelo protocolo HTTP. Dessa forma, os desenvolvedores de aplicações web devem estar preparados para trabalhar com o protocolo HTTP.

Acesso simultâneo

Além disso, na grande maioria dos casos, as aplicações web devem ser acessadas por diversos usuários ao mesmo tempo. Consequentemente, os desenvolvedores web devem criar ou utilizar algum mecanismo eficiente que permita esse tipo de acesso.

Conteúdo dinâmico

As páginas de uma aplicação web devem ser geradas dinamicamente. Por exemplo, quando um usuário de uma aplicação de email acessa a sua caixa de entrada, ele deseja ver todos os emails enviados até aquele momento. A página contendo a lista de emails deve ser gerada novamente toda vez que essa página for requisitada. Consequentemente, os desenvolvedores web devem criar ou utilizar algum mecanismo eficiente que permita que o conteúdo das páginas das aplicações web seja gerado dinamicamente.

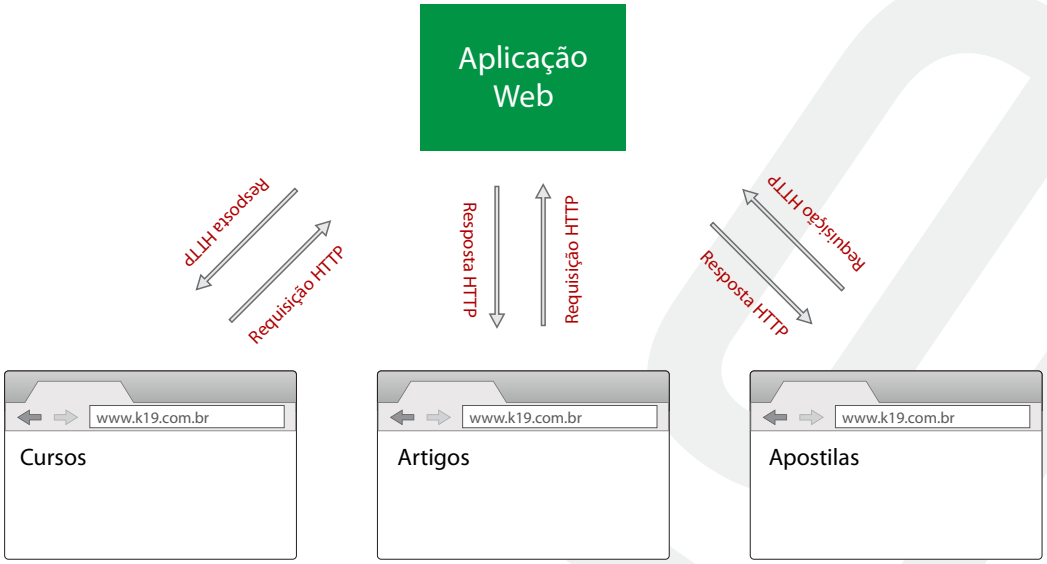


Figura 4.1: Necessidades de uma aplicação web

Solução

Resolver os três problemas apresentados tomaria boa parte do tempo de desenvolvimento, além de exigir conhecimentos técnicos extremamente específicos por parte dos desenvolvedores. Para facilitar o desenvolvimento de aplicações web, a plataforma .NET soluciona esses problemas fundamentais.

A plataforma .NET oferece os seguintes recursos fundamentais para o desenvolvimento de aplicações web:

- Envio e recebimento de mensagens HTTP.
- Acesso simultâneo das aplicações por vários usuários de uma maneira eficiente.
- Geração dinâmica das páginas das aplicações.

ASP.NET MVC

Atualmente, o ASP.NET MVC é o framework para desenvolvimento de aplicações web na plataforma .NET em maior ascensão. A documentação desse framework pode ser obtida em `http://www.asp.net/mvc`. O ASP.NET MVC é fortemente baseado nos padrões *MVC* e *Front Controller*.

MVC e Front Controller

O MVC (*model-view-controller*) é um padrão de arquitetura que tem por objetivo isolar a lógica de negócio da lógica de apresentação de uma aplicação.

Esse padrão (ou alguma variação) é amplamente adotado nas principais plataformas de desenvolvimento atuais. Em particular, ele é bastante utilizado no desenvolvimento de aplicações web.

O padrão MVC divide uma aplicação em três tipos de componentes: modelo, visão e controlador.

Modelo: encapsula os dados e as funcionalidades da aplicação.

Visão: é responsável pela exibição de informações, cujos dados são obtidos do modelo.

Controlador: recebe as requisições do usuário e aciona o modelo e/ou a visão.

Para mais detalhes sobre o padrão MVC, uma boa referência é o livro *Pattern-Oriented Software Architecture Volume 1: A System of Patterns* (editora Wiley, 1996) dos autores Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, Michael Stal e Michael Stal.

No padrão *Front Controller*, todas as requisições do usuário são recebidas pelo mesmo componente. Dessa forma, tarefas que devem ser realizadas em todas as requisições podem ser implementadas por esse componente. Isso evita a repetição de código e facilita a manutenção do sistema.

Para mais informações sobre esse padrão, consulte, por exemplo, o livro *Professional ASP.NET Design Patterns* (editora Wrox, 720) de Scott Millett.



Visual Studio

O primeiro passo para construir uma aplicação web utilizando o framework *ASP .NET MVC* é criar um projeto no *Visual Studio* a partir do modelo adequado. No nosso caso, o modelo de projeto que deve ser utilizado é o **ASP.NET Web Application**.

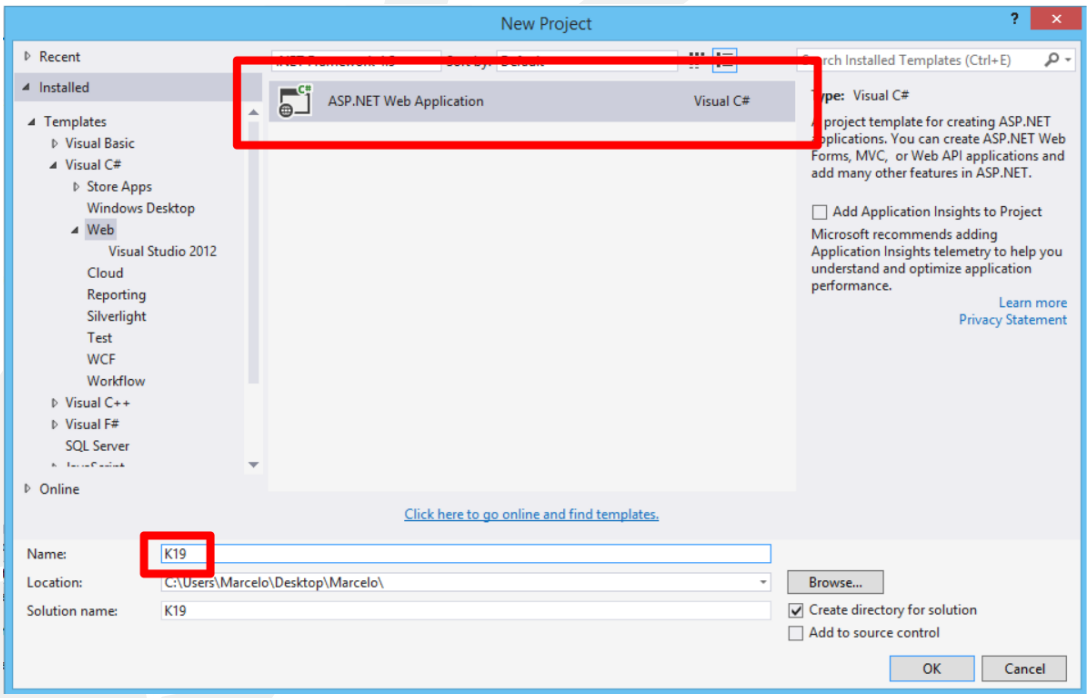


Figura 4.2: Criando um projeto

Devemos escolher *MVC* conforme figura abaixo:

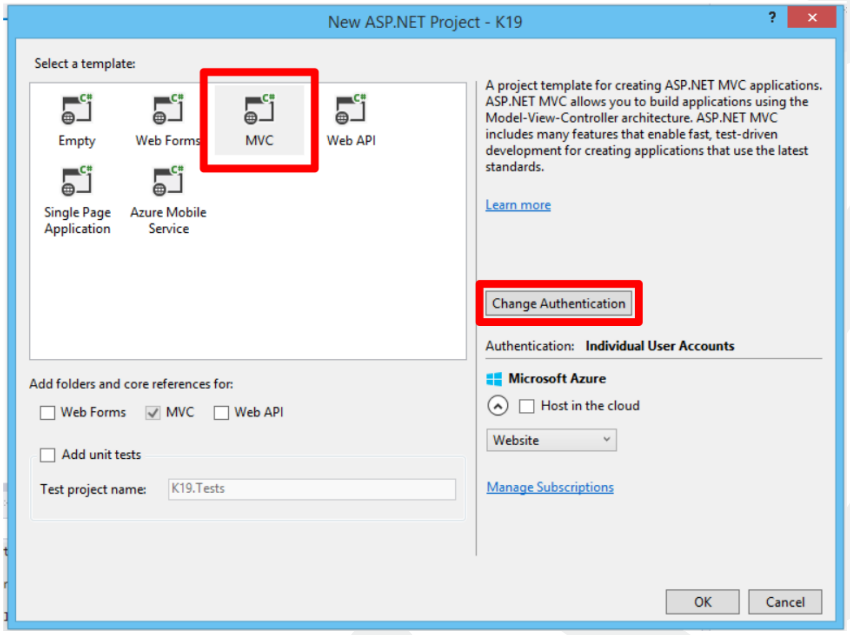


Figura 4.3: Criando um projeto

Além disso, devemos definir o tipo de autenticação. Nos nossos exemplos, por enquanto, não utilizaremos autenticação. Você deve escolher *No Authentication* conforme figura abaixo:

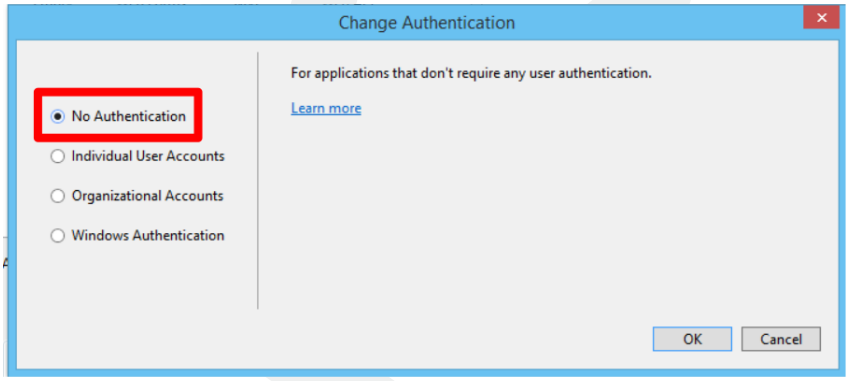


Figura 4.4: Criando um projeto

O projeto criado já vem com diversas pastas e arquivos. Ao longo dos próximos capítulos, a função de cada pasta e de cada arquivo será discutida.

Testando a aplicação

Para verificar o funcionamento do projeto, basta executá-lo através do menu: *Debug -> Start Debugging*. Um servidor será inicializado na máquina e a aplicação é implantada nesse servidor. Além disso, uma janela do navegador padrão do sistema é aberta na url principal da aplicação.

Trocando a porta do servidor

Para trocar a porta do servidor inicializado pelo *Visual Studio* utiliza, basta alterar as propriedades do projeto clicando com o botão direito do mouse no projeto e escolhendo o item *properties* e

depois a aba *web*.



Exemplo de uma Aplicação Web

Veremos a seguir um exemplo de uma simples aplicação web em ASP.NET MVC. Essa aplicação possuirá uma página principal que deverá exibir o seu número de visualização.

Primeiramente, criaremos um controlador para gerenciar o número de visualização da página principal. Os controladores são definidos por classes que derivam da classe `Controller`. Os nomes dos controladores devem terminar com a palavra `Controller`. Além disso, eles devem ser definidos na pasta `Controllers` da aplicação. Considere a classe `K19Controller` definida abaixo.

```
1  public class K19Controller : Controller
2  {
3    private static int NumeroDeAcessos { get; set; }
4
5    public ActionResult Index()
6    {
7      K19Controller.NumeroDeAcessos++;
8      ViewBag.Acessos = K19Controller.NumeroDeAcessos;
9      return View();
10   }
11 }
```

Código C# 4.1: K19Controller.cs

A classe `K19Controller` possui uma propriedade chamada `NumeroDeAcessos`. Essa propriedade armazenará o número de visualizações da página principal.

O método `Index()` será chamado toda vez que a url `http://<IP_SERVIDOR>:<PORTA_APP>/K19/Index` for requisitada por um navegador. Note que essa url é formada pelo nome do controlador (nome da classe que define o controlador sem o sufixo `Controller`) seguido pelo nome do método.

Cada vez que o método `Index()` é chamado, o valor da propriedade `NumeroDeAcessos` é incrementado. Além disso, o valor atualizado dessa propriedade é colocado na `ViewBag` para que possa ser acessado na camada de apresentação.

Para indicar que o fluxo de execução deve seguir para a camada de apresentação, o método `Index()` invoca o método `View()` e devolve a resposta obtida. Por padrão, o fluxo será direcionado para um arquivo chamado `Index.cshtml` que deve estar localizado na pasta `Views\K19\`.

Na camada de apresentação, vamos adicionar a página associada ao método `Index()`. Para isso, criaremos um arquivo chamado `Index.cshtml` na pasta `Views\K19\` da aplicação com o conteúdo abaixo.

```
1 @{
2   ViewBag.Title = "Index";
3 }
4 <h2>Acessos: @ViewBag.Acessos</h2>
```

Código CSHTML 4.1: Index.cshtml

Para acessar a `ViewBag`, devemos utilizar o caractere `@`, como no código acima.



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **K19** no **Visual Studio**. Utilize o template **MVC**.

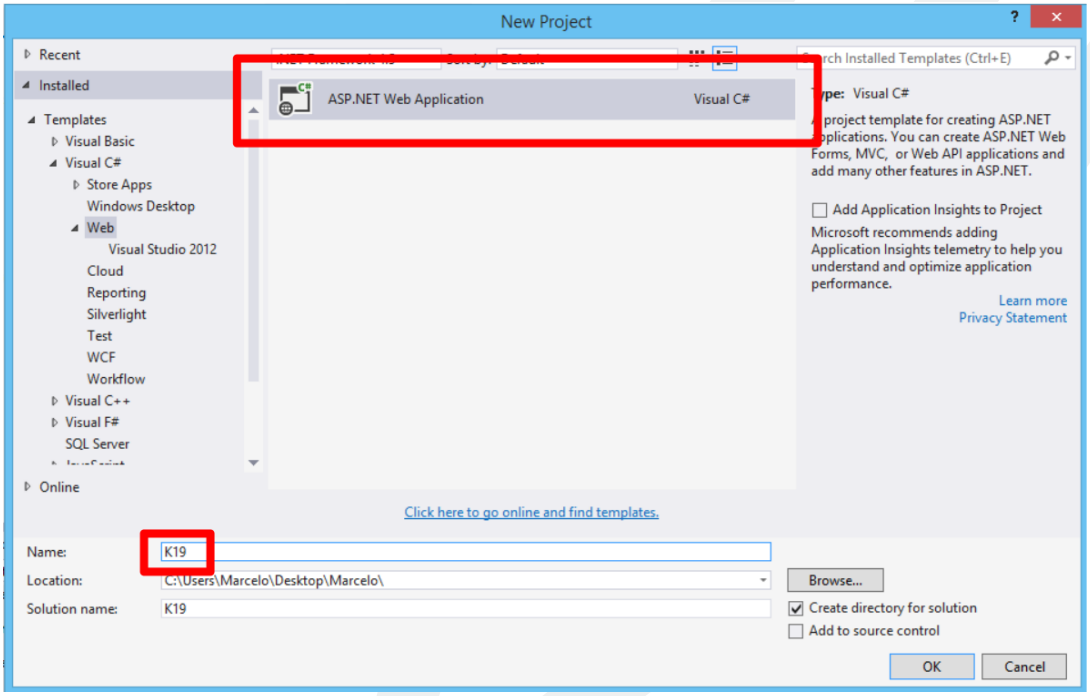


Figura 4.5: Criando um projeto

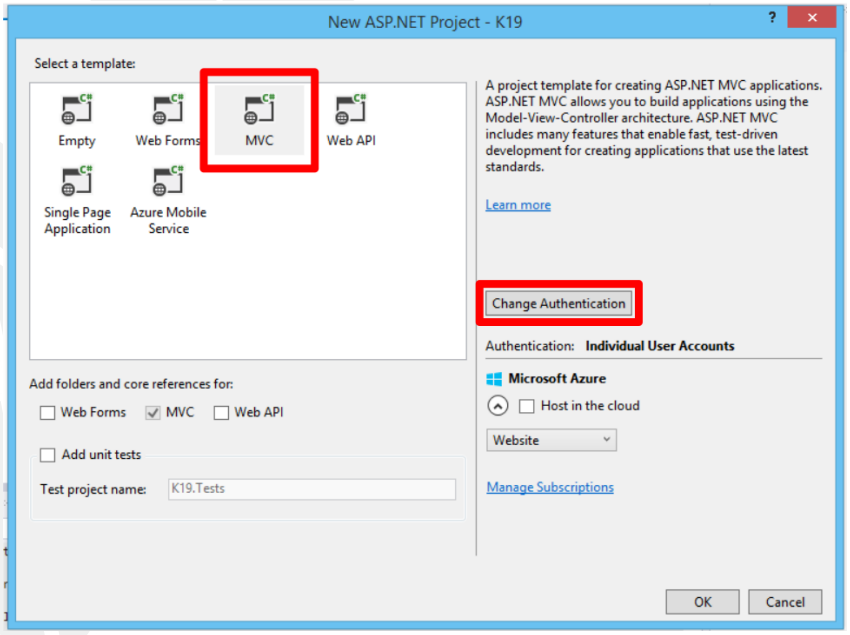


Figura 4.6: Criando um projeto

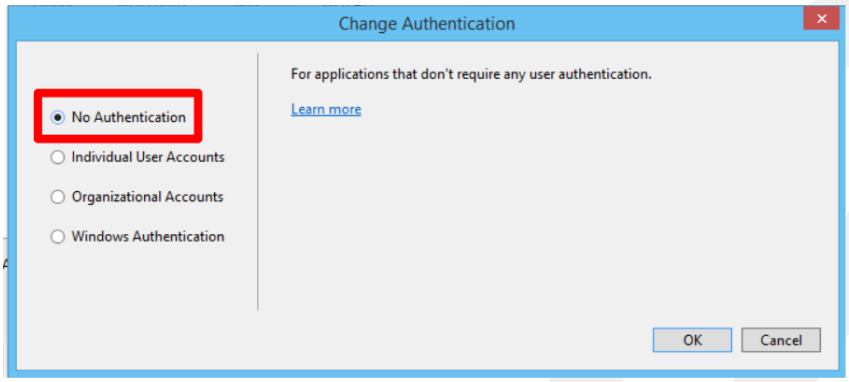


Figura 4.7: Criando um projeto

2 Crie um controlador chamado `K19` no projeto **K19**. Siga os passos apresentados nas imagens abaixo.

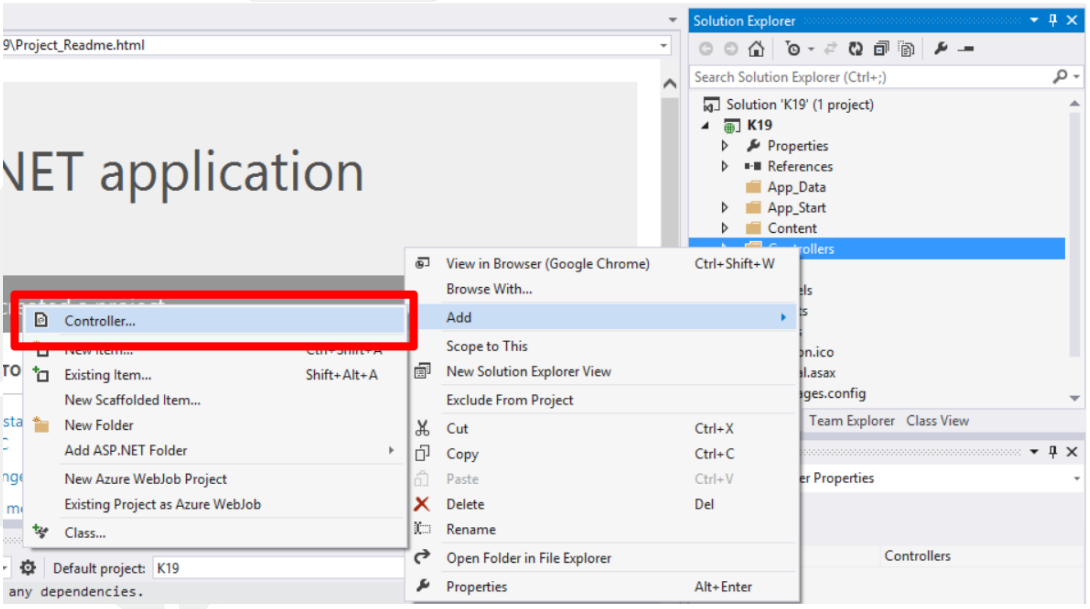


Figura 4.8: Criando um controlador

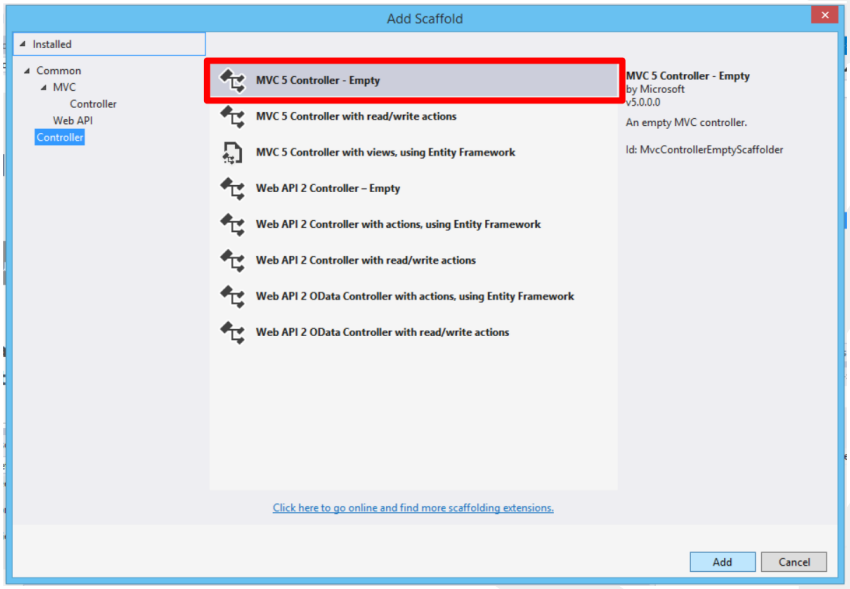


Figura 4.9: Criando um controlador

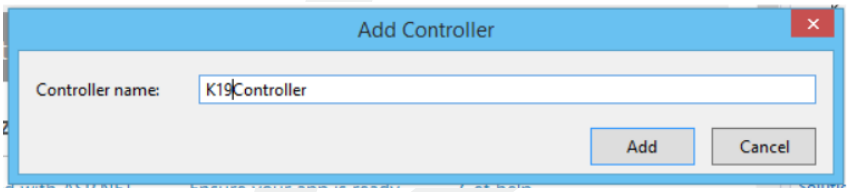


Figura 4.10: Criando um controlador

```
1  using System;
2  using System.Web.Mvc;
3
4  namespace K19.Controllers
5  {
6    public class K19Controller : Controller
7    {
8      public ActionResult Index()
9      {
10       Random random = new Random();
11       ViewBag.NumeroDaSorte = random.Next();
12       return View();
13     }
14   }
15 }
```

Código C# 4.2: www.k19.com.br/exercicios/k32/04/002/K19Controller.cs

3 Crie uma página na aplicação, adicionando um arquivo chamado **Index.cshtml** dentro da pasta **Views\K19**.

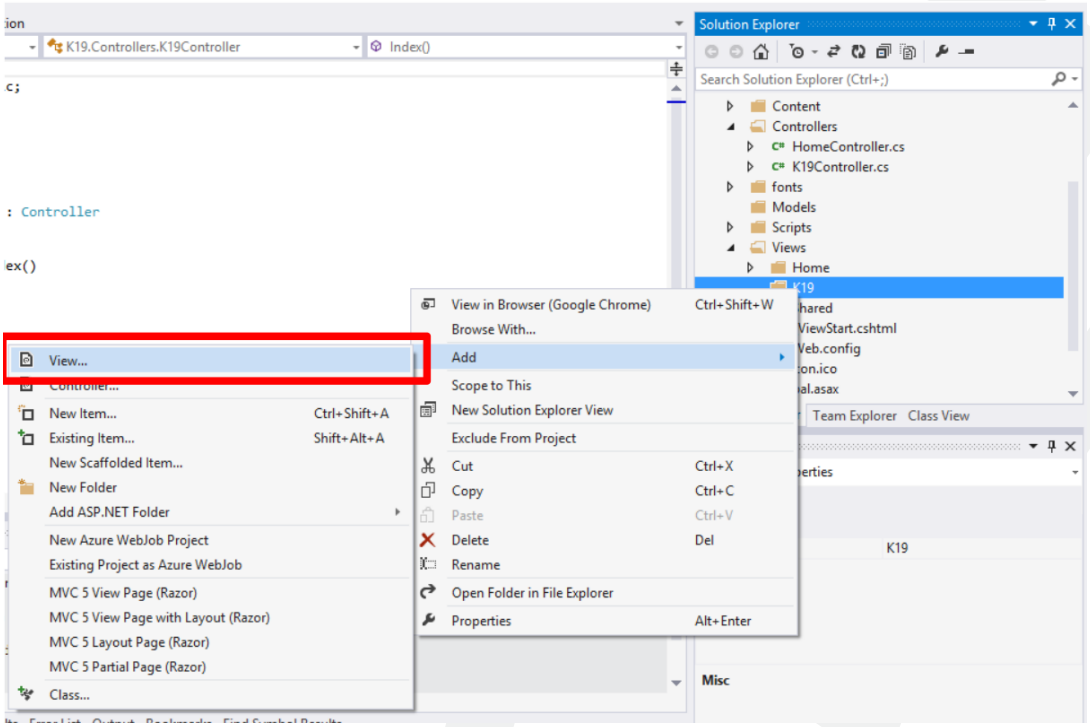


Figura 4.11: Adicionando uma tela

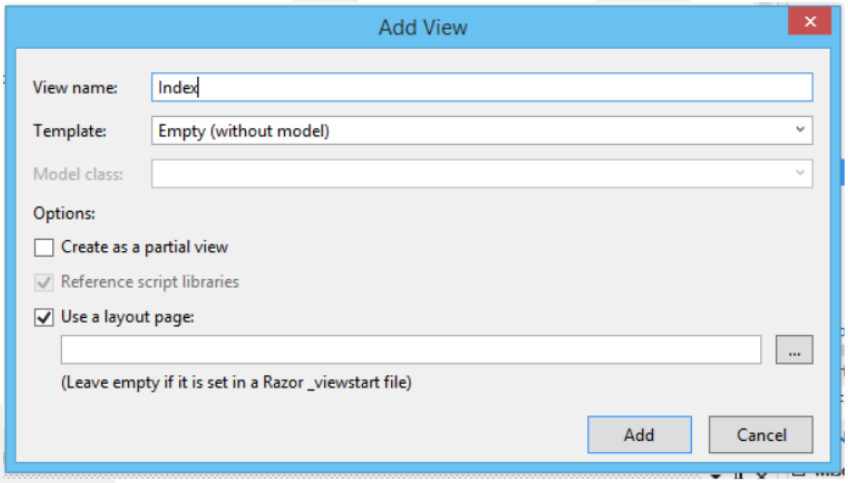


Figura 4.12: Adicionando uma tela

```
1 @{
2   ViewBag.Title = "Index";
3 }
4 <h2>Olá! O seu número da sorte é @ViewBag.NumeroDaSorte</h2>
```

Código CSHTML 4.2: www.k19.com.br/exercicios/k32/04/003/Index.cshtml

4. Para visualizar a página, basta executar o projeto no Visual Studio e acessar o endereço **http://localhost:<PORTA_APP>/K19/Index**

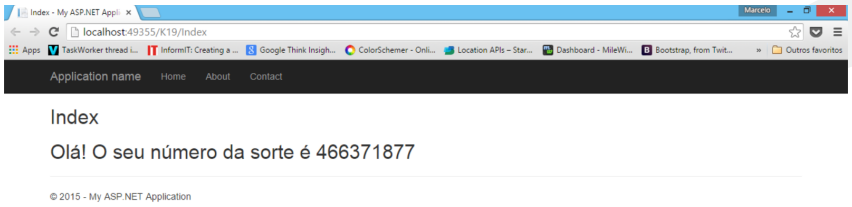


Figura 4.13: Acessando a página da aplicação

Integração com Entity Framework

A integração entre Entity Framework e ASP.NET MVC é realizada de maneira extremamente simples. No Visual Studio, ao criar um projeto ASP.NET Web Application do tipo *MVC*, a biblioteca do Entity Framework é adicionada automaticamente.

As configurações padrões do Entity Framework podem ser alteradas através do arquivo **Web.config**.

Por fim, bastaria implementar as entidades e mapeá-las como visto no Capítulo B.

Scaffold

O Visual Studio é capaz de gerar telas e controladores para as operações de inserção, leitura, alteração e remoção (CRUD) a partir de uma entidade de um projeto ASP.NET MVC. Os controladores gerados podem utilizar as funcionalidades do Entity Framework para interagir com o banco de dados.

Exercícios de Fixação

5 Vamos gerar o nosso primeiro scaffold. Para isto, defina a classe **Editora** na pasta **Models** do projeto **K19**.

```
1 namespace K19.Models
2 {
3   public class Editora
4   {
5     public int EditoraID { get; set; }
6
7     public string Nome { get; set; }
8
```

```
9       public string Email { get; set; }
10    }
11  }
```

Código C# 4.3: www.k19.com.br/exercicios/k32/04/005/Editora.cs

6 Antes de gerar o scaffold, é necessário fazer um build no projeto.

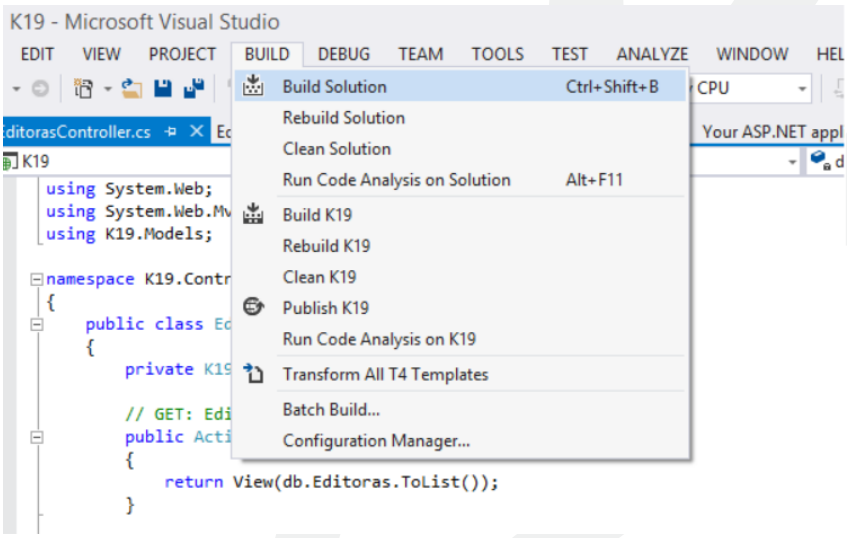


Figura 4.14: Build do projeto

7 Após o build do projeto, podemos gerar o scaffold. Para isso, utilize a opção **New Scaffolded Item** seguindo os passos apresentados nas imagens a seguir.

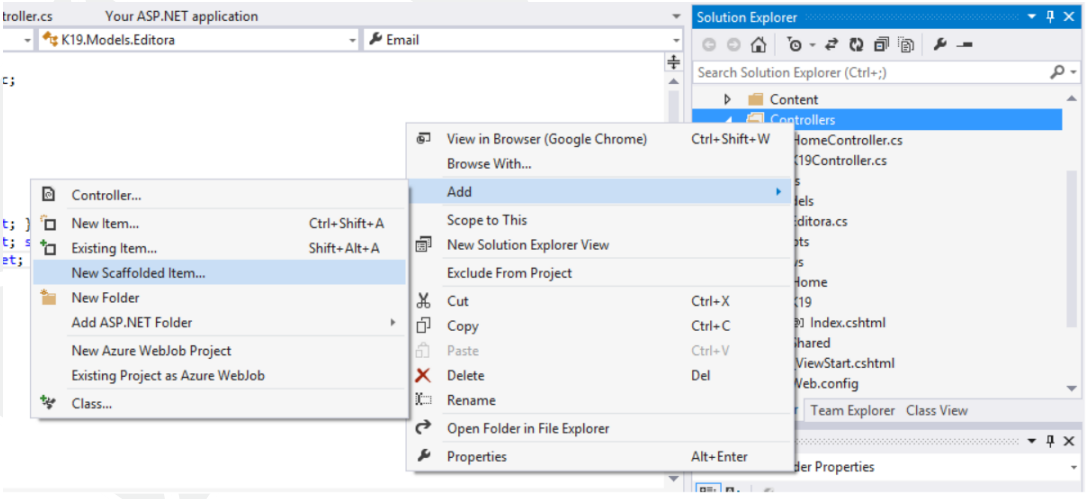


Figura 4.15: Gerando o scaffold

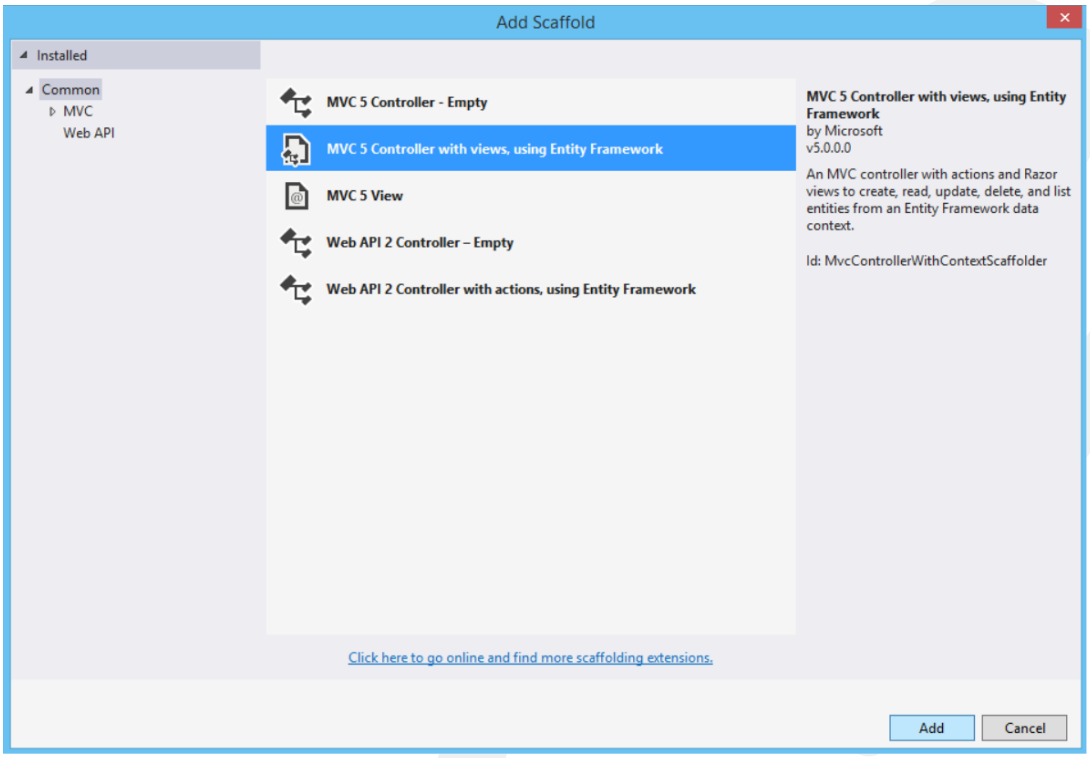


Figura 4.16: Gerando o scaffold

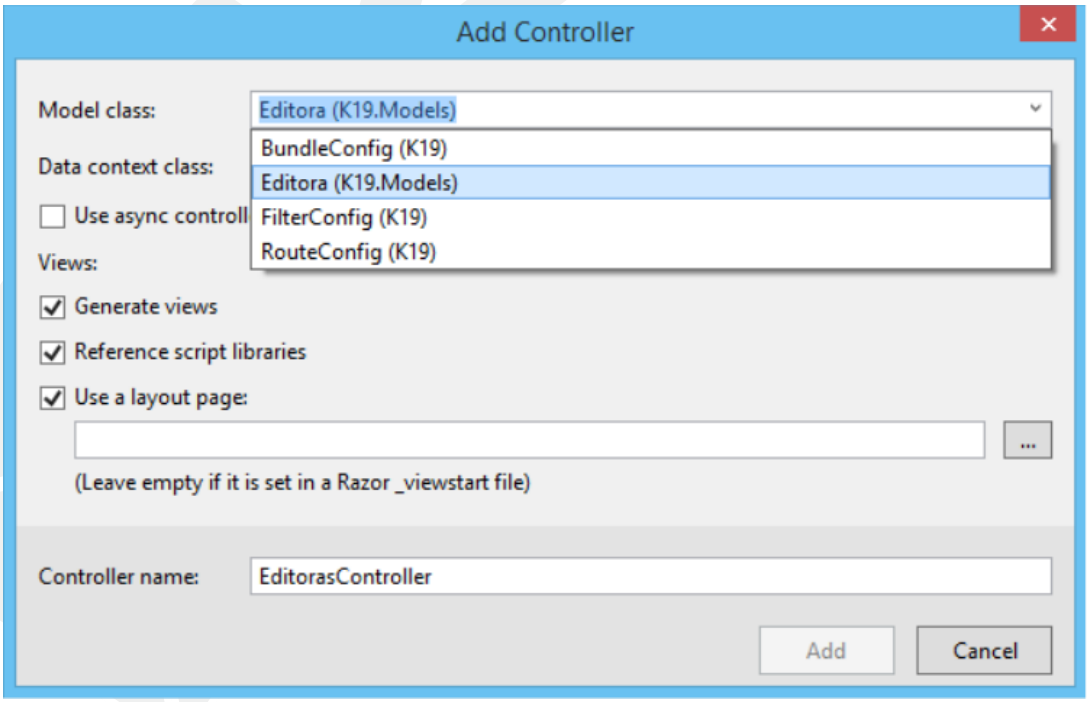


Figura 4.17: Gerando o scaffold

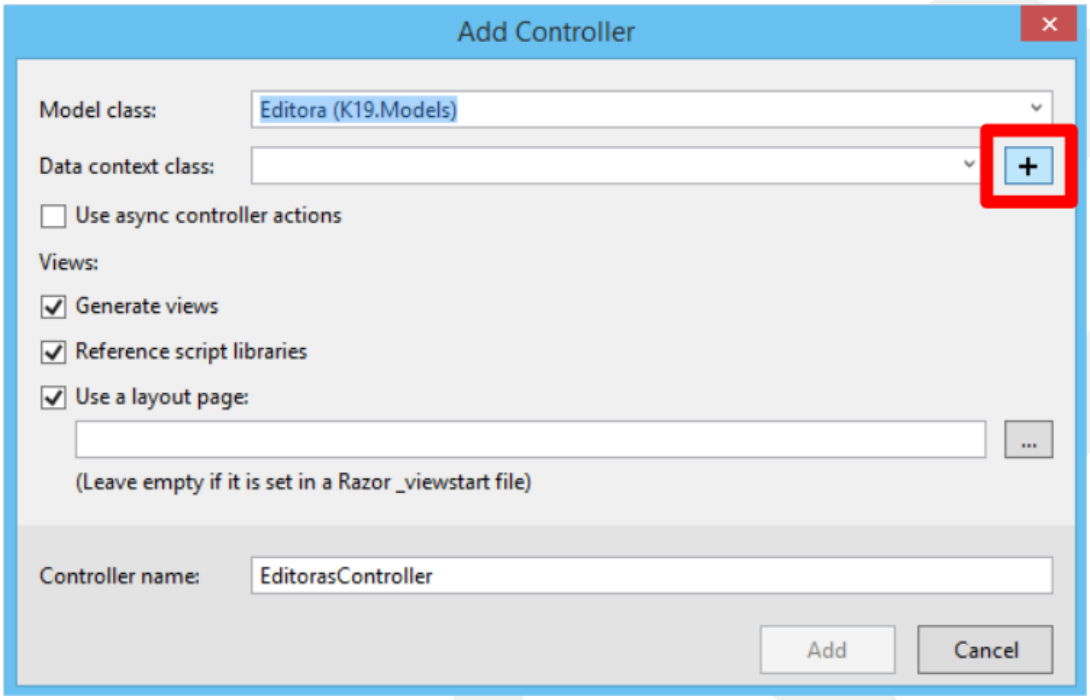


Figura 4.18: Gerando o scaffold

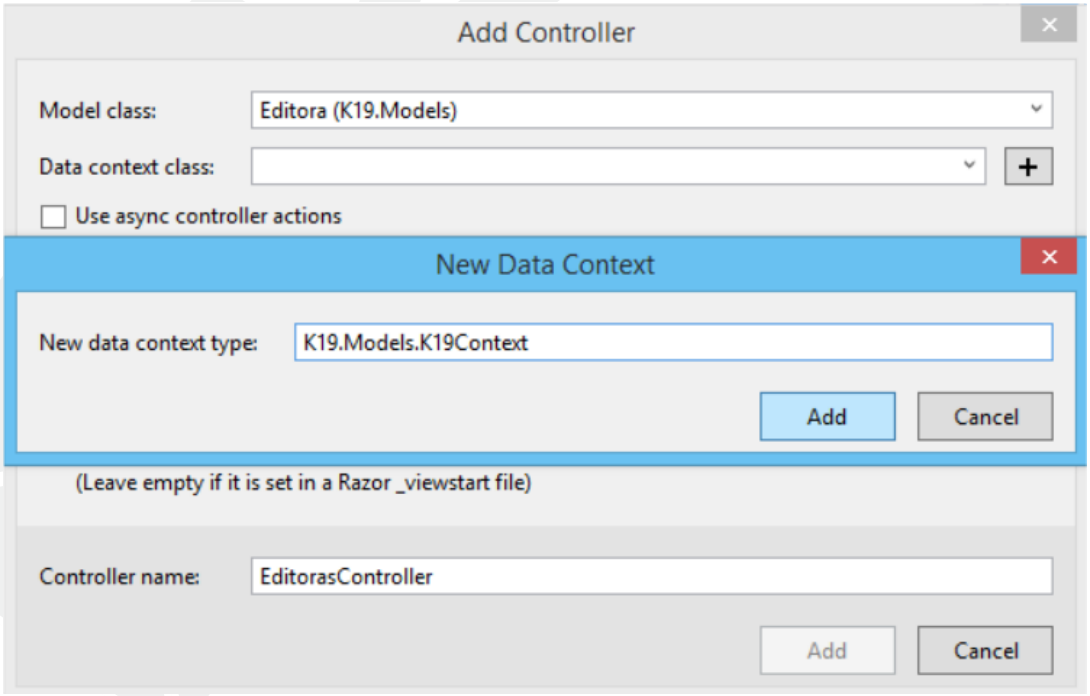


Figura 4.19: Gerando o scaffold

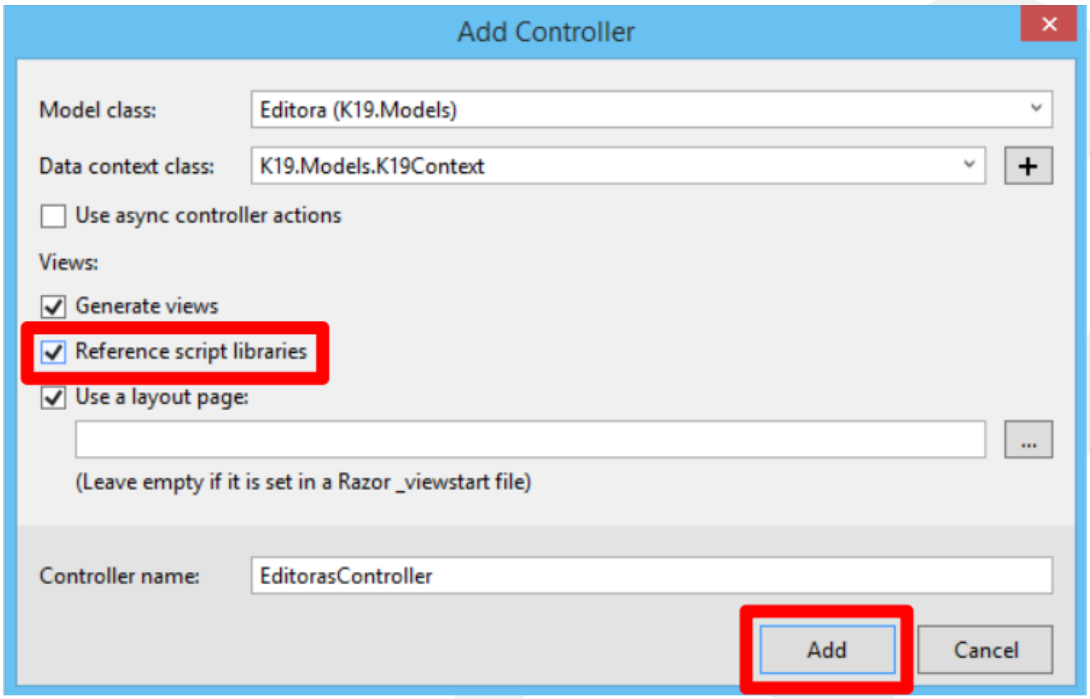


Figura 4.20: Gerando o scaffold

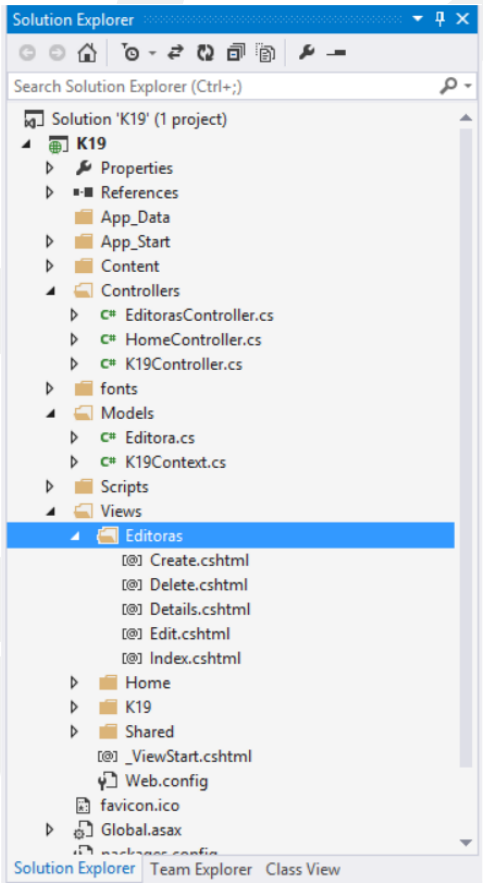


Figura 4.21: Scaffold gerado

8 Para testar, acesse o endereço **http://localhost:<PORTA_APP>/Editora/Index**.

9 Adicione uma entidade chamada **Livro** na pasta **Models**.

```
1  namespace K19.Models
2  {
3    public class Livro
4    {
5      public int LivroID { get; set; }
6
7      public string Titulo { get; set; }
8
9      public double Preco { get; set; }
10
11     public int EditoraID { get; set; }
12
13     public virtual Editora Editora { get; set; }
14   }
15 }
```

Código C# 4.4: www.k19.com.br/exercicios/k32/04/009/Livro.cs

10 Altere a classe **K19Context** para registrar a entidade `Livro`.

```
1  namespace K19.Models
2  {
3      public class K19Context : DbContext
4      {
5          public DbSet<Editora> Editoras { get; set; }
6
7          public DbSet<Livro> Livros { get; set; }
8      }
9  }
```

Código C# 4.5: www.k19.com.br/exercicios/k32/04/010/K19Context.cs

11 Faça novamente o build do projeto. Depois, gere o scaffold da entidade `Livro`. Utilize a opção **New Scaffolded Item** para fazer o scaffold. Basta seguir os mesmos passos do scaffold da Editora visto nos exercícios anteriores.

12 Para testar, acesse o endereço **http://localhost:<PORTA_APP>/Livro/Index**.

Apague a base de dados K19.Models.K19Context antes de testar a aplicação

CAPÍTULO 5

CAMADA DE APRESENTAÇÃO

A camada de apresentação é responsável por gerar as páginas de uma aplicação web. Os dados apresentados em uma página web são definidos na camada de modelo. A camada de controle recupera os dados da camada de modelo e os envia para a camada de apresentação. Basicamente, a camada de apresentação definirá como esses dados serão apresentados para os usuários da aplicação.

O fluxo inverso também ocorre. Ou seja, a camada de apresentação obtém dados inseridos pelos usuários e os envia para a camada de controle que, por sua vez, usa esses dados para alterar a camada de modelo.

Neste capítulo, mostraremos como a camada de apresentação de uma aplicação web é construída com a utilização de recursos do ASP.NET MVC como *Inline Code*, *HTML Helper*, *Layout* e *Partial Views*.



Razor e ASPX

Até a segunda versão do ASP.NET MVC, as telas (ou páginas) de uma aplicação web eram desenvolvidas em ASPX. A partir da terceira versão desse framework, podemos usar a linguagem Razor para construir essas telas. A principal característica da Razor é ser concisa e simples, diminuindo o número de caracteres e as tradicionais tags de scripts do ASP.NET MVC (`<% %>`).

Nos exemplos abaixo, mostraremos algumas diferença entre Razor e ASPX.

Criando Variáveis

Toda expressão em Razor começa com o caractere `@`. Para criarmos variáveis, precisamos declará-las dentro de um bloco Razor:

```
@{
  string nome = "K19";
  string telefoneDaK19 = "2387-3791";
  string enderecoDaK19 = "Av. Brigadeiro Faria Lima";
  int numeroDaK19 = 1571;
}
```

Código CSHTML 5.1: Criando variáveis em Razor

```
<%
  string nome = "K19";
  string telefoneDaK19 = "2387-3791";
  string enderecoDaK19 = "Av. Brigadeiro Faria Lima";
  int numeroDaK19 = 1571;
%>
```

Código ASPX 5.1: Criando variáveis em ASPX

Acessando variáveis

As variáveis podem ser facilmente acessadas.

```
<p>Telefone da K19: @telefoneDaK19</p>
```

Código CSHTML 5.2: Acessando uma variável

```
<p>Telefone da K19: <%= telefoneDaK19 %></p>
```

Código ASPX 5.2: Acessando uma variável

Condicionais (if e else)

Podemos utilizar os comandos de controle de fluxo `if` e `else`. Veja os exemplos abaixo.

```
@if(numero == numeroDaK19)
{
  <p>Chegou na K19!</p>
}
else
{
  <p>Continue andando.</p>
}
<!-- ou -->
@{
  int numero = 463;
  if(numero == numeroDaK19)
  {
    <p>Chegou na K19!</p>
  }
  else
  {
    <p>Continue andando.</p>
  }
}
```

Código CSHTML 5.3: Utilizando if e else em Razor

```
<%
if(numero == numeroDaK19)
{
%>
  <p>Chegou na K19!</p>
<%
}
else
{
%>
  <p>Continue andando.</p>
<%
}
%>
```

Código CSHTML 5.4: Utilizando if e else em ASPX

Laços

Podemos criar laços utilizando o comando `for`.

```
@for(int i = 0; i < 5; i++)
{
```

```
3   <p>i = @i</p>
4 }
```

Código CSHTML 5.5: Criando um laço em Razor

```
1 <%
2 for(int i = 0; i < 5; i++)
3 {
4 %>
5   <p>i = <%= i %> </p>
6 <%
7 }
8 %>
```

Código ASPX 5.3: Criando um laço em ASPX

Texto e código

```
1 @if (x == "nome")
2 {
3   @:O nome da editora é @editora.Nome
4 }
```

Código CSHTML 5.6: Texto e código em Razor

```
1 <%
2 if (x == "nome")
3 {
4 %>
5   O nome da editora é <%= editora.Nome %>
6 <%
7 }
8 %>
```

Código ASPX 5.4: Texto e código em ASPX

Comentários

Podemos adicionar comentários nas páginas.

```
1 @* Comentário *@
```

Código CSHTML 5.7: Comentários em Razor

```
1 <%-- Comentário --%>
```

Código ASPX 5.5: Comentários em ASPX



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **CamadaDeApresentacao** no **Visual Studio**. Utilize o template **MVC**.

2 Adicione um controlador chamado **Sorte** no projeto **CamadaDeApresentacao** utilizando o template **Empty MVC Controller**. Nesse controlador, defina uma ação chamada **Index**.

```
1  namespace CamadaDeApresentacao.Controllers
2  {
3    public class SorteController : Controller
4    {
5      public ActionResult Index()
6      {
7        Random random = new Random();
8        ViewBag.NumeroDaSorte = random.Next();
9        return View();
10     }
11   }
12 }
```

Código C# 5.1: www.k19.com.br/exercicios/k32/05/002/SorteController.cs

3 Adicione uma página associada à ação **Index** do controlador **Sorte** do projeto **CamadaDeApresentacao**.

```
1  @{
2    ViewBag.Title = "Index";
3  }
4
5  @for(int i = 0; i < 10; i++)
6  {
7    <h2>Olá! O seu número da sorte é @ViewBag.NumeroDaSorte</h2>
8  }
```

Código CSHTML 5.8: www.k19.com.br/exercicios/k32/05/003/Index.cshtml

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Sorte/Index`.

4 Altere o arquivo **Index.cshtml** do exercício anterior para imprimir uma mensagem a partir de uma variável.

```
1  @{
2    ViewBag.Title = "Index";
3    string mensagem = "Olá! O seu número da sorte é " + @ViewBag.NumeroDaSorte;
4  }
5
6  @for(int i = 0; i < 10; i++)
7  {
8    <h2>@mensagem</h2>
9  }
```

Código CSHTML 5.9: www.k19.com.br/exercicios/k32/05/004/Index.cshtml

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Sorte/Index`.



Exercícios Complementares

1 Altere o arquivo **Index.cshtml** para gerar a seguinte tela para o usuário:

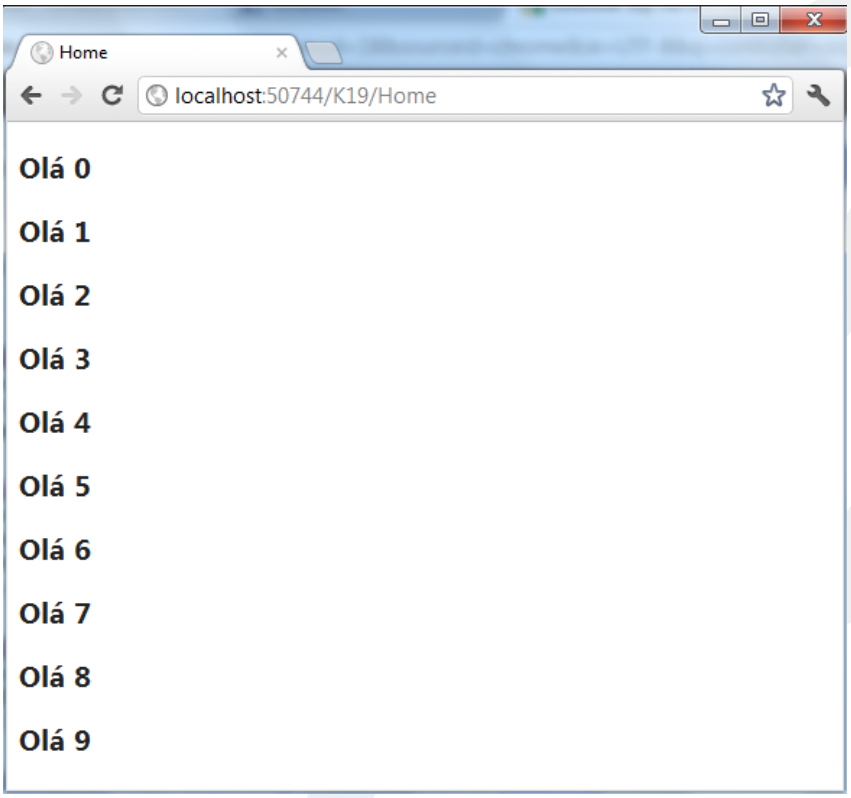


Figura 5.1: Inline Code



ViewBag e Strogly Typed Views

A `ViewBag` é utilizada para transmitir dados da camada de controle para a camada de apresentação. Para adicionar um item à propriedade `ViewBag`, devemos definir uma chave para esse item. Veja o código abaixo.

```
1 ViewBag.HoraDoServidor = DateTime.Now.ToShortTimeString();
```

Código C# 5.2: Adicionando um item na `ViewBag`

No código acima, criamos uma chave chamada `HoraDoServidor` e atribuímos um valor a essa chave.

Na camada de apresentação, os itens armazenados na `ViewBag` podem ser acessados facilmente através da sintaxe da Razor. Veja um exemplo abaixo.

```
1 A hora do servidor é @ViewBag.HoraDoServidor
```

Código CSHTML 5.11: Acessando um item da `ViewBag`

A `ViewBag` oferece uma forma simples de transmitir dados da camada de controle para a camada de apresentação, mas apresenta algumas desvantagens. Por exemplo, como as chaves são adicionadas dinamicamente, não é possível garantir a existência dessas chaves em tempo de compilação. Considere o código abaixo.

```
1 double numero = new Random().NextDouble();
2 if(numero < 0.5)
3 {
4   ViewBag.X = "K19 TREINAMENTOS";
5 }
6 else
7 {
8   ViewBag.X = 10;
9 }
```

Código C# 5.3: Adcicionando chaves dinamicamente na `ViewBag`

No código acima, o tipo do objeto associado à chave `X` depende do valor da variável `numero`, que é gerado aleatoriamente. Quando o valor gerado for menor do que 0.5, o tipo desse objeto será `string`. Caso contrário, o tipo desse objeto será `int`. Suponha que a chave `X` seja utilizada na camada de apresentação.

```
1 <p>@ViewBag.X.Lower()</p>
```

Código CSHTML 5.12: Utilizando a chave `X` `ViewBag`

O código acima não produz erros de compilação. Contudo, na execução, um erro poderá ocorrer. O problema é que esse código supõe que o objeto associado à chave `X` possui o método `Lower()`. No entanto, se a ação for acionada um número suficiente de vezes, em algum momento isso não será verdade, e um erro de execução ocorrerá.

Além disso, recursos do Visual Studio como *IntelliSense* (recurso para completar código) ou refatoração não podem ser aplicados com o uso da `ViewBag`.

Uma alternativa à utilização da `ViewBag` é o recurso **strongly typed views**. Com esse recurso podemos fixar, em tempo de compilação, o tipo do objeto que será transferido da camada de controle para a camada de apresentação. Uma vez que o tipo desse objeto foi fixado, o compilador é capaz de verificar a validade do código, evitando eventuais erros de execução. Além disso, o Visual Studio pode auxiliar o desenvolvimento da aplicação com os recursos de *IntelliSense* e de refatoração.

Para utilizar *strongly type views*, o primeiro passo é, na camada de controle, passar como parâmetro para o método `View()` um objeto do tipo esperado pela camada de apresentação.

```
1 public ActionResult Acao()
2 {
3   Editora editora = new Editora { Nome = "K19", Email = "contato@k19.com.br" };
4   return View(editora);
5 }
```

Código C# 5.4: Passando um objeto para a camada de apresentação

Para definir o tipo do objeto esperado na camada de apresentação, podemos utilizar a sintaxe da Razor. Veja o código abaixo.

```
1 @model K19.Models.Editora
```

Código CSHTML 5.13: Definindo o tipo do objeto esperado na camada de apresentação

Podemos também utilizar o comando `using` para evitar a escrita dos nomes completos das classes.

```
1 @using K19.Models
2 @model Editora
```

Código CSHTML 5.14: Utilizando o comando `using`

Para acessar o objeto transmitido da camada de controle para a camada de apresentação, devemos utilizar a propriedade `Model`. Observe o código a seguir.

```
1 @using K19.Models
2 @model Editora
3
4 <p>
5 Nome: @Model.Nome
6 <br />
7 Email: @Model.Email
8 </p>
```

Código CSHTML 5.15: Utilizando a propriedade `Model`

Se tentarmos acessar um método ou propriedade inexistentes, um erro de compilação ocorrerá, pois agora o compilador conhece o tipo do objeto transmitido. Dessa forma, o código abaixo produz um erro de compilação.

```
1 @using K19.Models
2 @model Editora
3
4 <p>
5 Nome: @Model.Nome
6 <br />
7 Telefone: @Model.Telefone
8 </p>
```

Código CSHTML 5.16: Produzindo um erro de compilação

Observe que apenas um objeto pode ser transmitido da camada de controle para a camada de apresentação através do recurso *strongly type views*. Já com a utilização de `ViewBag`, diversos objetos podem ser transmitidos. Note também que os dois recursos podem ser utilizados ao mesmo tempo.



Exercícios de Fixação

5 No projeto **CamadaDeApresentacao**, adicione um controlador chamado **Relogio** utilizanto o template **Empty MVC Controller**. Nesse controlador, defina uma ação chamada **Agora**. Essa ação deve armazenar a data e o horário atual na `ViewBag`.

```
1  namespace CamadaDeApresentacao.Controllers
2  {
3    public class RelogioController : Controller
4    {
5      public ActionResult Agora()
6      {
7        ViewBag.Agora  = DateTime.Now.ToString("dd/MM/yyyy HH:mm:ss");
8        return View();
9      }
10   }
11 }
```

Código C# 5.5: www.k19.com.br/exercicios/k32/05/005/RelogioController.cs

6 Adicione uma página associada à ação **Agora** do controlador **Relogio**.

```
@{
    ViewBag.Title = "Agora";
}

<h2>Agora: @ViewBag.Agora</h2>
```

Código HTML 5.1: www.k19.com.br/exercicios/k32/05/006/Agora.cshtml

Para testar, acesse: http://localhost:<PORTA_APP>/Relogio/Agora.

7 Adicione uma entidade chamada **Aluno** na pasta **Models** do projeto **CamadaDeApresentacao**.

```
namespace CamadaDeApresentacao.Models
{
  public class Aluno
  {
    public int AlunoID { get; set; }

    public string Nome { get; set; }

    public string Email { get; set; }
  }
}
```

Código C# 5.6: www.k19.com.br/exercicios/k32/05/007/Aluno.cs

8 Adicione um controlador chamado **Aluno** utilizando o template **Empty MVC Controller**. Nesse controlador, defina uma ação para transmitir para camada de apresentação um objeto da entidade **Aluno**. Utilize *strongly type views*.

```
namespace CamadaDeApresentacao.Controllers
{
  public class AlunoController : Controller
  {
    public ActionResult Detalhes()
    {
      Aluno a = new Aluno {
        AlunoID = 1,
        Nome = "Jonas Hirata",
        Email = "jonas@k19.com.br"
      };
      return View(a);
    }
  }
}
```

Código C# 5.7: www.k19.com.br/exercicios/k32/05/008/AlunoController.cs

9 Adicione uma tela associada à ação **Detalhes** do controlador **Aluno** com o seguinte conteúdo.

```
@model CamadaDeApresentacao.Models.Aluno
```

```
2
3  @{
4      ViewBag.Title = "Detalhes";
5  }
6
7  <h2>Detalhes</h2>
8
9  <p>
10     AlunoID: @Model.AlunoID
11 </p>
12
13 <p>
14     Nome: @Model.Nome
15 </p>
16
17 <p>
18     Email: @Model.Email
19 </p>
```

Código HTML 5.2: www.k19.com.br/exercicios/k32/05/009/Detalhes.cshtml

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Aluno/Detalhes`.

10 Adicione uma ação no controlador **Aluno**.

```
1  ...
2  public ActionResult Lista()
3  {
4    ICollection<Aluno> lista = new List<Aluno>();
5
6    for (int i = 0; i < 5; i++)
7    {
8      Aluno a = new Aluno {
9        AlunoID = i,
10       Nome = "Aluno " + i,
11       Email = "Email " + i
12     };
13     lista.Add(a);
14   }
15
16   return View(lista);
17 }
18 ...
```

Código C# 5.8: www.k19.com.br/exercicios/k32/05/010/AlunoController.cs

11 Adicione uma tela associada à ação **Lista** do controlador **Aluno** com o seguinte conteúdo.

```
1  @model ICollection<CamadaDeApresentacao.Models.Aluno>
2
3  @{
4      ViewBag.Title = "Lista";
5  }
6
7  <h2>Tabela de Aluno</h2>
8
9  <table>
10     <tr>
11         <th>AlunoID</th>
12         <th>Nome</th>
13         <th>Email</th>
14     </tr>
15
16     @foreach (var a in @Model)
```

```
17      {
18          <tr>
19              <td>@a.AlunoID</td>
20              <td>@a.Nome</td>
21              <td>@a.Email</td>
22          </tr>
23      }
24  </table>
```

Código HTML 5.3: www.k19.com.br/exercicios/k32/05/011/Lista.cshtml

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Aluno/Lista`.



HTML Helpers

A função das páginas CSHTML é gerar hipertexto XHTML para enviar aos navegadores dos usuários. Os arquivos `.cshtml` misturam tags XHTML com scripts de servidor escritos em C# (ou outra linguagem de programação suportada pelo .NET). Essa mistura pode prejudicar a legibilidade do código em alguns casos. Além disso, a manutenção da aplicação pode se tornar mais complexa.

Considere a criação de um link utilizando diretamente as tags HTML.

```
1  <a href="/Editora/Index/">Lista de editoras</a>
```

Código CSHTML 5.17: Link para a lista de editoras

Se o formato da url para acessar a ação `Index` do controlador `Editora` for alterado, o código acima deverá ser modificado. Veremos como o formato das urls pode ser configurado no Capítulo **??**.

Para facilitar a manutenção do código das páginas CSHTML, o ASP.NET MVC oferece os chamados **HTML Helpers**. A função de um HTML Helper é encapsular um código XHTML. Por exemplo, para adicionar um link, podemos usar o método **ActionLink** do objeto **Html**, ao invés de usar a tag <a> do HTML diretamente. No código abaixo, criamos um link para a ação responsável por listar as editoras.

```
1  @Html.ActionLink("Lista de editoras", "Index", "Editora")
```

Código CSHTML 5.18: Criando um link para a listagem de editoras

Agora, se o formato da url para acessar a ação `Index` do controlador `Editora` for alterado, o código acima não precisa ser alterado.

ActionLink Helper

O helper `ActionLink` é utilizado para gerar os links das páginas de uma aplicação web. Esse helper pode ser utilizado de diversas maneiras. A forma mais simples de utilizá-lo é passar a ele dois parâmetros: o texto e a ação associados ao link desejado. Nesse caso, o link gerado pelo helper `ActionLink` estará associado ao controlador correspondente a página na qual o link foi inserido.

```
1  @Html.ActionLink("TEXTO PARA O USUÁRIO", "ACTION" )
```

Código CSHTML 5.19: Criando um link com controlador implícito

Podemos definir o controlador desejado explicitamente. Para isso, é necessário passar um terceiro parâmetro para o método `ActionLink`.

```
1 @Html.ActionLink("TEXTO PARA O USUÁRIO", "ACTION", "CONTROLADOR" )
```

Código CSHTML 5.20: Criando um link com controlador explícito

O helper `ActionLink` permite que parâmetros sejam adicionados nos links gerados. Para isso, é necessário passar um array como parâmetro.

```
1 @Html.ActionLink("TEXTO PARA O USUÁRIO", "ACTION", new { Inicio = 0, Final = 10 })
```

Código CSHTML 5.21: Acrescenta parâmetros de url

BeginForm e EndForm Helpers

No ASP.NET MVC, há um conjunto de HTML Helpers para facilitar a criação de formulários nas páginas de uma aplicação web.

Para criar um formulário, utilizamos o helper `BeginForm`. Para terminar um formulário, utilizamos o helper `EndForm`. Veja o exemplo a seguir.

```
1 @{Html.BeginForm();}
2
3 <!-- elementos do formulário -->
4
5 @{Html.EndForm();}
```

Código CSHTML 5.22: Utilizando os helpers `BeginForm` *e* `EndForm`

Também podemos utilizar o comando `using` para garantir que os formulários sejam fechados. Nesse caso, não é necessário adicionar o helper `EndForm`. Observe o formulário abaixo.

```
1 @using(Html.BeginForm()) {
2
3 <!-- elementos de formulário -->
4
5 }
```

Código CSHTML 5.23: Utilizando o comando `using`

Por padrão, um formulário criado com o helper `BeginForm` enviará requisições à ação associada à url atual. Contudo, podemos definir explicitamente uma outra ação para receber essas requisições. Veja um exemplo no código abaixo.

```
1 @using(Html.BeginForm("ACTION", "CONTROLADOR")) {
2
3 <!-- elementos de formulário -->
4
5 }
```

Código CSHTML 5.24: Definindo uma ação e um controlador explicitamente

Por padrão, os formulários gerados pelo helper `BeginForm` fazem requisições do tipo `POST`. Nesse caso, as ações associadas a esses formulários devem ser anotadas com `HttpPost` para indicar o tipo de requisição HTTP esperado.

```
1  public class K19Controller : Controller
2  {
3    // POST: /K19/Home
4    [HttpPost]
5    public ActionResult Home()
6    {
7      ...
8      return View();
9    }
10 }
```

Código C# 5.9: Anotando uma ação com `HttpPost`

CheckBox

Podemos adicionar um checkbox em um formulário através do helper `CheckBox`.

```
1 @Html.CheckBox("Aceito", false)
```

Código CSHTML 5.25: Utilizando o helper `CheckBox`

O código acima produz o seguinte trecho de código HTML:

```
1 <input id="Aceito" name="Aceito" type="checkbox" value="true" />
2 <input name="Aceito" type="hidden" value="false" />
```

Código HTML 5.4: HTML gerado pelo helper `CheckBox`

Considere o formulário a seguir:

```
1 @using(Html.BeginForm("Cadastra", "Contrato"))
2 {
3   @Html.CheckBox("Aceito", false)
4   <input type="submit" value="Cadastra Contrato"/>
5 }
```

Código CSHTML 5.26: Formulário

Agora, considere a ação associada ao formulário criado no código acima.

```
1  public class ContratoController : Controller
2  {
3    // POST: /Contrato/Cadastra
4    [HttpPost]
5    public ActionResult Cadastra(Contrato contrato)
6    {
7      ...
8      return View();
9    }
10 }
```

Código C# 5.10: Ação

O valor enviado através do checkbox inserido no formulário será armazenado na propriedade `Aceito` do contrato enviado como parâmetro para a ação `Cadastra()`. Dessa forma, a classe `Contrato` deve possuir uma propriedade booleana chamada `Aceito`.

```
1 public class Contrato
2 {
```

```
3   public Boolean Aceito { get; set; }
4 }
```

Código C# 5.11: Contrato.cs

TextBox

Uma caixa de texto pode ser adicionada através do helper `TextBox`.

```
1 @Html.TextBox("Nome", "Digite o seu nome")
```

Código CSHTML 5.27: Utilizando o helper `TextBox`

O código acima produz o seguinte trecho de código HTML:

```
1 <input id="Nome" name="Nome" type="text" value="Digite o seu nome" />
```

Código HTML 5.5: HTML gerado pelo helper `TextBox`

TextArea

Uma caixa para textos maiores pode ser adicionada através do helper `TextArea`.

```
1 @Html.TextArea("Mensagem", "Digite uma mensagem")
```

Código CSHTML 5.28: Utilizando o helper `TextArea`

O código acima produz o seguinte trecho de código HTML:

```
1 <textarea cols="20" id="Mensagem" name="Mensagem" rows="2">Digite uma mensagem</↩
      textarea>
```

Código HTML 5.6: HTML gerado pelo helper `TextArea`

RadioButton

Um botão do tipo *radio* pode ser adicionado através do helper `RadioButton`.

```
1 @Html.RadioButton("CidadeNatal", "São Paulo", true)
2 @Html.RadioButton("CidadeNatal", "Natal", false)
3 @Html.RadioButton("CidadeNatal", "Piracicaba", false)
```

Código CSHTML 5.29: Utilizando o helper `RadioButton`

O código acima produz o seguinte trecho de código HTML:

```
1 <input checked="checked" id="CidadeNatal" name="CidadeNatal" type="radio" value="São ↩
      Paulo" />
2 <input id="CidadeNatal" name="CidadeNatal" type="radio" value="Natal" />
3 <input id="CidadeNatal" name="CidadeNatal" type="radio" value="Piracicaba" />
```

Código HTML 5.7: HTML gerado pelo helper `RadioButton`

Hidden

Um campo escondido pode ser adicionado através do helper `Hidden`.

```
1 @Html.Hidden("Id", 1)
```

Código CSHTML 5.30: Utilizando o helper `Hidden`

O código acima produz o seguinte trecho de código HTML:

```
1 <input id="Id" name="Id" type="hidden" value="1" />
```

Código HTML 5.8: HTML gerado pelo helper `Hidden`

Password

Uma caixa para senha pode ser adicionada através do helper `Password`.

```
1 @Html.Password("Password", "senha")
```

Código CSHTML 5.31: Utilizando o helper `Password`

O código acima produz o seguinte trecho de código HTML:

```
1 <input id="Password" name="Password" type="password" value="senha" />
```

Código HTML 5.9: HTML gerado pelo helper `Password`



Mais Sobre

Os dados preenchidos nos elementos de um formulário são recuperados da `ViewBag`. Considere o código abaixo:

```
1 @Html.TextBox("Nome")
```

Código CSHTML 5.32: Utilizando o helper `TextBox`

Caso exista uma chave chamada `Nome` na `ViewBag`, então o valor associado a essa chave será o valor inicial da caixa de texto. Podemos alterar esse comportamento, passando um segundo parâmetro para o helper `TextBox`, que será o valor inicial da caixa de texto. Observe o código a seguir.

```
1 @Html.TextBox("Nome", "NomeInicial")
```

Código CSHTML 5.33: Utilizando o helper `TextBox`

Strongly Typed Helpers

Se os HTML Helpers forem utilizados de maneira análoga à mostrada anteriormente, a probabilidade de ocorrer um erro de digitação é alta. A forma que os HTML Helpers foram aplicados não permite que o compilador verifique a existência das propriedades associadas aos elementos dos formulários. Por exemplo, considere o código a seguir.

```
1 @Html.CheckBox("Aceito", false)
```

Código CSHTML 5.34: Utilizando o helper `CheckBox`

O código anterior supõe que o objeto recebido como parâmetro pela ação associada ao formulário onde o checkbox foi inserido possua uma propriedade chamada `Aceito`. Essa propriedade pode não existir. Contudo, nenhum erro de compilação seria gerado nesse caso.

Para evitar esse tipo de problema, podemos utilizar a seguinte sintaxe em telas fortemente tipadas:

```
1 @model K19.Models.Contrato
2 ...
3 @Html.CheckBoxFor(c => c.Aceito)
4 ...
```

Código CSHTML 5.35: Utilizando o helper `CheckBoxFor`

Com essa sintaxe, o compilador tem condições de verificar a existência das propriedades. Sendo assim, ele produzirá um erro de compilação se uma propriedade inexistente for utilizada.

TextBoxFor

```
1 @model K19.Models.Empresa
2 ...
3 @Html.TextBoxFor(x => x.Nome)
4 ...
```

Código CSHTML 5.36: Utilizando o helper `TextBoxFor`

O código acima produz o seguinte trecho de código HTML:

```
1 <input id="Nome" name="Nome" type="text" value="K19 Treinamentos" />
```

Código HTML 5.10: HTML gerado pelo helper `TextBoxFor`

TextAreaFor

```
1 @model K19.Models.Empresa
2 ...
3 @Html.TextAreaFor(x => x.Descricao)
4 ...
```

Código CSHTML 5.37: Utilizando o helper `TextAreaFor`

O código acima produz o seguinte trecho de código HTML:

```
1 <textarea cols="20" id="Descricao" name="Descricao" rows="2">
2   K19 Treinamentos em Java e .NET
3 </textarea>
```

Código HTML 5.11: HTML gerado pelo helper `TextAreaFor`

RadioButtonFor

```
@model K19.Models.Pessoa
...
@Html.RadioButtonFor(x => x.CidadeNatal, "São Paulo")
@Html.RadioButtonFor(x => x.CidadeNatal, "Natal")
@Html.RadioButtonFor(x => x.CidadeNatal, "Piracicaba")
...
```

Código CSHTML 5.38: Utilizando o helper `RadioButtonFor`

O código acima produz o seguinte trecho de código HTML:

```
<input checked="checked" id="CidadeNatal" name="CidadeNatal" type="radio" value="São Paulo" />
<input id="CidadeNatal" name="CidadeNatal" type="radio" value="Natal"/>
<input id="CidadeNatal" name="CidadeNatal" type="radio" value="Piracicaba"/>
```

Código HTML 5.12: HTML gerado pelo helper `RadioButtonFor`

HiddenFor

```
@model K19.Models.Empresa
...
@Html.HiddenFor(e => e.Id)
...
```

Código CSHTML 5.39: Utilizando o helper `HiddenFor`

O código acima produz o seguinte trecho de código HTML:

```
<input id="Id" name="Id" type="hidden" value="1" />
```

Código HTML 5.13: HTML gerado pelo helper `HiddenFor`

PasswordFor

```
@model K19.Models.Usuario
...
@Html.PasswordFor(x => x.Senha)
...
```

Código CSHTML 5.40: Utilizando o helper `PasswordFor`

O código acima produz o seguinte trecho de código HTML:

```
<input id="Senha" name="Senha" type="password"/>
```

Código HTML 5.14: HTML gerado pelo helper `PasswordFor`



Mais Sobre

Os dados preenchidos nos elementos de um formulário de uma tela fortemente tipada são recuperados da propriedade `Model`. Considere o código abaixo:

```
@model K19.Models.Pessoa
...
@Html.TextBoxFor(x => x.Nome)
...
```

Código CSHTML 5.41: Utilizando o helper `TextBoxFor`

Caso exista a propriedade `Model.Nome`, o valor inicial dessa caixa de texto será o valor dessa propriedade.

DropDownList Helper

Considere uma aplicação para cadastrar livros de uma biblioteca. Quando um livro é cadastro, o usuário podem escolher a editora desse livro. A editora pode ser selecionada através de um *drop down list*. Para criar um drop down list, podemos utilizar o HTML Helper `DropDownList.cshtml`

O primeiro passo para utilizar o helper `DropDownList` é criar uma `SelectList` na camada de controle com as opções que o usuário poderá selecionar.

```
1 public class LivroController : Controller
2 {
3   public ActionResult Create()
4   {
5     List<Editora> editoras = editoraRepository.BuscaTodas();
6     ViewBag.Editoras = new SelectList(editoras,"Id","Nome");
7     return View();
8   }
9 }
```

Código C# 5.12: LivroController.cs

O segundo passo é adicionar o helper `DropDownList` na camada de apresentação.

```
1 @Html.DropDownList("Editoras")
```

Código CSHTML 5.42: Utilizando o helper `DropDownList`

EditorFor

Considere a seguinte entidade.

```
1 public class Editora
2 {
3   public int Id { get; set; }
4   public string Nome { get; set; }
5   public string Email { get; set; }
6   public bool IsAtivo { get; set; }
7   public string Descricao { get; set; }
8   public virtual ICollection<Livro> Livros { get; set; }
9 }
```

Código C# 5.13: Editora.cs

Para editar os dados de uma editora, temos uma página conforme o exemplo abaixo:

```
1 @model K19.Models.Editora
2
3 @{
4   ViewBag.Title = "Edição de Editora";
5 }
6
```

```
<h2>Edição de Editora</h2>

@using (Html.BeginForm()) {
  @Html.HiddenFor(model => model.Id)

  @Html.LabelFor(model => model.Nome)
  @Html.TextBoxFor(model => model.Nome)

  <br/>

  @Html.LabelFor(model => model.Email)
  @Html.TextBoxFor(model => model.Email)

  <br/>

  @Html.LabelFor(model => model.Descricao)
  @Html.TextAreaFor(model => model.Descricao)

  <br/>

  @Html.LabelFor(model => model.IsAtivo)
  @Html.CheckBoxFor(model => model.IsAtivo)

  <input type="submit" value="Salvar" />
}
```

Código CSHTML 5.43: Edit.cshtml

Para cada propriedade da entidade `Editora`, utilizamos um helper para gerar o código HTML necessário para a entrada de dados. Por exemplo, no caso das propriedades `Nome` e `Email`, utilizamos o Helper `TextBox`. Para a propriedade booleana `IsAtivo`, utilizamos o helper `CheckBox`.

Observe que a escolha do helper depende basicamente do tipo da propriedade associada a ele. Podemos deixar essa escolha a cargo do helper `EditorFor`. Para propriedades de tipo booleano, esse helper utilizará o `CheckBox`. Para propriedades do tipo `string`, esse helper utilizará o `TextBox`. Observe a utilização do helper `EditorFor` no código abaixo.

```
@model K19.Models.Editora

@{
  ViewBag.Title = "Edição de Editora";
}

<h2>Edição de Editora</h2>

@using (Html.BeginForm()) {
  @Html.HiddenFor(model => model.Id)

  @Html.LabelFor(model => model.Nome)
  @Html.EditorFor(model => model.Nome)

  <br/>

  @Html.LabelFor(model => model.Email)
  @Html.EditorFor(model => model.Email)

  <br/>

  @Html.LabelFor(model => model.Descricao)
  @Html.EditorFor(model => model.Descricao)

  <br/>

  @Html.LabelFor(model => model.IsAtivo)
  @Html.EditorFor(model => model.IsAtivo)

```

```
30   <input type="submit" value="Salvar" />
31 }
```

Código CSHTML 5.44: Edit.cshtml

EditorForModel

Também temos o helper `EditorForModel` para construir um formulário completo com base nas propriedades de uma entidade. Esse helper seguirá a mesma abordagem do `EditorFor`. Ou seja, para cada propriedade do `Model`, ele utilizará o helper apropriado.

```
1  @model K19.Models.Editora
2
3  @{
4    ViewBag.Title = "Edição de Editora";
5  }
6
7  <h2>Edição de Editora</h2>
8
9  @using (Html.BeginForm()) {
10   @Html.EditorForModel()
11
12   <input type="submit" value="Save" />
13 }
```

Código CSHTML 5.45: Edit.cshtml

Personalizando o EditorFor e o EditorForModel

Para personalizar o funcionamento do helper `EditorFor` e do `EditorForModel`, podemos utilizar algumas anotações. Veja o exemplo abaixo.

```
1  public class Usuario
2  {
3    [HiddenInput(DisplayValue = false)]
4    public int Id { get; set; }
5
6    [Display(Name="Nome do Usuário")]
7    public string Nome { get; set; }
8
9    [DataType(DataType.EmailAddress)]
10   public string Email { get; set; }
11
12   [DataType(DataType.Password)]
13   public string Senha { get; set; }
14
15   [DataType(DataType.MultilineText)]
16   public string Descricao { get; set; }
17
18   [DataType(DataType.Date)]
19   [DisplayFormat(ApplyFormatInEditMode = true, DataFormatString = "{0:yyyy-MM-dd}")]
20   public DateTime DataDeCadastro { get; set; }
21 }
```

Código C# 5.14: Usuario.cs



Exercícios de Fixação

12 Adicione uma ação no controlador **Aluno** com o seguinte código.

```
...
public ActionResult Editar()
{
  Aluno aluno = new Aluno
  {
    AlunoID = 1,
    Nome = "Jonas Hirata",
    Email = "jonas@k19.com.br"
  };
  return View(aluno);
}
...
```

Código C# 5.15: www.k19.com.br/exercicios/k32/05/012/AlunoController.cs

13 Adicione uma página associada à ação **Editar** do controlador **Aluno**.

```
@model CamadaDeApresentacao.Models.Aluno
@{
    ViewBag.Title = "Editar";
}

<h2>Editar</h2>

<form action="/Aluno/Editar" method="post">
  <input id="AlunoID" name="AlunoID" type="hidden" value="1" />

    <label for="Nome">Nome</label>
    <input id="Nome" name="Nome" type="text" value="@Model.Nome" />

    <label for="Email">E-mail</label>
    <input id="Email" name="Email" type="text" value="@Model.Email" />

    <input type="submit" value="Salvar" />
</form>
```

Código CSHTML 5.46: www.k19.com.br/exercicios/k32/05/013/Editar.cshtml

Para testar, acesse a url `http://localhost:<PORTA_APP>/Aluno/Editar`.

14 Altere o exercício anterior. Utilize HTML Helpers.

```
@model CamadaDeApresentacao.Models.Aluno
@{
    ViewBag.Title = "Editar";
}

<h2>Editar</h2>

@using (Html.BeginForm())
{
  @Html.Hidden("AlunoID")
  @Html.Label("Nome")
  @Html.TextBox("Nome")
  @Html.Label("Email")
  @Html.TextBox("Email")
  <input type="submit" value="Salvar" />
}
```

Código CSHTML 5.47: www.k19.com.br/exercicios/k32/05/014/Editar.cshtml

Para testar, acesse acesse a url http://localhost:<PORTA_APP>/Aluno/Editar.

15 Altere o exercício anterior e utilize strongly typed helpers.

```
@model CamadaDeApresentacao.Models.Aluno
@{
    ViewBag.Title = "Editar";
}

<h2>Editar</h2>

@using (Html.BeginForm())
{
  @Html.HiddenFor(x => x.AlunoID)
  @Html.LabelFor(x => x.Nome)
  @Html.TextBoxFor(x => x.Nome)
  @Html.LabelFor(x => x.Email)
  @Html.TextBoxFor(x => x.Email)
  <input type="submit" value="Salvar" />
}
```

Código CSHTML 5.48: www.k19.com.br/exercicios/k32/05/015/Editar.cshtml

Para testar, acesse a url http://localhost:<PORTA_APP>/Aluno/Editar.

16 Altere o exercício anterior e utilize o HTML Helper EditorFor.

```
@model CamadaDeApresentacao.Models.Aluno
@{
    ViewBag.Title = "Editar";
}

<h2>Editar</h2>

@using (Html.BeginForm())
{
  @Html.EditorFor(x => x.AlunoID)
  @Html.LabelFor(x => x.Nome)
  @Html.EditorFor(x => x.Nome)
  @Html.LabelFor(x => x.Email)
  @Html.EditorFor(x => x.Email)
  <input type="submit" value="Salvar" />
}
```

Código HTML 5.15: www.k19.com.br/exercicios/k32/05/016/Editar.cshtml

Para testar, acesse a url http://localhost:<PORTA_APP>/Aluno/Editar.

OBS: repare que o campo correspondente à propriedade `AlunoID` é exibido no formulário.

17 Altere a entidade `Aluno` para indicar que a propriedade `AlunoID` não deve ser exibida nos formulários que utilizarem o HTML Helper EditorFor e EditorForModel.

```
namespace CamadaDeApresentacao.Models
{
  public class Aluno
  {
    [HiddenInput(DisplayValue = false)]
    public int AlunoID { get; set; }

```

```
8       public string Nome { get; set; }
9
10      public string Email { get; set; }
11    }
12  }
```

Código C# 5.16: www.k19.com.br/exercicios/k32/05/017/Aluno.cs

Para testar, acesse a url http://localhost:<PORTA_APP>/Aluno/Editar.

OBS: repare que o campo correspondente à propriedade `AlunoID` não é exibido no formulário.

18 Altere o exercício anterior e utilize o HTML Helper EditorForModel.

```
1  @model CamadaDeApresentacao.Models.Aluno
2  @{
3      ViewBag.Title = "Editar";
4  }
5
6  <h2>Editar</h2>
7
8  @using (Html.BeginForm())
9  {
10     @Html.EditorForModel()
11   <input type="submit" value="Salvar" />
12 }
```

Código HTML 5.16: www.k19.com.br/exercicios/k32/05/018/Editar.cshtml

Para testar, acesse a url http://localhost:<PORTA_APP>/Aluno/Editar.

19 Adicione uma nova entidade chamada **Usuario** na pasta **Models** do projeto **CamadaDeApresentacao**.

```
1  namespace CamadaDeApresentacao.Models
2  {
3    public class Usuario
4    {
5      [HiddenInput(DisplayValue = false)]
6      public int Id { get; set; }
7
8      [Display(Name="Nome do Usuário")]
9      public string Nome { get; set; }
10
11     [DataType(DataType.EmailAddress)]
12     public string Email { get; set; }
13
14     [DataType(DataType.Password)]
15     public string Senha { get; set; }
16
17     [DataType(DataType.MultilineText)]
18     public string Descricao { get; set; }
19
20     [DataType(DataType.Date)]
21     [DisplayFormat(ApplyFormatInEditMode = true, DataFormatString = "{0:yyyy-MM-dd}")↩
           ]
22     public DateTime DataDeCadastro { get; set; }
23   }
24 }
```

Código C# 5.17: www.k19.com.br/exercicios/k32/05/019/Usuario.cs

20 Adicione um novo controlador chamado **Usuario**.

```
namespace CamadaDeApresentacao.Controllers
{
  public class UsuarioController : Controller
  {
    public ActionResult Editar()
    {
      Usuario usuario = new Usuario
      {
        Id = 1,
        Nome = "Rafael Cosentino",
        Email = "rafael@k19.com.br",
        Senha = "123",
        Descricao = "Sócio-fundador da K19 / Instrutor",
        DataDeCadastro = DateTime.Now
      };
      return View(usuario);
    }
  }
}
```

Código C# 5.18: www.k19.com.br/exercicios/k32/05/020/UsuarioController.cs

21 Adicione uma página associada à ação **Editar** do controlador **Usuario**.

```
@model CamadaDeApresentacao.Models.Usuario
@{
    ViewBag.Title = "Editar";
}

<h2>Editar</h2>

@using (Html.BeginForm())
{
    @Html.EditorForModel()
  <input type="submit" value="Salvar" />
}
```

Código CSHTML 5.49: www.k19.com.br/exercicios/k32/05/021/Editar.cshtml



Layouts

É comum que as páginas de uma aplicação web possuam conteúdo em comum (por exemplo, um cabeçalho ou um rodapé). O conteúdo em comum pode ser replicado em todas as páginas através do CTRL+C e CTRL+V. Porém, essa não é uma boa abordagem, pois quando alguma alteração precisa ser realizada, todos os arquivos devem ser modificados. Também é comum que as páginas de uma aplicação web possuam um certo padrão visual. Daí surge o conceito de **Layouts**.

Conteúdo comum

Tudo que é comum a todas as páginas de um determinado grupo pode ser definido em um Layout. Dessa forma, qualquer alteração é facilmente realizada modificando-se apenas um arquivo. Por exemplo, suponha que toda página de uma aplicação web deva ter o mesmo título e a mesma formatação. Podemos criar um Layout com o título desejado e com a referência ao arquivo CSS que define a formatação padrão.

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewBag.Title</title>
  @Styles.Render("~/Content/css")
  @Scripts.Render("~/bundles/modernizr")
</head>

<body>
  <div id="header">
    @Html.ActionLink("Editoras", "Index", "Editora")
    @Html.ActionLink("Livros", "Index","Livro")
  </div>
  @RenderBody()

  @Scripts.Render("~/bundles/jquery")
  @Scripts.Render("~/bundles/bootstrap")
  @RenderSection("scripts", required: false)
</body>
</html>
```

Código CSHTML 5.50: K19Layout.cshtml

No layout definido no código acima, utilizamos o método `Render()` das propriedades `Styles` e `Scripts` para adicionar conteúdo CSS e JavaScript. Na tag `<title>`, acrescentamos `@ViewBag.Title`, para que cada página possa definir o seu próprio título. Por fim, o método `RenderBody()` indica onde o conteúdo definido em cada página será adicionado.

É recomendado que os arquivos que definam layouts sejam colocados na pasta `Views\Shared`.



Mais Sobre

Suponha que devemos adicionar diversos arquivos CSS à uma página de uma aplicação ASP.NET MVC. Se esses arquivos forem adicionados um a um da maneira tradicional (com a tag `<link>`), o número de requisições que os navegadores terão de realizar para carregar essa página será alto, afetando o desempenho. Para contornar esse problema, poderíamos agrupar o código CSS de todos os arquivos em apenas um. Contudo, essa abordagem dificultaria a manutenção do código CSS. Dessa forma, o ASP.NET MVC possui um mecanismo capaz de agrupar o conteúdo CSS de diversos arquivos dinamicamente. Para usar esse mecanismo, basta invocar o método `Render()` da propriedade `Styles`.

```
@Styles.Render("~/Content/Arquivo1.css", "~/Content/Arquivo2.css", "~/Content/↩
    Arquivo3.css")
```

Código CSHTML 5.51: Utilizando o método `Render` *para adicionar conteúdo CSS*

Analogamente, a propriedade `Scripts` possui o método `Render()`, que também tem por objetivo agrupar o código JavaScript de diversos arquivos.

Com o layout definido, o próximo passo é indicar quais páginas utilizarão esse layout. Por exemplo, podemos atualizar a página de edição de editoras para utilizar `K19Layout.cshtml` como layout.

```
@model K19.Models.Editora

@{
  Layout = "~/Views/Shared/K19Layout.cshtml";

```

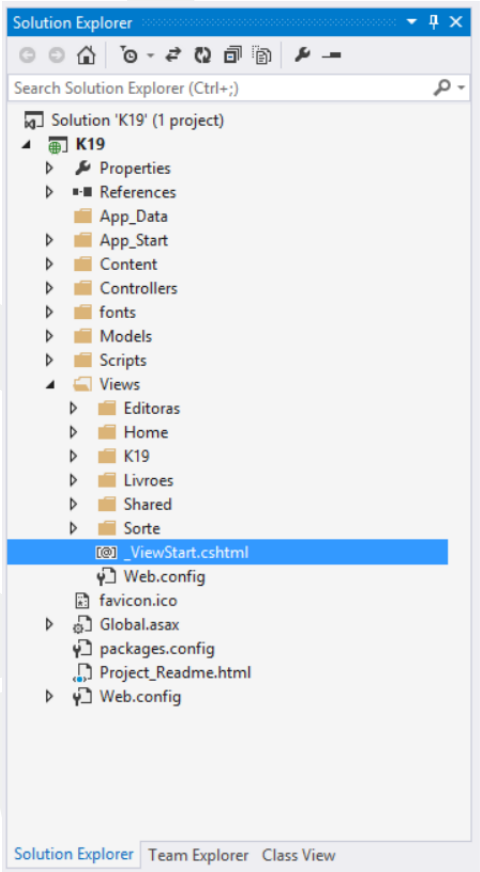
```
6   // Define o título específico desta página
7   ViewBag.Title = "Edição de Editora";
8 }
9
10 <h2>Edição de Editora</h2>
11
12 @using(Html.BeginForm()) {
13   @Html.EditorForModel()
14   <input type="submit" value="Salvar" />
15 }
```

Código CSHTML 5.52: Edit.cshtml

Quando a página de edição de editoras é requisitada, o arquivo `Edit.cshtml` é processado antes do arquivo `K19Layout.cshtml`, o que permite definir o valor de `ViewBag.Title` no arquivo `Edit.cshtml` e utilizá-lo no layout.

Para definir o layout de `Edit.cshtml`, foi necessário armazenar o caminho completo do arquivo que define o layout na propriedade `Layout`. Este procedimento não é muito prático, pois em cada página que deseja utilizar esse layout é necessário definir esta propriedade.

A partir da terceira versão do ASP.NET MVC, temos uma nova funcionalidade que permite definir um layout padrão para todas as páginas, não havendo necessidade de definir a propriedade `Layout` em cada página. Para isso, basta acrescentarmos o arquivo `_ViewStart.cshtml` à pasta `Views`:



O `_ViewStart.cshtml` permite definirmos um código que será executado antes de cada página ser renderizada. Nesse arquivo podemos definir, por exemplo, a propriedade `Layout`:

```
@{
  Layout = "~/Views/Shared/K19Layout.cshtml";
}
```

Código CSHTML 5.53: _ViewStart.cshtml

Como este código é processado antes dos arquivos específicos de cada página, não há mais necessidade de definir a propriedade `Layout` em cada arquivo específico.

Lacunas

Também podemos criar seções em um layout para serem preenchidas com conteúdos específicos definidos nas páginas. Considere o seguinte layout.

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewBag.Title</title>
  @Styles.Render("~/Content/css")
  @Scripts.Render("~/bundles/modernizr")
</head>

<body>
  <div id="header">
    @Html.ActionLink("Editoras", "Index", "Editora")
    @Html.ActionLink("Livros", "Index","Livro")
  </div>
  <div id="sidebar">@RenderSection("Sidebar",false)</div>
  <div id="content">@RenderBody()</div>
  <div id="footer">K19 Treinamentos</div>
  @Scripts.Render("~/bundles/jquery")
  @Scripts.Render("~/bundles/bootstrap")
  @RenderSection("scripts", required: false)
</body>
</html>
```

Código CSHTML 5.54: K19Layout.cshtml

No código acima, utilizamos o método `@RenderSection()` para criar uma seção no layout. O primeiro parâmetro ("Sidebar") é o nome da seção e o segundo é um booleano que indica se o preenchimento dessa seção é obrigatório ou não.

Para definir o conteúdo de uma seção, devemos utilizar o código *@section*. Observe o código de uma página que utiliza o layout criado anteriormente e define a seção `Sidebar`.

```
@model K19.Models.Editora

@{
  Layout = "~/Views/Shared/K19Layout.cshtml";

  // Define o título específico desta página
  ViewBag.Title = "Edição de Editora";
}

<h2>Edição de Editora</h2>

@using(Html.BeginForm()) {
  @Html.EditorForModel()
  <input type="submit" value="Salvar" />
}

@section Sidebar {
```

```
18    <p>Sidebar do cadastro de Edição de Editora</p>
19 }
```

Código CSHTML 5.55: Edit.cshtml



Exercícios de Fixação

22 Crie um arquivo chamado **K19Layout.cshtml** dentro da pasta **Shared** que fica na pasta **Views** do projeto **CamadaDeApresentacao**. Este arquivo servirá de modelo para as páginas da nossa aplicação.

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8" />
5      <meta name="viewport" content="width=device-width" />
6      <title>@ViewBag.Title</title>
7      @Styles.Render("~/Content/css")
8      @Scripts.Render("~/bundles/modernizr")
9  </head>
10 <body>
11     <header>
12         <img src="http://www.k19.com.br/css/img/main-header-logo.png"
13         alt="Logo vertical da K19" title="K19 Treinamentos" />
14         <span>K19 Treinamentos</span>
15     </header>
16
17     <section id="main">
18         @RenderBody()
19     </section>
20
21     <footer>&copy;K19 Treinamentos</footer>
22     @Scripts.Render("~/bundles/jquery")
23     @Scripts.Render("~/bundles/bootstrap")
24     @RenderSection("scripts", required: false)
25 </body>
26 </html>
```

Código CSHTML 5.56: www.k19.com.br/exercicios/k32/05/022/K19Layout.cshtml

23 Altere a página **Lista.cshtml** da pasta **Views/Aluno** para que ela utilize o layout definido no exercício anterior.

```
1  @model ICollection<CamadaDeApresentacao.Models.Aluno>
2
3  @{
4      ViewBag.Title = "Lista";
5      Layout = "~/Views/Shared/K19Layout.cshtml";
6  }
7
8  <h2>Tabela de Aluno</h2>
9
10 <table>
11     <tr>
12         <th>AlunoID</th>
13         <th>Nome</th>
14         <th>Email</th>
15     </tr>
16
17     @foreach (var a in @Model)
18     {
```

```
19          <tr>
20              <td>@a.AlunoID</td>
21              <td>@a.Nome</td>
22              <td>@a.Email</td>
23          </tr>
24      }
25  </table>
```

Código HTML 5.17: www.k19.com.br/exercicios/k32/05/023/Lista.cshtml

Teste acessando o endereço: http://localhost:<PORTA_APP>/Aluno/Lista.

24 Defina a página **K19Layout.cshtml** como layout padrão para todas as telas. Altere o arquivo **_ViewStart.cshtml** da pasta **Views**.

```
1  @{
2    Layout = "~/Views/Shared/K19Layout.cshtml";
3  }
```

Código CSHTML 5.57: www.k19.com.br/exercicios/k32/05/024/_ViewStart.cshtml

Teste acessando as páginas da aplicação.

- http://localhost:<PORTA_APP>/Home/Index
- http://localhost:<PORTA_APP>/Aluno/Detalhes
- http://localhost:<PORTA_APP>/Aluno/Editar
- http://localhost:<PORTA_APP>/Aluno/Lista
- http://localhost:<PORTA_APP>/Usuario/Editar
- http://localhost:<PORTA_APP>/Relogio/Agora

25 Altere o arquivo **Site.css** da pasta **Content** para aplicar algumas regras de formatação às nossas páginas.

```
1  ...
2  header,
3  #main,
4  footer {
5      width: 980px;
6      margin: 0 auto;
7  }
8
9  header {
10     border-bottom: 1px solid #666666;
11     padding: 0 0 10px 0;
12 }
13
14 header span {
15     font-size: 20px;
16     vertical-align: middle;
17 }
18
19 header img {
20     height: 80px;
21     margin: 0 20px 0 0;
22     vertical-align: middle;
23 }
24
```

```
25 footer {
26     border-top: 1px solid #666666;
27     padding: 10px 0 0 0;
28     margin-top: 20px;
29     text-align: center;
30     font-size: 10px;
31 }
32
33 #toolbar {
34     background: #999999;
35     width: 970px;
36     margin: 1px auto;
37     padding: 3px 5px;
38 }
39
40 #toolbar * {
41     margin: 3px 5px;
42 }
```

Código CSS 5.1: www.k19.com.br/exercicios/k32/05/025/Site.css

Teste acessando as páginas da aplicação (Dica: utilize o comando CTRL+F5 para garantir que o navegador solicite o novo conteúdo do arquivo `Site.css`).

- http://localhost:<PORTA_APP>/Home/Index
- http://localhost:<PORTA_APP>/Aluno/Detalhes
- http://localhost:<PORTA_APP>/Aluno/Editar
- http://localhost:<PORTA_APP>/Aluno/Lista
- http://localhost:<PORTA_APP>/Usuario/Editar
- http://localhost:<PORTA_APP>/Relogio/Agora

26 Altere o arquivo **K19Layout.cshtml** da pasta **Views\Shared**.

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8" />
5      <meta name="viewport" content="width=device-width" />
6      <title>@ViewBag.Title</title>
7      @Styles.Render("~/Content/css")
8      @Scripts.Render("~/bundles/modernizr")
9  </head>
10 <body>
11     <header>
12         <img src="http://www.k19.com.br/css/img/main-header-logo.png"
13         alt="Logo vertical da K19" title="K19 Treinamentos" />
14         <span>K19 Treinamentos</span>
15     </header>
16     <div id="toolbar">
17         @RenderSection("Toolbar", required: false)
18     </div>
19
20     <section id="main">
21         @RenderBody()
22     </section>
23
24     <footer>&copy;K19 Treinamentos</footer>
25     @Scripts.Render("~/bundles/jquery")
26     @Scripts.Render("~/bundles/bootstrap")
27     @RenderSection("scripts", required: false)
28 </body>
```

```
</html>
```

Código CSHTML 5.58: www.k19.com.br/exercicios/k32/05/026/K19Layout.cshtml

27 Defina a seção **Toolbar** nas seguintes páginas.

```
@model CamadaDeApresentacao.Models.Aluno
@{
    ViewBag.Title = "Editar";
}

@section Toolbar {
    <input type="button" value="Remover este cadastro" />
}

<h2>Editar</h2>

@using (Html.BeginForm())
{
    @Html.EditorForModel()
  <input type="submit" value="Enviar" />
}
```

Código CSHTML 5.59: www.k19.com.br/exercicios/k32/05/027/Editar.cshtml

```
@model CamadaDeApresentacao.Models.Aluno

@{
    ViewBag.Title = "Detalhes";
}

@section Toolbar {
    <input type="button" value="Remover este cadastro" />
}

<h2>Detalhes</h2>

<p>
    AlunoID: @Model.AlunoID
</p>

<p>
    Nome: @Model.Nome
</p>

<p>
    Email: @Model.Email
</p>
```

Código CSHTML 5.60: www.k19.com.br/exercicios/k32/05/027/Detalhes.cshtml

```
@model ICollection<CamadaDeApresentacao.Models.Aluno>

@{
    ViewBag.Title = "Lista";
    Layout = "~/Views/Shared/K19Layout.cshtml";
}

@section Toolbar {
  <input type="search" placeholder="Digite o nome do aluno" />
  <input type="button" value="Buscar" />
}

<h2>Tabela de Aluno</h2>

```

```
<table>
    <tr>
        <th>AlunoID</th>
        <th>Nome</th>
        <th>Email</th>
    </tr>

    @foreach (var a in @Model)
    {
        <tr>
            <td>@a.AlunoID</td>
            <td>@a.Nome</td>
            <td>@a.Email</td>
        </tr>
    }
</table>
```

Código CSHTML 5.61: www.k19.com.br/exercicios/k32/05/027/Lista.cshtml

Teste acessando as páginas da aplicação.

- http://localhost:<PORTA_APP>/Home/Index
- http://localhost:<PORTA_APP>/Aluno/Detalhes
- http://localhost:<PORTA_APP>/Aluno/Editar
- http://localhost:<PORTA_APP>/Aluno/Lista
- http://localhost:<PORTA_APP>/Usuario/Editar
- http://localhost:<PORTA_APP>/Relogio/Agora

28 Defina um conteúdo padrão para a seção **Toolbar** no layout.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width" />
    <title>@ViewBag.Title</title>
    @Styles.Render("~/Content/css")
    @Scripts.Render("~/bundles/modernizr")
</head>
<body>
    <header>
        <img src="http://www.k19.com.br/css/img/main-header-logo.png"
        alt="Logo vertical da K19" title="K19 Treinamentos" />
        <span>K19 Treinamentos</span>
    </header>
    <div id="toolbar">
        @if (IsSectionDefined("Toolbar"))
        {
            @RenderSection("Toolbar", required: false);
        }
        else
        {
            <input type="button"
            value="Veja o seu número da sorte"
            onclick="location.href='/Home/Index'" />
        }
    </div>

    <section id="main">
        @RenderBody()
    </section>
```

```
32
33      <footer>&copy;K19 Treinamentos</footer>
34      @Scripts.Render("~/bundles/jquery")
35      @Scripts.Render("~/bundles/bootstrap")
36      @RenderSection("scripts", required: false)
37  </body>
38  </html>
```

Código CSHTML 5.62: www.k19.com.br/exercicios/k32/05/028/K19Layout.cshtml

Teste acessando as páginas da aplicação.

- http://localhost:<PORTA_APP>/Home/Index
- http://localhost:<PORTA_APP>/Aluno/Detalhes
- http://localhost:<PORTA_APP>/Aluno/Editar
- http://localhost:<PORTA_APP>/Aluno/Lista
- http://localhost:<PORTA_APP>/Usuario/Editar
- http://localhost:<PORTA_APP>/Relogio/Agora



Partial views

Quanto mais elaborada é uma página de uma aplicação web, mais extenso é o seu código. Códigos muito extensos prejudicam a legibilidade e a manutenção da aplicação. Para organizar melhor o código, podemos dividir o conteúdo de uma página web em vários arquivos.

Suponha que desejamos dividir o conteúdo de uma página em duas partes. Devemos criar um arquivo para cada parte.

```
1  <h1> Parte 1 </h>
2  <p> Conteúdo da parte1</p>
```

Código CSHTML 5.63: _Parte1.cshtml

```
1  <h1> Parte 2 </h>
2  <p> Conteúdo da parte2</p>
```

Código CSHTML 5.64: _Parte2.cshtml



Mais Sobre
Por convenção, o nome dos arquivos que definem as telas parciais de uma aplicação ASP.NET MVC devem iniciar com o caractere "_".

Por fim, devemos criar um arquivo principal para agrupar essas partes. Utilizaremos o método `Partial()` para inserir o conteúdo dos arquivos secundários no arquivo principal.

```
1  <html>
2      <head>
3          <title>Exemplo de partial</title>
4      </head>
5
```

```
6       <body>
7           Html.Partial("_Parte1")
8           Html.Partial("_Parte2")
9       </body>
10  </html>
```

Código CSHTML 5.65: Principal.cshtml

O método `Partial()` procura os arquivos `_Parte1.cshtml` e `_Parte2.cshtml` no mesmo diretório do arquivo principal. Caso ele não encontre, ele continua procurando esses arquivos na pasta `Views\Shared`. Essa lógica também é aplicada no método *View* que utilizamos na camada de controle.

O recurso de páginas parciais permite a criação de conteúdo reutilizável de forma mais clara e concisa. Informações podem ser compartilhadas entre as páginas principais e as páginas parciais através da propriedade `ViewBag`. Por exemplo, podemos definir duas páginas principais, uma para criar e outra para editar editoras. Nessas duas páginas podemos acrescentar uma página parcial com o formulário que pode ser utilizado para cadastrar novas editoras ou editar editoras existentes.

Veja abaixo o exemplo da página parcial que contém o formulário de cadastro ou edição de editoras.

```
1   <!-- ~/Views/Editora/_Form.cshtml -->
2   @model K19.Models.Editora
3
4   @using (Html.BeginForm()) {
5     @Html.ValidationSummary(true)
6     <fieldset>
7       <legend>Editora</legend>
8       @if (Model != null)
9       {
10        @Html.EditorFor(model => model.Id)
11      }
12
13      <div class="editor-label">
14        @Html.LabelFor(model => model.Nome)
15      </div>
16      <div class="editor-field">
17        @Html.EditorFor(model => model.Nome)
18      </div>
19
20      <div class="editor-label">
21        @Html.LabelFor(model => model.Email)
22      </div>
23      <div class="editor-field">
24        @Html.EditorFor(model => model.Email)
25      </div>
26
27      <div class="editor-label">
28        @Html.LabelFor(model => model.IsAtivo)
29      </div>
30      <div class="editor-field">
31        @Html.EditorFor(model => model.IsAtivo)
32      </div>
33
34      <p>
35        <input type="submit" value="Save" />
36      </p>
37    </fieldset>
38    @section Scripts {
39      @Scripts.Render("~/bundles/jqueryval")
40    }
41  }
```

Código CSHTML 5.66: _Form.cshtml

Veja abaixo o exemplo das páginas principais que utilizam a página parcial definida no código acima.

```
<!-- ~/Views/Editora/Create.cshtml -->
@model K19.Models.Editora

@{
  ViewBag.Title = "Cadastro de Editora";
}

<h2>Cadastro de Editora</h2>

@Html.Partial("_Form")
```

Código CSHTML 5.67: Create.cshtml

```
<!-- ~/Views/Editora/Edit.cshtml -->
@model LivrariaVirtual.Models.Editora

@{
  ViewBag.Title = "Edição de Editora";
}

<h2>Edição de Editora</h2>

@Html.Partial("_Form")
```

Código CSHTML 5.68: Edit.cshtml



Exercícios de Fixação

29 Adicione uma nova entidade chamada **Post** na pata **Models**.

```
namespace CamadaDeApresentacao.Models
{
  public class Post
  {
    public string Autor { get; set; }

    public DateTime Data { get; set; }

    public string Titulo { get; set; }

    public string Categoria { get; set; }

    public string Texto { get; set; }
  }
}
```

Código C# 5.19: www.k19.com.br/exercicios/k32/05/029/Post.cs

30 Adicione um novo controlador chamado **Post** utilizando o template **Empty MVC Controller**.

```
namespace CamadaDeApresentacao.Controllers
{
  public class PostController : Controller
  {
    public ActionResult Lista()
    {
      ICollection<Post> posts = new List<Post>();

      for (int i = 0; i < 10; i++)
      {
        posts.Add(new Post
        {
          Autor = "Autor do post " + i,
          Categoria = "Categoria " + i,
          Data = DateTime.Now.Subtract(TimeSpan.FromDays(i)),
          Texto = i + " - Lorem ipsum dolor sit amet, consectetur adipiscing elit.",
          Titulo = "Título do post " + i
        });
      }
      return View(posts);
    }
  }
}
```

Código C# 5.20: www.k19.com.br/exercicios/k32/05/030/PostController.cs

31 Adicione uma página associada à ação **Lista** do controlador **Post**.

```
@model IEnumerable<CamadaDeApresentacao.Models.Post>
@{
    ViewBag.Title = "Lista";
}
<h2>Lista</h2>

@foreach (var post in Model) {
    @Html.Partial("_Post", post)
}
```

Código CSHTML 5.69: www.k19.com.br/exercicios/k32/05/031/Lista.cshtml

32 Adicione uma partial para exibir as informações de um Post. Crie um arquivo chamado **_Post.cshtml** na pasta **Views/Post**

```
@model CamadaDeApresentacao.Models.Post

<article>
    <header>
        <h1>@Model.Titulo</h1>
        <div><small>@Model.Autor - @Model.Data.ToString("dd/MM/yyyy")</small></div>
    </header>
    <p>@Model.Texto</p>
    <footer>
        Categoria: @Model.Categoria
    </footer>
</article>
```

Código CSHTML 5.70: www.k19.com.br/exercicios/k32/05/032/_Post.cshtml

Para testar, acesse a página `http://localhost:<PORTA_APP>/Post/Lista`

CAPÍTULO 6

CAMADA DE CONTROLE

No ASP.NET MVC as urls são mapeadas para métodos (ações) em classes que definem os chamadas `controladores`. As requisições enviadas pelos navegadores são processadas pelos controladores. O processamento realizado por um controlador para tratar uma requisição consiste basicamente em:

- Recuperar os dados enviados pelo usuário através de formulários.
- Interagir com a camada de modelo.
- Acionar a camada de apresentação para construir a página HTML que deve ser enviada para o usuário como resposta à sua requisição.

Para que uma classe seja considerada um controlador, ela deve seguir algumas regras.

- O nome da classe deve ter o sufixo "Controller".
- A classe deve implementar a interface `System.Web.Mvc.IController` ou herdar da classe `System.Web.Mvc.Controller`.

Raramente, você definirá uma classe para criar um controlador implementando a interface `IController`. Comumente as classes que definem controladores derivam de `Controller`. Diversas propriedade e métodos são herdados da classe `Controller`. Essas propriedades facilitam o processamento das requisições e a utilização dos recursos do ASP.NET MVC.



Actions

Os controladores e as ações são elementos fundamentais de uma aplicação ASP.NET MVC. Um controlador pode conter diversas ações. As ações são utilizadas para processar as requisições realizadas pelos navegadores. Para criar uma ação, é necessário definir um método `public` dentro da classe de um controlador. Os parâmetros desse método podem ser utilizados para receber os dados enviados pelos usuários através de formulários HTML. Esse método deve devolver um `ActionResult` que será utilizado pelo ASP.NET MVC para definir o que deve ser executado depois que a ação terminar.

Quando um usuário faz uma requisição HTTP através de um navegador, o ASP.NET MVC verifica na tabela de rotas o controlador e a ação associados à url da requisição realizada. Essa tabela é definida no arquivo `RouteConfig.cs` e inicializada no arquivo `Global.asax`.

Por padrão, quando criamos um projeto ASP.NET MVC no Visual Studio, uma rota com o formato `{controller}/{action}/{id}` é adicionada na tabela de rotas. Com essa rota, se uma requisição for

realizada para a url `http://www.k19.com.br/Editora/Listagem`, o controlador definido pela classe `EditoraController` e a ação implementada pelo método `Listagem()` dessa classe serão escolhidos para processar essa requisição.

Se uma requisição for realizada para a url `http://www.k19.com.br/Editora/Remove/1`, o controlador definido pela classe `EditoraController` e a ação implementada pelo método `Remove()` dessa classe serão escolhidos para processar a requisição realizada. Além disso, o valor `1` será passado como parâmetro para o método `Remove()`. Veremos mais sobre rotas no Capítulo **??**.



ActionResult

Quando uma ação termina, o método correspondente deve devolver um `ActionResult`. O valor devolvido indica para o ASP.NET MVC o que deve ser executado depois da ação. Veja abaixo uma lista com alguns tipos específicos de `ActionResult` que podem ser utilizados.

ViewResult: Devolve uma página da camada de apresentação. Considere o seguinte exemplo.

```
1 public class TesteController
2 {
3   public ActionResult Acao()
4   {
5     return View();
6   }
7 }
```

Considerando uma aplicação ASP.NET MVC em C# e Razor, o método `View()`, ao ser chamado sem parâmetros, executará o seguinte processo para determinar qual arquivo deve ser utilizado para construir a página de resposta.

1. Se o arquivo `Views\Teste\Acao.cshtml` existir, ele será utilizado.
2. Caso contrário, se o arquivo `Views\Shared\Acao.cshtml` existir, ele será utilizado.
3. Se nenhum desses arquivos existir, uma página de erro será devolvida.

Por outro lado, podemos especificar o nome do arquivo que define a página de resposta. Veja o exemplo abaixo.

```
1 return View("NomeDaView");
```

Além disso, podemos passar um objeto para a camada de apresentação. Esse objeto será utilizado nas páginas fortemente tipadas como vimos no Capítulo 5.

```
1 return View(editora);
```

Também é possível especificar o nome do arquivo que define a página de resposta e o objeto que deve ser transmitido para a camada de apresentação ao mesmo tempo.

```
1 return View("NomeDaView", editora);
```

PartialViewResult: Devolve uma página parcial da camada de apresentação. Exemplos:

```
return PartialView();
```

O método `PartialView()` utiliza a mesma abordagem do método `View()` para determinar o arquivo que deve ser utilizado para construir a página parcial de resposta.

```
return PartialView("NomeDaPartialView", editora);
```

RedirectResult: Redireciona o navegador para uma URL específica. Exemplo:

```
return Redirect("http://www.k19.com.br");
```

RedirectToAction: Redireciona para outra ação da camada de controle. Exemplo:

```
return RedirectToAction("OutraAction", "OutroController");
```

ContentResult: Devolve texto. Exemplo:

```
return Content("Texto","text\plain");
```

JsonResult: Devolve um objeto no formato JSON. Exemplo:

```
return Json(editora);
```

JavaScriptResult: Devolve código Javascript. Exemplo:

```
return JavaScript("\$('\#divResultText').html('JavaScript Passed');");
```

FileResult: Devolve dados binários. Exemplo:

```
return File(@"c:\relatorio.pdf", "application\pdf");
```

EmptyResult: Devolve uma resposta vazia. Exemplo:

```
return new EmptyResult();
```



Parâmetros

Os parâmetros enviados pelos usuários através de formulários HTML podem ser recuperados por meio da propriedade `Request`.

```
string nome = Request["Nome"];
```

Código C# 6.14: Recuperando o parâmetro `Nome` *enviado em uma requisição HTTP*

Esses parâmetros também podem ser recuperados através dos parâmetros da ação responsável pelo processamento da requisição HTTP realizada. Para isso, basta definir um parâmetro C# para cada parâmetro HTTP com o mesmo nome. Veja o exemplo abaixo:

```
<html>
<head>
  <title>Cadastro de Editora</title>
</head>
<body>
  <form action="/Editoras/Salva" method="post">
    Nome: <input type="text" name="nome"/>
    Email: <input type="text" name="email"/>
    <input type="submit" />
  </form>
</body>
</html>
```

Código CSHTML 6.1: Cadastra.csthml

No código acima, criamos um formulário HTML com os parâmetros `nome` e `email`. Esses parâmetros serão recuperados na ação `Salva` definida no código abaixo.

```
...
[HttpPost]
public ActionResult Salva(string nome, string email)
{
  Editora editora = new Editora { Nome = nome, Email = email };
  db.Editoras.Add(editora);
  return View();
}
...
```

Código C# 6.15: EditoraController.cs

O ASP.NET MVC também é capaz de montar objetos com os valores dos parâmetros HTTP enviados pelo usuário e passá-los como argumento para as ações dos controladores.

```
[HttpPost]
public ActionResult Salva(Editora editora)
{
  db.Editoras.Add(editora);
  return View();
}
```

Código C# 6.16: EditoraController.cs

As propriedades dos objetos recebidos como argumentos nas ações dos controladores precisam ter os mesmos nomes dos parâmetros HTTP ignorando-se letras maiúsculas ou minúsculas.



Exercícios de Fixação

1. Crie um projeto do tipo **ASP.NET Web Application** chamado **CamadaDeControle** no **Visual Studio**. Utilize o template **MVC**.

2. Adicione uma entidade chamada **Produto** na pasta **Models** do projeto **CamadaDeControle**.

```
namespace CamadaDeControle.Models
{
  public class Produto
  {
```

```
5       [HiddenInput(DisplayValue=false)]
6       public int ProdutoID { get; set; }
7
8       [Display(Name="Nome do Produto")]
9       public string Nome { get; set; }
10
11      [DataType(DataType.MultilineText)]
12      [Display(Name="Descrição do Produto")]
13      public string Descricao { get; set; }
14
15      [Display(Name="Preço do Produto")]
16      public double Preco { get; set; }
17    }
18  }
```

Código C# 6.17: www.k19.com.br/exercicios/k32/06/002/Produto.cs

3 Adicione um DbContext para registrar a entidade **Produto** e utilizar os recursos do Entity Framework. Crie uma classe chamada **K19Context** na pasta **Models**.

```
1  namespace CamadaDeControle.Models
2  {
3    public class K19Context : DbContext
4    {
5      public DbSet<Produto> Produtos { get; set; }
6    }
7  }
```

Código C# 6.18: www.k19.com.br/exercicios/k32/06/003/K19Context.cs

4 Adicione um controlador chamado **Produto** utilizando o template **Empty MVC Controller**.

```
1  namespace CamadaDeControle.Controllers
2  {
3    public class ProdutoController : Controller
4    {
5      public ActionResult Lista()
6      {
7        K19Context ctx = new K19Context();
8        return View(ctx.Produtos);
9      }
10
11     [HttpGet]
12     public ActionResult Cadastra()
13     {
14       return View();
15     }
16
17     [HttpPost]
18     public ActionResult Cadastra(string nome, string descricao, double preco)
19     {
20       Produto p = new Produto
21       {
22         Nome = nome,
23         Descricao = descricao,
24         Preco = preco
25       };
26
27       K19Context ctx = new K19Context();
28       ctx.Produtos.Add(p);
29       ctx.SaveChanges();
30
31       return RedirectToAction("Lista");
32     }
```

```
33    }
34 }
```

Código C# 6.19: www.k19.com.br/exercicios/k32/06/004/ProdutoController.cs

5 Adicione uma tela associada à ação **Lista** do controlador **Produto** com o seguinte conteúdo.

```
1  @using CamadaDeControle.Models
2  @model IEnumerable<Produto>
3
4  @{
5      ViewBag.Title = "Lista";
6  }
7
8  <h2>Lista</h2>
9
10 <table>
11   <tr>
12     <th>@Html.DisplayNameFor(m => m.Nome)</th>
13     <th>@Html.DisplayNameFor(m => m.Descricao)</th>
14     <th>@Html.DisplayNameFor(m => m.Preco)</th>
15   </tr>
16 @foreach(var produto in Model)
17 {
18   <tr>
19     <td>@Html.DisplayFor(m => produto.Nome)</td>
20     <td>@Html.DisplayFor(m => produto.Descricao)</td>
21     <td>@Html.DisplayFor(m => produto.Preco)</td>
22   </tr>
23 }
24 </table>
```

Código HTML 6.1: www.k19.com.br/exercicios/k32/06/005/Lista.cshtml

6 Adicione uma tela associada à ação **Cadastra** do controlador **Produto** com o seguinte conteúdo.

```
1  @model CamadaDeControle.Models.Produto
2
3  @{
4    ViewBag.Title = "Cadastra";
5  }
6
7  <h2>Cadastra</h2>
8
9  @using (Html.BeginForm())
10 {
11   @Html.EditorFor(m => m.ProdutoID)
12   <div>@Html.LabelFor(m => m.Nome)</div>
13   @Html.EditorFor(m => m.Nome)
14   <div>@Html.LabelFor(m => m.Descricao)</div>
15   @Html.EditorFor(m => m.Descricao)
16   <div>@Html.LabelFor(m => m.Preco)</div>
17   @Html.EditorFor(m => m.Preco)
18
19   <div><input type="submit" value="Cadastrar" /></div>
20 }
```

Código HTML 6.2: www.k19.com.br/exercicios/k32/06/006/Cadastra.cshtml

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Produto/Cadastra`.

7 Altere a ação **Cadastra** do controlador **Produto**.

```
namespace CamadaDeControle.Controllers
{
  public class ProdutoController : Controller
  {
    public ActionResult Lista()
    {
      K19Context ctx = new K19Context();
      return View(ctx.Produtos);
    }

    [HttpGet]
    public ActionResult Cadastra()
    {
      return View();
    }

    [HttpPost]
    public ActionResult Cadastra(Produto p)
    {
      K19Context ctx = new K19Context();
      ctx.Produtos.Add(p);
      ctx.SaveChanges();

      return RedirectToAction("Lista");
    }
  }
}
```

Código C# 6.20: www.k19.com.br/exercicios/k32/06/007/ProdutoController.cs

Para testar, acesse a url `http://localhost:<PORTA_APP>/Produto/Cadastra`.



TempData

Ao efetuar um redirecionamento, uma nova requisição é realizada pelo navegador. Nesta nova requisição, não temos mais acesso aos dados e objetos da requisição anterior ao redirecionamento. Caso haja a necessidade de preservar dados ao longo do redirecionamento, podemos utilizar a propriedade `TempData`.

Por exemplo, ao cadastrar uma editora, podemos efetuar um redirecionamento para a tela de listagem de editoras. Na propriedade `TempData`, podemos acrescentar uma mensagem indicando o eventual sucesso da operação. Veja o exemplo abaixo.

```
...
[HttpPost]
public ActionResult Salva(Editora editora)
{
    db.Editoras.Add(editora);
    db.SaveChanges();
    TempData["Mensagem"] = "Editora cadastrada com sucesso!";
    return RedirectToAction("Index");
}
...
```

Código C# 6.21: EditoraController.cs

Na camada de apresentação, podemos recuperar os dados armazenados na propriedade `TempData`. Veja o código a seguir:

```
@if (TempData["Mensagem"] != null)
{
    <p>@TempData["Mensagem"]</p>
}
```

Código CSHTML 6.2: Recuperando dados armazenados na propriedade `TempData`



Exercícios de Fixação

8 Altere a ação **Cadastra** do controlador **Produto**.

```
...
public ActionResult Cadastra(Produto p)
{
  K19Context ctx = new K19Context();
  ctx.Produtos.Add(p);
  ctx.SaveChanges();

  ViewBag.Mensagem = "Produto cadastrado com sucesso!";

  return RedirectToAction("Lista");
}
...
```

Código C# 6.22: www.k19.com.br/exercicios/k32/06/008/ProdutoController.cs

9 Modifique o arquivo **Lista.cshtml** da pasta **Views/Produto**.

```
@using CamadaDeControle.Models
@model IEnumerable<Produto>

@{
    ViewBag.Title = "Lista";
}

<h2>Lista</h2>

@if (ViewBag.Mensagem != null)
{
  <div>@ViewBag.Mensagem</div>
}

<table>
  <tr>
    <th>@Html.DisplayNameFor(m => m.Nome)</th>
    <th>@Html.DisplayNameFor(m => m.Descricao)</th>
    <th>@Html.DisplayNameFor(m => m.Preco)</th>
  </tr>
@foreach(var produto in Model)
{
  <tr>
    <td>@Html.DisplayFor(m => produto.Nome)</td>
    <td>@Html.DisplayFor(m => produto.Descricao)</td>
    <td>@Html.DisplayFor(m => produto.Preco)</td>
  </tr>
}
</table>
```

Código CSHTML 6.3: www.k19.com.br/exercicios/k32/06/009/Lista.cshtml

Teste a aplicação acessando a url http://localhost:<PORTA_APP>/Produto/Cadastra. Verifique que a mensagem de sucesso **não** aparecerá.

10 Altere novamente a ação **Cadastra** do controlador **Produto**.

```
...
public ActionResult Cadastra(Produto p)
{
  K19Context ctx = new K19Context();
  ctx.Produtos.Add(p);
  ctx.SaveChanges();

  TempData["Mensagem"] = "Produto cadastrado com sucesso!";

  return RedirectToAction("Lista");
}
...
```

Código C# 6.23: www.k19.com.br/exercicios/k32/06/010/ProdutoController.cs

11 Modifique o arquivo **Lista.cshtml** da pasta **Views/Produto**.

```
@using CamadaDeControle.Models
@model IEnumerable<Produto>

@{
    ViewBag.Title = "Lista";
}

<h2>Lista</h2>

@if (TempData["Mensagem"] != null)
{
  <div>@TempData["Mensagem"]</div>
}

<table>
  <tr>
    <th>@Html.DisplayNameFor(m => m.Nome)</th>
    <th>@Html.DisplayNameFor(m => m.Descricao)</th>
    <th>@Html.DisplayNameFor(m => m.Preco)</th>
  </tr>
@foreach(var produto in Model)
{
  <tr>
    <td>@Html.DisplayFor(m => produto.Nome)</td>
    <td>@Html.DisplayFor(m => produto.Descricao)</td>
    <td>@Html.DisplayFor(m => produto.Preco)</td>
  </tr>
}
</table>
```

Código CSHTML 6.4: www.k19.com.br/exercicios/k32/06/011/Lista.cshtml

Teste a aplicação acessando a url http://localhost:<PORTA_APP>/Produto/Cadastra. Verifique que a mensagem de sucesso aparecerá.



Rotas

Para acessar uma determinada ação da nossa aplicação, os usuários devem realizar uma requisição HTTP utilizando a url correspondente à ação. Por exemplo, para acessar a listagem de editoras, é necessário digitar na barra de endereço do navegador a url http://localhost:<PORTA_APP>/Editora/Index. Perceba que o o nome do controlador e o nome da ação desejados fazem parte da url. O formato dessas urls é definido por **rotas** criadas no arquivo RouteConfig.cs e carregadas no arquivo Global.asax. O código abaixo mostra a rota padrão inserida nos projetos ASP.NET MVC.

```
routes.MapRoute(
  name: "Default",
  url: "{controller}/{action}/{id}",
  defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
);
```

Código C# 6.24: RouteConfig.cs

O primeiro argumento do método MapRoute é o nome da rota, o segundo é a expressão que define o formato da rota e o terceiro é o conjunto de valores padrão dos parâmetros da rota.

A expressão que determina o formato da rota Default utiliza três parâmetros: controller, action e id. Dessa forma, se o usuário realizar uma requisição HTTP utilizando url http://localhost:<PORTA_APP>/Editora/Remove/1, o ASP.NET MVC criará uma instância do controlador Editora e executará o método Remove() passando o valor 1 como argumento.

Basicamente, as rotas associam urls e ações. Veja alguns exemplos de como as urls serão processadas de acordo com as regras de rotas.

URL	Mapeamento da URL
/	controller = "Home", action = "Index"
/Livro	controller = "Livro", action = "Index"
/Livro/Adiciona	controller = "Livro", action = "Adiciona"
/Livro/Remove/1	controller = "Livro", action = "Remove", id = 1

Adicionando uma rota

Para acrescentar uma rota, podemos utilizar o método MapRoute(). Suponha que a listagem de editoras deva ser acessada através da url http://localhost:<PORTA_APP>/Catalogo. Nesse caso, podemos adicionar uma rota no arquivo RoutesConfig.cs, como mostrado abaixo.

```
...
routes.MapRoute(
  name: "Nova Rota",
  url: "Catalogo",
  defaults: new { controller = "Editora", action = "Index" }
);
...
```

Código C# 6.25: RoutesConfig.cs



Importante
As rotas são processadas na ordem em que foram inseridas. Portanto, é importante definir as rotas mais específicas antes das rotas mais genéricas.

Adicionando Parâmetros nas Rotas

Podemos acrescentar parâmetros às rotas. Por exemplo, na rota criada anteriormente para a listagem de editoras, poderíamos adicionar um parâmetro para limitar a quantidade de editoras listadas.

```
routes.MapRoute(
  name: "Nova Rota",
  url: "Catalogo/{maximo}",
  defaults: new { controller = "Editora", action = "Index" }
);
```

Código C# 6.26: RoutesConfig.cs

Por padrão, os parâmetros adicionados a uma rota são obrigatórios. Dessa forma, no exemplo acima, a listagem de editoras só poderá ser acessada se o valor do parâmetro `maximo` for definido na url da requisição. Por exemplo, `http://localhost:<PORTA_APP>/Catalogo/20`. Se uma requisição for realizada para a url `http://localhost:<PORTA_APP>/Catalogo`, um erro ocorrerá. Para mudar esse comportamento, podemos tornar o parâmetro `maximo` opcional. Veja o código a seguir.

```
...
routes.MapRoute(
  name: "Nova Rota",
  url: "Catalogo/{maximo}",
  defaults: new { controller = "Editora", action = "Index",
    maximo = UrlParameter.Optional }
);
...
```

Código C# 6.27: RoutesConfig.cs

Os parâmetros das rotas podem ser recuperados nas ações. Observe o exemplo abaixo.

```
...
public ActionResult Index(int? maximo)
{
  // implementação
}
...
```

Código C# 6.28: EditoraController.cs

Ao definir parâmetros opcionais, devemos utilizar parâmetros do tipo `nullable type` nas ações, pois quando o parâmetro não estiver definido na url de uma requisição, o valor `null` será atribuído ao parâmetro da ação. Por exemplo, para os tipos `int` e `double`, devemos utilizar `int?` e `double?`, respectivamente.



Exercícios de Fixação

12 Altere a rota padrão do projeto **CamadaDeControle**. Para isso, modifique o arquivo **RouteConfig.cs** da pasta **App_Start**.

```
...
routes.MapRoute(
  name: "Default",
```

```
4   url: "{controller}/{action}/{id}",
5   defaults: new {
6     controller = "Produto",
7     action = "Lista",
8     id = UrlParameter.Optional
9   }
10 );
11 ...
```

Código C# 6.29: www.k19.com.br/exercicios/k32/06/012/RouteConfig.cs

Teste essa nova configuração acessando a url http://localhost:<PORTA_APP>/.

13 Adicione uma rota no projeto **CamadaDeControle**. Para isso, modifique o arquivo **RouteConfig.cs** da pasta **App_Start**.

```
1 ...
2 routes.MapRoute(
3   name: "ProdutoListaPorPreco",
4   url: "Produto/Lista/{PrecoMinimo}/{PrecoMaximo}",
5   defaults: new { controller = "Produto", action = "Lista" }
6 );
7 ...
```

Código C# 6.30: www.k19.com.br/exercicios/k32/06/013/RouteConfig.cs

OBS: A rota **ProdutoListaPorPreco** deve ser adicionada antes da rota **Default**.

14 Altere a ação **Lista** do controlador **Produto**.

```
1  ...
2  public ActionResult Lista(double? precoMinimo, double? precoMaximo)
3  {
4    K19Context ctx = new K19Context();
5    var produtos = ctx.Produtos.AsEnumerable();
6
7    if (precoMinimo != null && precoMaximo != null)
8    {
9      produtos = from p in produtos
10       where p.Preco >= precoMinimo & p.Preco <= precoMaximo
11       select p;
12   }
13
14   return View(produtos);
15 }
16 ...
```

Código C# 6.31: www.k19.com.br/exercicios/k32/06/014/ProdutoController.cs

Teste a aplicação acessando a url http://localhost:<PORTA_APP>/Produto/Lista/0/200. Altere os valores 0 e 200 para obter listas de produtos diferentes.

15 Adicione duas ações no controlador **Produto** para implementar a edição de produtos.

```
1 ...
2 public ActionResult Editar(int id = 0)
3 {
4   K19Context ctx = new K19Context();
5   Produto p = ctx.Produtos.Find(id);
```

```
6
7     if (p == null)
8     {
9       return HttpNotFound();
10    }
11
12    return View(p);
13  }
14
15  [HttpPost]
16  public ActionResult Editar(Produto p)
17  {
18    K19Context ctx = new K19Context();
19    ctx.Entry(p).State = System.Data.Entity.EntityState.Modified;
20
21    ctx.SaveChanges();
22
23    return RedirectToAction("Lista");
24  }
25  ...
```

Código C# 6.32: www.k19.com.br/exercicios/k32/06/015/ProdutoController.cs

16 Modifique a página de listagem de produtos. Para isso, altere o arquivo **Lista.cshtml** da pasta **Views/Produto**.

```
1  @using CamadaDeControle.Models
2  @model IEnumerable<Produto>
3
4  @{
5      ViewBag.Title = "Lista";
6  }
7
8  <h2>Lista</h2>
9
10 @if (TempData["Mensagem"] != null)
11 {
12   <div>@TempData["Mensagem"]</div>
13 }
14
15 <table>
16   <tr>
17     <th>@Html.DisplayNameFor(m => m.Nome)</th>
18     <th>@Html.DisplayNameFor(m => m.Descricao)</th>
19     <th>@Html.DisplayNameFor(m => m.Preco)</th>
20     <th></th>
21   </tr>
22 @foreach(var produto in Model)
23 {
24   <tr>
25     <td>@Html.DisplayFor(m => produto.Nome)</td>
26     <td>@Html.DisplayFor(m => produto.Descricao)</td>
27     <td>@Html.DisplayFor(m => produto.Preco)</td>
28     <td>@Html.ActionLink("Editar", "Editar", new { id = produto.ProdutoID })</td>
29   </tr>
30 }
31 </table>
```

Código C# 6.33: www.k19.com.br/exercicios/k32/06/016/Lista.cshtml

17 Adicione uma página associada à ação **Editar** do controlador **Produto**.

```
1  @model CamadaDeControle.Models.Produto
2
```

```
3  @{
4      ViewBag.Title = "Editar";
5  }
6
7  <h2>Editar</h2>
8
9  @using (Html.BeginForm())
10 {
11   @Html.EditorFor(m => m.ProdutoID)
12   <div>@Html.LabelFor(m => m.Nome)</div>
13   @Html.EditorFor(m => m.Nome)
14   <div>@Html.LabelFor(m => m.Descricao)</div>
15   @Html.EditorFor(m => m.Descricao)
16   <div>@Html.LabelFor(m => m.Preco)</div>
17   @Html.EditorFor(m => m.Preco)
18
19   <div><input type="submit" value="Editar" /></div>
20 }
```

Código C# 6.34: www.k19.com.br/exercicios/k32/06/017/Editar.cshtml

Teste a aplicação acessando a url `http://localhost:<PORTA_APP>/`. Altere os dados de alguns produtos.

VALIDAÇÃO

CAPÍTULO 7

Os usuários podem cometer erros ao preencher um formulário. Por exemplo, esquecer de preencher um campo que é obrigatório. Os parâmetros enviados pelos usuários devem ser validados pela aplicação com o intuito de não permitir o armazenamento de informações erradas.



Controller

O primeiro passo para implementar a validação dos parâmetros enviados através de formulários HTML é definir a lógica de validação na camada de controle.

```
1 if (editora.Nome == null || editora.Nome.Trim().Length == 0)
2 {
3     // Erro de Validação
4 }
```

Código C# 7.1: Definindo as regras de validação

O segundo passo é definir mensagens informativas para enviar aos usuários. O ASP.NET MVC possui um objeto especializado no armazenamento de mensagens de erros de validação. Esse objeto pode ser acessado através da propriedade `ModelState`.

```
1 if (editora.Nome == null || editora.Nome.Trim().Length == 0)
2 {
3   ModelState.AddModelError("Nome", "O campo Nome é obrigatório");
4 }
```

Código C# 7.2: Definindo as mensagens de erro de validação

As mensagens são armazenadas no `ModelState` através do método `AddModelError()`. Esse método permite que as mensagens sejam agrupadas logicamente, pois ele possui dois parâmetros: o primeiro é o grupo da mensagem e o segundo e a mensagem propriamente.

O código de validação pode ser colocado nos controladores, mais especificamente nas ações. Se algum erro for encontrado, o fluxo de execução pode ser redirecionado para uma página que mostre as mensagens informativas aos usuários. Normalmente, essa página é a mesma do formulário que foi preenchido incorretamente. O `ModelState` possui uma propriedade que indica se erros foram adicionados ou não. Essa propriedade chama-se `IsValid`.

```
1 [HttpPost]
2 public ActionResult Salva(Editora editora)
3 {
4     if (editora.Nome == null || editora.Nome.Trim().Length == 0)
5     {
6         ModelState.AddModelError("Nome", "O campo Nome é obrigatório.");
7     }
8     if (ModelState.IsValid)
9     {
```

```
10          db.Editoras.Add(editora);
11          TempData["mensagem"] = "A editora foi cadastrada com sucesso!";
12          return RedirectToAction("Index");
13      }
14      else
15      {
16          return View("Cadastra");
17      }
18  }
```

Código C# 7.3: EditoraController.cs

O ASP.NET MVC também pode adicionar mensagens no `ModelState` antes do controlador ser chamado. Normalmente, essas mensagens estão relacionadas a erros de conversão. Por exemplo, um campo que espera um número é preenchido com letras.



View

As mensagens de erros de validação podem ser acrescentadas em uma página através do método `ValidationSummary()` da propriedade `Html`. É importante salientar que esse método adiciona todas as mensagens de erro.

```
1  @model K19.Models.Editora
2
3  @{
4    ViewBag.Title = "Cadastro de Editora";
5  }
6
7  <h2>Cadastro de Editora</h2>
8
9  @using (Html.BeginForm()) {
10   @Html.ValidationSummary()
11
12   <br/>
13
14   @Html.LabelFor(model => model.Nome)
15   @Html.EditorFor(model => model.Nome)
16
17   <br/>
18
19   @Html.LabelFor(model => model.Email)
20   @Html.EditorFor(model => model.Email)
21
22   <br/>
23
24   <input type="submit" value="Enviar" />
25 }
```

Código CSHTML 7.1: Cadastra.cshtml

Podemos utilizar o método `ValidationMessageFor()` para mostrar somente as mensagens de erro de validação de um determinado grupo.

Para não mostrar erros dos grupos com o `ValidationSummary()`, devemos passar como parâmetro o valor `true`. Nesse caso, apenas os erros que não estão associados a um grupo específico serão apresentados pelo `ValidationSummary()`.

```
1  @model LivrariaVirtual.Models.Editora
2
3  @{
```

```
4   ViewBag.Title = "Cadastro de Editora";
5 }
6
7 <h2>Cadastro de Editora</h2>
8
9 @using (Html.BeginForm()) {
10  @Html.ValidationSummary(true)
11
12  <br/>
13
14  @Html.LabelFor(model => model.Nome)
15  @Html.EditorFor(model => model.Nome)
16  @Html.ValidationMessageFor(model => model.Nome)
17
18  <br/>
19
20  @Html.LabelFor(model => model.Email)
21  @Html.EditorFor(model => model.Email)
22  @Html.ValidationMessageFor(model => model.Email)
23
24  <br/>
25
26  <input type="submit" value="Enviar" />
27 }
```

Código CSHTML 7.2: Cadastra.cshtml



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **Validacao** no **Visual Studio**. Utilize o template **MVC**.

2 Adicione uma entidade chamada **Jogador** na pasta **Models** do projeto **Validacao**.

```
1  namespace Validacao.Models
2  {
3    public class Jogador
4    {
5      [HiddenInput(DisplayValue=false)]
6      public int JogadorID { get; set; }
7
8      public string Nome { get; set; }
9
10     public int? Numero { get; set; }
11
12     public double? Altura { get; set; }
13   }
14 }
```

Código C# 7.4: www.k19.com.br/exercicios/k32/07/002/Jogador.cs

3 Adicione um DbContext para registrar a entidade **Jogador** e utilizar os recursos do Entity Framework. Crie uma classe chamada **K19Context** na pasta **Models**.

```
1  namespace Validacao.Models
2  {
3    public class K19Context : DbContext
4    {
```

```
5       public DbSet<Jogador> Jogadores { get; set; }
6     }
7   }
```

Código C# 7.5: www.k19.com.br/exercicios/k32/07/003/K19Context.cs

4 Adicione um controlador chamado **Jogador** utilizando o template **Empty MVC Controller**.

```
1   namespace Validacao.Controllers
2   {
3     public class JogadorController : Controller
4     {
5       public ActionResult Lista()
6       {
7         K19Context ctx = new K19Context();
8         return View(ctx.Jogadores);
9       }
10
11      [HttpGet]
12      public ActionResult Cadastra()
13      {
14        return View();
15      }
16
17      [HttpPost]
18      public ActionResult Cadastra(Jogador j)
19      {
20        K19Context ctx = new K19Context();
21        ctx.Jogadores.Add(j);
22        ctx.SaveChanges();
23
24        return RedirectToAction("Lista");
25      }
26    }
27  }
```

Código C# 7.6: www.k19.com.br/exercicios/k32/07/004/JogadorController.cs

5 Adicione uma tela associada à ação **Lista** do controlador **Jogador** com o seguinte conteúdo.

```
1   @using Validacao.Models
2   @model IEnumerable<Jogador>
3
4   @{
5       ViewBag.Title = "Lista";
6   }
7
8   <h2>Lista</h2>
9
10  <table>
11    <tr>
12      <th>@Html.DisplayNameFor(m => m.Nome)</th>
13      <th>@Html.DisplayNameFor(m => m.Numero)</th>
14      <th>@Html.DisplayNameFor(m => m.Altura)</th>
15    </tr>
16  @foreach(var jogador in Model)
17  {
18    <tr>
19      <td>@Html.DisplayFor(m => jogador.Nome)</td>
20      <td>@Html.DisplayFor(m => jogador.Numero)</td>
21      <td>@Html.DisplayFor(m => jogador.Altura)</td>
22    </tr>
23  }
24  </table>
```

Código HTML 7.1: www.k19.com.br/exercicios/k32/07/005/Lista.cshtml

6 Adicione uma tela associada à ação **Cadastra** do controlador **Jogador** com o seguinte conteúdo.

```
@model Validacao.Models.Jogador

@{
  ViewBag.Title = "Cadastra";
}

<h2>Cadastra</h2>

@using (Html.BeginForm())
{
  <div>@Html.LabelFor(m => m.Nome)</div>
  @Html.EditorFor(m => m.Nome)
  <div>@Html.LabelFor(m => m.Numero)</div>
  @Html.EditorFor(m => m.Numero)
  <div>@Html.LabelFor(m => m.Altura)</div>
  @Html.EditorFor(m => m.Altura)

  <div><input type="submit" value="Cadastrar" /></div>
}
```

Código HTML 7.2: www.k19.com.br/exercicios/k32/07/006/Cadastra.cshtml

Para ver o resultado, acesse a url http://localhost:<PORTA_APP>/Jogador/Cadastra.

7 Altere a ação **Cadastra** do controlador **Jogador**. Aplique algumas regras de validação.

```
...
[HttpPost]
public ActionResult Cadastra(Jogador j)
{
  if (String.IsNullOrEmpty(j.Nome))
  {
    ModelState.AddModelError("Nome",
      "O nome do jogador é obrigatório");
  }
  if (j.Numero == null || j.Numero <= 0 || j.Numero >= 100)
  {
    ModelState.AddModelError("Numero",
      "O número do jogador deve ser maior que 0 e menor que 100");
  }
  if (j.Altura == null || j.Altura < 0)
  {
    ModelState.AddModelError("Altura",
      "A altura do jogador não pode ser negativa");
  }

  if (ModelState.IsValid)
  {
    K19Context ctx = new K19Context();
    ctx.Jogadores.Add(j);
    ctx.SaveChanges();
    return RedirectToAction("Lista");
  }
  else
  {
    return View("Cadastra", j);
  }
}
```

```
33 ...
```

Código C# 7.7: www.k19.com.br/exercicios/k32/07/007/JogadorController.cs

8 Altere o formulário de cadastro de jogador. Para isso, modifique o arquivo **Cadastra.cshtml** da pasta **Views/Jogador**.

```
1  @model Validacao.Models.Jogador
2
3  @{
4    ViewBag.Title = "Cadastra";
5  }
6
7  <h2>Cadastra</h2>
8
9  @using (Html.BeginForm())
10 {
11     @Html.ValidationSummary(true)
12
13   <div>@Html.LabelFor(m => m.Nome)</div>
14   @Html.EditorFor(m => m.Nome)
15     @Html.ValidationMessageFor(m => m.Nome)
16
17   <div>@Html.LabelFor(m => m.Numero)</div>
18   @Html.EditorFor(m => m.Numero)
19     @Html.ValidationMessageFor(m => m.Numero)
20
21   <div>@Html.LabelFor(m => m.Altura)</div>
22   @Html.EditorFor(m => m.Altura)
23     @Html.ValidationMessageFor(m => m.Altura)
24
25   <div><input type="submit" value="Cadastrar" /></div>
26 }
```

Código CSHTML 7.3: www.k19.com.br/exercicios/k32/07/008/Cadastra.cshtml

Teste as validações cadastrando alguns jogadores através da url `http://localhost:<PORTA_APP>/Jogador/Cadastra`.



Anotações

As lógicas de validação também podem ser definidas através de anotações adicionadas nas classes de modelo. Dessa forma, essas lógicas não estariam mais nos controladores, o que conceitualmente é o ideal, pois nos controladores só deveria existir lógica para controlar o fluxo da execução.

Required

Uma das validações mais comuns é a de campo obrigatório. Ela pode ser realizada através da anotação `Required` para propriedades do tipo string.

```
1  public class Editora
2  {
3    [Required]
4    public string Nome { get; set;}
5
6    ...
7  }
```

Código C# 7.8: Editora.cs

Com essa anotação, a lógica de validação pode ser retirada do controlador `Editora`.

```
...
[HttpPost]
public ActionResult Salva(Editora editora)
{
    if (ModelState.IsValid)
    {
        db.Editoras.Add(editora);
        return RedirectToAction("Index");
    }
    else
    {
        return View("Cadastra", editora);
    }
}
...
```

Código C# 7.9: EditoraController.cs

Alterando a mensagem

As anotações de validação possuem mensagens padrão que podem ser alteradas através do atributo `ErrorMessage`.

```
...
[Required(ErrorMessage="O campo Nome é obrigatório")]
public string Nome { get; set; }
...
```

Outros validadores

Há outras anotações para validação:

- Range
- RequlatExpression
- StringLength

Validação no lado do Cliente

As validações podem ser realizadas também nos navegadores para melhorar a interação com os usuários.

Antes da terceira versão do ASP.NET MVC, era necessário habilitar a validação no lado do cliente através do método `Html.EnableClientValidation()`. A partir da terceira versão do ASP.NET MVC, a validação no cliente está habilitada por padrão.

Para que a validação no lado cliente funcione corretamente, devemos acrescentar as bibliotecas javascript necessárias. No ASP.NET MVC, basta acrescentar a seção de scripts nas páginas.

```
@section Scripts {
  @Scripts.Render("~/bundles/jqueryval")
}
```



Exercícios de Fixação

9 Altere a entidade **Jogador** acrescentando as anotações de validação.

```
namespace Validacao.Models
{
  public class Jogador
  {
    [HiddenInput(DisplayValue = false)]
    public int JogadorID { get; set; }

    [Required(ErrorMessage="O nome do jogador é obrigatório")]
    public string Nome { get; set; }

    [Required(ErrorMessage="O número do jogador é obrigatório")]
    [Range(1, 99,
    ErrorMessage="O número do jogador deve ser maior que 0 e menor que 100")]
    public int? Numero { get; set; }

    [Required(ErrorMessage="A altura do jogador é obrigatória")]
    [Range(0, double.MaxValue,
    ErrorMessage="A altura do jogador não pode ser negativa")]
    public double? Altura { get; set; }
  }
}
```

Código C# 7.11: www.k19.com.br/exercicios/k32/07/009/Jogador.cs

10 Altere a ação **Cadastra** do controlador **Jogador**.

```
...
[HttpPost]
public ActionResult Cadastra(Jogador j)
{
  if (ModelState.IsValid)
  {
    K19Context ctx = new K19Context();
    ctx.Jogadores.Add(j);
    ctx.SaveChanges();
    return RedirectToAction("Lista");
  }
  else
  {
    return View("Cadastra", j);
  }
}
...
```

Código C# 7.12: www.k19.com.br/exercicios/k32/07/010/JogadorController.cs

Importante: Apague a base de dados **Validacao.Models.K19Context**

Para ver o resultado, acesse a url `http://localhost:<PORTA_APP>/Jogador/Cadastra`.

11 Para adicionar a validação no lado do cliente, basta alterar a página de cadastro de jogadores. Observe o código abaixo.

```
@model Validacao.Models.Jogador

@{
  ViewBag.Title = "Cadastra";
}

<h2>Cadastra</h2>

@using (Html.BeginForm())
{
    @Html.ValidationSummary(true)

  <div>@Html.LabelFor(m => m.Nome)</div>
  @Html.EditorFor(m => m.Nome)
    @Html.ValidationMessageFor(m => m.Nome)

  <div>@Html.LabelFor(m => m.Numero)</div>
  @Html.EditorFor(m => m.Numero)
    @Html.ValidationMessageFor(m => m.Numero)

  <div>@Html.LabelFor(m => m.Altura)</div>
  @Html.EditorFor(m => m.Altura)
    @Html.ValidationMessageFor(m => m.Altura)

  <div><input type="submit" value="Cadastrar" /></div>

@section Scripts {
  @Scripts.Render("~/bundles/jqueryval")
}
}
```

Código CSHTML 7.5: www.k19.com.br/exercicios/k32/07/011/Cadastra.cshtml

Para testar, acesse a url http://localhost:<PORTA_APP>/Jogador/Cadastra.

Sessão

CAPÍTULO 8

Considere a aplicação de uma loja virtual. Nessa aplicação, os clientes selecionam os produtos desejados e os adiciona no seu carrinho de compra. Cada cliente deve ter o seu próprio carrinho para que os seus produtos não se misturem com os produtos selecionados por outros clientes. A aplicação deve armazenar o carrinho de um cliente até que a compra seja finalizada ou até ela ter certeza que o cliente não precisa mais do carrinho.

Para resolver esse problema, podemos utilizar o conceito de **Sessão**. Para cada navegador conectado, o servidor manterá uma sessão aberta. Dessa forma, podemos separar os dados de cada usuário conectado.



Identificando os navegadores

Para aplicar a ideia de Sessão, é necessário ter a capacidade de identificar o navegador que está requisitando a aplicação a cada requisição. Uma primeira abordagem seria utilizar o endereço IP da máquinas para identificar os navegadores. Porém, nessa abordagem, dois navegadores executando na mesma máquina não poderiam ser identificados individualmente.

Outra abordagem é deixar a cargo do servidor a criação de um identificador único para cada navegador conectado. Quando um navegador faz a primeira requisição para a aplicação, o servidor deve gerar um identificador único para esse navegador e enviá-lo na resposta HTTP. A partir da segunda requisição, os navegadores devem enviar para a aplicação o identificador recebido na primeira requisição. Desta maneira, a aplicação saberá qual é o navegador que está realizando uma requisição. Os navegadores podem enviar os seus respectivos identificadores de diferentes formas. As mais utilizadas são:

Reescrita de URL Nesta abordagem, os identificadores são embutidos nos links e botões das páginas da aplicação. Quando os links ou botões são clicados pelo usuário, o identificador é enviado para a aplicação. Uma desvantagem é que todas as páginas devem ser geradas dinamicamente para adicionar o identificador em todos os links e botões.

Cookies Cookies são arquivos contendo informações. Eles são gerados nos servidores e enviados para os navegadores. Os navegadores armazenam os cookies localmente na máquina do usuário. Além disso, os navegadores enviam os cookies de volta para o servidor em todas as requisições. Os servidores podem armazenar os identificadores gerados em cookies. Dessa forma, a cada requisição, o servidor receberá um cookie contendo o identificador.



Sessões no ASP.NET MVC

No ASP.NET MVC, o objeto que representa uma sessão é um dicionário. Para armazenar informações, você deve adicionar uma chave e um valor na propriedade `Session`. Considere um objeto da classe `Usuario` que agrupa as informações sobre um determinado usuário. O código a seguir é um exemplo de como podemos guardar esse objeto na sessão após a realização da autenticação do usuário.

```
1  public class LoginController : Controller
2  {
3    ...
4    public ActionResult Login(Cliente cliente)
5    {
6      ...
7      Session["Cliente"] = cliente;
8      ...
9    }
10 }
```

Código C# 8.1: LonginController.cs

Você pode adicionar qualquer tipo de valor na sessão. De forma análoga, para resgatar as informações armazenadas, basta acessar a chave correspondente da propriedade `Session`. Veja o exemplo a seguir:

```
1  Cliente cliente = (Cliente)Session["Cliente"];
2  string saudacao = "Bem vindo " + cliente.Nome;
```

Código C# 8.2: Recuperando informações da sessão

Quando um usuário deslogar, podemos eliminar a informação armazenada em sua sessão. Para isso, podemos simplesmente remover todas as chaves do dicionário como no exemplo a seguir.

```
1  Session.RemoveAll();
```

Código C# 8.3: Eliminando todas as informações da sessão

Contudo, o método `RemoveAll()` não elimina a sessão. Ele apenas remove os dados contidos na sessão. Para eliminá-la, você deve utilizar o método `Abandon()`. Observe o exemplo a seguir.

```
1  public class LoginController : Controller
2  {
3    ...
4    public ActionResult Logout()
5    {
6      ...
7      Session.Abandon();
8      ...
9    }
10 }
```

Código C# 8.4: LoginController.cs



Session Mode

O ASP.NET MVC disponibiliza quatro modos de sessão (*Session Modes*):

- InProc

- StateServer
- SQLServer
- Custom

A diferença entre eles é o modo de armazenamento das sessões.

No modo `InProc`, todas as informações da sessão são armazenadas na memória do servidor. Esse é o modo mais simples e mais utilizado, e vem configurado como o padrão. Uma desvantagem desse modo é a possível sobrecarga de memória se forem armazenadas muitas informações na sessão. Por isso não é indicado para sistemas muito grandes, com muitos usuários navegando simultaneamente. Outro problema é que o servidor da aplicação não pode ser desligado, pois a informação armazenada na memória será perdida.

No modo `StateServer`, as informações são serializadas e enviadas para um servidor independente do servidor da aplicação. Isto possibilita que o servidor de aplicação possa ser reiniciado sem que as sessões sejam perdidas. Uma desvantagem é o tempo gasto para realizar a serialização e desserialização.

No modo `SQLServer`, as informações também são serializadas, mas são armazenadas em um banco de dados. Além de permitir que o servidor da aplicação seja reiniciado, este modo possibilita um maior controle de segurança dos dados da sessão. Uma desvantagem importante é que o processo é naturalmente lento.

No modo `Custom`, todo o processo de identificação de usuários e armazenamento de sessões pode ser personalizado.

Para selecionar algum dos modos disponíveis, basta adicionar a tag `<sessionState>` dentro da tag `<system.web>`, no arquivo `Web.Config` da raiz de um projeto ASP.NET MVC.

```
1 <sessionState mode="InProc"/>
```

Código XML 8.1: Web.Config

Podemos personalizar nossa sessão, modificando alguns atributos da tag `<sessionState>`. Por exemplo, podemos determinar o tempo de timeout das sessões. Veja o exemplo abaixo.

```
1 <sessionState mode="InProc" timeout="30"/>
```

Código XML 8.2: Web.Config

Outra possibilidade é desabilitar o uso de cookie com o atributo `cookieless`. Neste caso será utilizada a reescrita de URL.

```
1 <sessionState mode="InProc" cookieless="true"/>
```

Código XML 8.3: Web.Config



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **Sessao** no **Visual Studio**. Utilize o template **MVC**.

2 Adicione uma entidade chamada **Produto** na pasta **Models** do projeto **Sessao**.

```
1 namespace Sessao.Models
2 {
3   public class Produto
4   {
5     public int ProdutoID { get; set; }
6     public string Nome { get; set; }
7     public double Preco { get; set; }
8   }
9 }
```

Código C# 8.5: www.k19.com.br/exercicios/k32/08/002/Produto.cs

3 Adicione uma classe chamada **K19Context** na pasta **Models** para registrar a entidade **Produto** e utilizar os recursos do Entity Framework. Lembre-se de fazer com que sua classe herde da classe DbContext.

```
1 namespace Sessao.Models
2 {
3   public class K19Context : DbContext
4   {
5     public DbSet<Produto> Produtos { get; set; }
6   }
7 }
```

Código C# 8.6: www.k19.com.br/exercicios/k32/08/003/K19Context.cs

4 Adicione uma classe chamada **Carrinho** na pasta **Models** do projeto **Sessao**.

```
1  namespace Sessao.Models
2  {
3    public class Carrinho
4    {
5      public List<Produto> Produtos { get; set; }
6
7      public Carrinho()
8      {
9        this.Produtos = new List<Produto>();
10     }
11   }
12 }
```

Código C# 8.7: www.k19.com.br/exercicios/k32/08/004/Carrinho.cs

5 Adicione um controlador chamado **Produto** baseado na entidade **Produto** e no DbContext **K19Context** utilizando o recurso do *scaffolding*.

6 Altere a tela **Index** do controlador **Produto** de acordo com o código abaixo.

```
1 @model IEnumerable<Sessao.Models.Produto>
2
```

```
3  @{
4      ViewBag.Title = "Index";
5  }
6
7  <h2>Index</h2>
8
9  @if (TempData["Mensagem"] != null)
10 {
11   <div>@TempData["Mensagem"]</div>
12 }
13
14 <p>
15     @Html.ActionLink("Create New", "Create")
16 </p>
17 <table>
18     <tr>
19         <th>
20             @Html.DisplayNameFor(model => model.Nome)
21         </th>
22         <th>
23             @Html.DisplayNameFor(model => model.Preco)
24         </th>
25         <th></th>
26     </tr>
27
28 @foreach (var item in Model) {
29     <tr>
30         <td>
31             @Html.DisplayFor(modelItem => item.Nome)
32         </td>
33         <td>
34             @Html.DisplayFor(modelItem => item.Preco)
35         </td>
36         <td>
37             @Html.ActionLink("Edit", "Edit", new { id=item.ProdutoID }) |
38             @Html.ActionLink("Details", "Details", new { id=item.ProdutoID }) |
39             @Html.ActionLink("Delete", "Delete", new { id=item.ProdutoID }) |
40         @Html.ActionLink("Adicionar ao carrinho", "Adicionar", "Carrinho", new { id=↩
       item.ProdutoID }, null)
41         </td>
42     </tr>
43 }
44 </table>
```

Código CSHTML 8.1: www.k19.com.br/exercicios/k32/08/006/Index.cshtml

7 Adicione um controlador chamado **Carrinho** utilizando o template **Empty MVC Controller**. Em seguida altere-o de acordo com o código abaixo.

```
1  namespace Sessao.Controllers
2  {
3    public class CarrinhoController : Controller
4    {
5      //
6      // GET: /Carrinho/
7
8      public ActionResult Index()
9      {
10       return View(this.PegaCarrinhoDaSessao());
11     }
12
13     public ActionResult Cancelar()
14     {
15       Session.Abandon();
16       return RedirectToAction("Index", "Produto");
17     }
18
```

```
19      public ActionResult Adicionar(int id = 0)
20      {
21        K19Context ctx = new K19Context();
22        Produto p = ctx.Produtos.Find(id);
23
24        if (p == null)
25        {
26          return HttpNotFound();
27        }
28
29        Carrinho carrinho = this.PegaCarrinhoDaSessao();
30        carrinho.Produtos.Add(p);
31
32        TempData["Mensagem"] = "Produto adicionado ao carrinho com sucesso!";
33
34        return RedirectToAction("Index", "Produto");
35      }
36
37      public ActionResult Remover(int idx = 0)
38      {
39        Carrinho carrinho = this.PegaCarrinhoDaSessao();
40        carrinho.Produtos.RemoveAt(idx);
41
42        return RedirectToAction("Index");
43      }
44
45      private Carrinho PegaCarrinhoDaSessao()
46      {
47        if (Session["Carrinho"] == null)
48        {
49          Session["Carrinho"] = new Carrinho();
50        }
51
52        return Session["Carrinho"] as Carrinho;
53      }
54    }
55  }
```

Código C# 8.8: www.k19.com.br/exercicios/k32/08/007/CarrinhoController.cs

8 Adicione uma tela associada à ação **Index** do controlador **Carrinho** com o seguinte conteúdo.

```
1  @model Sessao.Models.Carrinho
2  @{
3    ViewBag.Title = "Index";
4  }
5
6  <h2>Index</h2>
7
8  @Html.ActionLink("Cancelar carrinho", "Cancelar")
9
10 <ul>
11   @{
12     int i = 0;
13     foreach (var produto in Model.Produtos)
14     {
15     <li>@produto.Nome (@produto.Preco) - @Html.ActionLink("Remover do carrinho",
16     "Remover", new { idx = i++ })</li>
17     }
18   }
19 </ul>
```

Código CSHTML 8.2: www.k19.com.br/exercicios/k32/08/008/Index.cshtml

9 Altere o layout **_Layout** de acordo com o código abaixo.

```
@using Sessao.Models;
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width" />
    <title>@ViewBag.Title</title>
    @Styles.Render("~/Content/css")
    @Scripts.Render("~/bundles/modernizr")
</head>
<body>
  <div>
    @{
      Carrinho carrinho = Session["Carrinho"] as Carrinho;

      if(carrinho != null){
        @Html.ActionLink("Meu carrinho (" + @carrinho.Produtos.Count + " produtos)",
        "Index", "Carrinho")
      }
    }
  </div>
    @RenderBody()

    @Scripts.Render("~/bundles/jquery")
    @RenderSection("scripts", required: false)
</body>
</html>
```

Código CSHTML 8.3: www.k19.com.br/exercicios/k32/08/009/_Layout.cshtml

FILTROS

CAPÍTULO 9

Muitas vezes, queremos restringir o acesso à determinadas áreas de uma aplicação, seja por questões de segurança ou por organização. Em nossa aplicação poderíamos, por exemplo, definir que para poder adicionar, alterar e remover editoras, o usuário deve estar logado no sistema. Caso contrário, o usuário apenas poderá listar as editoras.

```
1  ...
2  public ActionResult Cadastra()
3  {
4    if (Session["Cliente"] != null)
5    {
6      return base.View();
7    }
8    else
9    {
10     return base.RedirectToAction("Index", "Login");
11   }
12 }
13 ...
```

Código C# 9.1: EditoraController.cs

No exemplo acima, caso um usuário tente adicionar uma editora através do formulário de cadastro, a ação `Cadastra` verificará se o usuário já está logado. Essa verificação é realizada através do uso da sessão, como visto no capítulo 8. Se o usuário não estiver logado, ele será redirecionado para a página de Login. Apesar de funcionar, este código apresenta uma inconveniência. Temos que adicionar essa lógica em todas as ações que queremos que tenha o mesmo comportamento, ou seja, que apenas permitam o acesso de usuários logados.

Em outros casos, podemos desejar que algumas ações executem alguma tarefa em comum. Por exemplo, poderíamos adicionar uma mensagem em um arquivo de logging sempre que uma ação fosse executada. Desse modo, poderíamos guardar um histórico sobre o que a aplicação mais executou, qual foi a página mais visitada, etc. Mas novamente, teríamos que adicionar a mesma tarefa em todas as ações da nossa aplicação.

Nesses casos, em que várias ações possuem um comportamento em comum, podemos utilizar o conceito de **filtros**. Um **filtro** é semelhante a um método que é executado antes ou depois de uma ação.



Tipos de Filtros

O ASP.NET MVC possui 5 (quatro) tipos de filtros:

Tipo de Filtro	Interface	Implementação Padrão
Autorização	`IAuthorizationFilter`	`AuthorizeAttribute`
Autenticação	`IAuthenticationFilter`	`N/A`
Ação	`IActionFilter`	`ActionFilterAttribute`
Resultado	`IResultFilter`	`ActionFilterAttribute`
Exceção	`IExceptionFilter`	`HandleErrorAttribute`

Autenticação

Este filtro foi adicionado ao ASP .NET MVC na versão 5. Este filtro foi criado para verificar se o usuário está autenticado ou não.

Autorização

Filtro utilizado para restringir o acesso a ação do controlador a usuários autorizados. Caso o usuário não tenha permisão, o filtro retorna a resposta `401 HTTP Unauthorized`. Se o filtro for adicionado ao controlador, todas as ações terão o acesso restrito.

Ação

Este filtro é executado antes ou depois da ação ser executada. Este filtro é chamado depois do filtro de autorização e antes do filtro de resultado.

Resultado

O filtro de resultado é executado antes ou depois do resultado da ação ser processado. Este filtro permite alterarmos a maneira como o resultado da ação é gerado.

Exceção

Este filtro é utilizado para tratar as exceções. Este filtro é executado quando ocorre uma exceção no método ou nos filtros da ação.



Filtro de Autorização

Os filtros de autorização implementam a interface `IAuthorizationFilter`.

```
1 public interface IAuthorizationFilter
2 {
3   void OnAuthorization(AuthorizationContext filterContext);
4 }
```

Código C# 9.2: IAuthorizationFilter.cs

O método `OnAuthorization` é executado pelo ASP .NET MVC para realizar a autorização. A classe `AuthorizationContext` é derivada de `ControllerContext`, que permite o acesso aos dados do controlador, da rota e da requisição.

O ASP .NET MVC contém uma implementação padrão da `IAuthorizationFilter` que é a classe **AuthorizeAttribute**. A classe `AuthorizeAttribute` define a anotação `Authorize`, que pode ser adicionada a ação ou ao controlador para restringirmos o acesso a usuários autenticados e autorizados.

Caso um usuário não autorizado acesse uma ação ou um controlador marcado com a anotação `Authorize`, o MVC Framework retornará a resposta `401 HTTP status`. Se a aplicação estiver configurada para utilizar `ASP .NET Forms Authentication`, o usuário será redirecionado para uma página de login.

Para o nosso primeiro exemplo, podemos utilizar o filtro de autorização, onde exigimos que o usuário esteja logado (autenticado) para acessar determinadas áreas da aplicação. Para isso, precisamos adicionar o seguinte código no nosso método de login:

```
1 ...
2 FormsAuthentication.SetAuthCookie(usuario.Username, false);
3 ...
```

Código C# 9.3: AutenticadorController.cs

Isto adiciona um novo cookie utilizado para a autenticação do usuário. Este novo cookie é independente do cookie utilizado para armazenar informações da sessão. O primeiro parâmetro é referente ao nome do usuário (ou algo que o identifique). O segundo parâmetro é um booleano relativo ao tipo do cookie, se é permanente ou não. Caso seja `true`, ele sempre irá considerar que o usuário está autenticado após a primeira autenticação.

Para eliminar o cookie de autenticação, devemos adicionar o seguinte código na acão de logout:

```
1 ...
2 FormsAuthentication.SignOut();
3 ...
```

Código C# 9.4: AutenticadorController.cs

As ações que devem utilizar o filtro de autorização devem ser anotadas com `Authorize`.

```
1 ...
2 [Authorize]
3 public ActionResult Cadastra()
4 {
5   return base.View();
6 }
7 ...
```

Código C# 9.5: EditoraController.cs

Se queremos aplicar o mesmo filtro a todas as ações de um controlador, podemos adicionar a anotação `Authorize` na classe.

```
1 [Authorize]
2 public class EditoraController : Controller
3 {
4   ...
5 }
```

Código C# 9.6: EditoraController.cs

Desse modo, todas as ações presentes no controlador da `Editora` exigem que o usuário esteja autenticado.

Quando o filtro de autorização "barra" um usuário de acessar uma página, podemos redirecioná-lo para a página de login. Devemos incluir o seguinte código dentro da tag **<system.web>** do arquivo

`Web.Config`.

```
<authentication mode="Forms">
  <forms loginUrl="~/Autenticador/Logar" timeout="2880"/>
</authentication>
```

Código XML 9.1: Web.Config

Através da propriedade `User`, podemos obter o usuário associado a requisição atual. As informações do usuário podem ser obtidas através da propriedade `Identity`. Por exemplo, podemos verificar se o usuário associado à requisição atual está autenticado, basta utilizar a propriedade `IsAuthenticated` da propriedade `Identity`:

```
if (User.Identity.IsAuthenticated)
{
  ...
}
```

Código C# 9.7: Verificando se o usuário está autenticado

Para obter o nome do usuário autenticado, podemos utilizar a propriedade `Name` da propriedade `Identity`:

```
string nome = User.Identity.Name;
```

Código C# 9.8: Verificando quem é o usuário autenticado

A propriedade `Name` retorna o nome do usuário que passamos como parâmetro para o método `SetAuthCookie` na autenticação.



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **Filtros** no **Visual Studio**. Utilize o template **MVC**.

2 Adicione uma entidade chamada **Produto** na pasta **Models** do projeto **Filtros**.

```
namespace Filtros.Models
{
  public class Produto
  {
    public int ProdutoID { get; set; }
    public string Nome { get; set; }
    public double Preco { get; set; }
  }
}
```

Código C# 9.9: www.k19.com.br/exercicios/k32/09/002/Produto.cs

3 Adicione uma classe chamada **K19Context** na pasta **Models** para registrar a entidade **Produto** e utilizar os recursos do Entity Framework. Lembre-se de fazer com que sua classe herde da classe DbContext.

```
1 namespace Filtros.Models
2 {
3   public class K19Context : DbContext
4   {
5     public DbSet<Produto> Produtos { get; set; }
6   }
7 }
```

Código C# 9.10: www.k19.com.br/exercicios/k32/09/003/K19Context.cs

4 Adicione uma classe **Usuario** na pasta **Models**.

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace Filtros.Models
4  {
5    public class Usuario
6    {
7      public string Username { get; set; }
8      [DataType(DataType.Password)]
9      public string Password { get; set; }
10   }
11 }
```

Código C# 9.11: www.k19.com.br/exercicios/k32/09/004/Usuario.cs

5 Adicione o controlador **Autenticador** para permitir a autenticação do usuário.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Web;
5  using System.Web.Mvc;
6  using System.Web.Security;
7  using Filtros.Models;
8
9  namespace Filtros.Controllers
10 {
11   public class AutenticadorController : Controller
12   {
13
14     public ActionResult Logar()
15     {
16       return View();
17     }
18
19     [HttpPost]
20     public ActionResult Logar(Usuario usuario)
21     {
22       if (usuario.Username != null && usuario.Password != null &&
23           usuario.Username.Equals("K19") && usuario.Password.Equals("K19"))
24       {
25
26         Session["Usuario"] = usuario;
27         return RedirectToAction("Index", "Produto");
28       }
29       else
30       {
31         ViewBag.Mensagem = "usuário ou senha incorretos";
32         return View(usuario);
33       }
34     }
35
```

```
36      public ActionResult Sair()
37      {
38        Session.Abandon();
39        return RedirectToAction("Logar");
40      }
41    }
42  }
```

Código C# 9.12: www.k19.com.br/exercicios/k32/09/005/AutenticadorController.cs

```
1  @model Filtros.Models.Usuario
2
3  @{
4      ViewBag.Title = "Logar";
5  }
6
7  <h2>Login</h2>
8
9  @using (Html.BeginForm())
10 {
11     @Html.ValidationSummary(true)
12
13     <fieldset>
14         <legend>Usuario</legend>
15
16         <div class="editor-label">
17             @Html.LabelFor(model => model.Username)
18         </div>
19         <div class="editor-field">
20             @Html.EditorFor(model => model.Username)
21             @Html.ValidationMessageFor(model => model.Username)
22         </div>
23
24         <div class="editor-label">
25             @Html.LabelFor(model => model.Password)
26         </div>
27         <div class="editor-field">
28             @Html.EditorFor(model => model.Password)
29             @Html.ValidationMessageFor(model => model.Password)
30         </div>
31
32         <p>
33             <input type="submit" value="Logar" />
34         </p>
35     </fieldset>
36 }
37
38 <div>
39     @Html.ActionLink("Back to List", "Index")
40 </div>
41
42 @section Scripts {
43     @Scripts.Render("~/bundles/jqueryval")
44 }
```

Código CSHTML 9.1: www.k19.com.br/exercicios/k32/09/005/Logar.cshtml

6 Adicione um controlador para cadastrar um produto e listar os produtos. Altere o controlador para permitir que apenas os usuários logados possam cadastrar os produtos.

```
1  namespace Filtros.Controllers
2  {
3    public class ProdutoController : Controller
4    {
5      private K19Context db = new K19Context();
6
```

```
7       public ActionResult Index()
8       {
9         return View(db.Produtos.ToList());
10      }
11
12      public ActionResult Cadastrar()
13      {
14        if (Session["Usuario"] != null)
15        {
16          return View();
17        }
18        else
19        {
20          TempData["Mensagem"] = "Acesso não permitido";
21          return RedirectToAction("Index");
22        }
23
24      }
25
26      [HttpPost]
27      public ActionResult Cadastrar(Produto produto)
28      {
29        if (ModelState.IsValid)
30        {
31          db.Produtos.Add(produto);
32          db.SaveChanges();
33          return RedirectToAction("Index");
34        }
35        return View(produto);
36      }
37    }
38  }
```

Código C# 9.13: www.k19.com.br/exercicios/k32/09/006/ProdutoController.cs

```
1   @model IEnumerable<Filtros.Models.Produto>
2
3   @{
4       ViewBag.Title = "Index";
5   }
6
7   <h2>Index</h2>
8   @if (TempData["Mensagem"] != null)
9   {
10      <h3>@TempData["Mensagem"]</h3>
11  }
12  <h3></h3>
13  <p>
14      @Html.ActionLink("Cadastrar Produto", "Cadastrar")
15  </p>
16  <table>
17      <tr>
18          <th>
19              @Html.DisplayNameFor(model => model.Nome)
20          </th>
21          <th>
22              @Html.DisplayNameFor(model => model.Preco)
23          </th>
24          <th></th>
25      </tr>
26
27      @foreach (var item in Model)
28      {
29          <tr>
30              <td>
31                  @Html.DisplayFor(modelItem => item.Nome)
32              </td>
33              <td>
34                  @Html.DisplayFor(modelItem => item.Preco)
```

```
35              </td>
36
37          </tr>
38      }
39
40  </table>
```

Código CSHTML 9.2: www.k19.com.br/exercicios/k32/09/006/Index.cshtml

```
1   @model Filtros.Models.Produto
2
3   @{
4       ViewBag.Title = "Cadastrar";
5   }
6
7   <h2>Cadastrar</h2>
8
9   @using (Html.BeginForm())
10  {
11      @Html.ValidationSummary(true)
12
13      <fieldset>
14          <legend>Produto</legend>
15
16          <div class="editor-label">
17              @Html.LabelFor(model => model.Nome)
18          </div>
19          <div class="editor-field">
20              @Html.EditorFor(model => model.Nome)
21              @Html.ValidationMessageFor(model => model.Nome)
22          </div>
23
24          <div class="editor-label">
25              @Html.LabelFor(model => model.Preco)
26          </div>
27          <div class="editor-field">
28              @Html.EditorFor(model => model.Preco)
29              @Html.ValidationMessageFor(model => model.Preco)
30          </div>
31
32          <p>
33              <input type="submit" value="Create" />
34          </p>
35      </fieldset>
36  }
37
38  <div>
39      @Html.ActionLink("Back to List", "Index")
40  </div>
41
42  @section Scripts {
43      @Scripts.Render("~/bundles/jqueryval")
44  }
```

Código CSHTML 9.3: www.k19.com.br/exercicios/k32/09/006/Cadastrar.cshtml

Para testar, acesse a url `http://localhost:<PORTA_APP>/Produto/` e clique no link de cadastrar. Depois acesse o endereço `http://localhost:<PORTA_APP>/Autenticador/Logar/` para fazer a autenticação através do usuário **K19** e senha **K19**.

7 Ao invés de usar sessão, vamos utilizar o filtro de autorização do ASP.NET MVC. Primeiro, devemos alterar as ações `Logar` e `Sair` do controlador `Autenticador`.

```
1   using Filtros.Models;
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Security;

namespace Filtros.Controllers
{
  public class AutenticadorController : Controller
  {

    public ActionResult Logar()
    {
      return View();
    }

    [HttpPost]
    public ActionResult Logar(Usuario usuario)
    {
      bool resultado = FormsAuthentication.Authenticate(usuario.Username, usuario.↩
    Password);
      if (resultado)
      {

        FormsAuthentication.SetAuthCookie(usuario.Username,
        false); return RedirectToAction("Index", "Produto");
      }
      else
      {
        ViewBag.Mensagem = "usuário ou senha incorretos";
        return View(usuario);
      }
    }

    public ActionResult Sair()
    {
      FormsAuthentication.SignOut();
      return RedirectToAction("Logar");
    }
  }
}
```

Código C# 9.14: www.k19.com.br/exercicios/k32/09/007/AutenticadorController.cs

8 Altere o controlador `Produto` para que somente usuários logados possam acessar o formulário de cadastro.

```
...
[Authorize]
public ActionResult Cadastrar()
{
  return View();
}
...
```

Código C# 9.15: www.k19.com.br/exercicios/k32/09/008/ProdutoController.cs

9 Adicione no arquivo **Views/Shared/_Layout.cshtml** um link para ação **Sair**.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
```

```
5   <meta name="viewport" content="width=device-width" />
6   <title>@ViewBag.Title</title>
7   @Styles.Render("~/Content/css")
8   @Scripts.Render("~/bundles/modernizr")
9 </head>
10 <body>
11   @if (User.Identity.IsAuthenticated)
12   {
13     @:Olá @User.Identity.Name, @Html.ActionLink("Sair", "Sair", "Autenticador")
14   }
15   @RenderBody()
16
17   @Scripts.Render("~/bundles/jquery")
18   @RenderSection("scripts", required: false)
19 </body>
20 </html>
```

Código CSHTML 9.4: www.k19.com.br/exercicios/k32/09/009/_Layout.cshtml

10 Defina a url para a qual os usuários não logados serão redirecionados ao tentarem acessar o formulário de cadastro de editoras. Altere o arquivo `Web.config` da raiz do projeto **Filtros**. Defina o usuário e senha também no arquivo de configuração.

```
1 ...
2 <authentication mode="Forms">
3       <forms loginUrl="~/Autenticador/Logar">
4         <credentials passwordFormat="Clear">
5           <user name="K19" password="K19"/>
6         </credentials>
7       </forms>
8  </authentication>
9 ...
```

Código XML 9.2: www.k19.com.br/exercicios/k32/09/010/Web.config

Acesse o endereço `http://localhost:<PORTA_APP>/Produto/Cadastra`. Verifique o funcionamento do filtro de autorização que redirecionará os usuários que não estão logados para a url definida no arquivo `Web.config`.



Filtro de Autenticação

Os filtros de autenticação implementam a interface `IAuthenticationFilter`.

```
1 public interface IAuthenticationFilter
2 {
3   void OnAuthentication(AuthenticationContext filterContext);
4   void OnAuthenticationChallenge(AuthenticationChallengeContext context);
5 }
```

Código C# 9.16: IAuthenticationFilter.cs

O filtro de Autenticação é executado antes dos outros filtros. Este filtro pode ser utilizado em conjunto com o filtro de Autorização. Neste caso, o filtro de Autenticação é executado antes do filtro de Autorização e também depois. Quer dizer, o filtro de Autenticação sempre é executado antes dos outros filtros, mas caso a autorização do usuário falhe, o método `OnAuthenticationChallenge` é chamado posteriormente.

O filtro de Autenticação foi adicionada à versão 5 do ASP .NET MVC, ele permite que criemos uma autenticação própria na nossa aplicação.

Atualmente é bem comum termos outras formas de autenticação na aplicação. Um exemplo são as autenticações através de usuários de redes sociais como Facebook, Google e Twitter.

```
1  public class K19AutenticadorController : Controller
2  {
3    public ActionResult Logar()
4    {
5      return View();
6    }
7      // POST: K19
8    [HttpPost]
9      public ActionResult Logar(Usuario usuario)
10     {
11     if (usuario.Username.EndsWith("@k19.com.br") && usuario.Password == "k32")
12     {
13       FormsAuthentication.SetAuthCookie(usuario.Username, false);
14             return Redirect(Url.Action("Index", "Home"));
15         }
16         else
17         {
18             ModelState.AddModelError("", "Incorrect username or password");
19             return View();
20     }
21     }
22 }
```

Código C# 9.17: K19AutenticadorController.cs

```
1  public class K19AutorizacaoAttribute : FilterAttribute, IAuthenticationFilter
2  {
3    public void OnAuthentication(AuthenticationContext context)
4    {
5      IIdentity ident = context.Principal.Identity;
6      if (!ident.IsAuthenticated || !ident.Name.EndsWith("@k19.com.br"))
7      {
8        context.Result = new HttpUnauthorizedResult();
9      }
10   }
11   public void OnAuthenticationChallenge(AuthenticationChallengeContext context)
12   {
13     if (context.Result == null || context.Result is HttpUnauthorizedResult)
14     {
15       context.Result = new RedirectToRouteResult(new RouteValueDictionary {
16                 {"controller", "K19Autenticador"},
17                 {"action",  "Logar"},
18                 {"returnUrl", context.HttpContext.Request.RawUrl}
19       });
20     }
21   }
22 }
```

Código C# 9.18: K19AutorizacaoAttribute

No exemplo acima, eu criei um controlador chamado `K19Autenticador` que autentica qualquer usuário que tenha email k19.com.br. Foi criado também a classe `K19AutorizacaoAttribute` que herda da classe `FilterAttribute`, é uma classe base para todos os filtros, e implementa a interface `IAuthenticationFilter`.

Podemos utilizar a classe `K19AutorizacaoAttribute` como filtro nos controladores e ações da nossa aplicação. Este filtro restringiria a autenticação a apenas usuários com email da *k19.com.br* e senha *k32*.



Exercícios de Fixação

11 Crie um controlador chamado `K19Autenticador`.

```
public class K19AutenticadorController : Controller
 {
  public ActionResult Logar()
  {
    return View();
  }
    // POST: K19
  [HttpPost]
    public ActionResult Logar(Usuario usuario)
    {
    if (usuario.Username.EndsWith("@k19.com.br") && usuario.Password == "k32")
    {
      FormsAuthentication.SetAuthCookie(usuario.Username, false);
            return Redirect(Url.Action("Index", "Home"));
        }
        else
        {
            ModelState.AddModelError("", "Incorrect username or password");
            return View();
    }
    }
}
```

Código C# 9.19: www.k19.com.br/exercicios/k32/09/011/K19AutenticadorController.cs

12 Crie a View `Logar`.

```
@model Filtros.Models.Usuario

@{
    ViewBag.Title = "Logar";
}

<h2>Login</h2>

@using (Html.BeginForm())
{
    @Html.ValidationSummary(true)

    <fieldset>
        <legend>Usuario</legend>

        <div class="editor-label">
            @Html.LabelFor(model => model.Username)
        </div>
        <div class="editor-field">
            @Html.EditorFor(model => model.Username)
            @Html.ValidationMessageFor(model => model.Username)
        </div>

        <div class="editor-label">
            @Html.LabelFor(model => model.Password)
        </div>
        <div class="editor-field">
            @Html.EditorFor(model => model.Password)
            @Html.ValidationMessageFor(model => model.Password)
        </div>

```

```
32          <p>
33              <input type="submit" value="Logar" />
34          </p>
35      </fieldset>
36  }
37
38  <div>
39      @Html.ActionLink("Back to List", "Index")
40  </div>
41
42  @section Scripts {
43      @Scripts.Render("~/bundles/jqueryval")
44  }
```

Código CSHTML 9.5: www.k19.com.br/exercicios/k32/09/012/Logar.cshtml

13 Crie o filtro de Autenticação `K19Autorizacao`.

```
1  public class K19AutorizacaoAttribute : FilterAttribute, IAuthenticationFilter
2  {
3    public void OnAuthentication(AuthenticationContext context)
4    {
5      IIdentity ident = context.Principal.Identity;
6      if (!ident.IsAuthenticated || !ident.Name.EndsWith("@k19.com.br"))
7      {
8        context.Result = new HttpUnauthorizedResult();
9      }
10   }
11   public void OnAuthenticationChallenge(AuthenticationChallengeContext context)
12   {
13     if (context.Result == null || context.Result is HttpUnauthorizedResult)
14     {
15       context.Result = new RedirectToRouteResult(new RouteValueDictionary {
16                 {"controller", "K19Autenticador"},
17                 {"action",  "Logar"},
18                 {"returnUrl", context.HttpContext.Request.RawUrl}
19       });
20     }
21   }
22 }
```

Código C# 9.20: www.k19.com.br/exercicios/k32/09/013/K19AutorizacaoAttribute.cs

14 Altere o controlador `Home`. Acrescentar o filtro `K19Autorizacao`.

```
1  namespace Filtros.Controllers
2  {
3    public class HomeController : Controller
4    {
5      public ActionResult Index()
6      {
7        return View();
8      }
9
10     public ActionResult About()
11     {
12       ViewBag.Message = "Your application description page.";
13
14       return View();
15     }
16
17     [K19Autorizacao]
18     public ActionResult Contact()
19     {
```

```
20        ViewBag.Message = "Your contact page.";
21
22        return View();
23      }
24    }
25  }
```

Código C# 9.21: www.k19.com.br/exercicios/k32/09/014/HomeController.cs

Acessar a http://localhost:<PORTA_APP>/Home/Contact. Verificar que o usuário é redirecionado para a página Logar. Efetuar o login com um usuário *contato@k19.com.br* e senha *k32*, por exemplo.



Filtro de Ação

Os filtros de ação são executados antes ou depois de uma ação. Todos os filtros de ação devem implementar a interface IActionFilter. A interface IActionFilter contém dois métodos OnActionExecuted e OnActionExecuting.

```
1  public interface IActionFilter
2  {
3    void OnActionExecuted(ActionExecutedContext filterContext);
4    void OnActionExecuting(ActionExecutingContext filterContext);
5  }
```

Código C# 9.22: IActionFilter.cs

O método OnActionExecuting é chamado antes da ação ser executada e o método OnActionExecuted é chamado depois da ação ser executada. As classes ActionExecutingContext e ActionExecutedContext são derivadas da classe ControllerContext que provê acesso ao controlador e outras informações.

Através do filtro de ações podemos, por exemplo, salvar as informações das requisições num arquivo de log. Para isto, basta definirmos uma classe que implementa a interface IActionFilter:

```
1  public class LoggingFilter : IActionFilter
2  {
3    private static string filename = "RequisicoesInfo.log";
4
5    private static string logFilePath;
6
7    public void OnActionExecuted(ActionExecutedContext filterContext)
8    {
9
10   }
11
12   public void OnActionExecuting(ActionExecutingContext filterContext)
13   {
14     SaveInfo(filterContext);
15   }
16
17   private void SaveInfo(ControllerContext context)
18   {
19     string info = String.Format("Usuário: {0}, IP: {1}, Data/Hora: {2}, Url: {3}",
20       context.HttpContext.User.Identity.Name,
21       context.HttpContext.Request.UserHostAddress,
22       DateTime.Now,
23       context.HttpContext.Request.RawUrl);
24     var path = GetLogFilePath(context);
25     using (var logWriter = new StreamWriter(path, true))
26     {
```

```
27        logWriter.WriteLine(info);
28      }
29    }
30
31    private static string GetLogFilePath(ControllerContext context)
32    {
33      if (LoggingFilter.logFilePath == null)
34      {
35        var logPath = context.HttpContext.Server.MapPath("~/Logs");
36        if (!Directory.Exists(logPath))
37        {
38          Directory.CreateDirectory(logPath);
39        }
40        var path = Path.Combine(logPath, filename);
41        LoggingFilter.logFilePath = path;
42      }
43      return LoggingFilter.logFilePath;
44    }
45  }
```

Código C# 9.23: LoggingFilter.cs

Para aplicar o filtro a todos os controladores e todas ações, basta defini-lo como *global*. Para definir `LoggingFilter` como global, basta adicioná-lo à coleção de filtros globais do método `RegisterGlobalFilters` da classe `FilterConfig`.

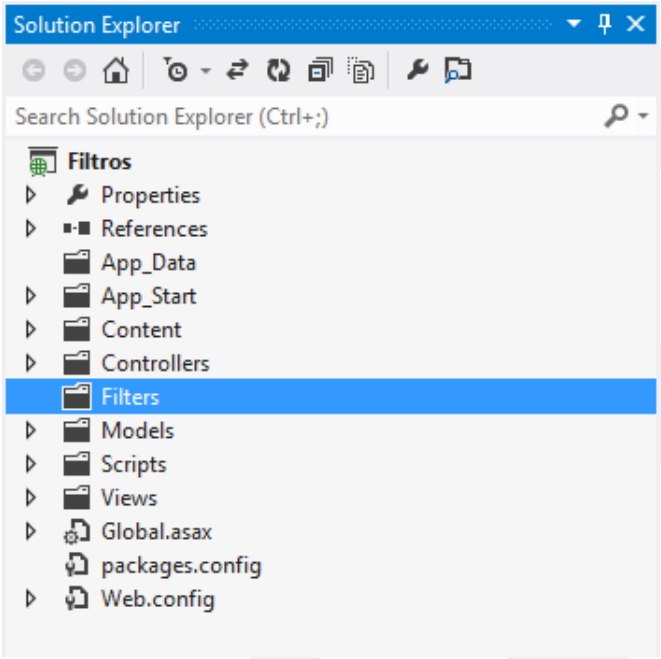
```
1  public class FilterConfig
2  {
3    public static void RegisterGlobalFilters(GlobalFilterCollection filters)
4    {
5      filters.Add(new HandleErrorAttribute());
6      filters.Add(new LoggingFilter());
7    }
8  }
```

Código C# 9.24: FilterConfig.cs



Exercícios de Fixação

15 Adicione uma pasta **Filters** ao projeto **Filtros**.



16 Crie um filtro de ação para registrar num arquivo de log as informações da requisição.

```
namespace Filtros.Filters
{
  public class LoggingFilter : IActionFilter
  {
    private static string filename = "RequisicoesInfo.log";

    private static string logFilePath;

    public void OnActionExecuted(ActionExecutedContext filterContext)
    {

    }

    public void OnActionExecuting(ActionExecutingContext filterContext)
    {
      SaveInfo(filterContext);
    }

    private void SaveInfo(ControllerContext context)
    {
      string info = String.Format("Usuário: {0}, IP: {1}, Data/Hora: {2}, Url: {3}",
        context.HttpContext.User.Identity.Name,
        context.HttpContext.Request.UserHostAddress,
        DateTime.Now,
        context.HttpContext.Request.RawUrl);
      var path = GetLogFilePath(context);
      using (var logWriter = new StreamWriter(path, true))
      {
        logWriter.WriteLine(info);
      }
    }

    private static string GetLogFilePath(ControllerContext context)
    {
      if (LoggingFilter.logFilePath == null)
      {
        var logPath = context.HttpContext.Server.MapPath("~/Logs");
        if (!Directory.Exists(logPath))
```

```
39          {
40            Directory.CreateDirectory(logPath);
41          }
42          var path = Path.Combine(logPath, filename);
43          LoggingFilter.logFilePath = path;
44        }
45        return LoggingFilter.logFilePath;
46      }
47    }
48  }
```

Código C# 9.25: www.k19.com.br/exercicios/k32/09/016/LoggingFilter.cs

17 Defina o filtro de ação criado no exercício anterior como global, para que ele seja aplicado a todos os controladores e ações.

```
1  namespace Filtros
2  {
3    public class FilterConfig
4    {
5      public static void RegisterGlobalFilters(GlobalFilterCollection filters)
6      {
7        filters.Add(new HandleErrorAttribute());
8        filters.Add(new LoggingFilter());
9      }
10   }
11 }
```

Código C# 9.26: www.k19.com.br/exercicios/k32/09/017/FilterConfig.cs

Acesse os seguintes endereços:

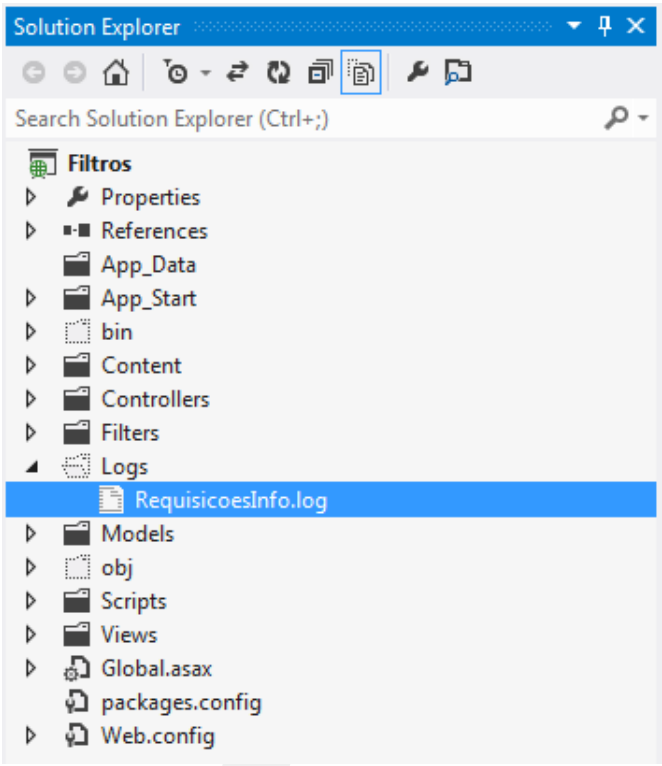
http://localhost:<PORTA_APP>/Produto

http://localhost:<PORTA_APP>/Autenticador/Logar

http://localhost:<PORTA_APP>/Produto/Cadastrar

Faça a autenticação com o username K19 e password K19.

E verifique, posteriormente, que as informações das requisições foram salvas no arquivo RequisicoesInfo.log dentro do diretório Logs.





Filtro de Resultado

O filtro de resultado é executado antes ou depois do resultado da ação ser gerado. O resultado da ação pode ser um JSON, uma página HTML ou qualquer outro resultado que derive da classe `ActionResult`. Para definir um filtro de resultado, devemos implementar a interface `IResultFilter` que contém dois métodos `OnResultExecuted` e `OnResultExecuting`.

```
1 public interface IResultFilter
2 {
3   void OnResultExecuted(ResultExecutedContext filterContext);
4   void OnResultExecuting(ResultExecutingContext filterContext);
5 }
```

Código C# 9.27: IResultFilter.cs

O método `OnResultExecuting` é executado antes do resultado ser gerado e o método `OnResultExecuted` é executado depois. As classes `ResultExecutingContext` e `ResultExecutedContext` são derivadas de `ControllerContext`, logo temos acesso ao controlador e às informações da requisição.

Através do filtro de resultado, podemos, por exemplo, mensurar o tempo entre a execução de uma ação e a geração do resultado. Para isto, podemos criar uma classe que é derivada de `ActionFilterAttribute`.



Importante
A classe `ActionFilterAttribute` é uma implementação padrão das interfaces

`IActionFilter` e `IResultFilter`, facilitando a criação de filtros que dependem da execução da ação e do resultado.

```
1  public class ElapsedTimeAttribute : ActionFilterAttribute
2  {
3    private Stopwatch duration;
4    public override void OnActionExecuting(ActionExecutingContext filterContext)
5    {
6      this.duration = Stopwatch.StartNew();
7    }
8
9    public override void OnResultExecuted(ResultExecutedContext filterContext)
10   {
11     this.duration.Stop();
12     string info = string.Format("Duração total: {0} ms", duration.ElapsedMilliseconds↩
           );
13     filterContext.HttpContext.Response.Write(info);
14   }
15 }
```

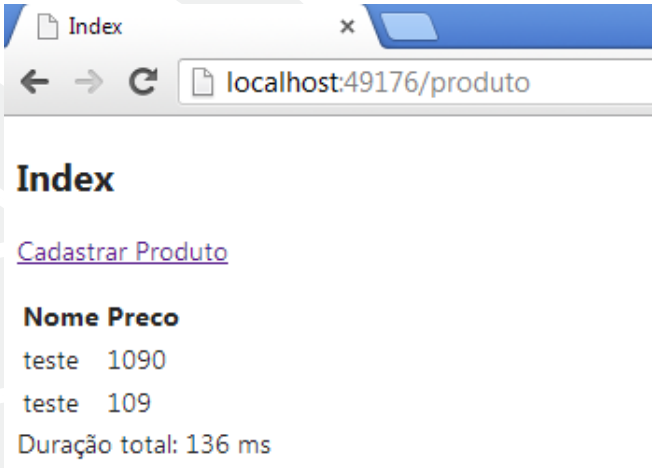
Código C# 9.28: ElapsedTimeAttribute.cs

Para aplicar o filtro, basta adicionar a anotação `ElapsedTime` no controlador ou na ação.

```
1  ...
2
3    [ElapsedTime]
4    public ActionResult Index()
5    {
6      return View(db.Produtos.ToList());
7    }
8
9  ...
```

Código C# 9.29: ProdutoController.cs

Ao acessar o seguinte endereço `http://localhost:<PORTA_APP>/produto`, podemos visualizar a duração entre a execução da ação e a geração do resultado.





Exercícios de Fixação

18 Adicione ao projeto **Filtros** um filtro para medir o tempo entre a execução de uma ação e a geração do resultado. **Dica:** Utilize a classe `ActionFilterAttribute` para criar o filtro.

```
1  namespace Filtros.Filters
2  {
3    public class ElapsedTimeAttribute : ActionFilterAttribute
4    {
5      private Stopwatch duration;
6      public override void OnActionExecuting(ActionExecutingContext filterContext)
7      {
8        this.duration = Stopwatch.StartNew();
9      }
10
11     public override void OnResultExecuted(ResultExecutedContext filterContext)
12     {
13       this.duration.Stop();
14       string info = string.Format("Duração total: {0} ms", duration.↩
             ElapsedMilliseconds);
15       filterContext.HttpContext.Response.Write(info);
16     }
17   }
18 }
```

Código C# 9.30: www.k19.com.br/exercicios/k32/09/018/ElapsedTimeAttribute.cs

19 Adicione o filtro `ElapsedTimeAttribute` à ação `Index` do controlador `Produto`. Para adicionar o filtro, basta acrescentar a anotação `[ElapsedTime]`.

```
1  public class ProdutoController : Controller
2  {
3    ...
4
5    [ElapsedTime]
6    public ActionResult Index()
7    {
8      return View(db.Produtos.ToList());
9
10   }
11
12   ...
13
14 }
```

Código C# 9.31: www.k19.com.br/exercicios/k32/09/019/ProdutoController.cs

Acesse o seguinte endereço: `http://localhost:<PORTA_APP>/produto`. Verifique que a duração do tempo entre a execução da ação e a geração do resultado é mostrada na tela.



Filtro de Exceção

Os filtros de exceção são executados quando ocorrem exceções nas ações ou nos filtros. Os filtros de exceção devem implementar a interface `IExceptionFilter`.

```
1 public interface IExceptionFilter
2 {
3   void OnException(ExceptionContext filterContext);
4 }
```

Código C# 9.32: IExceptionFilter.cs

O ASP .NET MVC contém a classe `HandleErrorAttribute` que é uma implementação padrão de `IExceptionFilter`. O filtro `HandleErrorAttribute` retorna a página `Error` localizada na pasta `Views/Shared` quando ocorre uma exceção. Através da `HandleErrorAttribute` podemos alterar a página de erro retornada de acordo com o tipo de exceção.

```
1 [HandleError(ExceptionType=typeof(InvalidOperationException),
2   View="OperacaoInvalida")]
3 [HandleError]
4 public ActionResult Index()
5 {
6   return View(db.Produtos.ToList());
7 }
```

Código C# 9.33: ProdutoController.cs

No exemplo acima, a primeira anotação filtrará os erros do tipo `InvalidOperationException` e retornará a página `OperacaoInvalida` da pasta `Views/Shared`. A segunda anotação filtrará todas as exceções e retornará a página padrão `Error` da pasta `Views/Shared`. É necessário habilitar o elemento **customErrors** no arquivo **Web.config** para o filtro `HandleErrorAttribute` funcionar.



Exercícios de Fixação

20 Altere o controlador `Produto` do projeto **Filtros**. Acrescente uma ação para visualizar um produto de uma lista através de uma posição. A ação deverá receber como parâmetro a posição, que será definida pelo usuário através da url.

```
1 public ActionResult Visualizar(int posicao)
2 {
3   List<Produto> produtos = db.Produtos.ToList();
4   return View(produtos[posicao]);
5 }
```

Código C# 9.34: www.k19.com.br/exercicios/k32/09/020/ProdutoController.cs

21 Defina uma página para mostrar os dados do produto que foi selecionado de acordo com a posição.

```
1  @model Filtros.Models.Produto
2
3  @{
4    ViewBag.Title = "Visualizar";
5  }
6
7  <h2>Visualizar</h2>
8
9  <fieldset>
10   <legend>Produto</legend>
```

```
11
12    <div class="display-label">
13      @Html.DisplayNameFor(model => model.Nome)
14    </div>
15    <div class="display-field">
16      @Html.DisplayFor(model => model.Nome)
17    </div>
18
19    <div class="display-label">
20      @Html.DisplayNameFor(model => model.Preco)
21    </div>
22    <div class="display-field">
23      @Html.DisplayFor(model => model.Preco)
24    </div>
25  </fieldset>
26  <p>
27    @Html.ActionLink("Back to List", "Index")
28  </p>
```

Código CSHTML 9.6: www.k19.com.br/exercicios/k32/09/021/Visualizar.cshtml

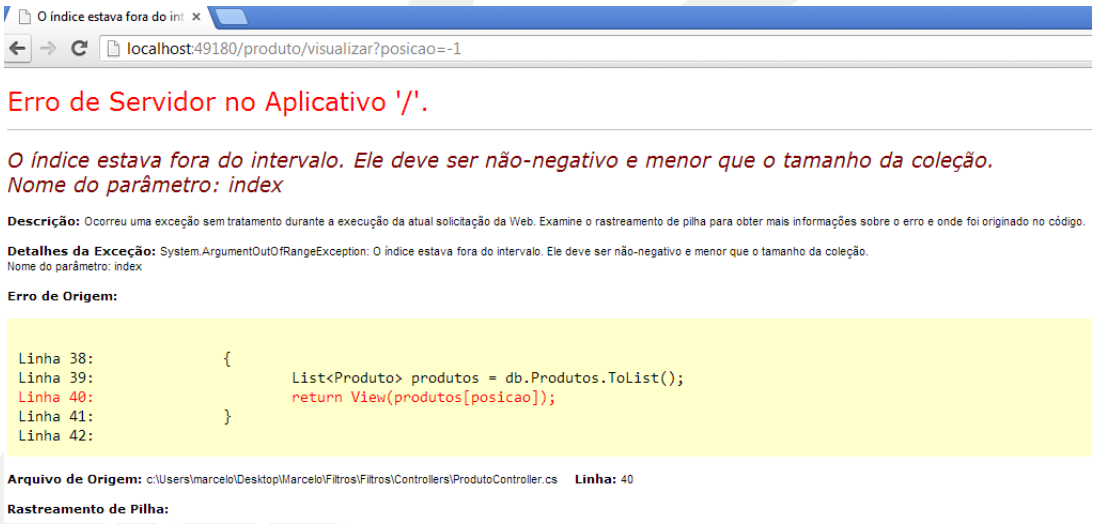
Verifique o funcionamento da ação **Visualizar**, acessando os seguintes endereços:

http://localhost:<PORTA_APP>/produto/visualizar?posicao=0

http://localhost:<PORTA_APP>/produto/visualizar?posicao=1

http://localhost:<PORTA_APP>/produto/visualizar?posicao=-1

Ao acessarmos uma posição inválida, ocorrerá a seguinte exceção:



22 Adicione um filtro para tratar a exceção do tipo `ArgumentOutOfRangeException`.

```
1 [HandleError(ExceptionType=typeof(ArgumentOutOfRangeException),
2 View ="PosicaoInvalida")]
3 public ActionResult Visualizar(int posicao)
4 {
5   List<Produto> produtos = db.Produtos.ToList();
6   return View(produtos[posicao]);
7 }
```

Código C# 9.35: www.k19.com.br/exercicios/k32/09/022/ProdutoController.cs

23 Crie a página `PosicaoInvalida` na pasta `Views/Shared`.

```
@{
  ViewBag.Title = "Posição Inválida";
}

<h2>Você acessou uma posição inválida.</h2>
```

Código CSHTML 9.7: www.k19.com.br/exercicios/k32/09/023/PosicaoInvalida.cshtml

24 Para o filtro `HandlerErrorAttribute` funcionar, habilite o elemento `customErrors` no arquivo `Web.config`.

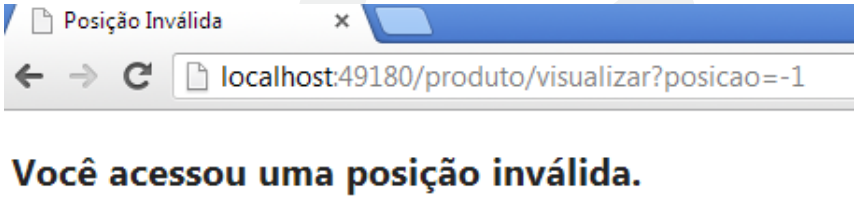
```
<system.web>
  <customErrors mode="On"/>
  ...
</system.web>
```

Código XML 9.3: www.k19.com.br/exercicios/k32/09/024/Web.config

Acesse o seguinte endereço:

`http://localhost:<PORTA_APP>/produto/visualizar?posicao=-1`

O filtro deverá redirecioná-lo para a tela `PosicaoInvalida`.



TRATAMENTO DE ERROS

CAPÍTULO 10

Inevitavelmente, as aplicações estão sujeitas a erros de várias naturezas. Por exemplo, erros podem ser gerados pelo preenchimento incorreto dos campos de um formulário. Esse tipo de erro é causado por falhas dos usuários. Nesse caso, é importante mostrar mensagens informativas com o intuito de fazer o próprio usuário corrigir os valores preenchidos incorretamente. Veja o Capítulo 7.

Por outro lado, há erros que não são causados por falhas dos usuários. Por exemplo, um erro de conexão com o banco de dados. Nesses casos, é improvável que os usuários possam fazer algo que resolva o problema. E mesmo que pudessem, provavelmente, não seria conveniente esperar que eles o fizessem.

Quando um erro desse tipo ocorre, o ASP.NET MVC cria uma página web com informações sobre o erro e a envia aos usuários. Para usuários locais, o ASP.NET MVC envia uma página web com informações detalhadas do erro ocorrido. Para usuários remotos, a página web enviada não contém informações detalhadas.

Em geral, não é conveniente que os usuários recebam detalhes técnicos sobre os erros gerados por falhas da aplicação. A primeira justificativa é que esses detalhes podem confundir os usuários. A segunda justificativa é que esses detalhes podem expor alguma falha de segurança da aplicação,

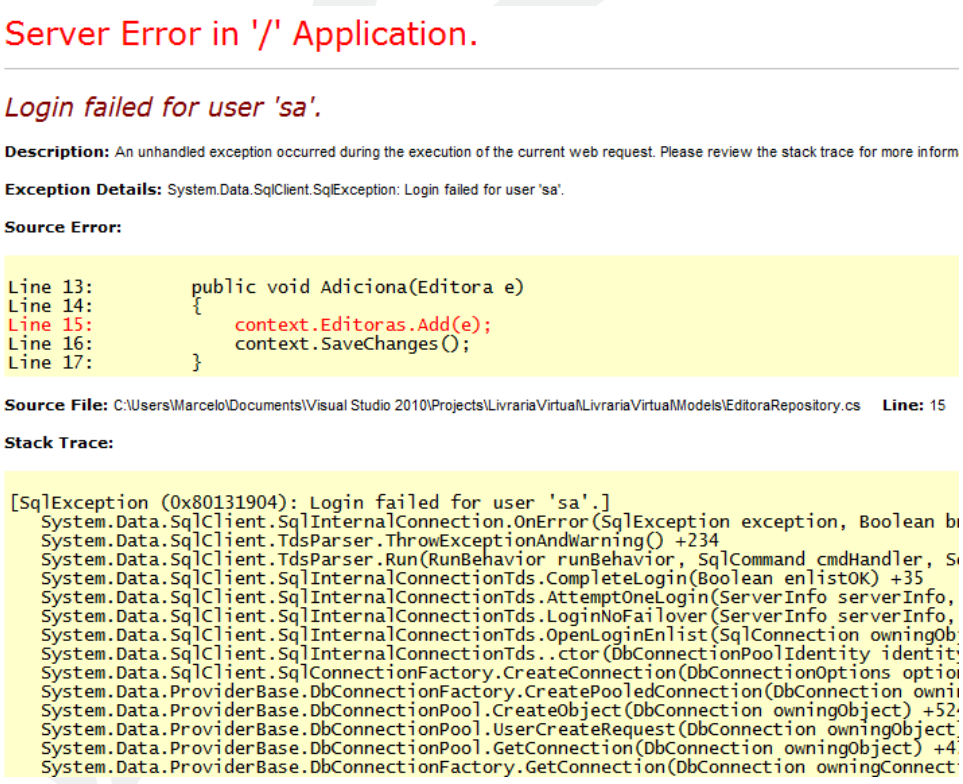


Figura 10.1: Exemplo de uma página web com informações sobre um erro

deixando-a mais vulnerável a ataques.



Try-Catch

Os erros de aplicação podem ser identificados através do comando `try-catch`, que pode ser colocados nos métodos que definem as ações dos controladores. Ao identificar a ocorrência de um erro, os controladores podem devolver uma página web com alguma mensagem para o usuário.

```
...
[HttpPost]
public ActionResult Salva(Editora editora)
{
  try
  {
    db.Editoras.Add(editora);
  }
  catch
  {
    return View("Error");
  }

  return RedirectToAction("Index");
}
...
```

Código C# 10.1: EditoraController.cs

Podemos criar uma página `Error.cshtml` na pasta **Views\Shared**. Dessa forma, todo controlador poderá devolver essa página.

```
@{
  Layout = null;
}

<!DOCTYPE html>
<html>
<head>
  <title>Erro</title>
</head>
<body>
  <h2>
    Servidor com problemas
  </h2>
  <p>
     Houve um problema em nosso servidor.<br/>
     Por favor tente novamente dentro de alguns instantes.
  </p>
</body>
</html>
```

Código CSHTML 10.1: Error.cshtml

As páginas de erro que serão mostradas pelos controladores teriam uma mensagem simples informando que houve um erro na aplicação e que não é possível atender a requisição do usuário naquele momento. Inclusive, seria conveniente padronizar a página de erro. Em outras palavras, todos os controladores teriam que mostrar a mesma página.



Custom Errors

Utilizar o comando `try-catch` nos controladores para lidar com os erros de aplicação não é uma boa alternativa, pois o código do controlador fica mais complexo. Além disso, haveria replicação de código nos controladores, pois provavelmente a página de erro seria padronizada.

Para lidar com os erros de aplicação de uma maneira mais prática e fácil de manter, podemos configurar o ASP.NET MVC para utilizar páginas de erro padrão. O primeiro passo é alterar o arquivo de configuração `Web.config`, acrescentando a tag `<customErrors>` dentro da tag `<system.web>`.

```
1 ...
2 <customErrors mode="On">
3
4 </customErrors>
5 ...
```

Código XML 10.1: Web.Config

O atributo `mode` da tag `<customErrors>` pode assumir três valores:

On: A página de erro padrão será enviada para usuários locais e remotos.

Off: A página de erro detalhada será enviada para usuários locais e remotos.

RemoteOnly: A página de erro detalhada será enviada para os usuários locais e a padrão para os remotos.

Por convenção, o ASP.NET MVC mantém uma página de erro padrão dentro da pasta **Views\Shared** com o nome `Error.cshtml`. Vamos alterar este arquivo. O conteúdo da página de erro é basicamente HTML.

```
1  @{
2    Layout = null;
3  }
4
5  <!DOCTYPE html>
6  <html>
7  <head>
8    <title>Erro</title>
9  </head>
10 <body>
11   <h2>
12     Servidor com problemas
13   </h2>
14   <p>
15      Houve um problema no nosso servidor.<br/>
16      Por favor tente novamente dentro de alguns instantes.
17   </p>
18 </body>
19 </html>
```

Código CSHTML 10.2: Error.cshtml



Http Errors

Um dos erros mais conhecidos do HTTP é o 404, que ocorre quando o navegador faz uma requisição para uma url que não existe. Basicamente, esse erro é gerado por falhas dos usuários ao tentarem digitar diretamente uma url na barra de endereço dos navegadores ou por links ou botões "quebrados" nas páginas da aplicação.

Quando o erro 404 ocorre, o ASP.NET MVC utiliza a página padrão para erros de aplicação configurada no `Web.config` através da tag `<customErrors>`. Porém, esse erro não deve ser considerado um erro de aplicação, pois ele pode ser gerado por falhas do usuário. Ele também não deve ser considerado um erro de usuário, pois ele pode ser gerado por falhas da aplicação. Consequentemente, é comum tratar o erro 404 de maneira particular, criando uma página de erro específica para ele.

```
@{
  Layout = null;
}

<!DOCTYPE html>
<html>
<head>
  <title>Arquivo não encontrado!</title>
</head>
<body>
  <h2>
    Esse arquivo não foi encontrado. Verifique se a url está correta.
  </h2>
</body>
</html>
```

Código CSHTML 10.3: NotFound.cshtml

No arquivo de configuração, podemos especificar uma página web particular para o erro 404 ou para os outros erros do HTTP.

```
...
<customErrors mode="On">
  <error statusCode="404" redirect="~/ErrorPage/NotFound"/>
</customErrors>
...
```

Código XML 10.2: Web.Config

De acordo com o código anterior, devemos definir um controlador chamado `ErrorPage` com uma ação chamada `NotFound`. Essa ação será acionada toda vez que o erro 404 do HTTP ocorrer.

```
namespace K19.Controllers
{
  public class ErrorPageController : Controller
  {
    public ActionResult NotFound()
    {
      return View();
    }
  }
}
```

Código C# 10.2: ErroPageController.cs



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **TratamentoDeErros** no **Visual Studio**. Utilize o template **MVC**.

2 Crie um controlador `Teste` no projeto **TratamentoDeErros**, conforme o código abaixo.

```
1  namespace TratamentoDeErros.Controllers
2  {
3    public class TesteController : Controller
4    {
5      public ActionResult TestaErro()
6      {
7        string[] nomes = new string[] { "Jonas Hirata", "Rafael Cosentino" };
8        string nome = nomes[2];
9        return View();
10     }
11   }
12 }
```

Código C# 10.3: www.k19.com.br/exercicios/k32/10/002/TesteController.cs

Acesse o seguinte endereço `http://localhost:<PORTA_APP>/Teste/TestaErro` e verifique a tela de erro.

Observação: No Visual Web Developer, quando executamos a aplicação, ele a executa em modo *debug*. Dessa forma, toda vez que um erro for gerado no processamento de uma requisição, a execução da aplicação é suspensa no ponto em que o erro ocorreu e detalhes sobre o problema são apresentados. Para continuar a execução da aplicação após a ocorrência de um erro, aperte a tecla F5.

3 Trate o erro do exercício anterior com o bloco `try-catch` e redirecione o usuário para uma tela com a seguinte mensagem “Sistema Temporariamente Indisponível”.

Altere a ação TestaErro do controlador Teste.

```
1  ...
2  public ActionResult TestaErro()
3  {
4    try
5    {
6      string[] nomes = new string[] { "Jonas Hirata", "Rafael Cosentino" };
7      string nome = nomes[2];
8      return View();
9    }
10   catch
11   {
12     return View("Error");
13   }
14 }
15 ...
```

Código C# 10.4: www.k19.com.br/exercicios/k32/10/003/TesteController.cs

4 Altere o código do arquivo `Error.cshtml` da pasta `Views\Shared` conforme o código abaixo:

```
1  @{
2    Layout = null;
3  }
4
```

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>Error</title>
</head>
<body>
  <h2>
    Sistema Temporariamente Indisponível...
  </h2>
</body>
</html>
```

Código CSHTML 10.4: www.k19.com.br/exercicios/k32/10/004/Error.cshtml

Acesse novamente o endereço http://localhost:<PORTA_APP>/Teste/TestaErro e verifique a nova página de erro.

5 Remova o bloco `try-catch` da ação `TestaErro` do controlador `Teste` que você adicionou no exercício anterior.

```
...
public ActionResult TestaErro()
{
  string[] nomes = new string[] { "Jonas Hirata", "Rafael Cosentino" };
  string nome = nomes[2];
  return View();
}
...
```

Código C# 10.5: www.k19.com.br/exercicios/k32/10/005/TesteController.cs

6 Altere o arquivo `Web.config` para configurar o ASP.NET MVC para utilizar páginas de erro padrão.

```
...
<system.web>
  <customErrors mode="On">
  </customErrors>
  ...
<system.web>
...
```

Código XML 10.3: www.k19.com.br/exercicios/k32/10/006/Web.config

Acesse o endereço http://localhost:<PORTA_APP>/Teste/TestaErro e verifique a página de erro.

7 Vamos definir uma página para ser exibida quando o erro 404 ocorrer. Primeiramente, crie um controlador chamado `ErrorPage` com uma ação chamada `NotFound`.

```
namespace TratamentoDeErros.Controllers
{
    public class ErrorPageController : Controller
    {
    public ActionResult NotFound()
    {
      return View();
```

```
8       }
9       }
10  }
```

Código C# 10.6: www.k19.com.br/exercicios/k32/10/007/ErroPageController.cs

8 Na pasta `Views\ErrorPage` do projeto **Erros**, adicione um arquivo chamado `NotFound.cshtml` com o seguinte conteúdo.

```
1  @{
2    Layout = null;
3  }
4
5  <!DOCTYPE html>
6  <html>
7  <head>
8    <title>Arquivo não encontrado!</title>
9  </head>
10 <body>
11   <h2>
12     Esse arquivo não foi encontrado. Verifique se a url está correta.
13   </h2>
14 </body>
15 </html>
```

Código CSHTML 10.5: www.k19.com.br/exercicios/k32/10/008/NotFound.cshtml

9 Altere o arquivo `Web.Config` para definir que quando um erro 404 ocorrer, a página exibida deve ser aquela que acabamos de criar.

```
1  ...
2  <system.web>
3    <customErrors mode="On">
4      <error statusCode="404" redirect="~/ErrorPage/NotFound"/>
5      </customErrors>
6    ...
7  <system.web>
8  ...
```

Código XML 10.4: www.k19.com.br/exercicios/k32/10/009/Web.config

Tente acessar páginas da aplicação que não existam. Acesse, por exemplo, o endereço `http://localhost:<PORTA_APP>/PaginaInexistente`.

ASP.NET Web API

APÊNDICE



Para permitir a implementação de webservices restful, o ASP.NET Web API foi adicionado à quarta versão do ASP.NET MVC. Webservices restful são baseados no estilo arquitetural REST. Discutiremos, a seguir, os principais conceitos desse estilo arquitetural.



REST

Resources

No estilo arquitetural REST, toda informação disponível é chamada de **resource**. Por exemplo, uma imagem é um resource. Uma lista de produtos é um resource. O cadastro de uma pessoa é um resource.

URIs

Todo resource possui um identificador único globalmente. Os identificadores são utililzados para acessar os respectivos resources. Particularmente, em uma rede HTTP, os resources são identificados por **URIs**(Uniform Resource Identifier - http://tools.ietf.org/html/rfc3986). Por exemplo, a URI www.k19.com.br/cursos identifica na internet a lista de cursos da K19.

Media Types

Um resource pode ser apresentado em diversos formatos. Na arquitetura REST, esses formatos são chamados de **media type**. Considere o cadastro de uma pessoa disponível em uma rede HTTP. Eventualmente, esse cadastro pode ser apresentado em **html**, **xml** ou **json**.

```
<html>
  <head>
    <title>Rafael Cosentino</title>
  <head>

  <body>
    <h1>Rafael Cosentino</h1>
    <p>Líder de treinamentos da K19</p>
  </body>
</html>
```

```
<pessoa>
  <nome>Rafael Cosentino</nome>
  <descricao>Líder de treinamentos da K19</descricao>
<pessoa>
```

```
{"nome": "Rafael Cosentino", "descricao": "Líder de treinamentos da K19"}
```

Operações

Em uma arquitetura REST, um conjunto pequeno e fixo de operações deve ser definido previamente. As operações são utilizadas para manipular os recursos de alguma forma.

Por exemplo, em uma rede HTTP, os recursos são manipulados pelos métodos do protocolo HTTP. Podemos atribuir uma semântica diferente para cada um desses métodos.

Resource	Método HTTP	Semântica
www.k19.com.br/cursos	GET	pega a lista de cursos
www.k19.com.br/cursos	POST	adiciona um curso na lista



Resources

No ASP.NET Web API, os resources são definidos por controladores de uma aplicação ASP.NET MVC que derivam de `System.Web.Http.ApiController`.

```
1 public class CursosController : ApiController
2 {
3   ...
4 }
```

Código C# A.1: CursosController.cs



URIs

As URIs dos resources são definidas através de rotas do ASP.NET Web API criadas no arquivo `RouteConfig.cs`. O Visual Web Developer adiciona a seguinte rota padrão do ASP.NET Web API nos projetos ASP.NET MVC:

```
1 routes.MapHttpRoute(
2   name: "DefaultApi",
3   routeTemplate: "api/{controller}/{id}",
4   defaults: new { id = RouteParameter.Optional }
5 );
```

Código C# A.2: RouteConfig.cs

De acordo com a rota padrão do ASP.NET Web API, a URI do resource correspondente ao controlador `Cursos` é `http://localhost:<PORTA_APP>/api/cursos`.



Operações

Podemos associar as operações HTTP aos métodos da classe `CursosController`. Essas associações são estabelecidas automaticamente através dos nomes dos métodos. Por exemplo, a operação

GET do HTTP é associada automaticamente a um método cujo nome possui o prefixo Get. Veja o exemplo abaixo.

```
1  public class CursosController : ApiController
2  {
3    ...
4    // GET http://localhost:<PORTA_APP>/api/cursos
5    public Curso[] GetCursos()
6    {
7      ...
8    }
9    ...
10 }
```

Código C# A.3: CursosController.cs

Na rota padrão do ASP.NET Web API, um parâmetro opcional chamado id foi definido. Quando esse parâmetro estiver presente em uma requisição do tipo GET, podemos recuperar o valor dele em um método cujo nome possua o prefixo Get e tenha um argumento chamado id. Observe o código abaixo.

```
1  public class CursosController : ApiController
2  {
3    ...
4    // GET http://localhost:<PORTA_APP>/api/cursos/5
5    public Curso[] GetCurso(int id)
6    {
7      ...
8    }
9    ...
10 }
```

Código C# A.4: CursosController.cs

Também podemos recuperar parâmetros de URL. Para isso, basta definir argumentos com os mesmos nomes desses parâmetros nos métodos associados às operações do HTTP. Veja o exemplo abaixo

```
1  public class CursosController : ApiController
2  {
3    ...
4    // GET http://localhost:<PORTA_APP>/api/cursos?sigla=K32
5    public Curso[] GetCurso(string sigla)
6    {
7      ...
8    }
9    ...
10 }
```

Código C# A.5: CursosController.cs

Considere o código a seguir.

```
1  public class CursosController : ApiController
2  {
3    ...
4    // POST http://localhost:<PORTA_APP>/api/cursos
5    public void PostCurso(Curso curso)
6    {
7      ...
8    }
9    ...
```

```
10 }
```

Código C# A.6: CursosController.cs

O método `PostCurso()` será associado a operação `POST` do HTTP. Os valores dos parâmetros enviados dentro de uma requisição do tipo `POST` à url `http://localhost:<PORTA_APP>/api/cursos` serão armazenados automaticamente pelo ASP.NET Web API nas propriedades do argumento `curso` do método `PostCurso()` de acordo com o nome dessas propriedades e dos nomes dos parâmetros enviados. Por exemplo, o valor do parâmetro `Nome` será armazenado dentro da propriedade `Nome` caso ambos existam.



Media Type

Por padrão, o ASP NET Web API utiliza os headers `Content-type` e `Accept` para definir o media type dos dados de entrada e saída. No exemplo anterior, se uma requisição do tipo `GET` for realizada à url `//POSThttp://localhost:<PORTA_APP>/api/cursos` com header `Accept: application/json`, o resource correspondente será apresentado em formato `JSON`.



Exercícios de Fixação

1 Crie um projeto do tipo **ASP.NET Web Application** chamado **WebApi** no **Visual Studio**. Utilize o template **MVC**.

2 Vamos definir o nosso primeiro webservice utilizando web api. Para isso, adicione uma classe para modelar cursos na pasta **Models**.

```
1  namespace WebApi.Models
2  {
3    public class Curso
4    {
5      static int contador;
6
7      public Curso()
8      {
9        Curso.contador++;
10       this.Id = contador;
11     }
12
13     public int Id { get; set; }
14     public string Sigla { get; set; }
15     public string Nome { get; set; }
16   }
17 }
```

Código C# A.7: www.k19.com.br/exercicios/k32/A/002/Curso.cs

3 Agora crie um controlador chamado `Cursos` para implementar as operações do webservice. Na criação desse controlador, selecione o template `Empty API Controller`.

```
1 using System;
```

```
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Net;
5  using System.Net.Http;
6  using System.Web.Http;
7  using LivrariaVirtual.Models;
8  using System.Web.Mvc;
9
10 namespace WebApi.Controllers
11 {
12   public class CursosController : ApiController
13   {
14     private static List<Curso> cursos = new List<Curso>
15     {
16       new Curso
17       {
18         Sigla = "K31",
19         Nome = "C# e Orientação a Objetos"
20
21       },
22       new Curso
23       {
24         Sigla = "K32",
25         Nome = "Desenvolvimento Web com ASP .NET MVC"
26
27       }
28     };
29
30     public List<Curso> Get()
31     {
32       return cursos;
33     }
34
35     public Curso Get(string sigla)
36     {
37       Curso curso = cursos.Find(c => c.Sigla.Equals(sigla));
38       return curso;
39     }
40
41     public Curso Get(int id)
42     {
43       Curso curso = cursos.Find(c => c.Id == id);
44       return curso;
45     }
46
47     public void Post(Curso curso)
48     {
49       cursos.Add(curso);
50     }
51   }
52 }
```

Código C# A.8: www.k19.com.br/exercicios/k32/A/003/CursosController.cs

4 Para testar o webservice que retorna a lista de cursos, basta acessar o seguinte endereço: http://localhost:<PORTA_APP>/api/cursos.

5 Teste o webservice que devolve o curso a partir de uma sigla. Basta acessar o seguinte endereço: http://localhost:<PORTA_APP>/api/cursos?sigla=K31.

6 Teste o webservice que devolve um curso a partir de um id. Basta acessar o seguinte endereço: http://localhost:<PORTA_APP>/api/cursos/1.

7 Defina uma página com um formulário para testar o webservice que adiciona curso. Para isso, crie um controlador Curso conforme o código abaixo:

```
using System.Web.Mvc;

namespace WebApi.Controllers
{
  public class CursoController : Controller
  {
    public ActionResult Cadastrar()
    {
      return View();
    }

  }
}
```

Código C# A.9: www.k19.com.br/exercicios/k32/A/007/CursoController.cs

8 Para testar o webservice que adiciona curso, devemos definir a página **Cadastrar.cshtml**.

```
@model WebApi.Models.Curso

@{
    ViewBag.Title = "Cadastrar";
}

<h2>Cadastrar</h2>

@using (Html.BeginForm(null, null, FormMethod.Post, new { @action = "/api/cursos" }))
{
    @Html.ValidationSummary(true)

    <fieldset>
        <legend>Curso</legend>

        <div class="editor-label">
            @Html.LabelFor(model => model.Sigla)
        </div>
        <div class="editor-field">
            @Html.EditorFor(model => model.Sigla)
            @Html.ValidationMessageFor(model => model.Sigla)
        </div>

        <div class="editor-label">
            @Html.LabelFor(model => model.Nome)
        </div>
        <div class="editor-field">
            @Html.EditorFor(model => model.Nome)
            @Html.ValidationMessageFor(model => model.Nome)
        </div>

        <p>
            <input type="submit" value="Create" />
        </p>
    </fieldset>
}

<div>
    @Html.ActionLink("Back to List", "Index")
</div>

@section Scripts {
    @Scripts.Render("~/bundles/jqueryval")
}
```

Código CSHTML A.1: www.k19.com.br/exercicios/k32/A/008/Cadastrar.cshtml

9 Teste o webservice que adiciona curso através do formulário criado no exercício anterior. Acesse o formulário através do seguinte endereço: `http://localhost:<PORTA_APP>/Curso/Cadastrar`. Para verificar se o curso foi adicionado, acesse `http://localhost:<PORTA_APP>/api/cursos/`.

CODE FIRST MIGRATIONS

APÊNDICE B

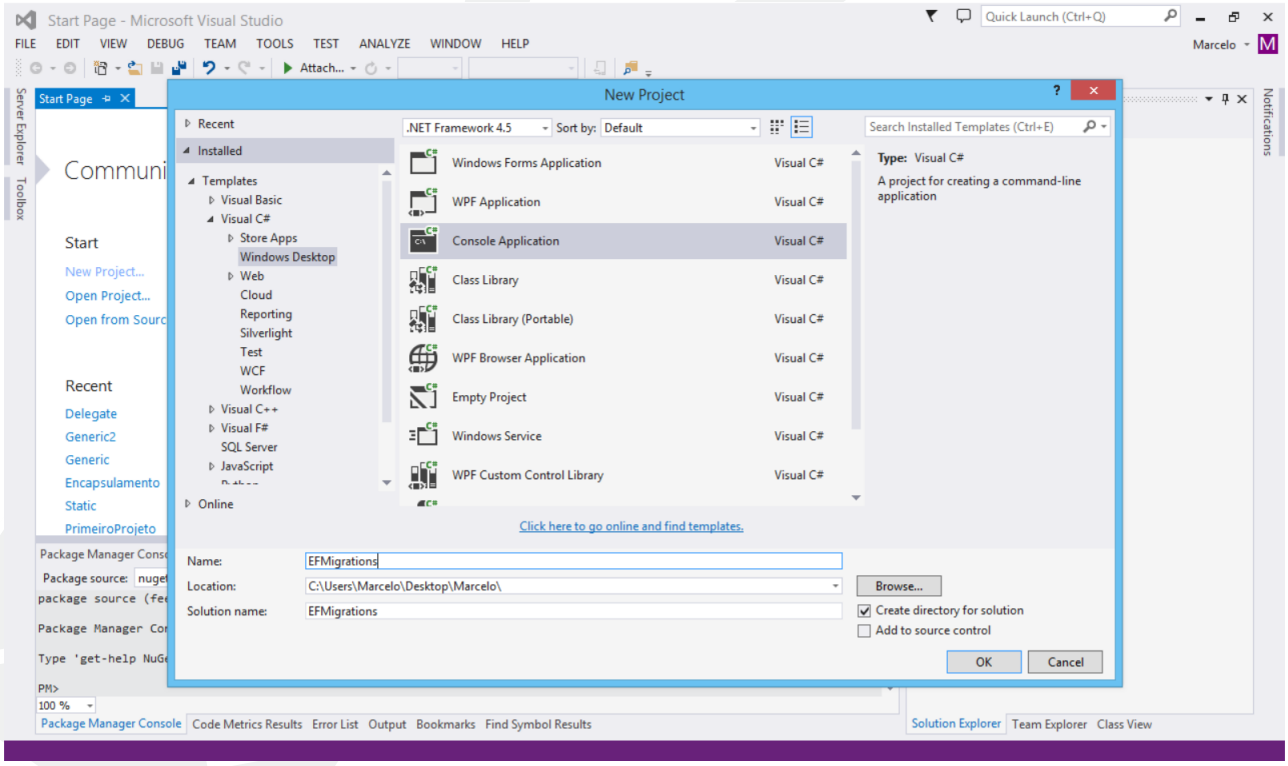
Code First Migrations

O Entity Framework Code First da Microsoft tem uma funcionalidade que permite controlar as mudanças no banco de dados que serão realizadas de acordo com as alterações na camada de modelo da aplicação. Essa funcionalidade é chamada de **Code First Migrations**.

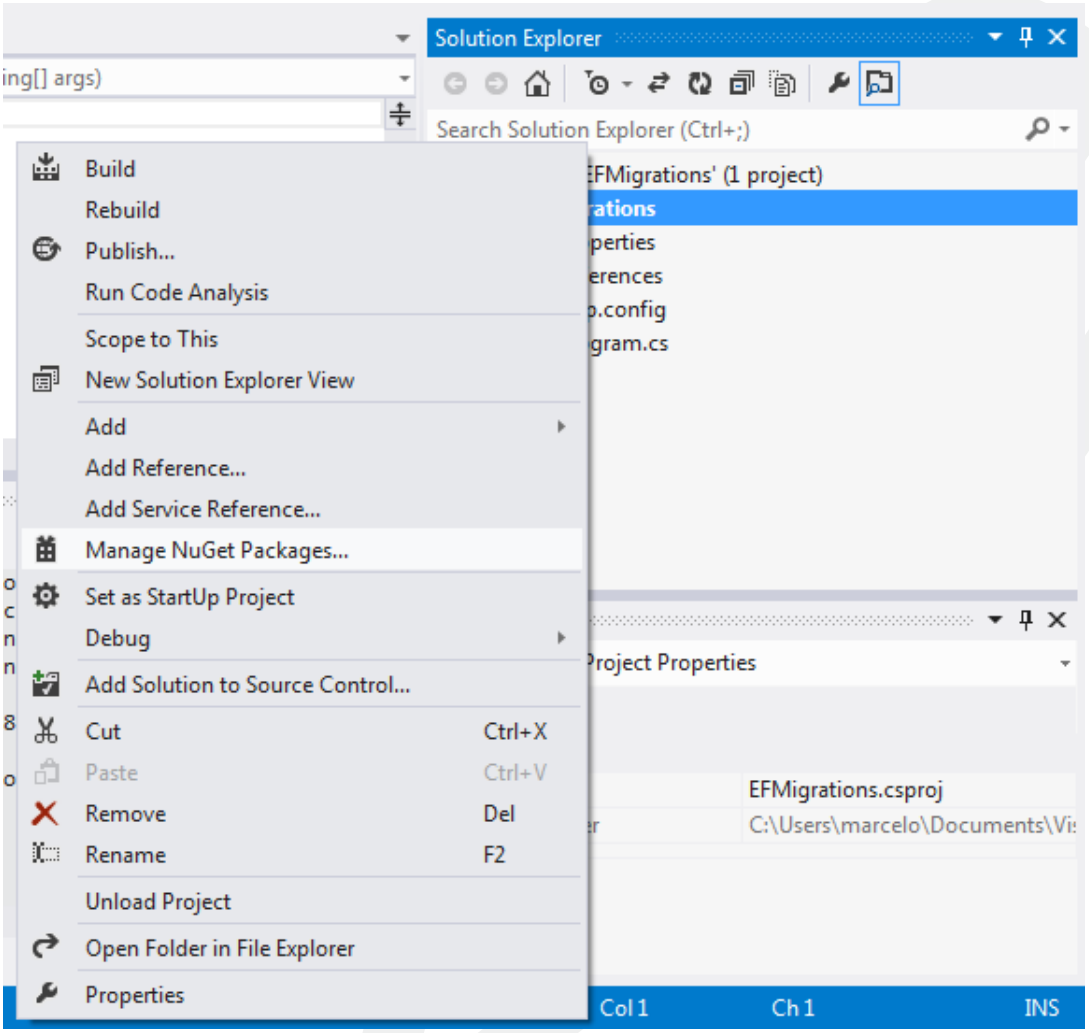
O Code First Migrations foi adicionado faz parte do Entity Framework.

Passo a Passo

Para testar o funcionamento do Code First Migrations, criaremos um projeto no Visual Studio.

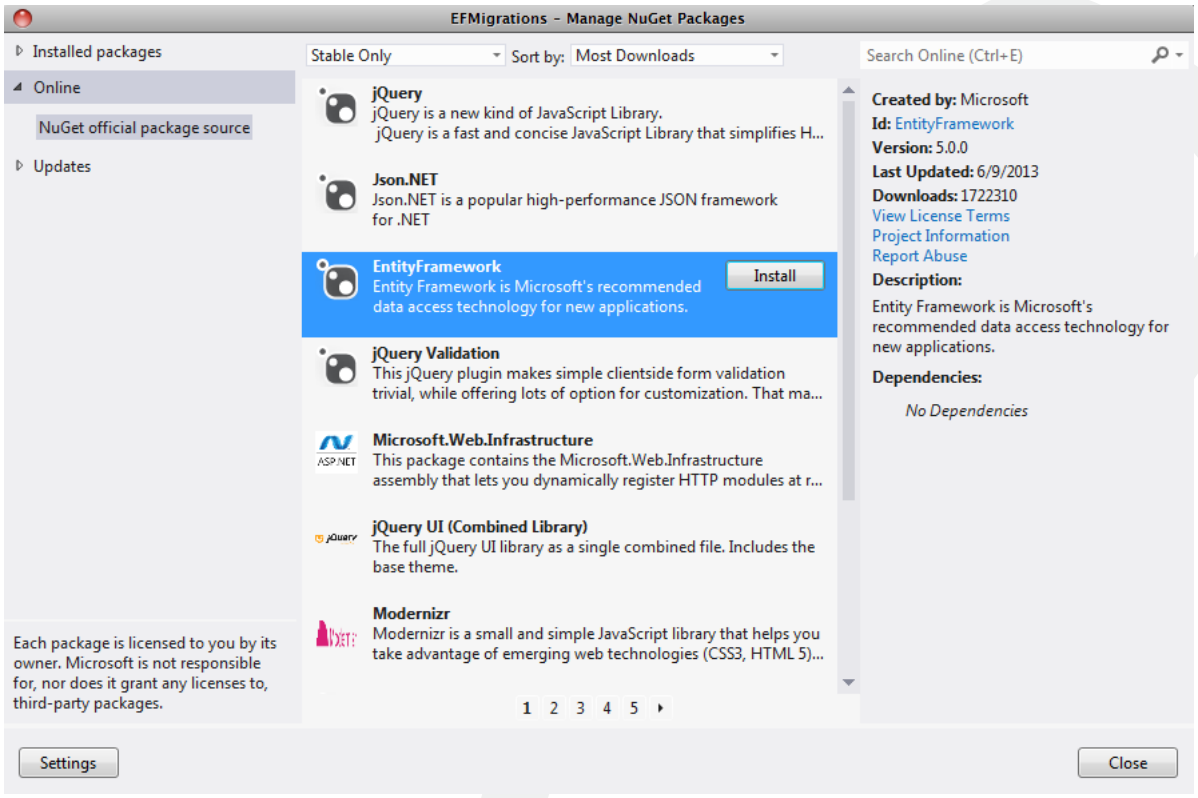


Agora, devemos instalar o Entity Framework. Para isso, utilizaremos o gerenciador de pacotes Nuget do Visual Studio.



O Nuget permite que bibliotecas e ferramentas sejam instaladas no Visual Studio. Ao instalar pacotes no seu projeto, ele adiciona as bibliotecas, referências e faz as alterações necessárias nos arquivos de configuração. Mais informações sobre o Nuget e os pacotes disponíveis, acesse o endereço http://nuget.org/packages

Para instalar o Entity Framework, basta selecionar o pacote **Entity Framework** e clicar em *Install*.



Após a instalação do Entity Framework, adicionaremos uma classe de modelo chamada `Editora`. Essa entidade será mapeada através de uma classe chamada `K19Context`. Para testar, criaremos uma classe com o método `Main()`. Observe o código dessas três classes.

```
1 namespace EFMigrations
2 {
3   public class Editora
4   {
5     public int Id { get; set; }
6     public string Nome { get; set; }
7     public string Email { get; set; }
8   }
9 }
```

Código C# B.1: Editora.cs

```
1 namespace EFMigrations
2 {
3   public class K19Context : DbContext
4   {
5     public DbSet<Editora> Editoras { get; set; }
6   }
7 }
```

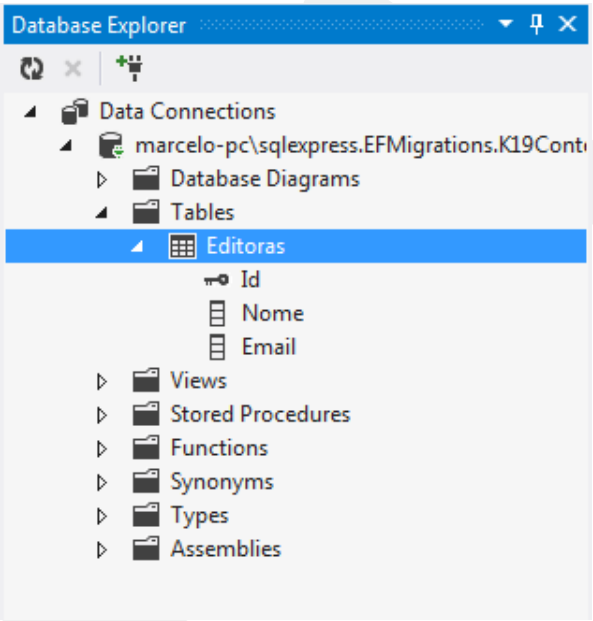
Código C# B.2: K19Context.cs

```
1 namespace EFMigrations
2 {
3   class Program
4   {
5     static void Main(string[] args)
6     {
7       using (var context = new K19Context())
```

```
8         {
9           Editora e = new Editora { Nome = "K19",
10            Email = "contato@k19.com.br" };
11          context.Editoras.Add(e);
12          context.SaveChanges();
13        }
14      }
15    }
16  }
```

Código C# B.3: Program.cs

Após a execução do projeto, teremos a seguinte tabela e banco de dados.

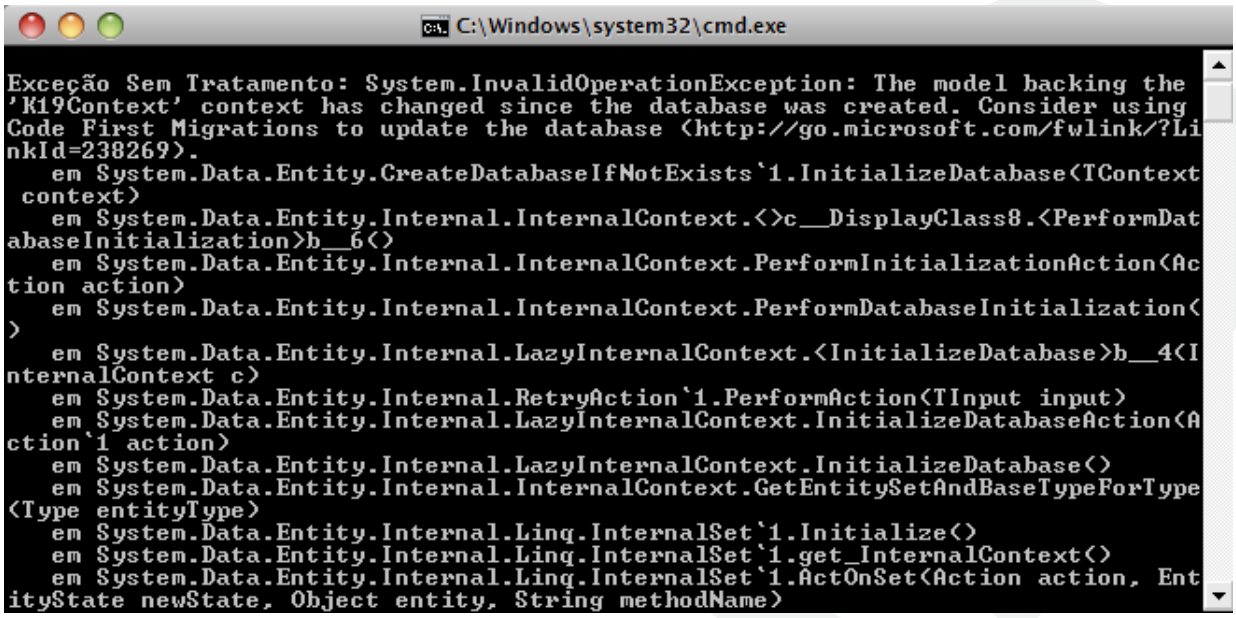


O próximo passo é alterar a classe de modelo `Editora`.

```
1  namespace EFMigrations
2  {
3    public class Editora
4    {
5      public int Id { get; set; }
6      public string Nome { get; set; }
7      public string Email { get; set; }
8
9      public string Telefone { get; set; }
10   }
11 }
```

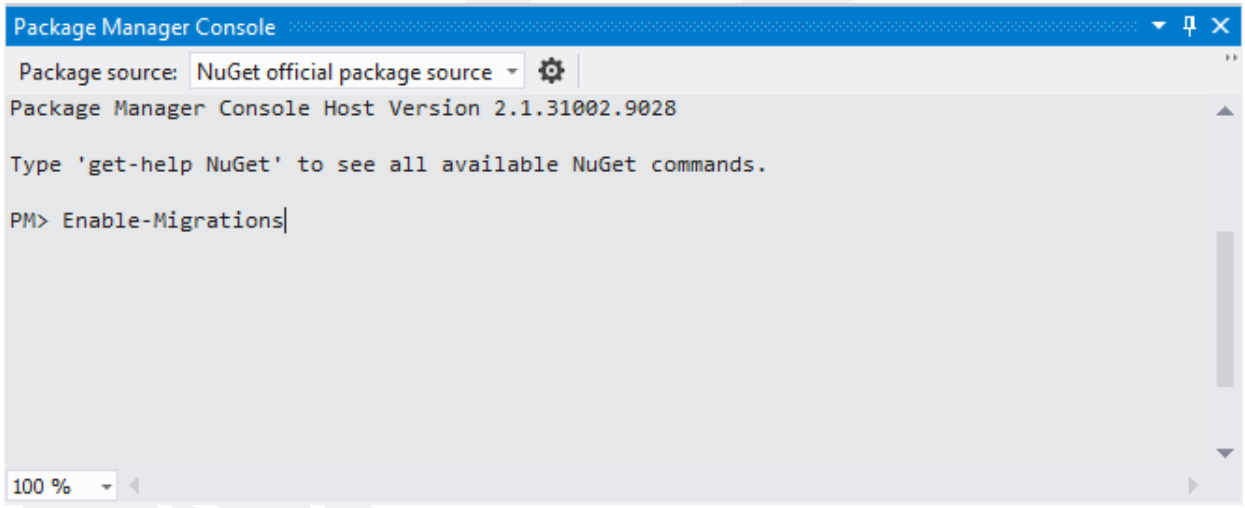
Código C# B.4: Editora.cs

Com a alteração na classe `Editora`, um exeção é gerada ao executar o projeto.



Para resolver esse problema, o banco de dados precisa ser atualizado. Para fazer essa atualização, o Entity Framework recomenda a utilização da ferramenta Code First Migrations.

O primeiro passo para utilizar Code First Migrations é habilitá-lo e adicioná-lo ao projeto através do Package Manager Console.



O comando `Enable-Migrations` adiciona uma pasta chamada **Migrations** no projeto. Esta pasta contém dois arquivos.

- A classe `Configuration` permite definir o comportamento do Code First Migrations para o nosso contexto.
- A classe parcial `InitialCreate` define a primeira versão das tabelas do banco de dados.

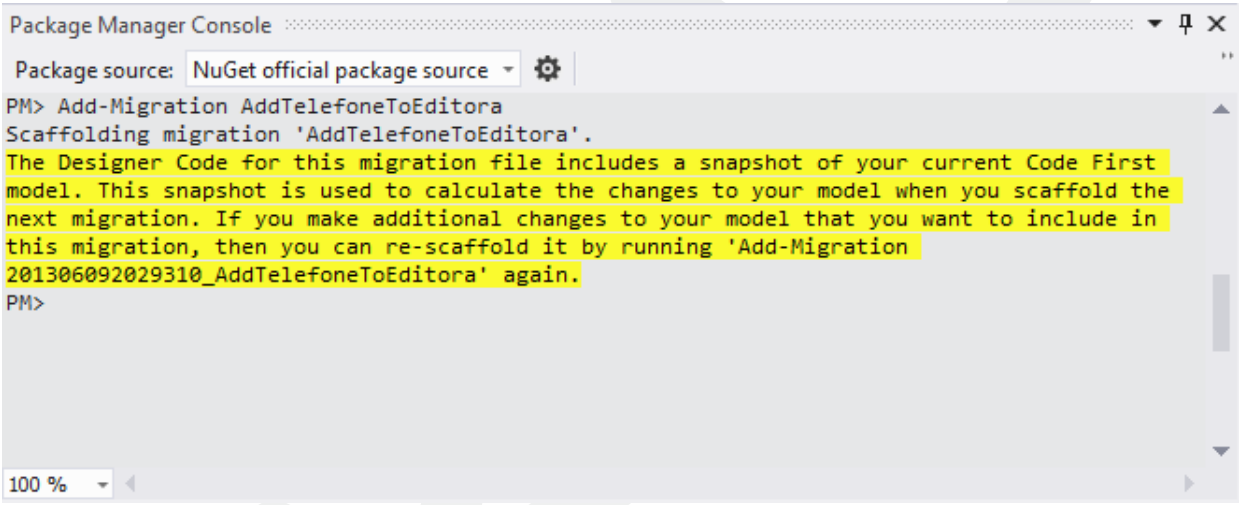
O Code First Migration tem dois comandos fundamentais.

- `Add-Migration` que gera o código necessário para atualizar o banco de dados de acordo com as alterações nas classes de modelo.
- `Update-Database` aplica as alterações pendentes no banco de dados.

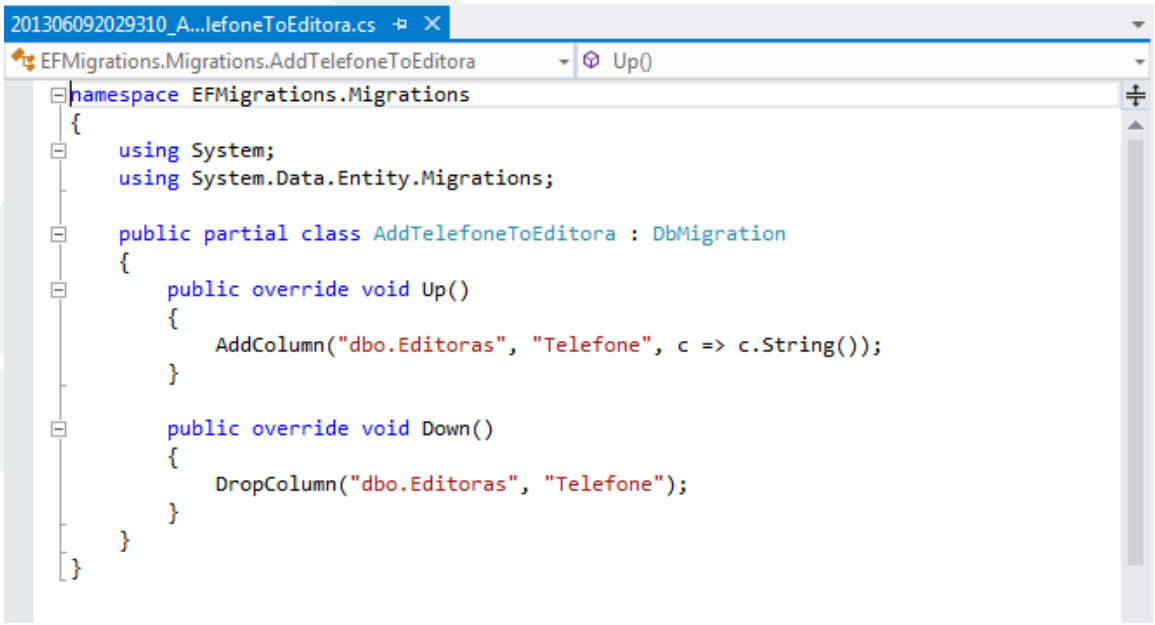
Após adicionarmos a propriedade `Telefone` à classe `Editora`, devemos adicionar uma *Migration* ao projeto para atualizar o banco de dados e adicionar uma nova coluna à tabela **Editoras**. Para adicionar uma *Migration*, devemos utilizar o comando `Add-Migration`. Para atualizar o banco de dados com as alterações definidas na *Migration*, devemos utilizar o comando `Update-Database`.

Para utilizar o comando `Add-Migration` é necessário definir o nome da `Migration` a ser adicionada.

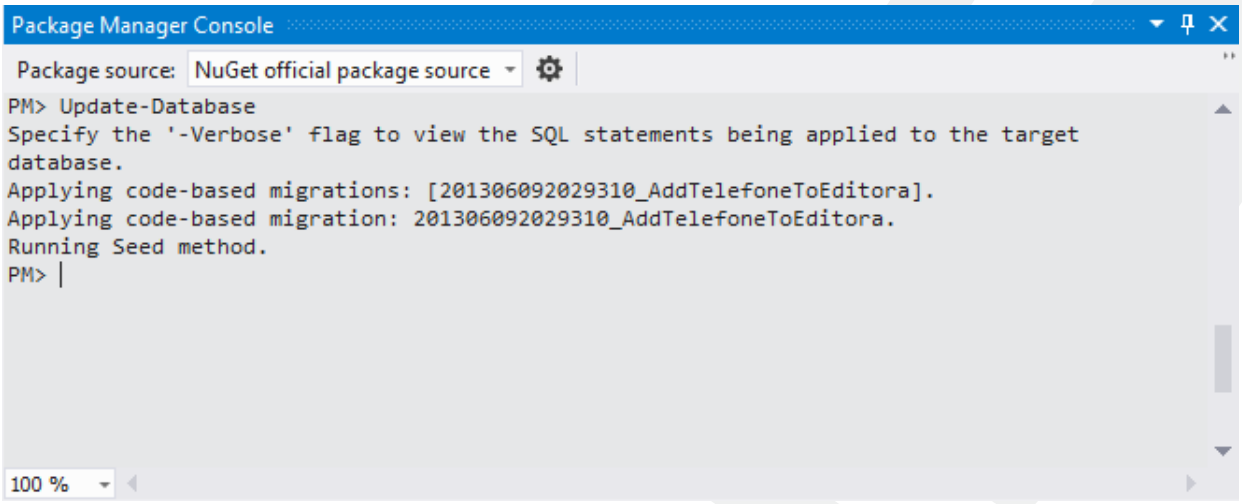
O nome da **Migration** que adicionaremos será `AddTelefoneToEditora`.



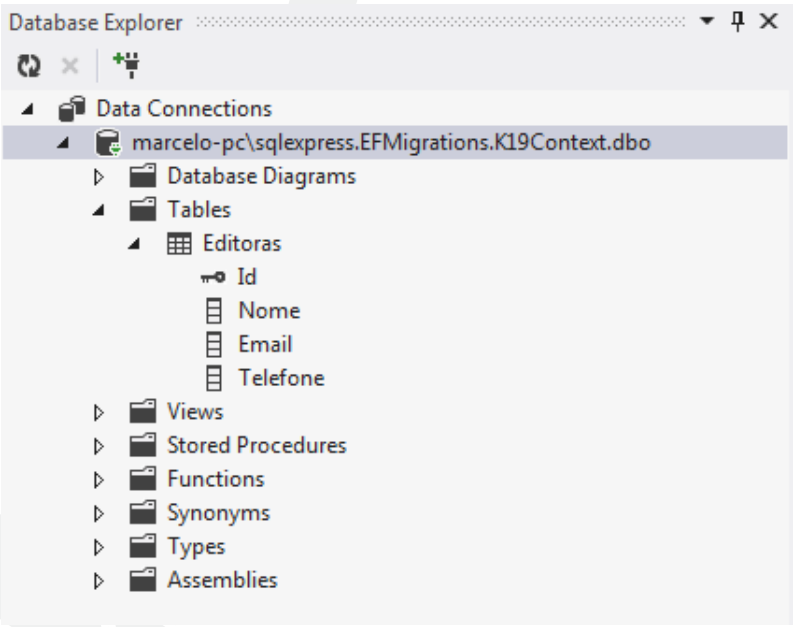
A classe que define a migração é adicionada à pasta **Migrations**.



Para atualizar o banco de dados com as alterações definidas na *Migration*, devemos executar o comando `Update-Database`.



Uma nova coluna é adicionada à tabela `Editoras` no banco de dados.



Exercícios de Fixação

1. Crie um projeto do tipo **Console Application** chamado **CodeFirstMigrations** no **Visual Studio**.

2. Adicione o Entity Framework ao projeto **CodeFirstMigrations** através do `Manage Nuget Packages`.

3 Após adicionar o Entity Framework ao projeto, crie a classe `Editora`.

```
using System.ComponentModel.DataAnnotations;

namespace CodeFirstMigrations
{
  class Editora
  {
    public int Id { get; set; }
    [Required(ErrorMessage = "O campo nome é obrigatório")]
    public string Nome { get; set; }
    [Required(ErrorMessage = "O campo email é obrigatório")]
    public string Email { get; set; }
  }
}
```

Código C# B.5: Editora.cs

4 Crie uma classe **K19Context** e adicione a propriedade **DbSet** para mapear a entidade **Editora**.

```
using System.Data.Entity;

namespace CodeFirstMigrations
{
  class K19Context : DbContext
  {
    public DbSet<Editora> Editoras { get; set; }
  }
}
```

Código C# B.6: K19Context.cs

5 Crie uma classe **Program** de teste para cadastrar as editoras.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CodeFirstMigrations
{
  class Program
  {
    static void Main(string[] args)
    {
      using (var context = new K19Context())
      {
        Console.WriteLine("Digite o nome da Editora:");
        string nome = Console.ReadLine();
        Console.WriteLine("Digite o email da Editora:");
        string email = Console.ReadLine();
        Editora e = new Editora { Nome = nome, Email = email };
        context.Editoras.Add(e);
        context.SaveChanges();
      }

    }
  }
}
```

Código C# B.7: Program.cs

Execute o projeto e cadastre uma editora.

6 Altere a classe **Editora** adicionando a propriedade **Telefone**.

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace CodeFirstMigrations
4  {
5    public class Editora
6    {
7      public int Id { get; set; }
8      [Required(ErrorMessage = "O campo nome é obrigatório")]
9      public string Nome { get; set; }
10     [Required(ErrorMessage = "O campo email é obrigatório")]
11     public string Email { get; set; }
12     public string Telefone { get; set; }
13   }
14 }
```

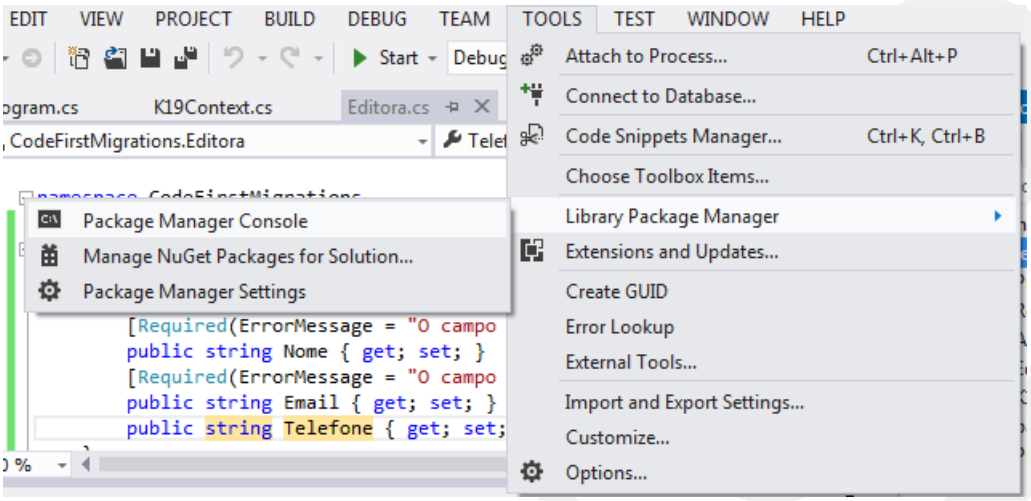
Código C# B.8: Editora.cs

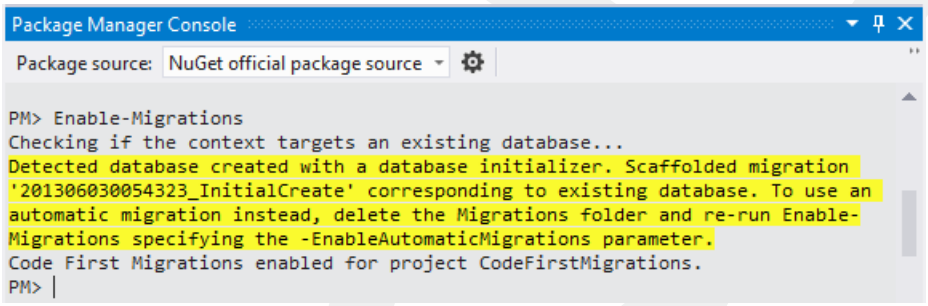
7 Execute novamente o projeto. Verifique que uma exceção do tipo **System.InvalidOperationException** ocorrerá.

```
'K19Context' context has changed since the database was created. Consider using
Code First Migrations to update the database (http://go.microsoft.com/fwlink/?Li
nkId=238269).
   em System.Data.Entity.CreateDatabaseIfNotExists`1.InitializeDatabase(TContext
 context)
   em System.Data.Entity.Internal.InternalContext.<>c__DisplayClass8.<PerformDat
abaseInitialization>b__6()
   em System.Data.Entity.Internal.InternalContext.PerformInitializationAction(Ac
tion action)
   em System.Data.Entity.Internal.InternalContext.PerformDatabaseInitialization(
)
   em System.Data.Entity.Internal.LazyInternalContext.<InitializeDatabase>b__4(I
nternalContext c)
   em System.Data.Entity.Internal.RetryAction`1.PerformAction(TInput input)
   em System.Data.Entity.Internal.LazyInternalContext.InitializeDatabaseAction(A
ction`1 action)
   em System.Data.Entity.Internal.LazyInternalContext.InitializeDatabase()
   em System.Data.Entity.Internal.InternalContext.GetEntitySetAndBaseTypeForType
(Type entityType)
   em System.Data.Entity.Internal.Linq.InternalSet`1.Initialize()
   em System.Data.Entity.Internal.Linq.InternalSet`1.get_InternalContext()
   em System.Data.Entity.Internal.Linq.InternalSet`1.ActOnSet(Action action, Ent
ityState newState, Object entity, String methodName)
   em System.Data.Entity.Internal.Linq.InternalSet`1.Add(Object entity)
   em System.Data.Entity.DbSet`1.Add(TEntity entity)
   em CodeFirstMigrations.Program.Main(String[] args) na c:\Users\marcelo\Deskto
p\Marcelo\CodeFirstMigrations\CodeFirstMigrations\Program.cs:linha 20
Pressione qualquer tecla para continuar. . .
```

8 Para corrigir o problema visto no exercício anterior, devemos habilitar o *Code First Migrations*. Para isso, execute o seguinte comando através do *Package Manager Console*.

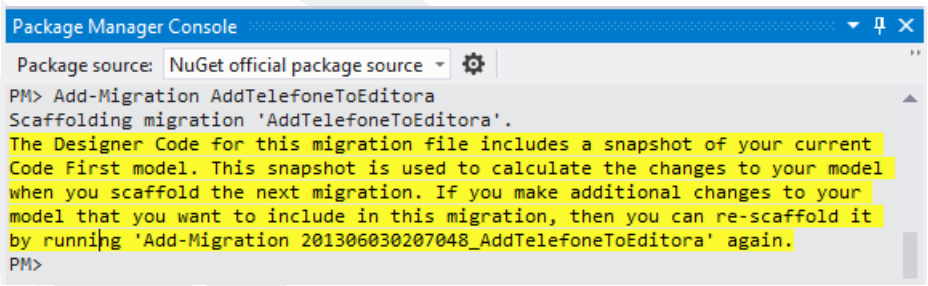
```
Enable-Migrations.
```





Verifique que uma pasta chamada **Migrations** com dois arquivos: *<DATA>_InitialCreate.cs* e *Configuration.cs* foi gerada no projeto **CodeFirstMigrations**.

9 Após habilitar o `Code First Migrations` no projeto **CodeFirstMigrations**, devemos adicionar uma migração para atualizar o banco de dados adicionando a coluna `Telefone` na tabela `Editoras`. Crie uma migração chamada *AddTelefoneToEditora* através do comando `Add-Migration`. Execute este comando através do `Package Manager Console`.



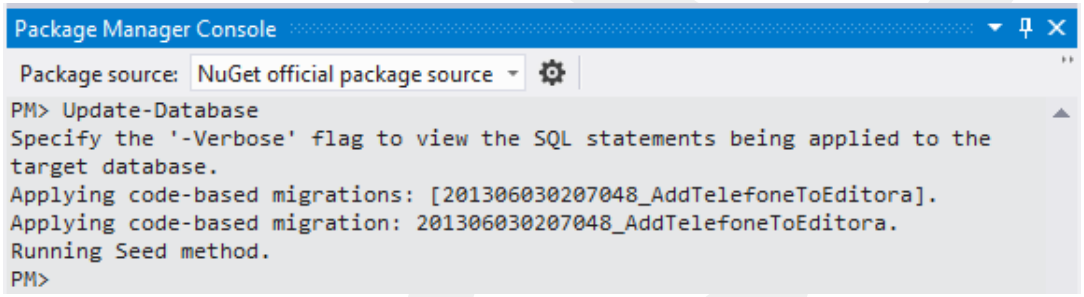
Verifique que um arquivo de migração foi criado na pasta **Migrations**:

```
1  namespace CodeFirstMigrations.Migrations
2  {
3      using System;
4      using System.Data.Entity.Migrations;
5
6      public partial class AddTelefoneToEditora : DbMigration
```

```
7       {
8           public override void Up()
9           {
10              AddColumn("dbo.Editoras", "Telefone", c => c.String());
11          }
12
13          public override void Down()
14          {
15              DropColumn("dbo.Editoras", "Telefone");
16          }
17      }
18  }
```

Código C# B.9: <DATA>_AddTelefoneToEditora.cs

10 Para atualizar a tabela Editoras no banco de dados, utilize o comando `Update-Database` que é executado através do Package Manager Console.



Verifique que a coluna *Telefone* foi adicionada a tabela **Editoras**.



Acrescentando uma nova entidade

O Code First Migrations gerencia as alterações no modelo e a adição de novas entidades. Ao adicionar uma nova entidade ao projeto, devemos criar uma **Migration** que adicionará uma nova tabela para o qual a classe será mapeada. Considere a seguinte classe para definir a entidade **Livro**.

```
1   namespace EFMigrations
2   {
3     public class Livro
4     {
5       public int Id { get; set; }
6       public string Titulo { get; set; }
7       public decimal Preco { get; set; }
8       public int EditoraId { get; set; }
9       public Editora Editora { get; set; }
10    }
11  }
```

Código C# B.10: Livro.cs

Para estabelecer o relacionamento entre as entidades **Editora** e **Livro**, a classe `Editora` deverá ser alterada.

```
1   namespace EFMigrations
2   {
```

```
3   public class Editora
4   {
5     public int Id { get; set; }
6     public string Nome { get; set; }
7     public string Email { get; set; }
8     public string Telefone { get; set; }
9
10    [ForeignKey("EditoraId")]
11    public virtual IList<Livro> Livros { get; set; }
12  }
13 }
```

Código C# B.11: Editora.cs

Devemos mapear a entidade **Livro** na classe **K19Context**.

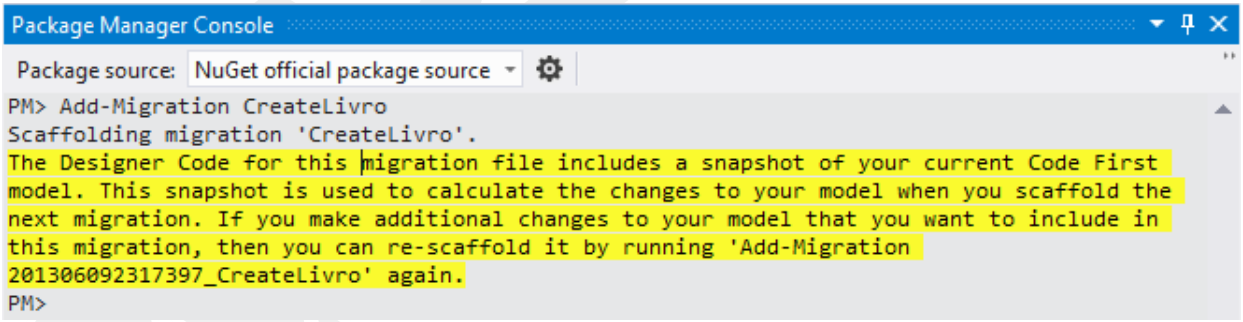
```
1 namespace EFMigrations
2 {
3   public class K19Context : DbContext
4   {
5     public DbSet<Editora> Editoras { get; set; }
6     public DbSet<Livro> Livros { get; set; }
7   }
8 }
```

Código C# B.12: Livraria.cs

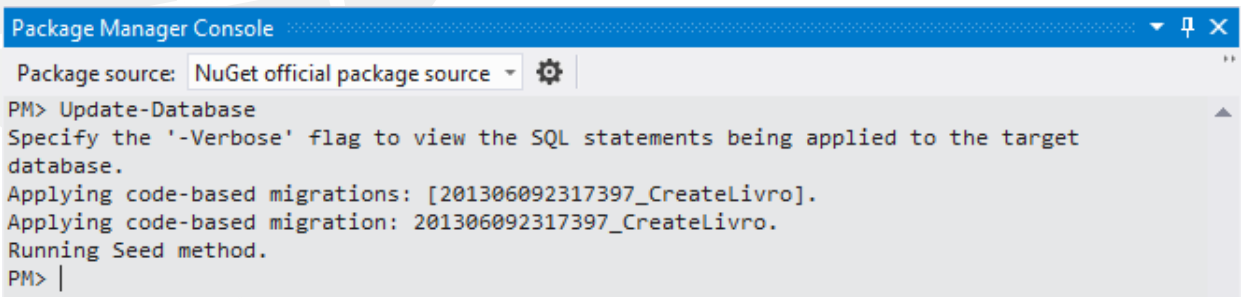
Após a criação da classe que define a entidade **Livro** e a alteração das classes `Editora` e `K19Context`, devemos criar uma **Migration** que atualizará o banco de dados criando uma nova tabela para o qual a classe **Livro** será mapeada.

Para adicionar a nova **Migration**, execute o seguinte comando:

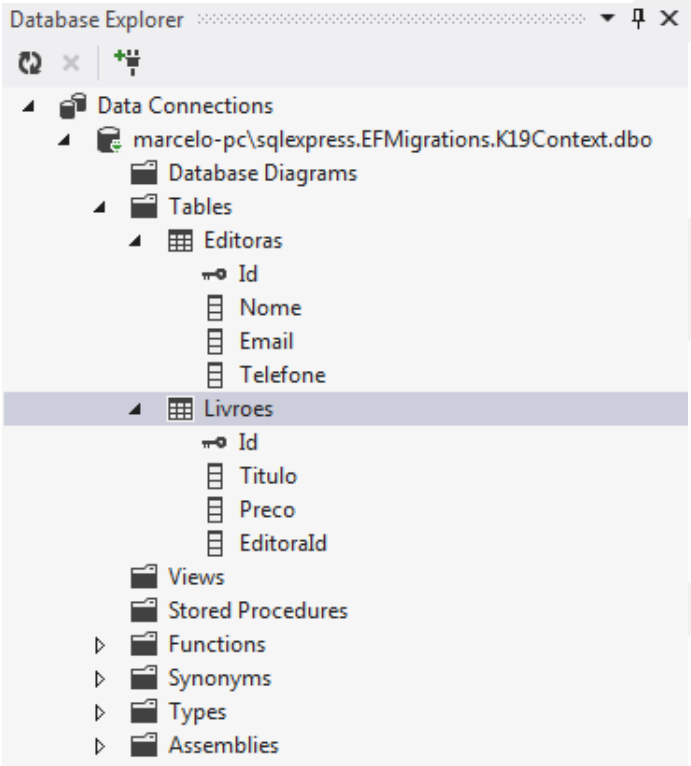
```
Add-Migration CreateLivro
```



Para atualizar o banco de dados e criar a nova tabela, devemos utilizar o comando `Update-Database`.



Após executar o comando `Update-Database`, verifique que a tabela para o qual a classe `Livro` foi mapeada foi criada no banco de dados.





Exercícios de Fixação

11 Acrescente a classe `Livro` conforme o código abaixo:

```
namespace CodeFirstMigrations
{
  class Livro
  {
    public int Id { get; set; }
    public string Titulo { get; set; }
    public decimal Preco { get; set; }
    public int EditoraId { get; set; }
    public Editora Editora { get; set; }
  }
}
```

Código C# B.13: Livro.cs

12 Altere a classe `K19Context` para mapear a classe `Livro` para uma tabela no banco de dados.

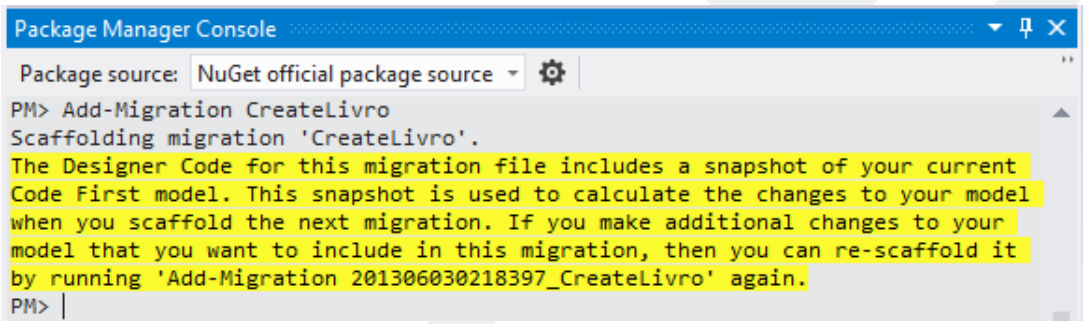
```
using System.Data.Entity;

namespace CodeFirstMigrations
{
  class K19Context : DbContext
```

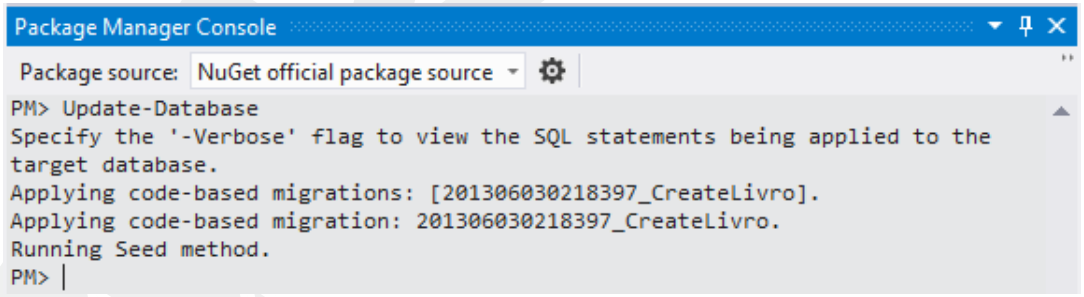
```
6   {
7       public DbSet<Editora> Editoras { get; set; }
8       public DbSet<Livro> Livros { get; set; }
9   }
10  }
```

Código C# B.14: K19Context.cs

13 Para gerar a tabela `Livros` no banco de dados, devemos criar uma migração. Crie uma migração chamada `CreateLivro` através do comando `Add-Migration`.



14 Execute o comando `Update-Database` através do `Package Manager Console` para gerar a tabela referente a classe `Livro` no banco de dados.



Verifique que a tabela referente a classe `Livro` foi gerada no banco de dados.

Operações do Code First Migrations

O Code First Migrations cria automaticamente as **Migrations** de acordo com as alterações realizadas nas classes que representam as entidades. Porém, as **Migrations** podem ser alteradas para realizar outras operações, como executar scripts SQL ou adicionar um indíce, por exemplo. Segue abaixo uma relação das operações que podem ser executadas durante uma **Migration**:

Operações	Descrição
AddColumn	Adiciona uma coluna a uma tabela existente
AddForeignKey	Cria uma chave estrangeira
AddPrimaryKey	Cria uma chave primária
AlterColumn	Altera a definição de uma coluna existente
CreateIndex	Cria um índice para uma coluna existente
CreateTable	Cria uma nova tabela no banco de dados
DropColumn	Remove uma coluna existente
DropForeignKey	Remove uma chave estrangeira existente
DropIndex	Remove um índice existente
DropPrimaryKey	Remove uma chave primária existente
DropTable	Remove uma tabela existente
MoveTable	Move a tabela para um novo *schema* de banco de dados
RenameColumn	Renomeia uma coluna existente
RenameTable	Renomeia uma tabela existente
Sql	Executa um comando SQL



Exercícios de Fixação

15 Crie um projeto do tipo **Console Application** chamado **CustomizandoCodeFirstMigrations** no **Visual Studio**.

16 Adicione ao projeto o Entity Framework através do *Manage Nuget Packages*.

17 Crie uma classe para representar a entidade **Pessoa**.

```
1  namespace CustomizandoCodeFirstMigrations
2  {
3    public class Pessoa
4    {
5      public int Id { get; set; }
6      public string Nome { get; set; }
7      public string Email { get; set; }
8      public string TelefoneFixo { get; set; }
9      public string Celular { get; set; }
10   }
11 }
```

Código C# B.15: Pessoa.cs

18 Crie uma classe **K19Context** para mapear a entidade **Pessoa** para o banco de dados.

```
1  namespace CustomizandoCodeFirstMigrations
2  {
3    public class K19Context : DbContext
4    {
5      public DbSet<Pessoa> Pessoas { get; set; }
6    }
7  }
```

Código C# B.16: K19Context.cs

19 Crie uma classe de teste que persiste um objeto da classe **Pessoa** no banco de dados.

```
namespace CustomizandoCodeFirstMigrations
{
  class Program
  {
    static void Main(string[] args)
    {
      using (var db = new K19Context())
      {
        Pessoa p = new Pessoa { Nome = "K19", Email = "contato@k19.com.br",
          Celular = "(11) 2387-3791", TelefoneFixo = "(11) 2387-3792" };
        db.Pessoas.Add(p);
        db.SaveChanges();
      }
    }
  }
}
```

Código C# B.17: Pessoa.cs

20 Altere a classe **Pessoa** para tornar obrigatório a propriedade **TelefoneFixo**. Renomeie também a propriedade **Celular** para **TelefoneCelular**.

```
namespace CustomizandoCodeFirstMigrations
{
  public class Pessoa
  {
    public int Id { get; set; }
    public string Nome { get; set; }
    public string Email { get; set; }
    [Required]
    public string TelefoneFixo { get; set; }
    public string TelefoneCelular { get; set; }
  }
}
```

Código C# B.18: Pessoa.cs

21 Para que as alterações sejam refletidas no banco de dados, é necessário habilitar o **Code First Migrations** no projeto. Execute o comando `Enable-Migrations` através do `Package Manager Console`.

22 Após habilitar o **Code First Migrations** no projeto, crie uma **Migration** que efetuará as operações de alteração no banco de dados. Execute o comando **Add-Migration AlteraTelefoneFixoERenomeiaCelularDePessoa**

23 Após a execução do comando `Add-Migration`, verifique que o arquivo criado irá remover a coluna *Celular* da tabela, ao invés de renomeá-la.

```
namespace CustomizandoCodeFirstMigrations.Migrations
```

```
2  {
3    using System;
4    using System.Data.Entity.Migrations;
5
6    public partial class AlteraTelefoneFixoERenomeiaCelularDePessoa : DbMigration
7    {
8      public override void Up()
9      {
10       AddColumn("dbo.Pessoas", "TelefoneCelular", c => c.String());
11       AlterColumn("dbo.Pessoas", "TelefoneFixo", c => c.String(nullable: false));
12       DropColumn("dbo.Pessoas", "Celular");
13     }
14
15     public override void Down()
16     {
17       AddColumn("dbo.Pessoas", "Celular", c => c.String());
18       AlterColumn("dbo.Pessoas", "TelefoneFixo", c => c.String());
19       DropColumn("dbo.Pessoas", "TelefoneCelular");
20     }
21   }
22 }
```

Código C# B.19: yyyyymmdds_AlteraTelefoneFixoERenomeiaCelularDePessoa.cs

Altere o arquivo de `Migration` para renomear a coluna, utilizando o método `Rename`.

```
1  namespace CustomizandoCodeFirstMigrations.Migrations
2  {
3    using System;
4    using System.Data.Entity.Migrations;
5
6    public partial class AlteraTelefoneFixoERenomeiaCelularDePessoa : DbMigration
7    {
8      public override void Up()
9      {
10       RenameColumn("dbo.Pessoas", "Celular", "TelefoneCelular");
11       AlterColumn("dbo.Pessoas", "TelefoneFixo", c => c.String(nullable: false));
12     }
13
14     public override void Down()
15     {
16       RenameColumn("dbo.Pessoas", "TelefoneCelular", "Celular");
17       AlterColumn("dbo.Pessoas", "TelefoneFixo", c => c.String());
18     }
19   }
20 }
```

Código C# B.20: yyyyymmdds_AlteraTelefoneFixoERenomeiaCelularDePessoa.cs

24 Após a alteração do arquivo de **Migration**, execute o comando `Update-Database` para alterar as colunas no banco de dados e, posteriormente, verique que as colunas foram renomeadas.



Migração para uma versão específica

O `Code First Migrations` permite ao desenvolvedor fazer a migração para uma versão específica. É possível, por exemplo, desfazer as alterações e voltar para uma versão anterior através do comando **Update-Database**. Basta definir o nome da versão através do parâmetro **-TargetMigration**:

- **Update-Database -TargetMigration:***{NomeDaMigrationAnterior}*

É possível também voltar para o estado inicial do banco de dados, executando o comando **Update-Database -TargetMigration:$InitialDatabase**.



Gerando Script SQL

O `Code First Migrations` executa as alterações no banco de dados através do comando **Update-Database**. O comando **Update-Database** gera script SQL também, basta adicionar o parâmetro `-Script`, que ao invés de executar o comando diretamente no banco de dados, ele irá gerar o script SQL que poderá ser visualizado ou salvo pelo usuário através do Visual Studio.

- **Update-Database -Script -TargetMigration:AddTelefoneToEditora**

Através do comando **Update-Database** é possível definir o estado inicial e a versão final para que o script SQL contenha todas as alterações necessárias para atualizar o banco de dados.

- **Update-Database -Script -SourceMigration:$InitialDatabase -TargetMigration:AddTelefone-ToEditora**

RESPOSTAS

APÊNDICE C

Exercício Complementar 2.1

```
using System.Data.Odbc;

namespace Odbc
{
  class CriaTabelaLivro
  {
    static void Main(string[] args)
    {
      string stringDeConexao = @"driver={SQL Server};
        server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";

      using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
      {
        conexao.Open();

        string sql =
          "CREATE TABLE Livros (" +
            "id BIGINT IDENTITY(1,1)," +
            "titulo VARCHAR (255) NOT NULL," +
            "preco FLOAT NOT NULL," +
            "editora_id BIGINT NOT NULL," +
            "CONSTRAINT PK_Livros PRIMARY KEY CLUSTERED (id asc)," +
            "CONSTRAINT FK_Livro_Editora FOREIGN KEY (editora_id) " +
            "REFERENCES Editoras (id)" +
          ")";
        OdbcCommand command = new OdbcCommand(sql, conexao);
        command.ExecuteNonQuery();
      }
    }
  }
}
```

Código C# 2.16: www.k19.com.br/exercicios/k32/02/C001/CriaTabelaLivro.cs

Exercício Complementar 2.2

```
namespace Odbc
{
  class Livro
  {
    public long? Id { get; set; }
    public string Titulo { get; set; }
    public double? Preco { get; set; }
    public long? EditoraId { get; set; }
  }
}
```

Código C# 2.17: www.k19.com.br/exercicios/k32/02/C002/Livro.cs

Exercício Complementar 2.3

```
using System;
using System.Data.Odbc;

namespace Odbc
{
  class InsereLivro
  {
    static void Main(string[] args)
    {
      string stringDeConexao = @"driver={SQL Server};
        server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";

      Livro livro = new Livro();
      System.Console.Write("Digite o Título do Livro:");
      livro.Titulo = System.Console.ReadLine();

      System.Console.Write("Digite o Preço do Livro:");
      livro.Preco = Convert.ToDouble(System.Console.ReadLine());

      System.Console.Write("Digite o Id da Editora do Livro:");
      livro.EditoraId = Convert.ToInt64(System.Console.ReadLine());

      string sql =
        @"INSERT INTO Livros (Titulo, Preco, Editora_Id)
        OUTPUT INSERTED.id
        VALUES('" + livro.Titulo + @"', '" + livro.Preco + @"', "
            + livro.EditoraId + ")";

      using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
      {
        OdbcCommand command = new OdbcCommand(sql, conexao);
        conexao.Open();
        livro.Id = command.ExecuteScalar() as long?;

        System.Console.WriteLine("Livro cadastrado com id: " + livro.Id);
      }
    }
  }
}
```

Código C# 2.18: www.k19.com.br/exercicios/k32/02/C003/InsereLivro.cs

Selecione a classe **InsereLivro** como Startup Object do projeto **Odbc**. Execute esse projeto e verifique se o registro foi inserido com sucesso na tabela Livros.

Exercício Complementar 2.5

```
using System;
using System.Data.Odbc;

namespace Odbc
{
  class InsereLivro
  {
    static void Main(string[] args)
    {
      string stringDeConexao = @"driver={SQL Server};
```

```
11          server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
12
13        Livro livro = new Livro();
14        System.Console.Write("Digite o Título do Livro:");
15        livro.Titulo = System.Console.ReadLine();
16
17        System.Console.Write("Digite o Preço do Livro:");
18        livro.Preco = Convert.ToDouble(System.Console.ReadLine());
19
20        System.Console.Write("Digite o Id da Editora do Livro:");
21        livro.EditoraId = Convert.ToInt64(System.Console.ReadLine());
22
23        string sql =
24          @"INSERT INTO Livros (Titulo, Preco, Editora_Id)
25          OUTPUT INSERTED.id
26          VALUES(?, ?, ?)";
27
28        using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
29        {
30          OdbcCommand command = new OdbcCommand(sql, conexao);
31
32          command.Parameters.AddWithValue("@Titulo", livro.Titulo);
33          command.Parameters.AddWithValue("@Preco", livro.Preco);
34          command.Parameters.AddWithValue("@EditoraId", livro.EditoraId);
35
36          conexao.Open();
37          livro.Id = command.ExecuteScalar() as long?;
38
39          System.Console.WriteLine("Livro cadastrado com id: " + livro.Id);
40        }
41      }
42    }
43  }
```

Código C# 2.23: www.k19.com.br/exercicios/k32/02/C005/InsereLivro.cs

Exercício Complementar 2.7

```
1  using System.Data.Odbc;
2  using System.Collections.Generic;
3
4  namespace Odbc
5  {
6    class ListaLivros
7    {
8      static void Main(string[] args)
9      {
10       string stringDeConexao = @"driver={SQL Server};
11         server=.\SQLEXPRESS;database=livraria;Trusted_Connection=Yes";
12
13       using (OdbcConnection conexao = new OdbcConnection(stringDeConexao))
14       {
15         string sql = "SELECT * FROM Livros";
16         OdbcCommand command = new OdbcCommand(sql, conexao);
17         conexao.Open();
18         OdbcDataReader resultado = command.ExecuteReader();
19
20                 List<Livro> lista = new List<Livro>();
21
22         while (resultado.Read())
23         {
24           Livro livro = new Livro();
25
26           livro.Id = resultado["Id"] as long?;
```

```
27            livro.Titulo = resultado["Titulo"] as string;
28            livro.Preco = resultado["Preco"] as double?;
29            livro.EditoraId = resultado["Editora_Id"] as long?;
30
31            lista.Add(livro);
32          }
33
34          foreach(Livro livro in lista)
35          {
36            System.Console.WriteLine("{0} : {1} - {2} - {3}\n",
37              livro.Id, livro.Titulo, livro.Preco, livro.EditoraId);
38          }
39        }
40      }
41    }
42  }
```

Código C# 2.29: www.k19.com.br/exercicios/k32/02/C007/ListaLivros.cs

Selecione a classe **ListaEditoras** como Startup Object do projeto **Odbc**. Execute esse projeto e veja a saída do programa.

Exercício Complementar 2.8

```
1  using System.Data.Odbc;
2
3  namespace Odbc
4  {
5    class CriaTabelaLivro
6    {
7      static void Main(string[] args)
8      {
9        using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
10       {
11         conexao.Open();
12
13         string sql =
14           "CREATE TABLE Livros (" +
15             "id BIGINT IDENTITY(1,1)," +
16             "titulo VARCHAR (255) NOT NULL," +
17             "preco FLOAT NOT NULL," +
18             "editora_id BIGINT NOT NULL," +
19             "CONSTRAINT PK_Livros PRIMARY KEY CLUSTERED (id asc)," +
20             "CONSTRAINT FK_Livro_Editora FOREIGN KEY (editora_id) " +
21             "REFERENCES Editoras (id)" +
22           ")";
23         OdbcCommand command = new OdbcCommand(sql, conexao);
24         command.ExecuteNonQuery();
25       }
26     }
27   }
28 }
```

Código C# 2.38: www.k19.com.br/exercicios/k32/02/C008/CriaTabelaLivro.cs

```
1  using System;
2  using System.Data.Odbc;
3
4  namespace Odbc
5  {
6    class InsereLivro
7    {
```

```
8     static void Main(string[] args)
9     {
10      Livro livro = new Livro();
11      System.Console.Write("Digite o Título do Livro:");
12      livro.Titulo = System.Console.ReadLine();
13
14      System.Console.Write("Digite o Preço do Livro:");
15      livro.Preco = Convert.ToDouble(System.Console.ReadLine());
16
17      System.Console.Write("Digite o Id da Editora do Livro:");
18      livro.EditoraId = Convert.ToInt64(System.Console.ReadLine());
19
20      string sql =
21        @"INSERT INTO Livros (Titulo, Preco, Editora_Id)
22        OUTPUT INSERTED.id
23        VALUES(?, ?, ?)";
24
25      using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
26      {
27        OdbcCommand command = new OdbcCommand(sql, conexao);
28
29        command.Parameters.AddWithValue("@Titulo", livro.Titulo);
30        command.Parameters.AddWithValue("@Preco", livro.Preco);
31        command.Parameters.AddWithValue("@EditoraId", livro.EditoraId);
32
33        conexao.Open();
34        livro.Id = command.ExecuteScalar() as long?;
35
36        System.Console.WriteLine("Livro cadastrado com id: " + livro.Id);
37      }
38    }
39  }
40 }
```

Código C# 2.39: www.k19.com.br/exercicios/k32/02/C008/InsereLivro.cs

```
1  using System.Data.Odbc;
2  using System.Collections.Generic;
3
4  namespace Odbc
5  {
6    class ListaLivros
7    {
8      static void Main(string[] args)
9      {
10       using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
11       {
12         string sql = "SELECT * FROM Livros";
13         OdbcCommand command = new OdbcCommand(sql, conexao);
14         conexao.Open();
15         OdbcDataReader resultado = command.ExecuteReader();
16
17                 List<Livro> lista = new List<Livro>();
18
19         while (resultado.Read())
20         {
21           Livro livro = new Livro();
22
23           livro.Id = resultado["Id"] as long?;
24           livro.Titulo = resultado["Titulo"] as string;
25           livro.Preco = resultado["Preco"] as double?;
26           livro.EditoraId = resultado["Editora_Id"] as long?;
27
28           lista.Add(livro);
29         }
30
31         foreach(Livro livro in lista)
32         {
33           System.Console.WriteLine("{0} : {1} - {2} - {3}\n",
```

```
34              livro.Id, livro.Titulo, livro.Preco, livro.EditoraId);
35          }
36        }
37      }
38    }
39  }
```

Código C# 2.40: www.k19.com.br/exercicios/k32/02/C008/ListaLivros.cs

Exercício Complementar 2.9

```
1  using System;
2  using System.Data.Odbc;
3
4  namespace Odbc
5  {
6    class AlteraLivro
7    {
8      static void Main(string[] args)
9      {
10       Livro livro = new Livro();
11
12       System.Console.Write("Digite o id do Livro que deseja alterar:");
13       livro.Id = Convert.ToInt64(System.Console.ReadLine());
14
15       System.Console.Write("Digite o novo Título do Livro:");
16       livro.Titulo = System.Console.ReadLine();
17
18       System.Console.Write("Digite o novo Preço do Livro:");
19       livro.Preco = Convert.ToDouble(System.Console.ReadLine());
20
21       System.Console.Write("Digite o Id da nova Editora do Livro:");
22       livro.EditoraId = Convert.ToInt64(System.Console.ReadLine());
23
24       string sql =
25         @"UPDATE Livros SET Titulo = ?, Preco = ?, Editora_Id = ? WHERE id = ?";
26
27       using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
28       {
29         OdbcCommand command = new OdbcCommand(sql, conexao);
30
31         command.Parameters.AddWithValue("@Titulo", livro.Titulo);
32         command.Parameters.AddWithValue("@Preco", livro.Preco);
33         command.Parameters.AddWithValue("@EditoraId", livro.EditoraId);
34         command.Parameters.AddWithValue("@Id", livro.Id);
35
36         conexao.Open();
37         command.ExecuteNonQuery();
38       }
39     }
40   }
41 }
```

Código C# 2.41: www.k19.com.br/exercicios/k32/02/C009/AlteraLivro.cs

Exercício Complementar 2.10

```
1  using System;
```

```
using System.Data.Odbc;

namespace Odbc
{
  class RemoveLivro
  {
    static void Main(string[] args)
    {
      using (OdbcConnection conexao = ConnectionFactory.CreateConnection())
      {
        System.Console.WriteLine("Digite o id do livro que deseja remover: ");
        long id = Convert.ToInt64(System.Console.ReadLine());

        string sql = "DELETE FROM Livros WHERE id = ?";

        OdbcCommand command = new OdbcCommand(sql, conexao);

        command.Parameters.AddWithValue("@Id", id);

        conexao.Open();

        command.ExecuteNonQuery();
      }
    }
  }
}
```

Código C# 2.42: www.k19.com.br/exercicios/k32/02/C010/RemoveLivro.cs

Exercício Complementar 5.1

```
@{
  ViewBag.Title = "Index";
}

@for(int i = 0; i < 10; i++)
{
  <h2>Olá @i</h2>
}
```

Código CSHTML 5.10: Index.cshtml